DX Framework

Большой учебник по своему движку и космическому симулятору

Математика с нуля · исходный C++ код · ECS · NPC и ремонт · стрельба и пули

Bullet Physics · DirectX 11 · вершинные и пиксельные шейдеры · системы корабля

Практические работы · выпускная сцена · один план развития на 14 дней

Издание 8.0 · 29 октября 2026 года

Подготовлено по исходникам текущего репозитория DX Framework

Содержание

Перед началом: как превратить этот учебник в знание своего кода 7
Как читать книгу по кругам 7
Практика чтения исходника: шесть вопросов к каждой строке 7
Где искать первоисточник 8
Что считать усвоенным 8
Как читать и менять DX Framework с нуля 9
1. Как разобрать строку C++ и не потерять смысл 9
2. Разбираем `ECSRegistry.h` буквально по шагам 10
3. Главный кадр: читаем порядок, а не названия систем 12
4. Как добавлять систему так, чтобы она заработала во всей игре 13
Математика DX Framework с нуля: учебник для человека, который начинает с арифметики 15
1. Число в игре: значение, единица, диапазон и точность 15
2. Координаты: как назвать место в двумерном и трёхмерном мире 16
3. Вектор: стрелка с направлением и длиной 17
4. Скалярное произведение: узнать угол и направление без рисунка 19
5. Векторное произведение и проекция: сторона, нормаль, поверхность 20
6. Матрица: таблица чисел, а не загадочный графический объект 21
7. Сложение, вычитание и умножение матрицы на число 22
8. Умножение матриц: как заполнить одну клетку результата 23
9. Обратная матрица: действие, которое отменяет другое действие 24
10. Матрицы 4×4: точка, направление, масштаб, поворот и перенос 25
11. Угол, окружность, тригонометрия и ориентация 26
12. Камера, перспектива и луч из пикселя 28
13. Время, скорость, ускорение и численное движение 30
14. Математика прямо в системах космического симулятора 32
15. Большой практикум: реши до просмотра ответов 35
16. Карта источников и правила дальнейшего изучения 37
Математика и алгоритмы для игры: второй курс с выводами и расчётами 39
1. Линейная интерполяция и экспоненциальное сглаживание 39
2. Численные производная и интеграл: скорость из позиции, путь из скорости 39
3. Алгоритм маршрута: граф, стоимость и A* 40
4. Выбор работы экипажа как задача оптимизации 40
5. Вероятность, разброс оружия и воспроизводимость 41
6. Пространственные алгоритмы: AABB и отсев 41
7. Численная устойчивость и тесты свойств 41
8. Луч, плоскость и сфера: вычисляем попадание до Bullet 42
9. Столкновение прямоугольников и разделяющая ось 42
10. Управление дверью или камерой: ошибка, скорость и устойчивость 43

Практический курс: от первого кадра до своей сцены 44
Урок 1. От запуска приложения до одного игрового кадра 44
Урок 2. ECS как модель игрового мира 45
Урок 3. Сцена, редактор и первый игровой уровень 46
Урок 4. Математика как инструмент, а не список формул 47
Урок 5. Bullet Physics от формы до игрового события 48
Урок 6. Рендер DirectX 11 и смысл каждого шейдера 49
Урок 7. Корабль как связанная система, а не набор HUD чисел 51
Урок 8. Дружественный NPC от аварии до завершённого ремонта 52
Урок 9. Вражеский NPC от зрения и поиска до попадания 54
Урок 10. NavMesh, двери и движение персонажа 56
Урок 11. Как добавлять системы без разрушения архитектуры 56
Урок 12. Анимация, оружие и согласование позы 58
Урок 13. Сохранение, фоновые задачи и устойчивость сцены 58
Урок 14. Интерфейс, звук и понятная обратная связь 59
Урок 15. Контент, производительность и граница почти готовой игры 60
Урок 16. Выпускная работа на одном корабле 61
Системы DX Framework: чтение реального кода от кадра до пикселя 63
1. Один кадр целиком: как читать `ApplicationClass::Frame` 63
2. ECS: сущность, компонент, система и владение состоянием 64
3. Сцена и сохранение: путь от Entity к JSON и обратно 66
4. Bullet и ECS: два представления одного физического объекта 67
5. Вода, глубина, пробоины и атмосферное давление: не смешивать среды 69
6. Электросеть, генератор и зависимые системы корабля 71
7. Renderer и шейдеры: от Transform до цвета пикселя 73
8. Выстрел, Bullet raycast и `DamageEvent`: что происходит с «пулей» 75
9. Фоновые задачи и ассеты: когда результат готов к использованию 77
10. Как добавлять систему, чтобы она стала частью игры 79
11. Атмосфера комнат: уже работающая игровая модель и её границы 80
12. Секции корабля, масса и разрушаемые соединения 81
Дополнительные системы DX Framework: от нажатия клавиши до события и эффекта 82
1. Ввод игрока: от физической клавиши к игровому действию 82
2. Триггерные зоны: геометрия, память пересечения и действие 83
3. Инвентарь: одна Entity оружия и три слота 83
4. Анимация: игровое время, поза и вид от первого лица 84
5. Атмосфера комнат: простая игровая модель и физические пределы 84
6. Частицы: рождение, жизнь и отрисовка 85
7. Взрыв: видимый эффект и импульс для физических тел 85
8. Тревога корабля: выбор самого серьёзного сообщения 86

9. Room culling: какие комнаты видит камера 86
10. Показатели игрока: перевод комнаты в здоровье и выносливость 87
Рендер и физика: объяснение от данных до кадра 88
18. CPU, GPU и жизнь одного draw call 88
19. Пиксельный шейдер и физический смысл материала 89
20. Как добавить новый шейдер безопасно 89
Bullet Physics и симуляция мира: углублённый разбор 91
21. Тело, форма и мир физики 91
22. Персонаж, дверь и разрушение 92
23. Физические проверки, которые нужны перед игрой 92
Шейдеры с нуля: от одного треугольника до материала DX Framework 93
1. Один треугольник до первого HLSL слова 93
2. Читаем простейший vertex shader почти по строкам 93
3. Что делает rasterizer между VS и PS 94
4. Простейший pixel shader и ловушка текущего файла 94
5. Текстура: UV, sampler и прозрачный вырез 94
6. Нормаль, Lambert и первая формула освещения 95
7. От обычного света к PBR без заклинаний 95
8. Как добавить свой маленький шейдерный эффект в движок 96
9. Буфер констант — точное соглашение о байтах 96
10. Какие ещё бывают шейдеры и нужны ли они здесь 96
11. Цвет, яркость и тональная коррекция 97
12. Задачи для самостоятельной проверки шейдеров 97
Корабельные сети и среда: углублённый разбор 98
Разбор 24. Корабельные сети как связанные правила 98
Разбор 25. Давление, затопление и пространство 99
Разбор 26. Авария, ремонт и наблюдаемый результат 100
NPC в DX Framework: от данных ECS до ремонта, поиска и выстрела 101
1. Сначала поймём один игровой кадр 101
2. Карта данных: кому принадлежит какое состояние 102
3. Мирный NPC: как задача ремонта превращается в назначение 104
4. Исполнение ремонта почти по строкам 106
5. Навигация: граф комнат, локальная сетка и Bullet капсула 109
6. Противник: выбор, зрение, память и состояния боя 111
7. Укрытие, оружие и очередь урона 113
8. Прогоним данные руками: два проверяемых сценария 116
9. Как добавлять поведение, не разрывая цепочку 118
10. Диагностика по наблюдаемым данным 119
11. Что сейчас является пределом реализации 121

Редактор, сцена и самостоятельная работа с кодом 122
1. Как объект проходит путь от кнопки редактора до мира 122
2. `SaveScene` построчно: почему в JSON попадает не всё 123
3. `LoadScene` построчно: пересоздание мира 123
4. Практикум: один новый компонент проходит весь путь 124
5. Завершённый эпизод вместо бесконечного набора механик 125
Новая механика и дорога к выпуску 126
Разбор 44. Спецификация новой механики до кода 126
Разбор 45. Вставка в кадр и интеграционные границы 126
Разбор 46. Сохранение и восстановление состояния игры 126
Разбор 47. UI и звук как часть правил 127
Разбор 48. Производство ресурсов 127
Разбор 49. Миссия и проверяемый вертикальный срез 127
Разбор 50. Сборка, внешний запуск и критерий завершения 128
Производственная часть игры 129
Урок 17. Конвейер моделей, материалов и анимации 129
Урок 18. Миссия и правила игрового эпизода 130
Урок 19. Наблюдаемость и диагностика без догадок 131
Урок 20. Сборка, упаковка и внешний прогон 132
Математический задачник для движка 133
Задача 1. Движение NPC к рабочей точке 133
Задача 2. Угол зрения через dot product 133
Задача 3. Выбор правильной стороны поворота 133
Задача 4. Порядок матриц на точке 134
Задача 5. Нормаль при неравномерном масштабе 134
Задача 6. Экранная точка и picking ray 134
Задача 7. Порог прибытия и гистерезис 135
Задача 8. Полезность ремонта и цена маршрута 135
Задача 9. Прицеливание и движение цели 135
Задача 10. Физика взрыва и импульс 135
Задача 11. Аккумулятор и мощность 136
Задача 12. Согласованность симуляции после сохранения 136
Практикум создания космического симулятора 137
Лабораторная 1. Генератор питает дверь и кислородный прибор 137
Лабораторная 2. Две пробоины и два инженера 138
Лабораторная 3. Враг видит, ищет и стреляет честно 139
Лабораторная 4. Новый пожар и тушение экипажем 139
Лабораторная 5. Релизная сборка и проверка вне исходников 140
Единственный план развития: 14 дней 142

78. День 1. Воспроизводимая сцена и базовая диагностика 142
79. День 2. Назначение и освобождение ремонта 142
80. День 3. Достижимость и двери до выбора цели 142
81. День 4. Роли экипажа и расход материалов 143
82. День 5. Пожар как часть симуляции корабля 143
83. День 6. Экипаж тушит пожар 143
84. День 7. Восприятие врага без всеведения 144
85. День 8. Огонь, анимация и перезарядка 144
86. День 9. Укрытия и выход из тупиков 145
87. День 10. Первая командная тактика 145
88. День 11. Авария корабля меняет решения 145
89. День 12. События, звук и тревога 145
90. День 13. Сохранение, нагрузка и устойчивость 146
91. День 14. Вертикальный срез и кандидат к выпуску 146

В PDF оглавление обновляется при экспорте. В Word его можно обновить клавишей F9.

Перед началом: как превратить этот учебник в знание своего кода

Эта книга написана для чтения без открытой IDE. Её задача — дать тебе достаточно последовательное объяснение, чтобы во время работы ты мог понять незнакомую часть своего движка, а вечером открыть нужный метод и проверить понимание. Текст идёт от числа и строки C++ к Entity, кадру, физике, рендеру, NPC и законченной игровой сцене. Для важных путей выполнения указаны конкретные файлы и методы. Номер строки — ориентир на 29 октября 2026 года; после правок он может сдвинуться. Ищи имя функции, а не считай номер строки вечным адресом.

У книги есть два вида утверждений. Формулировка «в текущем коде» означает, что поведение найдено в исходниках этого репозитория. Формулировка «предлагается», «добавь», «проектное улучшение» означает задачу для развития игры. Это особенно важно для корабельной среды и NPC. В существующих `PressureSystem` и `FloodingSystem` есть водная модель глубины и затопления; полноценная разгерметизация в вакуум описывается как новая система. В главном игровом кадре вызов `NpcSquadCombatSystem::Update` закомментирован; групповая тактика остаётся проектной работой. Смешивать готовое и задуманное опасно: можно потратить вечер на поиск функции, которой ещё нет.

Как читать книгу по кругам

Первый круг — понять язык задачи. Пройди главы об арифметике, векторах и матрицах с карандашом. Каждое действие сделай на маленьких целых числах. Например, при сложении двух матриц 2×2 сложи четыре пары чисел отдельно и убедись, что размеры совпали. Когда сможешь объяснить, почему точка переносится, а направление не переносится, переходи к преобразованиям мира и камеры. До этого `world * view * projection` будет только заклинанием.

Второй круг — проследить один объект. Создай Entity, добавь Transform и нужные компоненты, найди систему, которая их читает, посмотри, в каком месте `ApplicationClass::Frame` она вызывается. Проследи результат до Inspector, рендера, Bullet и JSON сцены. Не пытайся запомнить весь репозиторий сразу. У каждого механизма найдётся цепь «входные данные → решение → изменение состояния → видимый результат → сохранение». Если потерялся, возвращайся к этой цепи.

Третий круг — пройти сквозной сценарий. Один инженер видит пробоину, получает работу, строит путь, открывает дверь, подходит к рабочей точке, наращивает прогресс и завершает ремонт. Затем один враг видит игрока, выбирает цель, целится, создаёт выстрел, Bullet определяет попадание, а DamageSystem меняет здоровье. Эти сценарии связывают главы между собой. Для каждого перехода выпиши ответственного владельца: данные компонента, систему, вызов в кадре, событие или UI. Если два класса одновременно считают себя владельцем одного решения, это повод уточнить архитектуру.

Четвёртый круг — изменить один механизм. Прежде чем писать код, опиши наблюдаемое правило для игрока и единицы чисел. Потом укажи компонент, систему, место вызова, редактор, сериализацию, диагностику и проверку. Главы о добавлении системы используют пожар как пример такого полного пути. Даже маленькое поле `float` является частью нескольких договоров: кто его задаёт, кто меняет, когда оно обновляется и переживает ли загрузку.

Практика чтения исходника: шесть вопросов к каждой строке

Когда встретишь условие вроде `if (!target || !target->active) return;`, переведи его в обычную речь: «если цели нет либо она неактивна, функция прекращает действие». Затем спроси, какой код создаёт `target`, может ли указатель стать недействительным после удаления Entity и какое состояние

останется у NPC после `return`. Одной расшифровки синтаксиса недостаточно: строка имеет смысл только вместе с жизненным циклом данных.

К каждой важной строке задавай шесть вопросов. **Что читает?** Перечисли аргументы и поля. **Что пишет?** Назови компонент или внешний ресурс. **В каких единицах?** Отличи метры от секунд и мощность от энергии. **Когда выполняется?** Найди место в кадре и режим Editor/Play. **Что может отсутствовать?** Проверь `nullptr`, удалённую Entity, пустой путь и отсутствующий GPU ресурс. **Как увидеть результат?** Найди debug поле, лог, Inspector или простую тестовую сцену. Если ответы записаны, строчка перестаёт быть магией.

Возьми выражение `progress += speed * dt`. При `speed=10` единиц работы в секунду и `dt=0,1` секунды добавится 1 единица. Если обновление выполняется десять раз, прогресс вырастет на 10. Если убрать `dt`, на 30 FPS работа будет идти вдвое медленнее, чем на 60 FPS, при одинаковом реальном времени. Если выполнить тот же update и в `StateSystem`, и в `RepairSystem`, работа ускорится вдвое. Поэтому после понимания формулы найди единственного владельца записи `progress`.

Где искать первоисточник

Код движка находится в `src`. Начни с `src/Core/ApplicationClass.cpp` для порядка кадра и режимов, `src/ECS/ECSRegistry.h` для сущностей и компонентов, `src/Scene/SceneManager.cpp` для сохранения и загрузки, `src/Editor/EditorUI.cpp` для создания и редактирования объектов. NPC живут в `src/AI` и связанных системах; физика, рендер и компоненты находятся в соответствующих каталогах `src`. Для неизвестного символа ищи точное имя методом поиска по проекту. Если имя встречается и в декларации, и в реализации, сначала прочитай публичный контракт, затем тело метода, затем все вызовы.

Внешние правила проверяй по первоисточникам, а не по случайному примеру из другой версии движка. Соглашение DirectXMath о векторах-строках и матрицах описано в документации Microsoft (https://learn.microsoft.com/en-us/windows/win32/dxmath/pg-xnamath-getting-started); подготовка матриц для shader buffers — в разделе оптимизации DirectXMath (https://learn.microsoft.com/en-us/windows/win32/dxmath/pg-xnamath-optimizing). Назначение стадий Direct3D объясняет официальный материал о rendering pipeline (https://learn.microsoft.com/en-us/windows/uwp/gaming/tutorial--assembling-the-rendering-pipeline). Для деталей Bullet используй официальное руководство и исходный код (https://github.com/bulletphysics/bullet3/blob/master/docs/Bullet_User_Manual.pdf), а для NavMesh — документацию Recast Navigation (https://github.com/recastnavigation/recastnavigation/blob/main/Docs/_1_Introduction.md). Эти источники объясняют библиотеки; окончательный ответ о поведении твоей игры даёт вызов в текущем коде.

Что считать усвоенным

Тема освоена, когда ты без подсказки проходишь четыре шага: объясняешь правило обычными словами; решаешь маленький численный пример; показываешь путь данных по файлам; воспроизводишь результат в минимальной сцене. Для матрицы это означает не только помнить запись M^{-1}, но и проверить на бумаге $M \cdot M^{-1} = I$, понять, зачем она нужна лучу камеры, и получить ожидаемую точку после преобразования. Для NPC это означает не только знать названия состояний, но и объяснить, почему он стоит, кому принадлежит задача и почему выстрел разрешён или запрещён.

Книга рассчитана на самостоятельную работу, но исходники продолжают изменяться. После правки проверяй инварианты: удалённая Entity не остаётся целью; энергия не создаётся из воздуха; прогресс зависит от игрового времени; видимость требует геометрического подтверждения; сохранённые ссылки восстанавливаются на новые Entity; важное игровое состояние не живёт только в shader или UI. Такой способ проверки останется полезным, даже когда имена классов изменятся.

Как читать и менять DX Framework с нуля

Этот раздел рассчитан на человека, который умеет открыть проект, но хочет заново понять собственный C++ код. Здесь не требуется предварительно помнить названия паттернов. Мы будем идти от одного значения в памяти к компоненту, от компонента к системе, затем к целому кадру. Если встретилось незнакомое слово, сначала выясни, какие данные оно обозначает и кто ими владеет. Только после этого переходи к следующему методу. Файлы и имена ниже описывают состояние проекта на 29 октября 2026 года; строки могут сдвинуться после изменения исходников, поэтому ищи указанное имя метода.

1. Как разобрать строку C++ и не потерять смысл

1.1. Тип, имя, значение и время жизни

Возьми простую строку `float deltaTime = 0.016f;`. `float` задаёт тип числа с плавающей точкой, `deltaTime` — имя, `0.016f` — начальное значение. Суффикс `f` означает литерал типа float, иначе дробный литерал в C++ обычно имеет тип double. Но тип не сообщает единицы измерения. В игровом кадре `deltaTime` — секунды, а `position` обычно измеряется в метрах. Если прибавить `deltaTime` к позиции без скорости, объект будет двигаться на численно похожее расстояние, но игровое правило окажется неявным. При чтении каждой арифметической строки подпиши единицы аргументов и результата.

Локальная переменная создаётся при входе в область видимости и уничтожается при выходе из неё. Поле объекта живёт столько же, сколько объект. Динамически созданный ресурс живёт до освобождения владельцем. Это различие важно в `ApplicationClass::Frame`: локальный `registry` как ссылка на `m_sceneManager.GetECSRegistry()` не создаёт второй мир. Когда функция заканчивается, сама ссылка исчезает, но Registry остаётся полем SceneManager. Нельзя вернуть указатель на локальную временную переменную из функции и продолжать пользоваться им в следующем кадре.

1.2. Значение, ссылка и указатель

`Entity e` передаёт числовой идентификатор по значению: изменение локальной копии не меняет ID у вызывающего кода. `ECSRegistry& registry` означает ссылку на существующий Registry: система работает с тем же объектом, а не копирует все компоненты. `const RepairableComponent* repairable` — указатель, который может быть `nullptr`, и через который нельзя менять компонент, пока действует `const`. Звёздочка в объявлении указывает тип указателя; `*repairable` в выражении разыменовывает его. `repairable->progress` — доступ к полю через указатель. Разыменовать `nullptr` нельзя.

Если `GetComponent<T>(entity)` возвращает `T*`, следующий безопасный шаг — проверить результат. Сам `Entity` может быть живым, но компонента T у него нет. Для примера, у декоративного ящика есть Transform, но нет Repairable. Правильное условие `if (auto* repairable = registry.GetComponent<RepairableComponent>(e))` связывает проверку и использование. Однако даже ненулевой указатель не обещает вечной жизни: после `DestroyEntity`, `RemoveComponent` или структурной операции с pool проверь его заново. Особо опасно хранить `T&` между вызовами систем, которые могут удалить цель.

1.3. `const`, `auto` и `enum`

`const` не означает, что весь игровой мир заморожен; оно ограничивает изменение через конкретный доступ. `const auto* brain = registry.GetComponent<NpcBrainComponent>(e)` позволяет читать brain, но не записывать state через этот указатель. `auto` выводит тип выражения, а не превращает язык в динамический: если справа `T*`, тип остаётся указателем. При чтении сложного кода при необходимости выпиши настоящий тип `auto`, иначе легко перепутать копию с ссылкой. `auto&` создаёт ссылку, `auto` в range loop может копировать элемент.

`enum class` задаёт именованные варианты состояния: `NpcState::Idle`, `NpcState::Attacking` и так далее. Он помогает компилятору отличать состояние NPC от произвольного числа. Но enum сам не определяет переходы: их задают условия систем. Если персонаж одновременно хранит `state=Attacking` и `target=INVALID_ENTITY`, это логически противоречивое состояние, даже если оба поля по отдельности имеют допустимые типы. Составь инварианты: при Attacking цель жива и враждебна; при Repairing есть действительная цель и достижимая рабочая точка. Claim нужен для автоматического назначения; для оборудования может использоваться WorkZone, а пробоина в текущем коде обрабатывается без него.

1.4. `std::vector`, `unordered_map` и итерация

`std::vector<T>` хранит последовательность. Добавление элемента иногда перераспределяет память, поэтому старые указатели и ссылки на элементы могут стать недействительными. `std::unordered_map<Key,Value>` хранит значения по ключу; изменение структуры контейнера влияет на итераторы по правилам конкретной операции. В ECS Registry каждый тип компонента имеет pool, а `ForEach` итерирует его. Поэтому не удаляй текущий компонент из того же pool прямо во время `ForEach`, если контракт `ComponentManager` этого явно не гарантирует. Безопасный подход: во время обхода складывай Entity ID в `pendingDestroy`, после обхода применяй удаления. Это не просто стиль — так сохраняется корректность итератора и ссылок.

1.5. Лямбда и захват

`[&](Entity e, auto& component) { ... }` — функция без имени, которую можно передать `ForEach`. `[&]` разрешает использовать внешние локальные переменные по ссылке. Это удобно для накопления результатов, но опасно для фоновой задачи: после выхода из вызывающей функции захваченная ссылка может стать висячей. В синхронном `ForEach`, который вызывает лямбду до возврата, захват по ссылке обычно соответствует времени жизни. При `JobSystem::SubmitJob` код может выполниться позже; копируй нужные входные данные и применяй итог в главном потоке после проверки живости Entity и версии сцены.

1.6. RAII и владение ресурсом

RAII означает, что освобождение ресурса связано с уничтожением объекта-владельца. `std::unique_ptr<IComponentPool>` внутри `ECSRegistry` ясно выражает единоличное владение pool. Если Registry уничтожается, unique_ptr освободит pool. Bullet body, GPU buffer и job queue живут по другим правилам: удалить C++ объект недостаточно, если он всё ещё зарегистрирован в physics world или привязан к GPU command. Для каждого такого ресурса составь пару «создание — удаление» и момент отписки от внешнего мира. Если scene reload оставил collider без Entity, это нарушение жизненного цикла, а не визуальный баг.

Проверочное упражнение. Прочитай сигнатуру `void StateSystem::Update(ECSRegistry& registry, float deltaTime, JobSystem& jobs)`. Назови, какие аргументы копируются, какие ссылаются на существующий объект, где ожидаются единицы, какие изменения возможны. Затем найди один вызов функции из `ApplicationClass::Frame`. Проверь, передан ли `simulationDeltaTime`, когда Play поставлен на паузу. Не переходи к телу метода, пока сигнатура не понятна.

2. Разбираем `ECSRegistry.h` буквально по шагам

2.1. `CreateEntity`: от ID до живой сущности

Файл `src/ECS/ECSRegistry.h`, метод `CreateEntity` (в текущем файле около строк 15–22). Строка с `if (nextEntity == INVALID_ENTITY)` защищает от выдачи зарезервированного значения. `throw std::overflow_error` останавливает операцию явной ошибкой; без этой проверки переполнение ID могло бы дать новое значение, совпадающее с «нет сущности». `const Entity entity = nextEntity++;` сначала берёт прежний номер, затем увеличивает счётчик. `m_liveEntities.insert(entity)` фиксирует, что ID существует. Возврат отдаёт номер вызывающему коду. В этот момент у сущности ещё нет

Transform, RenderComponent или NPCBrain: CreateEntity создаёт идентичность, а не готовый игровой объект.

Проверь на бумаге: если `nextEntity=10`, первый вызов возвращает 10 и оставляет 11; второй возвращает 11. `IsAlive(10)` после первого вызова истинно, но `HasComponent<TransformComponent>(10)` может быть ложно. Это объясняет, почему одно лишь создание Entity не рисует объект. Если игра удалит 10, текущая реализация не переиспользует этот номер немедленно; generation handle здесь не реализован. При добавлении повторного использования в будущем понадобятся поколения, чтобы старый сохранённый ID не указывал на новую сущность.

2.2. `DestroyEntity`: удаление из набора и pools

Около строк 24–28 `DestroyEntity` сначала делает `m_liveEntities.erase(e)`, затем проходит `for (auto& [ti, pool] : m_pools)` и вызывает `pool->Remove(e)` у каждого типа компонента. Это универсальная очистка ECS данных: Registry не обязан знать заранее, какие компоненты были у Entity. Но у внешних систем есть дополнительные ресурсы. Если компонент содержал `btRigidBody*`, перед уничтожением нужно убедиться, что тело исключено из Bullet world и освобождено по договору владельца. Если async job хранит ссылку на компонент, удаление до завершения job опасно. Поэтому в gameplay коде зачастую нужен более высокий `DespawnEntity` путь, который согласует ECS, physics, renderer, jobs и временные reservations.

На проверочной сцене создай Entity с двумя компонентами, вызови Destroy, затем проверь `IsAlive=false`, `GetComponent<T>=nullptr` для обоих. Потом проверь физический мир: число тел тоже должно уменьшиться, если Entity владело телом. Если не уменьшилось, Registry выполнил свой контракт, а вызывающий lifecycle путь — нет. Важно различать эти уровни ответственности.

2.3. `AddComponent` и две перегрузки

Около строк 33–43 есть две формы. `AddComponent(Entity e, const T& comp)` копирует объект и разрешена только для copy-constructible T. `AddComponent(Entity e, T&& comp)` пересылает временное значение или объект, который можно переместить; это нужно для некопируемых компонентов, например тех, что владеют физическим ресурсом. `GetOrCreatePool<T>()` находит pool этого типа либо создаёт его. Обрати внимание: в этих строках не видно проверки `IsAlive(e)`. Следовательно, вызывающий код должен соблюдать контракт живой Entity или нужно усилить Registry; не предполагай невидимую защиту. Если компонент добавлен на несуществующий ID, pool может хранить мусорную запись, которая не соответствует `m_liveEntities`.

Для новичка полезна аналогия: Entity ID — номер папки, component pool — отдельная таблица одного вида данных. `AddComponent` добавляет строку в таблицу по номеру папки. Если номер папки неверен, таблица не обязательно сама поймёт это. Поэтому фабрика объекта должна создавать Entity один раз, затем последовательно добавлять обязательные компоненты и валидировать готовую конфигурацию.

2.4. `GetComponent`, `HasComponent`, `RemoveComponent`

`GetComponent<T>` находит pool типа T. Если pool отсутствует, возвращает `nullptr`; иначе вызывает `pool->Get(e)`. `HasComponent<T>` проверяет наличие pool и Entity в нём. `RemoveComponent<T>` удаляет только этот тип у выбранной Entity. Ни один из этих методов не меняет `m_liveEntities`. Следовательно, NPC может оставаться живым, но временно потерять оружие, и системы должны обрабатывать это как допустимое или ошибочное состояние по своему правилу. Нельзя писать `*registry.GetComponent<T>(e)` без проверки, если отсутствие T возможно.

`GetComponents<T>()` в отличие от `GetComponent` создаёт pool, если его ещё не было. Это полезно для системного обхода, но имеет побочный эффект создания пустого pool. При диагностике «почему появился pool» учитывай это. Для read-only запроса к одному объекту предпочитай `GetComponent`/`HasComponent`, чтобы чтение не изменяло структуру Registry без необходимости.

2.5. `ForEach<T1,T2>` построчно

Около строк 45–59 берутся два указателя на pools: `FindPool<T1>()` и `FindPool<T2>()`. Если один отсутствует, `return` заканчивает обход без вызова callback. Затем цикл проходит каждый элемент первого pool. Для текущего Entity проверяется `pool2->Has(entity)`. Если второй компонент найден, берётся ссылка `comp2` и вызывается `fn(entity, comp1, comp2)`. Это значит, что порядок типов влияет на количество итераций: `ForEach<Редкий,Частый>` обычно делает меньше проверок, чем `ForEach<Частый,Редкий>`. Но менять порядок нужно только если callback аргументы и логика соответственно переставлены.

Ключевое ограничение: callback получает ссылки на элементы pools. Если внутри callback удалить текущий компонент или Entity, цикл может потерять валидность итератора. Даже если конкретный контейнер иногда переживает удаление другого элемента, не строй на этом неявную гарантию. Собирай команды на структурные изменения отдельно. Вопрос для самопроверки: `ForEach<NpcBrainComponent,TransformComponent>` найдёт ли Entity только с Transform? Нет: она отсутствует в первом pool и не участвует в цикле. Найдёт ли Entity только с Brain? Цикл её увидит, но проверка второго pool не пропустит callback.

2.6. `type_index` и закрытая часть класса

В `private` находятся `nextEntity`, набор живых ID и `unordered_map<std::type_index,std::unique_ptr<IComponentPool>>`. `typeid(T)` превращается в ключ вида компонента. `FindPool<T>` ищет ключ и приводит базовый `IComponentPool*` к конкретному `ComponentManager<T>*`; корректность зависит от того, что под данным ключом всегда лежит pool именно T. `GetOrCreatePool<T>` создаёт его при первом обращении. Это простая реализация ECS, которую можно понять без магии: каждый тип имеет собственную таблицу, а Registry связывает таблицы общим Entity ID.

Упражнение: вручную нарисуй три pools — Transform у Entity 1 и 2, Repairable у Entity 2 и 3, NPCBrain у Entity 1. Что вызовет `ForEach<TransformComponent,RepairableComponent>`? Только Entity 2. Что произойдёт после `DestroyEntity(2)`? ID 2 удалён из набора живых и обоих pools. После этого попытка `GetComponent<RepairableComponent>(2)` должна вернуть null. Проверь тем же набором в маленьком тестовом запуске.

3. Главный кадр: читаем порядок, а не названия систем

3.1. Вход `Frame(float dt)` и режимы времени

`ApplicationClass::Frame` начинается около строки 1023 `src/Core/ApplicationClass.cpp`. Параметр `dt` — длительность кадра. В отдельном режиме проверки оружия код принудительно ставит `dt=1/60`, поэтому измерение FPS в этой ветке не отражает реальное время. Затем обновляется DebugHud, input manager и очищаются события текущего кадра. Уже здесь читатель должен спросить: `m_events.Clear()` удаляет события прошлого кадра до или после того, как их потребители обработали? Ответ ищи в местах публикации и чтения событий, а не предполагай. Для нового события выбери конкретную фазу кадра, чтобы оно не исчезало сразу после создания.

Переключение F5 проверяет не только `IsKeyDown`, но и `!m_wasF5Down`: так одно долгое удержание клавиши не переключает режим каждый кадр. При входе в Play `EnterPlayMode` проверяет сцену, затем сбрасываются timer, event log, trigger state, открытый UI и режим камеры. В AI-only сцене без игрока код отдельно не создаёт фиктивный инвентарь для `INVALID_ENTITY`. Это пример защитной проверки на границе режимов. При выходе в Editor нужно очистить накопленное намерение движения character controller; иначе физика продолжает шагать, используя последнюю команду Play. Этот комментарий в коде показывает, что режим Editor не является полной остановкой всех подсистем.

3.2. От ввода игрока до намерения движения

В Play при работающей симуляции `m_gameplayController.Update` получает Registry, `simulationDeltaTime`, input, режим захвата UI и yaw камеры. Он не обязан непосредственно перемещать mesh. Его задача — превратить действие игрока и направление камеры в намерение. Далее системы движения и Bullet определят фактическое положение. Если игрок нажал W, а персонаж стоит, проверь по цепи: получен ли GameAction; заблокировал ли его UI; записана ли желаемая скорость; вызван ли character controller; есть ли контакт с collider; изменился ли Transform после physics. Если первый разрыв на входе, физику менять рано.

3.3. Почему NPC pipeline стоит именно так

В текущем кадре перед выбором и выстрелами уменьшаются таймеры оружия (`WeaponSystem::UpdateTimers`). Затем `NpcUtilitySystem` рассчитывает автоматическое действие; `CrewTaskCoordinator` закрепляет работу за конкретным членом экипажа; `NpcCombatSystem` обновляет боевую видимость и память; `NpcCoverSystem` выбирает индивидуальное укрытие; `StateSystem` исполняет итоговое состояние, строит путь и заполняет движение; `CharacterFacingSystem` поворачивает персонажа. Групповой `NpcSquadCombatSystem::Update` в этой ревизии закомментирован. Позднее, после обновления позы и оружейных данных, `NpcRangedCombatSystem` может фактически стрелять. Не говори «NpcCombatSystem стреляет» только потому, что он выбирает Attacking; выстрел имеет отдельные условия и место кадра.

На бумаге проследи одного инженера. До кадра у него Idle и нет цели. Utility видит пробоину и рассчитывает полезность; Coordinator ставит claim; StateSystem в этом же кадре может увидеть назначение и запросить путь; Facing поворачивает модель. Фактическая капсула может сдвинуться позже на physics step. Если поменять местами Coordinator и StateSystem, первое движение задержится до следующего кадра. Это не обязательно катастрофа, но меняет наблюдаемое поведение. Любая перестановка вызовов должна сопровождаться таблицей «кто пишет поле → кто читает».

3.4. Кто владеет состоянием и что значит «на один кадр старое»

В одном кадре разные поля могут отражать разные моменты. У NPC `brain.state` уже новый, `desiredVelocity` вычислена по новому маршруту, а `Transform` ещё хранит позицию до physics step. Renderer после physics может увидеть обновлённое положение, а HUD, если был заполнен раньше, покажет старую дистанцию. Это обычный вопрос фаз, пока задержка определена и мала. Для диагностики показывай timestamp/fixed tick рядом с данными. Если поле используется через несколько кадров без инвалидации, это уже ошибка контракта, особенно для raycast muzzle и lastKnownPosition.

4. Как добавлять систему так, чтобы она заработала во всей игре

4.1. Сперва сформулируй правило без C++

Выберем пожар. Правило для игрока: повреждённый прибор может загореться; огонь расходует ресурс и нагревает комнату; инженер с огнетушителем может потушить его; закрытая дверь влияет на доступность; HUD сообщает причину, если работа невозможна. Запиши, как игрок видит начало, развитие и конец события. Затем выпиши состояния `Igniting`, `Burning`, `Dying`, `Extinguished`; для каждого — кто меняет его, что видно, можно ли сохранять. Если эту схему нельзя изложить словами, реализация тоже будет противоречивой.

Данные огня принадлежат `FireComponent` на Entity очага: интенсивность, топливо, room ID, рабочая зона и временной параметр. Температура комнаты принадлежит комнате, заряд огнетушителя — инвентарю NPC. Не дублируй одно и то же число в нескольких местах без явного правила синхронизации. Например, если и FireComponent, и RoomComponent хранит «текущую температуру пожара», один из них неизбежно станет устаревшим. Выбери источник правды и производные показатели.

4.2. Реализация минимального тика

Создай систему, которая работает с одним очагом без NPC и рендера. Пример ниже — учебный псевдокод, потому что конкретных `FireComponent` и `RoomTemperatureComponent` в текущем проекте ещё может не быть. Он показывает контракт обновления, а не готовую вставку:

```
void FireSystem::Update(ECSRegistry& ecs, float dt) {
    ecs.ForEach<FireComponent>([&](Entity fireEntity, FireComponent& fire) {
        if (fire.state != FireState::Burning) return;
        const float burn = std::min(fire.fuel, fire.burnRate * dt);
        fire.fuel -= burn;
        fire.intensity = std::clamp(fire.intensity + fire.growthRate * dt,
                                    0.0f, fire.maxIntensity);
        if (fire.fuel <= 0.0f) fire.state = FireState::Dying;
    });
}
```

Разбор. Проверка state предотвращает рост потушенного очага. `burnRate*dt` переводит расход «в секунду» в расход за шаг. `std::min` не позволяет забрать больше топлива, чем есть. Clamp удерживает интенсивность в допустимых границах. Переход в Dying выполняется при исчерпании топлива. Но этот код ещё не нагревает комнату, не наносит урон и не создаёт задачу NPC. Добавляй по одному звену и каждый раз проверяй инвариант, чтобы при сбое знать, на каком этапе он появился.

4.3. Место в кадре и зависимости

Если Coordinator должен выбирать пожар по текущей интенсивности, `FireSystem::Update` должен выполняться до него либо решение сознательно использует предыдущий tick. Если тушение снижает интенсивность после работы NPC, порядок должен не удваивать изменение за один кадр. Отдельно реши, когда атмосфера реагирует на кислородный расход: до или после FireSystem, и нужна ли задержка. Напиши таблицу входов и выходов каждой функции. Не вставляй новую систему «куда удобно» между случайными вызовами `ApplicationClass::Frame`.

4.4. Сцена, Inspector, сохранение, UI

После работающего тика нужно дать автору игры создать очаг в Editor. Добавь сериализацию только authoring полей, например начальная интенсивность, запас топлива и room ID. Runtime указатель на комнату, временные claim, path cache, Bullet pointer и GPU handle не записывай как числа в JSON. Для продолжаемой игры отдельно реши, какие runtime поля нужно сохранять: оставшееся топливо и текущий state обычно важны. После LoadScene восстанови Entity references и runtime ресурсы. В Inspector показывай и редактируемые authoring поля, и read-only live данные, чтобы пользователь не путал сохранённый параметр со значением текущего кадра.

4.5. Связь с NPC и контроль отказов

Новая работа требует обнаружения очага, оценки полезности, claim, доступной WorkZone, маршрута, расхода огнетушителя, прогресса тушения и события завершения. Каждое из этих звеньев имеет отдельную причину отказа: нет очага, он уже занят, путь закрыт, нет ресурса, комната опасна, WorkZone внутри collider. Покажи причину в debug UI. Начни с одного NPC и одного очага, затем два NPC и две цели. Смерть исполнителя должна освободить claim. Ручная команда должна вытеснить automatic работу по ясному правилу. После тушения удалённый Entity не должен оставаться в чужом `repairTarget`.

4.6. Проверка результата в игре

Построй маленькую сцену с двумя комнатами, дверью и одним очагом. Запиши прогноз на 10 секунд: сколько топлива сгорит, до какой интенсивности вырастет пожар, когда инженер войдёт в WorkZone, сколько заряда потратит и когда огонь перейдёт в Extinguished. Выполни прогон при 30, 60 и 120 FPS на одинаковое simulation time. Если числа кратно различаются, проверь `dt`. Сохрани игру посередине, загрузи и убедись, что огонь не создался второй раз, ресурс не вернулся, claim не дублировался. Это минимальный стандарт новой системы до включения её в большую миссию.

Математика DX Framework с нуля: учебник для человека, который начинает с арифметики

Эту часть можно читать отдельно от остальной книги. Она рассчитана на человека, который помнит, что такое сложение, но ещё не работал с векторами, матрицами и трёхмерной графикой. Здесь сначала разбирается действие над числами на бумаге, затем его геометрический смысл, затем применение к конкретной игровой задаче. Формулу не нужно запоминать до того, как ты понимаешь, какие величины в неё входят и какого ответа ждёшь. Для каждого примера сначала попробуй предсказать результат, потом проверь вычисление и лишь после этого сравни с программой.

Соглашение записи: десятичная дробь в русском тексте использует запятую: `0,5`. Координаты отделяются точкой с запятой: `(0,5; 0; 0,8)`. Иначе запись `(0,5,0,0,8)` можно ошибочно прочитать как пять координат. В C++ десятичный разделитель — точка: `0.5f`. Расстояния ниже указаны в метрах, время в секундах, если прямо не оговорено другое. Вектор-строка и матрицы DirectXMath будут разобраны после обычной арифметики матриц; не пытайся начинать с HLSL.

Факты о проекте помечены конкретными путями. Например, `src/Render/Shader/Vs/pbr.vs` действительно умножает `finalPos` на `worldMatrix`, затем результат на `viewMatrix` и `projectionMatrix`. `src/Render/Shader/PBRShaderClass.cpp` действительно транспонирует эти матрицы перед записью в constant buffer. Примеры вроде комнаты 10×10 м или NPC в точке `(2;0;3)` — учебные сцены; их координаты не утверждают, что такая сцена уже есть в репозитории. Внешние правила DirectXMath сверены с документацией Microsoft: https://learn.microsoft.com/en-us/windows/win32/dxmath/pg-xnamath-getting-started и https://learn.microsoft.com/en-us/windows/win32/dxmath/pg-xnamath-optimizing .

1. Число в игре: значение, единица, диапазон и точность

1.1. Почему `2` само по себе почти ничего не сообщает

Число 2 может означать два патрона, 2 метра, 2 метра в секунду, 2 секунды, удвоение интенсивности света или идентификатор команды. Операции с такими величинами различаются. Можно сложить 2 метра и 3 метра и получить 5 метров. Нельзя осмысленно сложить 2 метра и 3 секунды. В C++ оба значения могут храниться как `float`, и компилятор не остановит ошибку. Поэтому рядом с параметром всегда отвечай на четыре вопроса: что измеряется; в каких единицах; в каком диапазоне; кто владеет изменением.

В учебной сцене NPC движется со скоростью 2 м/с. Это означает: если не было препятствий и скорость постоянна, за одну секунду он пройдёт 2 м. За четверть секунды — `2 м/с × 0,25 с = 0,5 м`. Единица «секунда» сокращается, остаётся «метр». Такая проверка называется проверкой размерности. Если написать `2 м/с + 0,25 с`, единицы не совпадают; выражение неверно ещё до вычисления. Если `repairRate` равен 10 единиц работы/с, прибавка за кадр равна `10 × dt`, а не просто 10.

Практические единицы для движка: положение и длина — м; площадь — м²; объём — м³; время — с; скорость — м/с; ускорение — м/с²; масса — кг; сила — Н = кг·м/с²; импульс — Н·с; давление — Па = Н/м²; мощность — Вт = Дж/с. В проекте некоторые поля энергосистемы названы с суффиксом `Mw`, то есть мегаватты. Мегаватт — мощность, не запас энергии. Запас энергии можно выразить в джоулях либо ватт-часах: 1 Wh = 3600 Дж. Ошибка в 3600 раз для батареи выглядит как игровая поломка, хотя формула может быть записана синтаксически правильно.

Решённый пример. Генератор даёт 3 МВт, потребитель требует 2 МВт. За 5 с при полном питании потребитель использует `2 МВт × 5 с = 10 МДж`. Нельзя сказать, что у генератора осталось «1 МВт энергии»: 1 МВт — не остаток в баке, а мгновенный резерв мощности. Для вывода времени

автономной работы нужна ёмкость аккумулятора. Если она 20 МДж и генератор выключен, такой потребитель в идеализированной модели проработает `20 МДж / 2 МВт = 10 с`. Проверка размерности: `(МДж)/(МДж/с)=с`.

1.2. Десятичные дроби, проценты и интервал 0…1

В Inspector часто встречается число вроде 0,75. Оно может обозначать 75% здоровья, 75% мощности или вес одного критерия. Перевод процента в долю: `75% = 75/100 = 0,75`. Если у устройства здоровье 60 из максимальных 80, доля здоровья `60/80=0,75`, доля повреждения `1−0,75=0,25`. Если UI показывает 25% повреждения, а система ремонта считает 0,25 единицы работы, это ещё не значит, что работа займёт 0,25 с. Между долей повреждения и временем нужен отдельный коэффициент скорости.

`clamp(x,0,1)` ограничивает результат диапазоном от 0 до 1: если x<0, возвращает 0; если x>1, возвращает 1; иначе x. Он нужен, например, чтобы `powerRatio` не оказался −0,2 или 1,4 из-за неверного входа. Но clamp не исправляет причину некорректного результата. Если battery charge стал −500 Дж, сначала найди ошибку знака или двойное списание, затем используй ограничение как защиту. Не делай clamp настолько рано, что debug перестанет показывать исходную проблему.

1.3. `float` — приближение, а не бесконечно точное число

Число `0,1` нельзя точно представить обычным двоичным `float`. После многократного сложения итог может быть немного больше или меньше ожидаемого десятичного значения. Поэтому `if (progress == 1.0f)` — плохой критерий завершения; практичнее `progress >= requiredProgress` с ограничением результата. Для сравнения геометрии выбирают допустимую погрешность в единицах задачи: например, 1 мм для близких позиций. Нельзя использовать один универсальный `epsilon` для положения в метрах, нормализованного направления и угла в радианах: их масштабы различаются.

Проверка. Если две точки отличаются по X на `0,00001 м`, для прохода через дверь эта разница несущественна; для shader depth она может быть заметной. Точность определяется тем, какой игровой вопрос ты задаёшь. Сравнение `distanceSquared <= radiusSquared` обычно и быстрее, и устойчивее, чем вычисление `sqrt` с последующим точным равенством радиусу. Граничный случай нужно решить заранее: касание круга считается «внутри» (`<=`) или «снаружи» (`<`)?

1.4. Короткая проверка главы

Дано: персонаж идёт со скоростью 1,5 м/с и находится в движении 4 с. Ответ: 6 м по прямой без столкновений. Дано: насос откачивает 0,04 м³/с в течение 5 с. Ответ: 0,2 м³, если вода есть и насос всё время запитан. Дано: источник выдаёт 500 Вт 20 с. Ответ: 10 000 Дж. Если в любом ответе не получается вывести единицу результата, вернись к размерности, а не к коду.

2. Координаты: как назвать место в двумерном и трёхмерном мире

2.1. Числовая прямая, оси и начало координат

На числовой прямой положение задаётся одним числом: 0 — выбранное начало, +3 — на три единицы в положительном направлении, −2 — на две единицы в отрицательном. На плоскости нужны две независимые оси. Точка `(3;4)` означает: от начала пройти 3 единицы по X и 4 по Y. В трёхмерном мире добавляется Z; точка `(3;4;5)` хранит по одному числу для каждой оси. Само число координат не говорит, где находится «верх». В этом проекте многие примеры и функции используют Y как вертикаль: `PressureSystem` получает `pos.getY()` и вычисляет глубину относительно уровня воды. Для конкретной камеры и импортированной модели направления forward нужно проверить отдельно.

Точка `(0;0;0)` называется началом координат. Игровая сцена может содержать корабль вдали от него, и это не ошибка. Если корабль сдвинут целиком, локальные координаты его деталей должны оставаться прежними, а мировые — измениться. Это различие между локальным и мировым

пространством станет важным при матрицах. Пока достаточно представить локальную систему как линейку, прикреплённую к объекту, а мировую — как линейку, прикреплённую к сцене.

2.2. Точка и смещение — разные виды величин

Пусть инженер стоит в A=(2;0;3) м, а пробоина находится в B=(8;0;11) м. Чтобы получить смещение от A к B, вычитаем координаты B−A по осям: (8−2;0−0;11−3)=(6;0;8) м. Смысл ответа: пройти 6 м по +X, 0 м по Y и 8 м по +Z. Это вектор смещения. Если прибавить его к A, получим B: (2+6;0+0;3+8)=(8;0;11). Так проверяется знак вычитания. Если ошибочно вычислить A−B, получится направление от пробоины к инженеру; NPC побежит от задачи.

Точка отвечает на вопрос «где?», смещение — «на сколько и куда?». Сложение двух точек (A+B) может иметь специальный смысл, например перед делением на 2 для середины, но обычно не обозначает положение игрового объекта само по себе. Разность двух точек даёт вектор; точка плюс вектор даёт новую точку. Это различие может быть потеряно в типе XMFLOAT3, поэтому в именах переменных пиши worldPosition, toTarget, desiredDisplacement.

2.3. Координатное пространство как часть типа

Точка дула (0;0;0,6) в локальном пространстве оружия не равна мировой точке (0;0;0,6) в палубе. Первая ещё должна пройти transform оружия, кости руки и персонажа. Если сравнить её напрямую с targetWorldPosition, результат не имеет геометрического смысла, хотя C++ разрешит вычесть два XMFLOAT3. То же относится к направлению взгляда NPC и позиции на экране: пиксель (960;540) не измеряется в метрах. Перед вычислением все величины должны быть в одном пространстве.

Проверка в редакторе: поставь персонажа у мирового нуля, отметь debug точкой его muzzle world position; перенеси персонажа на +10 м по X. Локальный offset дула должен остаться тем же, мировая точка — сместиться примерно на 10 м. Поверни персонажа на месте: мировая точка дула движется вокруг него, но расстояние до руки остаётся тем же. Если это не выполняется, проблема в цепочке преобразований, не в AI.

2.4. Горизонтальная и полная дистанция

Для перемещения по палубе иногда рассматривают только X и Z. Если от NPC до цели разница (3;4;4) м, полная трёхмерная дистанция равна √(3²+4²+4²)=√41≈6,403 м; горизонтальная — √(3²+4²)=5 м. Они отвечают на разные вопросы. При ремонте на одной палубе горизонтальное расстояние может быть полезным. Для выстрела между этажами потерять Y опасно: цель прямо над NPC имеет горизонтальную дистанцию 0, но не находится вплотную. StateSystem::UpdateRepairTask проекта содержит проверку ремонтной дистанции; перед изменением её правила проверь WorkZone, этаж и collider.

Практика: поставь две цели на одинаковых X и Z, но разной высоты. Посмотри значения полной и горизонтальной дистанции. Объясни, какая из них нужна для обхода по палубе, какая — для дальности оружия, а какая — для проверки «можно работать руками». Если тебе понадобились обе, храни два явно названных вычисления вместо одного неоднозначного distance.

3. Вектор: стрелка с направлением и длиной

3.1. Что записано в `(x; y; z)`

Вектор можно нарисовать стрелкой. Её начало для свободного вектора можно переносить, не меняя самого вектора; важны направление и длина. Вектор (3;0;4) означает три единицы по X, ноль по Y и четыре по Z. Он может быть смещением в метрах, скоростью в м/с, силой в ньютонах или направлением без единицы. Поэтому слово «вектор» сообщает форму данных, но не физический смысл. В документации методов всегда указывай и то, и другое.

Равенство векторов означает равенство всех соответствующих координат. (3;0;4) и (3;0;−4) не равны. Нулевой вектор (0;0;0) имеет длину ноль и не задаёт направление. Отрицательный вектор −v

получаем сменой знака каждой координаты: если v=(3;0;4), то −v=(−3;0;−4); стрелка смотрит в противоположную сторону той же длины.

3.2. Сложение и вычитание по компонентам

Пусть NPC сначала проходит a=(2;0;1) м, затем b=(−1;0;3) м. Совокупное смещение получается сложением соответствующих координат: a+b=(2+(−1);0+0;1+3)=(1;0;4) м. Это можно увидеть геометрически: поставь начало стрелки b в конец стрелки a, тогда стрелка от первого начала к последнему концу — сумма. Порядок двух свободных смещений не меняет результат: a+b=b+a. Но при столкновениях реальный путь через мир может отличаться, если пройти смещения в ином порядке; векторное равенство описывает конечное геометрическое смещение без препятствий.

Вычитание также выполняется по компонентам: a−b=(2−(−1);0−0;1−3)=(3;0;−2) м. Его можно понимать как a+(−b). Нельзя сложить вектор из метров с вектором скорости в м/с: форма одинаковая, единицы различаются. Чтобы прибавить скорость к позиции, сначала умножь её на время и получи смещение. Поэтому корректная строка `position += velocity * dt` состоит из двух шагов с понятным смыслом.

Ручная проверка знака. Если ты находишься в A=(2;0;3) и хочешь B=(8;0;11), вычисли B−A=(6;0;8). После прибавления к A обязан получить B. Эта проверка обратной операцией полезна в коде: укажи debug значения A, B, `toTarget` и `A+toTarget`; последнее должно совпасть с B с небольшим численным допуском.

3.3. Умножение вектора на число

Умножить вектор на число k означает умножить каждую координату: k(x;y;z)=(kx;ky;kz). Если скорость v=(1,2;0;1,6) м/с и прошло 0,25 с, смещение v·dt=(0,3;0;0,4) м. Число k>1 растягивает стрелку; 0<k<1 сокращает; k<0 ещё и меняет направление. Например, −2(1;0;3)=(−2;0;−6).

В игре умножение на число встречается повсюду: `velocity*dt`, `force*multiplier`, `normal*distance`, `color*brightness`. Но смысл результата различен. `speed*direction` допустимо, когда direction нормирован и безразмерен: результат имеет единицу скорости. Если direction содержит расстояние до цели, дальняя цель увеличит длину скорости. Поэтому перед движением к цели часто нужна нормализация.

3.4. Длина из прямоугольного треугольника

Для плоского вектора (3;4) длина равна √(3²+4²)=5 по теореме Пифагора. В трёх измерениях сначала найди длину горизонтальной проекции, затем добавь третью ось; итог |v|=√(x²+y²+z²). Для (2;3;6) получим √(4+9+36)=√49=7. Квадраты делают знак координат несущественным для длины: (−3;0;4) тоже имеет длину 5. Длина всегда неотрицательна.

Когда нужен только ответ «цель в радиусе R?», сравни x²+y²+z² с R². Для R=5 и вектора (3;0;4) обе стороны равны 25, значит цель на границе. Эта проверка не вычисляет квадратный корень. Если после неё нужен точный метраж для HUD или времени движения, вычисли корень один раз. Проверяй, что левая и правая части обе измеряются в м².

3.5. Нормализация: оставить направление, убрать длину

Нормировать вектор означает разделить каждую координату на его длину. Для d=(6;0;8) длина 10, значит n=d/10=(0,6;0;0,8). Проверим результат: |n|=√(0,6²+0,8²)=√(0,36+0,64)=1. Число 1 здесь безразмерно. Для NPC со скоростью 2 м/с желаемая velocity n*2=(1,2;0;1,6) м/с, и её длина снова 2 м/с. Если взять d*2, длина станет 20 м/с; NPC бежал бы быстрее к более далёкой цели.

Нулевой вектор нормировать нельзя: деление на ноль не задаёт направление. Почти нулевой вектор тоже опасен, потому что маленькие ошибки координат усилятся после деления. Выбери маленький порог в единицах задачи и проверь квадрат длины до `sqrt`. Пример защитного псевдокода:

```
XMVECTOR delta = XMVectorSubtract(target, position);
float lenSq = XMVectorGetX(XMVector3LengthSq(delta));
```

```
if (lenSq <= epsilonMeters * epsilonMeters) {
    // Цель уже достигнута: направление не требуется.
    return;
}
XMVECTOR direction = XMVectorScale(delta, 1.0f / std::sqrt(lenSq));
```

Этот фрагмент учебный: `target` и `position` предполагаются мировыми точками, а существующая система персонажа может иметь собственные типы, ограничения и способ движения через Bullet. Не копируй его в проект без проверки контракта.

3.6. Три мини-задачи для самопроверки

Первая: `(1;2;3)+(4;−2;0)=(5;0;3)`. Вторая: `(5;0;3)−(1;2;3)=(4;−2;0)`; здесь вычитание проверило сложение. Третья: вектор `(0;0;−7)` имеет длину 7 и нормированную форму `(0;0;−1)`. Если получил `(0;0;1)`, потерял знак при делении. В игре это развернёт NPC или луч на 180°.

4. Скалярное произведение: узнать угол и направление без рисунка

4.1. Определение через координаты

Для векторов a=`(ax;ay;az)` и b=`(bx;by;bz)` скалярное произведение равно `dot(a,b)=ax·bx+ay·by+az·bz`. Умножаем компоненты с одинаковыми индексами, затем складываем три полученных числа. Например, a=`(1;2;3)`, b=`(4;−1;2)`: `dot=1·4+2·(−1)+3·2=4−2+6=8`. Результат — одно число, не вектор, поэтому операция называется скалярной. Если a в метрах, b тоже в метрах, единица результата м². Для двух единичных направлений результат безразмерен.

Попробуй простые направления. Вперёд `(0;0;1)` и вправо `(1;0;0)` дают dot 0. Вперёд и вперёд дают 1. Вперёд и назад `(0;0;−1)` дают −1. Эти три результата уже говорят об угле: совпадают, перпендикулярны, противоположны. Геометрическая формула связывает их с углом θ: `dot(a,b)=|a||b|cosθ`. Для единичных a и b длины равны 1, поэтому `dot=cosθ`. Так число от −1 до 1 кодирует угол без вызова `acos`.

4.2. Почему нормализация перед сравнением обязательна

Пусть NPC смотрит вперёд `f=(0;0;1)`, а цель расположена в направлении `t=(3;0;4)` м. Прямое `dot(f,t)=4`. Значение 4 невозможно интерпретировать как косинус: длина t равна 5, она включилась в результат. Нормируем t: `t_unit=(0,6;0;0,8)`, теперь `dot(f,t_unit)=0,8`. Угол `acos(0,8)≈36,87°`. Если цель отодвинуть в пять раз вдоль того же луча, t станет `(15;0;20)`, но после нормализации dot останется 0,8. Видимость по углу не должна меняться только из-за дистанции; дальность проверяется отдельным правилом.

4.3. Полный угол обзора и половинный угол

Поле зрения 90° обычно означает полный конус 90°, то есть от направления взгляда разрешены 45° влево и 45° вправо. Порог для dot — `cos(45°)≈0,707`. Направление с dot 0,8 внутри; с dot 0,6 снаружи. Для полного FOV 70° половина 35°, порог `cos35°≈0,819`; направление с dot 0,8 чуть снаружи. Нельзя сравнить dot напрямую с 35: dot единичных векторов находится примерно в [−1;1]. Нельзя взять `cos70°`, если 70° было полным конусом: такой код фактически допустил бы ±70°, то есть 140°.

Быстрая проверка на бумаге: цель ровно впереди даёт dot 1 и должна проходить любой разумный FOV; ровно сбоку даёт 0 и не проходит FOV меньше 180°; сзади даёт −1 и не проходит фронтальный FOV. Если код нарушает эти три случая, сначала ищи неправильный forward, ненормированный вектор или перепутанные градусы и радианы.

После FOV обязательно проверь препятствия raycast. Стена внутри конуса не становится прозрачной от правильного dot. `NpcCombatSystem` в проекте использует геометрические проверки восприятия; не подменяй личную видимость результатом одного углового фильтра. Отдельно храни lastKnownPosition, чтобы NPC не «видел» скрытого игрока через обновляемую позицию цели.

4.4. Dot для движения, освещения и попадания

Если фактическое смещение NPC за секунду `m=(0,5;0;0)`, а желаемое единичное направление `d=(1;0;0)`, `dot(m,d)=0,5 м`: он продвинулся в сторону цели на полметра. Если `m=(0;0;0,5)`, dot 0: он двигался поперёк. Если `m=(-0,5;0;0)`, dot −0,5: ушёл назад. Это полезнее для диагностики, чем просто длина движения, потому что персонаж может много бегать в сторону, не приближаясь к задаче.

Для диффузного освещения `max(dot(N,L),0)` измеряет, насколько нормаль поверхности направлена к свету. Если N и L единичные и совпадают, результат 1; если перпендикулярны — 0; если свет позади поверхности — отрицательный результат обрезается до нуля. Здесь dot используется той же математикой, что и обзор NPC, хотя смысл в игре другой. Если нормаль не нормирована, освещение станет зависеть от её длины.

4.5. Упражнение с ответом

Forward=`(1;0;0)`, направление на цель до нормализации `(1;0;1)`. Длина цели `√2≈1,414`, единичное направление `(0,707;0;0,707)`, dot≈0,707. Угол около 45°. При полном FOV 80° половина 40°, цель снаружи; при 100° половина 50°, внутри. Проверь: если использовать исходный `(1;0;1)`, dot=1, и ошибочный код решил бы, что цель точно впереди.

5. Векторное произведение и проекция: сторона, нормаль, поверхность

5.1. Почему результат cross снова вектор

Векторное произведение `cross(a,b)` возвращает вектор, перпендикулярный обоим исходным. В координатах он вычисляется так: `cross(a,b)=(ay·bz−az·by; az·bx−ax·bz; ax·by−ay·bx)`. Это может выглядеть страшно, но сначала посмотри на три простых осевых направления. `cross((1;0;0),(0;1;0))=(0;0;1)`. Поменяем порядок: `cross((0;1;0),(1;0;0))=(0;0;−1)`. Значит, cross меняет знак при перестановке аргументов. Правило порядка особенно важно для нормали треугольника и знака поворота.

Рассчитаем треугольник на палубе. A=`(0;0;0)`, B=`(1;0;0)`, C=`(0;0;1)`. Первое ребро B−A=`(1;0;0)`, второе C−A=`(0;0;1)`. Cross даёт `(0;-1;0)`. Если поменять B и C, нормаль станет `(0;1;0)`. Визуально геометрические точки те же, но порядок вершин изменился; rasterizer может скрыть треугольник при back-face culling, а освещение перевернётся. Поэтому «стена исчезла» иногда означает неверный winding, а не проблему загрузки текстуры.

5.2. Нулевой cross и площадь треугольника

Если два вектора параллельны, cross равен нулю. Например, a=`(1;0;0)`, b=`(2;0;0)` дают `(0;0;0)`. Для треугольника это означает нулевую площадь: три точки лежат на прямой, нормаль не определена. Длина cross двух рёбер равна площади соответствующего параллелограмма; площадь треугольника вдвое меньше. Для перпендикулярных рёбер длиной 3 и 4 м длина cross равна 12 м², площадь треугольника 6 м². Это можно использовать для диагностики вырожденного mesh и расчёта геометрического веса.

5.3. Сторона цели на плоскости XZ

Пусть NPC смотрит в +Z: forward=`(0;0;1)`, а цель справа в +X: toTarget=`(1;0;0)`. `cross(forward,toTarget)=(0;1;0)` по приведённой формуле. Если поменять порядок, Y будет −1. Значит, Y-компонента cross может указать сторону, но названия «плюс = вправо» нельзя копировать слепо в любой проект: модель может иметь локальный forward −Z, а вращение yaw может использовать другой знак. Сделай тест с четырьмя целями по осям; выведи crossY и фактическое направление поворота. Только после проверки зафиксируй конвенцию в одном месте кода.

5.4. Проекция на направление

Для единичного вектора n проекция v на n равна `proj_n(v)=dot(v,n)·n`. Она возвращает часть v, направленную вдоль n. Допустим, скорость v=`(3;2;0)` м/с, нормаль пола n=`(0;1;0)`. Dot=2 м/с. Нормальная часть `(0;2;0)` м/с; часть вдоль поверхности `v−proj=(3;0;0)` м/с. Это помогает ограничить желаемое движение персонажа плоскостью пола. Для прыжка удалять вертикальную часть нельзя: она принадлежит физике.

Если n не единичный, формула меняется: `proj_n(v)=dot(v,n)/dot(n,n)·n`. Например, n=`(0;2;0)` и v=`(3;2;0)`: числитель 4, знаменатель 4, результат `(0;2;0)`. Если ошибочно взять `dot(v,n)·n`, получится `(0;8;0)`. Поэтому перед использованием упрощённой формулы явно нормируй направление или используй полную форму.

5.5. Нормаль поверхности и отражение

Нормаль — единичное направление, перпендикулярное поверхности. Для отражения вектора движения v от идеальной плоскости с единичной нормалью n используется `r=v−2·dot(v,n)·n`. Проверь на простом случае: падающий вектор `(1;−1;0)`, горизонтальная поверхность с n=`(0;1;0)`. Dot=−1, отражённый r=`(1;−1;0)−2·(−1)·(0;1;0)=(1;1;0)`. Горизонтальная компонента осталась, вертикальная сменила знак. Реальное тело в Bullet имеет трение, restitution и ограничения, поэтому эту формулу не нужно вручную применять к каждому столкновению. Она объясняет направление идеального отражения и помогает проверить нормали и визуальный эффект.

6. Матрица: таблица чисел, а не загадочный графический объект

6.1. Строки, столбцы и размер

Матрица — прямоугольная таблица чисел. Например, матрица A размером 2×3 содержит две строки и три столбца:

```
A = [ 1  2  3 ]
    [ 4  5  6 ]
```

Число в первой строке и втором столбце — 2; его можно обозначить `A[1,2]`, если индексация в книге начинается с 1. В C++ индексация массива обычно начинается с 0, поэтому та же ячейка могла бы иметь индексы `[0][1]`. Не смешивай эти два соглашения в одном вычислении. Размер записывается «число строк × число столбцов», а не наоборот. Если повернуть эту таблицу при транспонировании, получится матрица 3×2.

Матрицы нужны не только графике. Таблица отношений между четырьмя фракциями тоже может быть матрицей; коэффициенты переходов состояний и система уравнений — тоже. В 3D графике особенно полезны матрицы 3×3 для вращения/масштаба и 4×4 для преобразования точек с переносом. Сначала научись арифметике на 2×2 и 2×3: правила для 4×4 те же, просто ячеек больше.

6.2. Равенство и нулевая матрица

Матрицы равны, если их размеры одинаковы и соответствующие ячейки равны. `[1 2; 3 4]` и `[1 2; 3 5]` различаются последней ячейкой. Матрица 2×2 не равна матрице 2×3 даже если первые четыре числа совпали. Нулевая матрица содержит нули во всех ячейках; её размер всё равно важен. Если прибавить нулевую матрицу того же размера, исходная матрица не изменится.

Это похоже на векторное сложение, но матрица хранит несколько строк и столбцов. Не называй нулевую матрицу «нулевым transform»: для преобразований нейтральной операцией служит единичная матрица, а умножение точки на полностью нулевую 4×4 матрицу уничтожит её координаты и w. Сравни это с числами: для сложения нейтрален 0, для умножения нейтральна 1.

7. Сложение, вычитание и умножение матрицы на число

7.1. Сложение по соответствующим ячейкам

Возьмём две матрицы одинакового размера:

```
A = [ 1  2 ]    B = [ 5  6 ]
    [ 3  4 ]        [ 7  8 ]
```

Сложение выполняется по ячейкам: верхний левый результат `1+5=6`, верхний правый `2+6=8`, нижний левый `3+7=10`, нижний правый `4+8=12`. Поэтому `A+B=[6 8; 10 12]`. Проверь результат вычитанием: `(A+B)-B=A`. Нельзя сложить матрицу 2×2 с матрицей 3×2, потому что у одной отсутствует третья строка, которой должна соответствовать строка другой. Это не ограничение библиотеки, а невозможность выполнить правило по ячейкам.

В графике матрицы преобразований обычно **составляют умножением**, а не сложением. Прибавить матрицу переноса к матрице вращения не значит «сначала повернуть, потом перенести». Сложение может использоваться при промежуточном линейном вычислении или численном методе, но путь вершины через model, view, projection составляют матричным произведением. Это различие объясняет, почему знание элементарного сложения не делает формулу `W·V·P` очевидной: это другая операция.

7.2. Вычитание по ячейкам

Для тех же A и B: `A-B=[1-5, 2-6; 3-7, 4-8]=[-4 -4; -4 -4]`. Вычитание не коммутативно: `B-A=[4 4; 4 4]`. Если сначала обновить положение камеры, а потом захотеть разницу между двумя transform матрицами, простое `M_new-M_old` даёт таблицу разностей коэффициентов, но не обязательно удобный геометрический «поворот между ними». Для относительного преобразования обычно нужна обратная матрица: `M_old⁻¹·M_new` или соответствующий порядок выбранной конвенции. Смысл операции выбирается по вопросу, а не по доступному оператору C++.

7.3. Умножение на скаляр

Умножить A на число 3 — значит умножить каждую ячейку: `3A=[3 6; 9 12]`. Если умножить на 0, получится нулевая матрица. Если на −1, все знаки поменяются. Скалярное умножение полезно при линейной комбинации матриц, но `0,5·M_world` обычно **не** означает «половина пространственного преобразования»: для поворота и переноса требуется интерполировать соответствующие параметры или использовать специально разработанную интерполяцию transform. Например, усреднение двух rotation matrices по ячейкам может нарушить ортогональность, превратив чистый поворот в смесь со scale/shear.

Проверь вручную: `2(A+B)=2[6 8;10 12]=[12 16;20 24]`. Отдельно `2A+2B=[2 4;6 8]+[10 12;14 16]=[12 16;20 24]`. Это распределительное свойство полезно для контроля арифметики. Оно не даёт разрешения складывать несовместимые размеры или физические единицы.

7.4. Транспонирование шаг за шагом

Транспонирование меняет строки и столбцы местами: ячейка с номером `(i,j)` переходит на `(j,i)`. Для нашей матрицы 2×3:

```
A   = [ 1  2  3 ]
      [ 4  5  6 ]

Aᵀ  = [ 1  4 ]
      [ 2  5 ]
      [ 3  6 ]
```

Посмотри не на готовый ответ, а на строительство: первая строка A `[1 2 3]` стала первым столбцом A^T; вторая `[4 5 6]` стала вторым столбцом. Новый размер 3×2. Если транспонировать ещё раз, получится исходная таблица: $(A^T)^T=A$. Для квадратной матрицы размер сохраняется, но числа вне главной диагонали обмениваются местами. Например, $[1\ 2;\ 3\ 4]^T=[1\ 3;\ 2\ 4]$; диагональные 1 и 4 остались на месте.

Транспонирование имеет две разные роли, которые легко спутать. Математически оно участвует, например, в преобразовании нормалей после неравномерного масштаба. Технически `PBRShaderClass.cpp` проекта вызывает `XMMatrixTranspose(world/view/projection)` перед записью в HLSL constant buffer из-за соглашения размещения матриц в памяти. Второе **не** меняет задуманного порядка геометрических действий. Официальная документация DirectXMath отдельно подтверждает row-major хранение на CPU и необходимость транспонирования при обычном column-major HLSL без `row_major`.

7.5. Задачи на арифметику матриц

Дано C=`[2 −1; 0 5]`, D=`[−3 4; 7 1]`. Ответ: `C+D=[−1 3; 7 6]`, `C−D=[5 −5; −7 4]`, `2C=[4 −2; 0 10]`, C^{T}=`[2 0; −1 5]`. Проверь каждую ячейку отдельно. Если при транспонировании получил `[2 −1;0 5]`, ты только переписал исходную матрицу; на этом примере две внедиагональные ячейки −1 и 0 обязаны обменяться местами.

8. Умножение матриц: как заполнить одну клетку результата

8.1. Почему размеры проверяются до арифметики

Матрицу A размера m×n можно умножить на B размера n×p; внутренние размеры n должны совпадать. Результат имеет размер m×p. Умножение A 2×3 на B 3×2 допустимо и даст 2×2. Умножение B 3×2 на A 2×3 тоже допустимо, но даст 3×3 — уже другой размер и другая операция. Для A 2×3 и C 4×2 произведение A·C не определено: первая матрица выдаёт три числа в строке, а вторая ждёт четыре числа в столбце для каждой ячейки результата.

Каждая ячейка результата — dot одной **строки** первой матрицы и одного **столбца** второй. Не умножай просто одинаково расположенные ячейки, как при сложении: это было бы поэлементное произведение, а не обычное матричное умножение. Чтобы не ошибиться, выделяй пальцем нужную строку и нужный столбец, перемножай попарно, складывай.

8.2. Полностью заполненная матрица 2×2

Пусть A=`[1 2; 3 4]`, B=`[5 6; 7 8]`. Результат C=A·B тоже 2×2. Заполним все четыре клетки:

```
C11 = [1 2] · [5;7] = 1·5 + 2·7 = 19
C12 = [1 2] · [6;8] = 1·6 + 2·8 = 22
C21 = [3 4] · [5;7] = 3·5 + 4·7 = 43
C22 = [3 4] · [6;8] = 3·6 + 4·8 = 50

A·B = [19 22]
      [43 50]
```

Проверим иной порядок: B·A=`[23 34;31 46]`. Например, его верхняя левая ячейка `5·1+6·3=23`, а не 19. Следовательно, в общем случае `A·B≠B·A`. Это главное практическое правило матриц в движке: поменяв порядок world и view или scale и translation, ты выполняешь иную операцию, а не просто переписываешь формулу.

8.3. Пример с прямоугольными размерами

Пусть A имеет размер 2×3, B — 3×2:

```
A = [ 1  2  3 ]    B = [ 7  8 ]
    [ 4  5  6 ]        [ 9 10 ]
                       [11 12 ]
```

Верхняя левая ячейка C: `1·7+2·9+3·11=7+18+33=58`. Верхняя правая: `1·8+2·10+3·12=8+20+36=64`. Нижняя левая: `4·7+5·9+6·11=28+45+66=139`. Нижняя правая: `4·8+5·10+6·12=32+50+72=154`. Итак, `A·B=[58 64;139 154]`. Проверь размер: две строки A остались двумя строками результата, два столбца B — двумя столбцами результата.

8.4. Единичная матрица

Единичная матрица I размера 2×2 выглядит `[1 0;0 1]`. Проверь на A=`[3 4;5 6]`: A·I=A и I·A=A. Например, первая клетка A·I равна `3·1+4·0=3`; следующая `3·0+4·1=4`. Для 4×4 единицы стоят на четырёх диагональных местах, остальные ячейки нули. В геометрии I означает «ничего не изменить». Это причина, по которой новый Transform без масштаба, вращения и переноса должен давать единичную матрицу.

8.5. Ассоциативность и порядок действий

Умножение матриц ассоциативно: `(A·B)·C=A·(B·C)`, если размеры совместимы. Это значит, что можно заранее вычислить `W·V·P` и затем умножать на него множество вершин. Но нельзя переставлять множители: `W·V·P` и `P·V·W` обычно различны. Порядок скобок влияет на удобство и стоимость, порядок самих преобразований — на геометрический смысл. Для диагностики vertex shader проекта умножает вершину на world, затем view, затем projection отдельно; так легче увидеть первое место расхождения.

9. Обратная матрица: действие, которое отменяет другое действие

9.1. Обратное число как понятная аналогия

Для ненулевого числа 4 обратное число 1/4: `4·(1/4)=1`. Единица нейтральна для умножения чисел. Для квадратной матрицы M обратная матрица `M⁻¹` удовлетворяет `M·M⁻¹=I` и `M⁻¹·M=I`, где I — единичная матрица. Если M переносит точку из локального пространства в мировое, `M⁻¹` возвращает мировую точку в локальное. Но не каждая матрица обратима. У числа 0 нет обратного; у матрицы с потерянным измерением, например scale X=0, тоже нет.

9.2. Детерминант 2×2 и ручное вычисление обратной

Для M=`[a b; c d]` детерминант `det(M)=ad−bc`. Если он не равен нулю, обратная матрица `M⁻¹=(1/det)[d −b; −c a]`. Запоминать формулу проще после смысла: мы хотим получить такую таблицу, которая при умножении с M даст I. Проверим на конкретном примере M=`[2 1;1 1]`. Детерминант `2·1−1·1=1`; значит `M⁻¹=[1 −1;−1 2]`. Умножаем M·M⁻¹: первая строка/первый столбец `2·1+1·(−1)=1`; первая строка/второй столбец `2·(−1)+1·2=0`; вторая строка/первый `1·1+1·(−1)=0`; вторая строка/второй `1·(−1)+1·2=1`. Получили I.

Теперь M=`[2 4;1 2]`: determinant `2·2−4·1=0`. Вторая строка равна половине первой: матрица сплющивает разные исходные данные в один и тот же результат, поэтому восстановить вход по выходу невозможно. Для 4×4 случаи нулевого масштаба или вырожденной проекции аналогично делают inverse невозможным. Перед `XMMatrixInverse` проверяй, не создал ли Transform нулевой scale; после инверсии не допускай NaN в raycast.

9.3. Почему транспонирование не равно обращению

Для M=`[2 1;1 1]` транспонированная матрица совпадает с самой M, потому что она симметрична. Но обратная, как мы вычислили, `[1 −1;−1 2]` — совсем другая. Значит, `transpose` не «отменяет» любое преобразование. Для чистой ортонормальной матрицы вращения обратная действительно равна транспонированной, но это специальное свойство, которое теряется при произвольном масштабе и переносе. Ошибка «взял transpose вместо inverse» может незаметно работать только на простом тесте с чистым вращением, а потом ломаться на растянутом объекте.

9.4. Где обратная нужна в игре

World matrix переводит локальную вершину в мир. При попадании пули Bullet сообщает мировую точку. Чтобы разместить decal в локальных координатах корпуса, применяют inverse world к точке попадания. View matrix переводит мир в координаты камеры; при построении луча из пикселя нужно сделать обратный путь из clip/view в world. Для дочернего объекта relative transform также

находится с помощью обратного преобразования родителя, а не поэлементным вычитанием матриц.

Учебный пример: объект перенесён на +5 по X без других изменений. Мировая точка попадания X=7 соответствует локальной X=2, потому что inverse переносит на −5. Если вместо inverse применить ещё один перенос +5, получишь 12 и decal окажется далеко от места удара. Сначала проверяй этот простейший случай, потом добавляй rotation и scale.

10. Матрицы 4×4: точка, направление, масштаб, поворот и перенос

10.1. Зачем нужна четвёртая координата w

Матрица 3×3 умеет линейно масштабировать и вращать трёхмерный вектор, но постоянный перенос вроде «прибавь 5 к X» нельзя выразить одной 3×3 матрицей для всех точек. Поэтому к `(x;y;z)` добавляют w: точка становится `(x;y;z;1)`, направление — `(x;y;z;0)`. Матрица 4×4 может разместить перенос в последней строке при соглашении векторов-строк. При умножении точка с w=1 получает перенос; направление с w=0 — нет. Это не произвольное правило: направление не находится в конкретном месте, поэтому сдвиг объекта не должен менять его ориентацию.

Для переноса на `(5;0;2)` в соглашении вектора-строки можно написать:

```
T = [ 1  0  0  0 ]
    [ 0  1  0  0 ]
    [ 0  0  1  0 ]
    [ 5  0  2  1 ]
```

Строка точки `p=(1;0;0;1)` после p·T станет `(6;0;2;1)`. Строка направления `d=(1;0;0;0)` после d·T останется `(1;0;0;0)`. Самостоятельно перемножь каждую из четырёх ячеек результата как dot строки p с соответствующим столбцом T. Для первой координаты точки: `1·1+0·0+0·0+1·5=6`; для направления последний член `0·5`, поэтому остаётся 1.

10.2. Масштаб как диагональная матрица

Масштаб `(2;3;4)` для вектора-строки задаётся диагональной матрицей с 2, 3, 4 и 1 на диагонали. Точка `(1;2;3;1)` перейдёт в `(2;6;12;1)`. Масштаб по X в 2 раза удваивает расстояния вдоль X относительно локального начала. Если scale X=0, все X-координаты станут 0 и обратная матрица невозможна. Если scale X=−1, объект зеркально отразится; вместе с геометрией может измениться winding и видимость при back-face culling. Для физического тела масштаб collision shape нужно согласовать отдельно — render transform не всегда автоматически масштабирует Bullet форму.

10.3. Почему `S·R·T` и `T·R·S` не одно и то же

Пусть локальная точка X=1, масштаб X=2 и перенос +5 по X. При векторе-строке `p·S·T` сначала масштаб: X=2, затем перенос: X=7. При `p·T·S` сначала перенос: X=6, затем масштаб: X=12. Числа 7 и 12 оба получены правильным умножением, но описывают разные инструкции. Выбери игровую инструкцию прежде, чем кодировать порядок матриц. Для модели нужно обычно масштабировать и повернуть вокруг локального pivot, а затем расположить объект в мире. Именно такую композицию `scaleMatrix * rotationMatrix * translationMatrix` возвращает `TransformComponent::GetWorldMatrix()` в `src/ECS/Components/Components.h`.

В том же методе rotation строится как `rotX * rotY * rotZ`. Значит, при принятом соглашении сначала применяется X, потом Y, потом Z. Если ты изменишь порядок на Z·Y·X, модель с несколькими ненулевыми углами окажется в иной ориентации. Не называй это просто «Euler rotation» без порядка: порядок осей — часть контракта сериализации и Inspector. При проверке используй не симметричный куб, а стрелку, у которой можно различить перед, верх и правую сторону.

10.4. Иерархия: ребёнок движется вместе с родителем

Если предмет прикреплён к руке NPC, его локальное положение задаётся относительно руки. Мировой путь вершины по соглашению векторов-строк можно представить как `p_mesh · M_itemLocal`

· M_hand · M_characterWorld с поправкой на конкретный skinning/bind контракт. Сначала применяется ближайший локальный transform, потом родительский. Поверни персонажа вокруг Y: расстояние между рукой и предметом должно остаться постоянным, хотя обе мировые координаты изменились. Если предмет улетает по дуге вокруг мирового нуля, вероятно, родитель и ребёнок перемножены в неправильном порядке или pivot расположен не там.

10.5. Нормали и обратная транспонированная матрица

Нормаль поверхности должна остаться перпендикулярной преобразованным касательным. Возьмём t=(1;1;0) и n=(1;-1;0); dot(t,n)=1-1=0. Масштабируем X в 2 раза: касательная t′=(2;1;0). Если нормаль тоже просто масштабировать, n_bad=(2;-1;0) и dot(t′,n_bad)=4-1=3: она больше не перпендикулярна поверхности. Применение inverse transpose к нормали даёт n_correct=(1/2;-1;0), после чего dot(t′,n_correct)=1-1=0. Нормализуй результат для освещения.

Это **математическое** применение inverse transpose, отличное от технического XMMatrixTranspose при загрузке world/view/projection в HLSL buffer. В PBRShaderClass.cpp есть и отдельный расчёт normalMatrix = XMMatrixTranspose(XMMatrixInverse(nullptr, worldMatrix)) для локальных нормалей трещины. Одинаковое слово «transpose» в двух строках кода не означает одинаковую причину. При неравномерном масштабе проверь не только normal, но и tangent/binormal basis normal map.

10.6. Три пространственных инварианта

Первый: перенос меняет точку, но не направление. Второй: чистый поворот сохраняет длину вектора и расстояние между двумя точками. Третий: world transform объекта должен одинаково переводить debug вершину, render mesh и socket, если все они описаны в одной локальной системе. Если первый нарушен, проверь w. Если второй нарушен, к rotation добавился scale или численная ошибка. Если третий нарушен, у разных подсистем различные модели parent/pivot или неполная цепочка матриц.

11. Угол, окружность, тригонометрия и ориентация

11.1. Градус — удобное название, радиан — удобное число

Один полный оборот — 360°. Половина оборота — 180°, четверть — 90°. Это понятная человеку шкала. Для вычислений удобнее радианы: полный оборот равен 2π радиан, половина — π, четверть — π/2. Число π приблизительно 3,14159265. Радиан определяется через окружность: угол в один радиан стягивает дугу, длина которой равна радиусу. Поэтому многие формулы движения и производных оказываются простыми именно в радианах.

Перевод из градусов в радианы: rad = deg·π/180. Обратный: deg = rad·180/π. Например, 90° дают 90·π/180=π/2≈1,5708 радиана. Для 45° получаем π/4≈0,7854. Если функция ожидает радианы, а передать 90 как число, она увидит угол 90 радиан, то есть много оборотов. Результат sin/cos будет выглядеть случайным. В D3DClass.cpp FOV 75° превращается в радианы перед XMMatrixPerspectiveFovLH; в ApplicationClass.cpp угол сопровождения камеры тоже переводится из градусов. Это два настоящих места в проекте, где единицы угла нельзя смешивать.

Практическое правило для нового API: в имени параметра или комментарии пиши единицу (yawRadians, fovDegrees), если она не очевидна. Конфигурационный файл может быть удобен в градусах, а математическое ядро — в радианах. Преобразуй единицу на одной границе и не чередуй её внутри алгоритма. В инспекторе отображай градусы, если так удобнее художнику; в TransformComponent::rotation при вызове XMMatrixRotationX/Y/Z должны быть согласованные радианы.

11.2. Прямоугольный треугольник и sin/cos

Представь стрелку длиной 1, которая лежит на плоскости XZ и поворачивается вокруг вертикальной Y. Её проекция на ось X и ось Z зависит от принятой «нулевой» ориентации. Если нулевой yaw направлен вдоль +Z, часто записывают direction=(sin(yaw);0;cos(yaw)). При yaw=0 получаем

(0;0;1); при yaw=π/2 — (1;0;0). Если нулевую ориентацию выбрать вдоль +X, компоненты поменяются местами: (cos(yaw);0;sin(yaw)). Обе формулы математически корректны. Для кода двигателя проверь базис модели, камеру и матрицу вращения, а не копируй формулу по памяти.

В прямоугольном треугольнике sin(θ)=противолежащий катет/гипотенуза, cos(θ)=прилежащий катет/гипотенуза. Если длина перемещения L=10 и угол к оси X равен 30°, то компоненты x=10 cos30°≈8,6603, y=10 sin30°=5. Проверка по Пифагору: sqrt(8,6603²+5²)≈10. Это полезно для веерной атаки, направления взгляда, орбитальной камеры и распределения точек вокруг объекта. Не смешивай результат sin() с градусным числом: передай π/6, а не 30.

Синус и косинус — координаты точки на единичной окружности. Из этого следует sin²θ+cos²θ=1. Если направление построено этими функциями на плоскости и sin/cos вычислены для одного угла, его длина около 1. «Около», потому что float хранит приближение. Проверять строгое равенство длины единице опасно; сравнивай с допуском, например abs(length−1)<1e−5 там, где такая точность соответствует задаче.

11.3. `atan2`: получить угол из двух координат

Обычный atan(y/x) теряет информацию о четверти окружности: отношение 1/1 равно отношению (−1)/(−1). Кроме того, при x=0 деление невозможно. Функция atan2(y,x) получает обе компоненты отдельно и выбирает правильную четверть. На математической плоскости XY: atan2(1,1)=45°; atan2(1,−1)=135°; atan2(−1,−1)=−135° (или эквивалентные 225° после приведения диапазона). Угол возвращается в радианах.

Учебный NPC стоит в (2;0;3), цель — в (5;0;7). Горизонтальный вектор к цели (3;0;4). При соглашении «нулевой yaw вдоль +Z» нужен atan2(dx,dz)=atan2(3,4)≈0,6435 радиана, или ≈36,87°. Проверка: горизонтальная длина 5, значит нормализованное направление (0,6;0;0,8). sin(36,87°)≈0,6, cos(36,87°)≈0,8. Если модель смотрит своей локальной осью −Z, добавится поправка к базису модели. Если знак yaw в движке противоположный, сверяй результат на одной отладочной стрелке.

11.4. Кратчайшая разность углов

Переход от 179° к −179° — это поворот на +2°, а простое вычитание даёт −358°. Угол периодичен: к нему можно прибавить или отнять 360°, не меняя направления. Поэтому для плавного поворота ищут разность в диапазоне [-π;π]. Один способ: delta=atan2(sin(target−current), cos(target−current)). В примере target−current=−358°; после sin/cos и atan2 получим +2°. Затем ограничь шаг maxTurnRateRadiansPerSecond·dt: если допускается 90°/с и dt=0,0167 с, за кадр можно повернуться примерно на 1,5°, а не мгновенно на всю разницу.

Простой lerp(currentYaw,targetYaw,t) через границу ±180° пойдёт длинной дорогой. Обычная линейная интерполяция ещё и не гарантирует постоянную угловую скорость. Для короткого yaw используй приведённую разность и ограничение скорости. Для полной трёхмерной ориентации лучше кватернион с подходящей интерполяцией.

11.5. Три угла Эйлера: интуиция и ограничение

Поворот вокруг X меняет направление «верх/низ» относительно объекта, вокруг Y разворачивает его горизонтально, вокруг Z наклоняет боком. Тройка чисел удобна для инспектора и сохранения. Но порядок важен: после первого поворота локальные оси уже изменились. В TransformComponent::GetWorldMatrix() проект явно составляет rotX * rotY * rotZ. Поэтому тройка (90°;90°;0°) не описывает уникальное действие без этой договорённости. Необдуманная перестановка множителей изменит существующие сцены.

Углы Эйлера имеют проблему gimbal lock: при определённой ориентации две степени свободы совпадают, и разные изменения чисел дают одинаковое вращение. Это не «баг функции», а ограничение представления последовательными поворотами вокруг осей. Для UI углы Эйлера остаются удобными; для хранения и плавного смешивания анимаций часто подходят кватернионы.

При этом кватернион тоже должен соответствовать той же системе координат и локальному forward модели.

11.6. Кватернион без магии

Кватернион имеет четыре числа `(x;y;z;w)`. Поворот на угол θ вокруг единичной оси `a=(ax;ay;az)` кодируется как `q=(ax sin(θ/2); ay sin(θ/2); az sin(θ/2); cos(θ/2))`. Половина угла здесь — часть математического способа представления трёхмерного вращения; не «ошибка в два раза». Для поворота на 90° вокруг Y ось `(0;1;0)`, θ/2=45°, `sin45°=cos45°≈0,7071068`; значит `q≈(0;0,7071068;0;0,7071068)`. Проверка длины: `sqrt(0²+0,7071²+0²+0,7071²)≈1`.

Кватернион `(0;0;0;1)` обозначает отсутствие вращения. Кватернионы `q` и `−q` обозначают один и тот же поворот, что важно при интерполяции: прежде чем смешивать их, проверь знак dot между ними и при необходимости поменяй знак второго, чтобы идти коротким путём. Умножение кватернионов объединяет повороты, но, как и матричное, зависит от порядка и API соглашения. Не вводи собственный оператор композиции по одному примерному рисунку; сверь его с `XMQuaternionMultiply` и реальной цепочкой матриц проекта.

Для плавного поворота камеры или персонажа обычно используют `slerp(q0,q1,t)`: интерполяцию по кратчайшей дуге на сфере единичных кватернионов. `t=0` даёт начальную ориентацию, `t=1` конечную. Если `t` каждый кадр просто ставить равным `speed·dt`, движение будет экспоненциально приближаться к цели, а не завершаться за фиксированное время. Если нужен поворот ровно за 0,4 с, храни прошедшее время и вычисляй `t=clamp(elapsed/0,4;0;1)`. Для ограниченной угловой скорости заранее измеряй угол между ориентациями и выводи допустимую долю шага.

11.7. Задачи главы

1. Переведи 120° в радианы. Ответ: `120π/180=2π/3≈2,0944`.
2. Вектор `(3;0;4)` направлен к цели. Найди yaw от +Z к нему. Ответ: `atan2(3,4)≈36,87°`.
3. NPC сейчас повернут на 170°, а цель на −170°. Каков кратчайший поворот? Ответ: +20°.
4. Напиши кватернион для 180° вокруг единичной Y. Половина угла 90°, поэтому ответ `(0;1;0;0)`. Его отрицание кодирует тот же поворот.

12. Камера, перспектива и луч из пикселя

12.1. Камера — преобразование координат, а не нарисованный объект

Мировые координаты отвечают на вопрос «где находится объект в сцене». Координаты вида отвечают «где находится он относительно камеры». Если камера стоит в мировой точке `(0;0;5)` и смотрит вдоль +Z, мировая точка `(0;0;8)` находится на 3 единицы перед ней. В простом случае без поворота view-преобразование вычитает положение камеры: `z_view=8−5=3`. Когда камера повернута, нужно ещё выразить разность в её базисе right/up/forward. Поэтому view matrix есть обратное преобразование камеры: перемещение камеры вправо заставляет мир в координатах вида уйти влево.

Для конструктора view matrix нужны позиция, цель и направление «верх». `XMMatrixLookAtLH` в `src/Render/Camera/cameraclass.cpp` применяет левосторонний вариант. Если камера и target совпали, направление взгляда нулевое и базис не определить. Если forward параллелен выбранному up, right через cross тоже может оказаться нулевым. В свободной камере защити эти случаи, иначе матрица наполнится NaN. При вращении камеры сохраняй ортонормальность базиса; при необходимости нормализуй forward, пересчитай right и up.

12.2. Почему удалённое кажется меньше

Перспективная проекция по сути связывает размер изображения с отношением размера объекта к расстоянию: `видимый размер ∝ размер/z`. Если одинаковые щиты имеют ширину 2 и стоят на расстояниях 5 и 10, дальний займет примерно вдвое меньше пикселей при прочих равных.

Проекционная матрица переводит view-координаты в clip-координаты. После деления на w получаются нормализованные координаты NDC. Без этого деления перспективы не будет; `clip.xyz` нельзя напрямую считать экранными координатами.

В `D3DClass.cpp` движок создаёт `XMMatrixPerspectiveFovLH` с вертикальным FOV 75° и отношением ширины к высоте. Для вертикального FOV 90° на расстоянии z=1 половина видимой высоты `tan(45°)·1=1`, полная высота 2. На z=10 половина высоты 10. Объект высотой 1 занимает половину высоты ближней плоскости при z=1 и только 1/20 при z=10. Это показывает связь FOV, расстояния и размера без необходимости угадывать элементы 4×4 матрицы.

Если соотношение сторон `a=width/height=16/9`, то при вертикальном FOV 90° половина видимой ширины на z=1 равна `a·tan(45°)=16/9≈1,7778`. Горизонтальный FOV тогда `2 atan(16/9)≈121,28°`. Горизонтальный и вертикальный FOV различны. Не подставляй «75°» одновременно в обе оси. При изменении окна нужно обновить aspect и projection, иначе объекты растянутся.

12.3. Near, far, глубина и отсечение

Проекция принимает ближнюю и дальнюю плоскости: область вне них отсекается. Слишком близкий near ухудшает распределение точности depth buffer по расстоянию; это может проявиться как мерцание почти совпадающих поверхностей (`z-fighting`). Для космического симулятора с размером корабля в метры и далёкими объектами нельзя автоматически поставить near почти в ноль и far на астрономический масштаб, ожидая одинаковой точности везде. Подбирай диапазоны по камере и сцене, применяй отдельные решения для очень дальнего фона, а расстояния вычисляй в согласованных единицах. Точные пределы проекта всегда проверяй в коде настроек проекции.

В Direct3D после деления clip на w глубина NDC лежит в диапазоне `[0;1]`. X и Y лежат приблизительно в `[-1;1]` для видимой области. Фрагменты за пределами clip volume отбрасываются или обрезаются. Координата z в depth buffer — нелинейное представление расстояния при перспективе; поэтому нельзя считать «depth=0,5 значит половина расстояния между near и far». Для восстановления линейной глубины нужно знать формулу используемой проекции либо применить inverse projection.

12.4. Путь вершины в реальном shader

В `src/Render/Shader/Vs/pbr.vs` вершина последовательно проходит `mul(finalPos, worldMatrix)`, затем `mul(worldPos, viewMatrix)`, затем `mul(viewPos, projectionMatrix)`. На CPU `PBRShaderClass.cpp` транспонирует world/view/projection перед записью в constant buffer. Эти две части надо читать вместе. Математическая композиция движка — world → view → projection для вектора-строки. Транспонирование перед HLSL здесь связано с хранением и соглашением матриц между DirectXMath и HLSL; оно не меняет намеренный порядок пространств. Для локализации ошибки нарисуй одну вершину и запиши на каждом этапе значения: local, world, view, clip, NDC. Если world уже неверен, перестройка projection не поможет.

Пример для простого случая: локальная точка `(1;0;0;1)`, объект перенесён на +2 по X, камера стоит в `(0;0;-5)` и смотрит вдоль +Z. После world точка `(3;0;0;1)`, после view — `(3;0;5;1)`. Проекция зависит от FOV, aspect, near и far. Уже до неё видно, что точка находится справа от оптической оси и на положительной глубине 5. Это хорошая промежуточная проверка для разработчика, который пока не умеет перемножать полную projection matrix вручную.

12.5. Как превратить клик мыши в мировой луч

Пусть окно имеет ширину W и высоту H, а курсор — пиксель `(px;py)` от левого верхнего угла. Начни с нормализованных экранных координат: `x_ndc=2px/W−1`, `y_ndc=1−2py/H`. Центр окна `(W/2;H/2)` превращается в `(0;0)`. Верхний левый угол `(0;0)` превращается в `(-1;1)`. Знак Y перевёрнут, потому что экранная ось Y растёт вниз, а NDC — вверх. Если есть DPI scaling, viewport внутри окна или render target другой величины, используй размеры именно viewport в пикселях изображения.

Сконструируй два положения в clip/NDC: `near=(x_ndc;y_ndc;0;1)` и `far=(x_ndc;y_ndc;1;1)` для Direct3D диапазона глубины. Применяй inverse матрицы `view·projection` по соглашению векторов-строк. После умножения на inverse у полученных однородных точек раздели x,y,z на w. Результат — две мировые точки. Направление луча `normalize(farWorld−nearWorld)`; начало можно взять в nearWorld либо в положении камеры, если схема и тип проекции это допускают. Обе точки требуются: одна экранная координата не задаёт уникальную глубину.

Учебный численный контроль без матриц: viewport 800×600, клик `(600;150)`. Тогда `x_ndc=2·600/800−1=0,5`, `y_ndc=1−2·150/600=0,5`. Клик вправо и вверх от центра, значит мировой луч должен идти в верхнюю правую часть изображения. Если после inverse он идёт вниз, проверь инверсию экранной Y и порядок матриц. Если движок использует `XMVector3Unproject`, проверь, что переданные viewport и near/far согласованы с теми же матрицами.

12.6. Луч, плоскость и объект

Луч записывают как `P(t)=O+tD`, где O — начало, D — нормализованное направление, `t≥0` — расстояние вдоль него. Плоскость через точку Q с нормалью N описывается условием `(P−Q)·N=0`. Подставляем луч: `(O+tD−Q)·N=0`; раскрываем dot и получаем `t=((Q−O)·N)/(D·N)`. Если знаменатель около нуля, луч почти параллелен плоскости. Если t<0, пересечение за началом луча.

Пример: начало `(0;2;0)`, направление `(0;−1;0)`, пол `y=0` с Q=`(0;0;0)` и N=`(0;1;0)`. Числитель `(Q−O)·N=(0;−2;0)·(0;1;0)=−2`; знаменатель `D·N=−1`; t=2. Точка `O+2D=(0;0;0)`. Для реальной геометрии чаще используется Bullet ray test или collision query. Математика остаётся той же в основе, но Bullet дополнительно выбирает ближайшую форму, учитывает фильтры столкновений и возвращает нормаль. В UI/gameplay проверь фильтры, чтобы луч не попадал сначала в невидимый proxy или собственный корпус игрока.

12.7. Ортографическая проекция и миникарта

Ортографическая проекция сохраняет размер объекта независимо от глубины. Это удобно для интерфейса, схемы отсека или вида сверху, где метр должен занимать одинаковое число пикселей. Если видимый мировой прямоугольник имеет ширину 20 метров, а viewport 1000 пикселей, масштаб равен 50 пикселей на метр. Значит, объект шириной 2 метра занимает 100 пикселей и на ближнем, и на дальнем краю видимого диапазона. В перспективе это не так. Для пиксельного интерфейса ортографическая проекция избавляет от неожиданного изменения масштаба, но координаты Y и origin всё равно нужно согласовать.

12.8. Задачи главы

5. Камера без поворота стоит в `(2;0;0)`. Какая view-X у мировой точки `(5;0;0)`? Ответ: 3.
6. Для viewport 1000×500 курсор `(250;125)`. Найди NDC X,Y. Ответ: `x=−0,5`, `y=0,5`.
7. Луч из `(0;5;0)` направлен вертикально вниз. Где он встретит пол y=0 и чему равно t для единичного D? Ответ: `(0;0;0)`, t=5.
8. Почему farWorld−nearWorld нормализуют? Чтобы направление луча имело длину 1; тогда t удобно интерпретировать как расстояние в мировых единицах.

13. Время, скорость, ускорение и численное движение

13.1. `dt` измеряется в секундах

Игра обновляется шагами. Между двумя обновлениями проходит время `dt` (delta time). Если предыдущий кадр закончился в момент 2,000 с, а текущий в 2,016 с, то `dt=0,016` с. При 60 кадрах/с средний dt около `1/60≈0,01667` с, при 30 кадрах/с — `1/30≈0,03333` с. Число кадров само по себе не является временем. Код `position.x += 0,1f` каждый кадр перемещает объект вдвое быстрее на 60 FPS, чем на 30 FPS. Если скорость задана как 6 метров в секунду, пиши `position.x += 6·dt`: на 60 FPS прибавка около 0,1 м, на 30 FPS около 0,2 м, за одну реальную секунду оба режима пройдут около 6 м.

Размерностная проверка спасает от ошибок: `[м/с]·[с]=[м]`. Если `speed` — м/с, `acceleration` — м/с², то `velocity += acceleration·dt` имеет единицу м/с. А выражение `position += acceleration·dt` даёт м/с и не может быть корректной прибавкой к метрам. Прежде чем искать ошибку в коде, подпиши единицы рядом с каждым промежуточным значением. Для углового движения `[рад/с]·[с]=[рад]`.

Не подменяй dt константой `0,016f`, если движок может работать при переменном FPS. Не умножай величину на dt дважды: если velocity уже обновлена с `acceleration·dt`, прибавка позиции `velocity·dt` использует dt ещё раз потому, что это другая физическая операция; но `position += speed·dt·dt` для постоянной скорости даст неверный результат. При паузе симуляции игровое dt должно стать нулём или вызовы симуляции должны прекратиться, даже если реальные кадры UI продолжаются.

13.2. Средняя скорость и мгновенная скорость

Средняя скорость на промежутке `Δt` — `Δposition/Δt`. NPC прошёл от X=2 м до X=5 м за 1,5 с: `Δx=3 м`, средняя скорость `3/1,5=2 м/с`. Он мог ускоряться и тормозить, поэтому мгновенная скорость в конкретный момент может отличаться. Векторная скорость содержит направление. Если за 2 с позиция изменилась от `(1;0;1)` до `(5;0;7)`, перемещение `(4;0;6)` м, средний вектор скорости `(2;0;3)` м/с. Его модуль `sqrt(2²+3²)=sqrt13≈3,606 м/с`. Это не то же самое, что «суммарный путь» при сложном маршруте с поворотами: средняя скорость по перемещению может быть нулём, когда NPC обошёл комнату и вернулся в исходную точку.

Ускорение — изменение скорости за время: `a=Δv/Δt`. Например, скорость корабля выросла от 4 до 10 м/с за 3 с при постоянном ускорении: `a=(10−4)/3=2 м/с²`. При постоянном ускорении аналитические формулы `v(t)=v0+a·t`, `x(t)=x0+v0·t+(a·t²)/2`. Для x0=0, v0=4, a=2 и t=3 получаем `x=0+4·3+2·9/2=21 м`. Это эталон, с которым можно сравнить численный интегратор.

13.3. Явный Эйлер: простейший интегратор

В реальном кадре ускорение может изменяться из-за управления и столкновений. Мы приближаем движение малыми шагами. Явный Euler использует скорость в начале шага: `p_next=p+v·dt`, затем `v_next=v+a·dt`. Возьмём p=0 м, v=0 м/с, a=2 м/с², dt=1 с. После первого шага p=0, v=2; после второго p=2, v=4. Точная аналитическая позиция в t=2 с при постоянном ускорении — 4 м. Ошибка 2 м велика, потому что шаг огромный и метод отбрасывает движение, возникшее из ускорения внутри каждого шага. При dt=0,01 с ошибка меньше, но полностью не исчезает.

Это объясняет, почему физический результат чувствителен к размеру шага. Если написать собственную кинематику NPC, можно использовать явный Euler для простого движения, но не следует ожидать от него идеального сохранения энергии или точных траекторий. Для Bullet не подменяй его внутренний шаг своим обновлением `Transform` после симуляции: реши, какая система владеет позицией конкретного тела.

13.4. Полунеявный Эйлер

Полунеявный, или symplectic, Euler сначала обновляет скорость: `v_next=v+a·dt`, затем позицию новой скоростью: `p_next=p+v_next·dt`. Для тех же p=0, v=0, a=2, dt=1: после первого шага v=2, p=2; после второго v=4, p=6. Точное p=4 находится между результатами явного и полунеявного методов. Это не доказательство, что один всегда точнее другого. При длительной симуляции пружин и орбит свойства устойчивости отличаются, а для столкновений важны ограничения физического движка. Выбор метода нужно проверять на конкретном сценарии и фиксированном шаге.

Если ускорение постоянно в пределах шага, можно использовать кинематическую формулу `p_next=p+v·dt+0,5a·dt²`, `v_next=v+a·dt`, которая для примера даст p=1 после первого шага и p=4 после второго. Но если ускорение зависит от позиции, столкновений или тяги, формула перестаёт быть точной без дополнительных условий. Не переносите «точную» формулу постоянного ускорения на любой симулятор.

13.5. Фиксированный физический шаг и аккумулятор

Рендер может идти с переменным временем кадра. Физике часто нужен повторяемый фиксированный шаг h, например 1/60 с. Схема: прибавить реальное время кадра в аккумулятор; пока там хотя бы h — выполнить один `simulate(h)` и вычесть h; остаток использовать для интерполяции изображения. Если рендер кадр занял 0,050 с и h≈0,01667, получится три физических шага. Если кадр занял 0,010 с, физического шага в этот кадр может не быть — аккумулятор сохранит остаток до следующего.

```
accumulator += frameDtSeconds;
const float fixedStep = 1.0f / 60.0f;
while (accumulator >= fixedStep) {
    SimulateOneStep(fixedStep);
    accumulator -= fixedStep;
}
const float alpha = accumulator / fixedStep;
RenderInterpolated(alpha);
```

Это **учебный образец**, а не утверждение, что текущий цикл приложения именно так устроен. В реальном коде нужно ограничить число шагов после долгой остановки или сворачивания окна, иначе программа будет бесконечно «догонять» физику. При этом простое обрезание dt теряет игровое время, поэтому решение зависит от требований: одиночная игра, сеть, replay, пауза. Для воспроизводимого теста поведения NPC фиксируй не только h, но и порядок обновления систем, источник случайных чисел и начальное состояние.

13.6. Интерполяция и экстраполяция

`lerp(a,b,t)=a+(b−a)t`. При a=10, b=20, t=0,25 получаем 12,5. Если t от 0 до 1, результат лежит на отрезке между ними. Для отображения между двумя физическими состояниями используй предыдущую и текущую позицию: `renderPos=lerp(previous,current,alpha)`. Это сглаживает видимое движение, но изображение обычно отстаёт от текущей симуляции на часть шага. Для сетевой игры используют и более сложные буферы состояний. Экстраполяция пытается предсказать будущее, например `predicted=p+v·delay`; она уменьшает видимую задержку, но может ошибаться при внезапном повороте или ударе.

Численный пример для сети: известная позиция цели `(10;0;0)` и скорость `(2;0;0)` м/с, предполагаемая задержка 0,1 с. Простейшее предсказание даёт `(10,2;0;0)`. Если цель в этот момент резко остановилась, предсказание промахнётся на 0,2 м. Поэтому прогноз — допущение, а не измеренный факт. Для атаки NPC лучше отличать настоящее наблюдение, последнюю видимую позицию и предсказанную позицию; иначе AI будет «знать» будущее.

13.7. Задачи главы

9. Персонаж идёт 3 м/с, dt=0,02 с. Сколько он проходит за кадр? Ответ: 0,06 м.
10. Скорость выросла с 1 до 5 м/с за 2 с. Постоянное ускорение? Ответ: 2 м/с².
11. Для `lerp(4,12,0,75)` вычисли результат. Ответ: `4+(12−4)·0,75=10`.
12. Объект с v=6 м/с движется 0,5 с без ускорения. Ответ: 3 м.
13. Чем опасен один шаг dt=0,5 с для тела, летящего к стене? Между старой и новой позициями оно может проскочить тонкую стену; нужны подходящие настройки непрерывного обнаружения столкновений, ограничение шага и корректная геометрия.

14. Математика прямо в системах космического симулятора

14.1. Дальность и квадрат расстояния в состоянии атаки NPC

В `StateSystem.h` при атаке NPC код сначала вычисляет `dx`, `dy`, `dz` как координаты цели минус координаты NPC, затем `distanceSquared=dx²+dy²+dz²`. Это квадрат длины вектора к цели. Для проверки «ближе ли цель чем attackRange» квадратного корня не нужно: при неотрицательных расстояниях `distance≤R` равносильно `distance²≤R²`. Например, разность `(3;0;4)` даёт

`distanceSquared=9+16=25`; дальность 6 даёт `rangeSquared=36`. Цель внутри 6 м, потому что 25≤36. Сравнение 25≤6 было бы ошибкой единиц: слева м², справа м.

Этот приём экономит `sqrt`, но его главная ценность здесь — явный контракт. Если для рукопашной атаки измеряется 3D расстояние, противник на другом этаже может оказаться вне дальности, хотя горизонтальная дистанция мала. Для ремонта `StateSystem.h` специально использует `HorizontalDistance` и не учитывает Y, потому что персонаж ходит по палубе. Это два разных правила. Перед копированием функции расстояния сформулируй игровое условие: «насколько близки по полу», «дотянется ли рука в 3D», «видит ли камера через портал». Одно слово `distance` не сообщает, какие оси участвовали.

Когда NPC вооружён, `StateSystem.h` берёт минимум из `rangedAttackRange` NPC и `range` оружия и ограничивает значение снизу нулём. Численный пример: в настройке NPC 15 м, у оружия 10 м; практический предел 10 м. Если цель на 12 м, персонаж ещё должен сближаться, хотя его собственный параметр 15. Этот минимум нужен, чтобы боевой AI не ожидал выстрела на расстоянии, которого оружие не поддерживает. Проверку линии огня минимум не заменяет: в 5 м от цели может стоять стена или дружественный NPC.

14.2. Поворот NPC к цели в настоящем коде

Для цели относительно NPC `dx`, `dz` задают горизонтальный вектор. В состоянии `Attacking` проект считает `targetYaw=atan2(dx,dz)+π`. Добавление π — поправка к базовой ориентации модели/системы в этом участке кода, которую нельзя выбросить, если хочешь воспроизвести нынешнее поведение. Затем разность между targetYaw и текущим yaw приводится через `atan2(sin(diff),cos(diff))` к кратчайшему углу. Максимальный шаг равен `movement.rotationSpeed·deltaTime`, а `std::clamp` ограничивает изменение yaw. После поворота код считает **оставшийся** угол и допускает атаку лишь при отклонении не более 8°.

Разберём числа без добавленной π, чтобы видеть механику ограничения. Текущий yaw=0°, требуемый=30°, скорость 90°/с, dt=0,1 с. Максимальный шаг 9°, новый yaw=9°, остаток 21°; удар ещё нельзя запускать при допуске 8°. После трёх шагов yaw=27°, остаток 3°, можно. Если бы проверка использовала разность **до** поворота, атака началась бы на кадр позже или раньше в зависимости от состояния и порога. В исходном `StateSystem.h` комментарий рядом с `remainingYaw` специально объясняет повторное вычисление после изменения rotation.

Для новых моделей настройку π лучше выразить как явно названный `modelForwardYawOffset`, если архитектура допускает это изменение. Но перед рефакторингом зафиксируй тесты для существующих NPC: направь их к целям по +X, −X, +Z, −Z; сравни видимую ось лица и направление ударного/оружейного socket. Иначе «упрощение» формулы сломает модель, которая ориентирована локальным −Z.

14.3. Ремонт: прогресс как величина, скорость как отношение

В состоянии ремонта `StateSystem.h` устанавливает `duration=max(brain.repairDuration,0,05)` и ограничивает `repairSkill` экипажа диапазоном `[0,25;2]`. Прибавка прогресса равна `requiredProgress·dt·repairSkill/duration`, затем передаётся в общий `RepairSystem::ApplyRepairProgress`. Это фактическая формула проекта. Если requiredProgress=1, duration=8 с, dt=0,5 с, skill=1,5, то прибавка за шаг `1·0,5·1,5/8=0,09375`. За 16 таких шагов, то есть 8 с, накопилось бы 1,5 единицы прогресса. Непрерывная оценка времени завершения равна `8/1,5≈5,333 с`, а при отдельном обновлении раз в 0,5 с порог будет достигнут на 11-м шаге, то есть через 5,5 с. Это и означает «на 50% быстрее» в комментарии к коду: скорость прогресса умножена на 1,5, время поделено на 1,5. Время не сокращается на 50%; непрерывная оценка сокращается на треть.

Если requiredProgress=10, duration=8 с, skill=1, то скорость `10/8=1,25` единиц прогресса в секунду. За dt=0,2 с добавится 0,25 единицы. Нормализованное значение для UI `progress/requiredProgress`: 2,5/10=0,25 или 25%. При переходе от 0 до 1 арифметика удобна, но не забывай хранить в данных реальный требуемый прогресс и условия завершения. Если требуются инструмент, запчасть,

давление в отсеке или доступность электросети, один только счётчик не решает задачу. Такие условия должны быть явными в отдельной логике и сообщать причину ожидания игроку.

При проектировании новой работы экипажа вычисляй `timeToComplete = remainingProgress / effectiveRate`, если rate>0. Это позволяет AI выбирать между несколькими задачами с учётом навыка и пути. Например, пробоина A требует 5 единиц, текущий прогресс 1, темп экипажа 2 ед/с: чистая работа займёт `(5−1)/2=2 с`. Пробоина B требует 10, осталось 10, темп 2: 5 с. Но если до A идти 12 с, а до B — 1 с, B может быть предпочтительнее при близкой опасности. Это **предлагаемая модель выбора**, не утверждение о текущем `StateSystem.h`.

14.4. Путь и цена задачи: пример математической модели AI

Кратчайшее евклидово расстояние не равно времени пути через двери. Два отсека могут быть в двух метрах по прямой, но между ними — закрытая стена; путь через коридор займёт 20 м. Для выбора работы используй длину маршрута по навигационному графу или оценку времени, а затем добавляй стоимость риска. Учебная функция оценки: `cost = routeSeconds + workSeconds + riskPenaltySeconds − urgencyBonusSeconds`. Все слагаемые приведены к секундам оценки, поэтому их можно складывать осмысленно. Для задачи A: 8+2+3−5=8; для B: 2+5+0−1=6; B дешевле по выбранной модели. В реальной игре веса надо проверять наблюдением: не допускай, чтобы бонус срочности делал cost бесконечно отрицательным и NPC бесконечно переключался между задачами.

Для предотвращения постоянного переключения введи гистерезис: новую задачу принимай только если она существенно лучше текущей, например `newCost < currentCost−switchPenalty`. При currentCost=8 и switchPenalty=2 новая задача с cost=7 не вытеснит текущую, а с cost=5 — вытеснит. Это поведенческое решение опирается на простое сравнение, но заметно влияет на правдоподобие экипажа. Переходы состояния и приоритет команды игрока должны оставаться отдельными правилами: арифметическая оценка не должна отменять прямой приказ без осознанного дизайна.

14.5. Луч Bullet и физические единицы

`src/Physics/PhysicsRaycast.h` перед `world->rayTest` нормализует входное направление и строит конец луча как `origin + forward·distance`. Если origin=`(1;2;3)`, direction=`(0;0;10)`, distance=5, нормализация даст forward=`(0;0;1)`, а конец `(1;2;8)`. Без нормализации конец был бы `(1;2;53)`, и параметр distance перестал бы означать пять мировых единиц. В том же коде есть проверка `distance<=0`, отсутствие world и нечисловые компоненты forward. При нулевом направлении нормализовать нечего: такой отрезок не описывает осмысленный выстрел и должен отсеиваться до вызова Bullet.

Проверка `isfinite` после нормализации не доказывает, что длина исходного направления была больше нуля: нулевой вектор может остаться нулевым. Поэтому при добавлении нового вызова следует отдельно требовать `dot(direction,direction)>epsilon²`, прежде чем строить отрезок. `rayTest` отвечает на геометрический вопрос «с чем пересекается отрезок», а не на игровой «можно ли наносить урон». После hit ещё нужны отношения фракций, тип поверхности, фильтр собственных частей, состояние двери, зона уязвимости и политика проникновения снаряда. При отладке рисуй начало и конец луча, точку попадания и нормаль; подписывай длину и единицы. Когда дальнобойный NPC видит цель, но не стреляет, математическая диагностика должна показать, какой именно луч и какую преграду использует система.

Для баллистического снаряда начальная скорость `v0` и гравитация `g` дают учебную траекторию `p(t)=p0+v0t+0,5gt²` при постоянной g и без сопротивления. Например, горизонтальная скорость 10 м/с, вертикальная 5 м/с, g=`(0;−10;0)` м/с²: за 1 с смещение `(10;0;0)` м по X и `5−5=0` м по Y, снаряд вернулся на высоту старта. Это не описание текущего оружия проекта: hitscan и физический projectile требуют разных реализаций. Bullet может обновлять динамическое тело со своими настройками гравитации; не добавляй к такому телу вторую ручную гравитацию в gameplay update.

14.6. Математика vertex и pixel shader

Vertex shader получает отдельные вершины. Его главная пространственная работа в `pbr.vs` — превратить позицию из локального пространства через world и view в clip. Pixel shader вычисляет

видимую окраску фрагмента. В `src/Render/Shader/Ps/pbr.ps` освещение использует, например, `NdotL=saturate(dot(N,L))`: N — нормаль поверхности, L — направление к свету. Если N и L совпадают, dot=1, поверхность смотрит прямо на свет. Если они перпендикулярны, dot=0, прямой вклад равен нулю. Если dot отрицателен, свет находится за поверхностью, а `saturate` обрежет значение к нулю. Нормализация N и L важна, иначе dot зависит ещё и от их длины.

Численный пример: N=`(0;1;0)`, L=`(0;0,6;0,8)`. Оба единичны, `dot=0,6`; простой диффузный фактор 0,6. Если по ошибке L=`(0;6;8)` без нормализации, dot=6, после `saturate` станет 1 — свет покажется максимальным, хотя направление то же самое. Это объясняет, зачем shader содержит безопасную нормализацию. PBR модель в коде сложнее простого Lambert: там есть `DistributionGGX`, геометрический фактор и Fresnel, roughness/metalness. Но их входные NdotL, NdotV и NdotH по-прежнему состоят из тех же векторных операций.

Нормаль из normal map сначала хранится как цвет в диапазоне `[0;1]` по каналам. Обычное преобразование к направлению — `n=2·texel−1`, то есть цвет `(0,5;0,5;1)` становится `(0;0;1)`. Но это направление в tangent space, а не сразу в мире: нужен базис tangent/bitangent/normal, построенный из геометрии и согласованный с UV. Если normal map выглядит «вывернутой», проверь соглашение канала Y и порядок базиса, а не меняй освещение наугад. Код `pbr.ps` дополнительно ортогонализует tangent относительно N через `T−N·dot(N,T)`: это проекция из главы 5. После вычитания T становится перпендикулярным N, а нормализация возвращает длину 1.

14.7. Числа, которые нельзя складывать без смысла

В игровом коде легко увидеть выражение вроде `health + distance + lightIntensity` и принять его за «score». Математически сложить числа можно, но физически они измеряют разные вещи. Перед составлением оценки переведи их в безразмерные доли или в общую шкалу. Например, `health01=health/maxHealth`, `distance01=clamp(distance/maxRelevantDistance;0;1)`, `danger01=1−health01`. Тогда `score=0,6·danger01+0,4·(1−distance01)` — понятная, хотя всё ещё дизайнерская, формула. При health=25/100, distance=2/10: danger=0,75, близость=0,8, score=`0,45+0,32=0,77`. Проверяй крайние случаи: здоровье 0, дистанция 0, дистанция вне диапазона, maxHealth=0. `clamp` не исправляет деление на ноль.

15. Большой практикум: реши до просмотра ответов

Эти задачи соединяют арифметику с решениями, которые придётся принимать в движке. Перед расчётом подпиши единицы измерения, размерность матрицы и пространство координат. Если ответ выглядит правдоподобно, всё равно проверь его обратной операцией или крайним случаем. Особенно полезно вести расчёт на бумаге: умение найти одну неверную клетку матрицы иногда сэкономит день отладки shader.

15.1. Задачи

14. NPC находится в `(2;1;−3)` м, цель в `(8;1;5)` м. Найди вектор к цели, его длину, единичное направление и горизонтальную длину.
15. Два единичных направления `a=(1;0;0)` и `b=(0,6;0;0,8)`. Найди dot, угол между ними и ответь, попадёт ли b в конус с полным FOV 100° вокруг a.
16. Даны `A=[2 −1 3;0 4 5]` и `B=[−2 6 1;7 −4 0]`. Найди `A+B`, `A−B`, `3A` и A^{T}. Подпиши размер каждого ответа.
17. Даны `C=[1 2;3 1]` и `D=[2 0;−1 4]`. Вычисли `C·D` и `D·C`, показывая четыре суммы для каждого ответа. Одинаковы ли произведения?
18. Найди determinant и inverse для `M=[3 1;2 1]`. Перемножь M с найденной обратной и объясни, почему ответ — I.
19. Локальная точка `(2;0;0;1)` сначала масштабируется по X в 3 раза, затем переносится на +4 по X. Каков её X в мире? Что получится, если поменять порядок? Что произойдёт с направлением `(1;0;0;0)` при одном только переносе?

20. Переведи 135° в радианы. Направление к цели на плоскости XZ равно `(-4;0;4)` при нулевом yaw вдоль +Z. Найди угол `atan2(dx,dz)` в градусах. Почему одного `atan(dx/dz)` может быть недостаточно в других четвертях?
21. Окно 1280×720 пикселей. Клик в `(960;540)`. Найди x,y в NDC. Укажи, где курсор относительно центра.
22. Ремонт требует 12 единиц прогресса. Уже сделаны 3. `repairDuration=6` с, skill=1,5, `dt=0,25` с. Вычисли прибавку за кадр, нормализованный прогресс после него и оценку оставшегося времени при постоянном темпе. Это учебное предположение без задержек и конкурирующих действий.
23. Стрелок стоит в `(0;1;0)`, направление выстрела `(3;0;4)`, дальность 10 м. Вычисли нормализованное направление и конечную точку отрезка ray test. Если между ними враг на 8 м и стена на 5 м, какой объект геометрически должен быть первым попаданием без фильтров?
24. Скорость NPC `(4;0;0)` м/с, ускорение `(-2;0;0)` м/с², dt=0,5 с, стартовая позиция `(0;0;0)`. Вычисли один шаг явного и полунеявного Euler. Почему позиции различаются?
25. В vertex shader локальная позиция `(1;0;0;1)` проходит перенос world +3 по X, затем view для камеры без поворота в `(2;0;-4)`. Найди world и view координаты до projection. Что обязательно нужно сделать с clip координатами после projection, чтобы получить NDC?

15.2. Подробные решения

1. Направление и дистанция. Вычитаем «цель минус NPC»: `(8-2;1-1;5-(-3))=(6;0;8)` м. Квадрат длины `6²+0²+8²=36+64=100` м², длина 10 м. Делим каждую компоненту на 10: `(0,6;0;0,8)`. Горизонтальная длина по XZ тоже 10 м, потому что dy=0. Проверка: `0,6²+0,8²=0,36+0,64=1`. Если бы Y цели был другим, полная и горизонтальная длины разошлись бы.

2. Угол и поле зрения. `dot(a,b)=1·0,6+0·0+0·0,8=0,6`. Оба вектора единичны, поэтому `cosθ=0,6`, θ=`acos(0,6)≈53,13°`. Полный FOV 100° означает половинный угол 50°. `53,13°>50°`, следовательно b вне конуса. Эквивалентная проверка без acos: порог `cos50°≈0,6428`; `dot=0,6<0,6428`, вне. Если бы кто-то сравнил с 100°, он ошибочно включил бы цель.

3. Четыре операции с прямоугольной матрицей. Обе исходные матрицы 2×3, поэтому сумма, разность и скалярное произведение тоже 2×3. Сумма первой строки `[2+(-2), -1+6, 3+1]=[0,5,4]`, второй `[0+7,4+(-4),5+0]=[7,0,5]`; `A+B=[0 5 4;7 0 5]`. Разность первой `[2-(-2),-1-6,3-1]=[4,-7,2]`, второй `[0-7,4-(-4),5-0]=[-7,8,5]`; `A-B=[4 -7 2;-7 8 5]`. `3A=[6 -3 9;0 12 15]` размером 2×3. При транспонировании первый столбец A `[2;0]` станет первой строкой, второй `[-1;4]` второй, третий `[3;5]` третьей: `Aᵀ=[2 0;-1 4;3 5]`, размер 3×2. Проверка: `(A+B)-B=A` по каждой клетке.

4. Произведения. Для C·D: верхняя левая `1·2+2·(-1)=0`, верхняя правая `1·0+2·4=8`, нижняя левая `3·2+1·(-1)=5`, нижняя правая `3·0+1·4=4`. Значит `C·D=[0 8;5 4]`. Для D·C: верхняя левая `2·1+0·3=2`, верхняя правая `2·2+0·1=4`, нижняя левая `(-1)·1+4·3=11`, нижняя правая `(-1)·2+4·1=2`; `D·C=[2 4;11 2]`. Оба результата 2×2, но не равны. Поэлементное произведение, например `[2 0;-3 4]`, было бы третьей операцией, которую вопрос не спрашивал.

5. Обращение. `det(M)=3·1-1·2=1`. Формула для `[a b;c d]` даёт `M⁻¹=(1/1)[1 -1;-2 3]=[1 -1;-2 3]`. Проверка M·M⁻¹: клетки `3·1+1·(-2)=1`, `3·(-1)+1·3=0`, `2·1+1·(-2)=0`, `2·(-1)+1·3=1`. Получили `[1 0;0 1]`, поэтому найденное преобразование действительно отменяет M.

6. Порядок. Сначала scale: 2→6, затем translation: 6→10. При обратном порядке: 2→6 после переноса, затем 6→18 после масштаба. Числа 10 и 18 отвечают двум разным цепочкам. Направление с w=0 при одном только переносе остаётся `(1;0;0;0)`, потому что в переносе участвует множитель w и он равен нулю. Для вектора-строки первая цепочка записывается `p·S·T`, вторая `p·T·S`.

7. Углы. `135°·π/180=3π/4≈2,35619` радиана. Для `(dx,dz)=(-4,4)` `atan2(-4,4)=-45°` или `-π/4`. В другой четверти, например `(dx,dz)=(4,-4)`, отношение dx/dz=−1 такое же, но истинный угол от +Z — 135°, а не −45°. `atan2` видит оба знака и выбирает четверть; деление теряет этот факт и ещё ломается при dz=0.

8. Пиксель в NDC. `x=2·960/1280−1=1,5−1=0,5`. `y=1−2·540/720=1−1,5=−0,5`. Положительный x — правее центра; отрицательный y — ниже центра. Верх экрана имеет положительный NDC Y после указанного преобразования, несмотря на то что пиксельный Y растёт вниз.

9. Ремонт. Скорость прогресса `required·skill/duration=12·1,5/6=3` единицы/с. За 0,25 с прибавится `3·0,25=0,75`. Новый прогресс 3,75 из 12, нормализованный `3,75/12=0,3125`, или 31,25%. Осталось `12−3,75=8,25`; при скорости 3 ед/с понадобится `8,25/3=2,75` с. Здесь предполагается, что ремонт не заблокирован и никакая другая система не изменит прогресс.

10. Bullet луч. Длина исходного направления `sqrt(3²+4²)=5`, нормализованное направление `(3/5;0;4/5)=(0,6;0;0,8)`. Конец `origin+direction·10=(0;1;0)+(6;0;8)=(6;1;8)`. Если без фильтров стена находится в 5 м, а враг в 8 м на том же луче, первое пересечение — стена. Геометрия не сообщает, можно ли пробить её, поэтому политика урона рассматривается после hit.

11. Два интегратора. Ускорение за шаг изменяет скорость на `a·dt=(−1;0;0)` м/с, конечная скорость `(3;0;0)` м/с у обоих методов. Явный Euler сначала сдвигает по старой скорости: `(0;0;0)+(4;0;0)·0,5=(2;0;0)` м. Полунеявный использует новую скорость: `(0;0;0)+(3;0;0)·0,5=(1,5;0;0)` м. Точное положение для постоянного ускорения за полсекунды `4·0,5+0,5·(−2)·0,5²=2−0,25=1,75` м лежит между ними. Различие вызвано приближением непрерывного изменения скорости одним дискретным значением на шаг.

12. Пространства vertex shader. После world переноса локальная X=1 становится мировой X=4: `(4;0;0;1)`. Камера в `(2;0;−4)` без поворота задаёт простую view разность `world−camera=(2;0;4)`, значит view координаты `(2;0;4;1)`. После projection получаем clip `(x;y;z;w)`. Для NDC нужно выполнить perspective divide `(x/w;y/w;z/w)` при допустимом ненулевом w; до этого нельзя сравнивать clip X/Y с экранным диапазоном `[-1;1]`.

15.3. Самоконтроль перед работой с новой системой

Попробуй объяснить вслух без подсказки следующие переходы. Почему `position` и `direction` должны иметь разные w? Почему сравнение квадратов дальности допустимо, но сравнение `distanceSquared` с обычным радиусом — нет? Почему нормаль после неравномерного масштаба нельзя преобразовать как позицию? Как определить размер результата A(m×n)·B(n×p), прежде чем перемножать числа? Как найти клетку `(i,j)` через строку и столбец? Что изменится, если в `S·R·T` переставить множители? Как из пикселя получить мировой луч и зачем две глубины? Почему `velocity·dt` измеряется в метрах, а `acceleration·dt` — в метрах в секунду? Если на любой вопрос нет уверенного ответа, вернись к соответствующей главе и пересчитай её численный пример вручную.

16. Карта источников и правила дальнейшего изучения

Факты о проекте в этой главе сверены с файлами `src/ECS/Components/Components.h` (`TransformComponent::GetWorldMatrix`), `src/Render/Shader/Vs/pbr.vs`, `src/Render/Shader/Ps/pbr.ps`, `src/Render/Shader/PBRShaderClass.cpp`, `src/Render/D3DClass.cpp`, `src/Render/Camera/cameraclass.cpp`, корневым `StateSystem.h` и `src/Physics/PhysicsRaycast.h` внутри `DX_FrameWork`. Описания нового оценщика задач, фиксированного шага, простых баллистических траекторий и абстрактных матриц помечены как учебные модели; это варианты реализации, которые нужно согласовать с реальным циклом систем и физикой проекта.

Для проверки библиотечного соглашения использованы официальные материалы Microsoft: DirectXMath Programming Guide (https://learn.microsoft.com/en-us/windows/win32/dxmath/pg-xnamath-getting-started), DirectXMath Optimization (https://learn.microsoft.com/en-us/windows/win32/dxmath/pg-xnamath-optimizing), структура XMMATRIX (https://learn.microsoft.com/en-us/windows/win32/api/directxmath/ns-directxmath-xmmatrix) и HLSL transpose (https://learn.microsoft.com/en-us/windows/win32/direct3dhlsl/dx-graphics-hlsl-transpose). Они объясняют соглашения хранения и операций, но фактический порядок world/view/projection для этой игры следует прежде всего из её `GetWorldMatrix`, CPU-записи constant buffer и `pbr.vs`. Если

поменяются shader, режим компиляции HLSL или соглашение матриц, пересчитай простую точку `(1;0;0;1)` через весь pipeline и обнови эту главу вместе с кодом.

Математика и алгоритмы для игры: второй курс с выводами и расчётами

Первый математический курс дал арифметику векторов и матриц. Теперь цель — научиться превращать игровое правило в алгоритм, заранее оценивать его стоимость и проверять результат числами. Примеры ниже учебные, если явно не назван метод движка. Алгоритм хорош, когда понятны входные данные, единицы, инвариант, крайние случаи и цена выполнения. Формула без этих пяти вещей легко превращается в красивый, но неисправный код.

1. Линейная интерполяция и экспоненциальное сглаживание

Пусть дверь находится в начале хода в позиции 0 м и должна за секунду дойти до 2 м. Линейная интерполяция между `a` и `b` при параметре `t` равна `lerp(a,b,t)=a+(b-a)t`, где `t` ограничен [0,1]. При `t=0` результат 0, при `t=0.25` — 0.5 м, при `t=1` — 2 м. Почему формула устроена так? Разность `b-a` показывает полный путь, умножение на долю показывает пройденную часть, прибавление `a` возвращает нас к началу координат. Для вектора действие выполняется по каждой координате. Если `A=(1;0;2)`, `B=(5;0;6)`, `t=0.25`, получаем `(2;0;3)`.

Частая ошибка — каждый кадр вызывать `position=lerp(position,target,0.1)`. Это не ход длительностью десять кадров. После первого кадра остаётся 90% расстояния, после второго 81%, после n кадров `0.9^n` исходного расстояния. На 120 FPS за секунду шагов больше, поэтому движение приблизится к цели сильнее, чем на 30 FPS. Если нужен путь за фиксированную секунду, храни стартовую позицию и `elapsed/duration`. Если нужно именно плавное приближение, используй коэффициент, зависящий от времени: `alpha=1-exp(-lambda*dt)`, затем `x += (target-x)*alpha`. При `lambda=2/c` и `dt=0.5c` получаем `alpha=1-e^-1≈0.632`, оставшаяся доля 0.368. Два шага по 0.25 с дают ту же общую долю `1-e^-1`, если нет округления и target не менялся. Здесь размерность `lambda*dt` должна быть безразмерной.

В движке подобные операции встречаются при визуальном сглаживании, цветах частиц и повороте. Но угол нельзя бездумно интерполировать как обычное число через границу ±π: от 179° к −179° арифметическая середина 0° идёт длинным путём. Сначала вычисли кратчайшую разницу `atan2(sin(b-a),cos(b-a))`, затем прибавляй её долю. Для полной 3D ориентации вместо смешения коэффициентов матриц применяй интерполяцию кватернионов с проверкой нормировки и выбора короткой дуги.

2. Численные производная и интеграл: скорость из позиции, путь из скорости

Производная отвечает на вопрос «как быстро меняется величина сейчас?». Если позиция в t=1 с равна 2 м, а в t=1.1 с — 2.3 м, средняя скорость на интервале `(2.3-2)/0.1=3 м/с`. Чем меньше интервал, тем ближе оценка к мгновенной скорости, но слишком малый шаг усиливает ошибки `float` и шум измерения. Поэтому для velocity физического тела лучше читать согласованное значение Bullet, чем каждый кадр вычитать два почти одинаковых Transform без учёта телепорта и физического шага.

Интеграл собирает маленькие вклады за время. Если скорость постоянна 3 м/с в течение 0.1 с, путь 0.3 м. В коде это `position += velocity*dt`. Если ускорение 2 м/с², скорость за шаг меняется на `acceleration*dt`. Для начальных `v=0`, `x=0`, `dt=0.5`, два полунеявных шага дают: первый `v=1`, `x=0.5`; второй `v=2`, `x=1.5`. Аналитическое положение при постоянном ускорении за 1 с равно `x=0+0·1+0.5·2·1²=1 м`. Разница 0.5 м — численная погрешность крупного шага, а не другая физика. При меньшем фиксированном шаге ошибка падает. Bullet решает больше задач, чем эта простая формула, но понимание интегратора помогает объяснить влияние `dt`.

Проверяй размерности: `a` в м/с², `a*dt` в м/с, `v*dt` в м. Не прибавляй силу в ньютонах сразу к скорости: сначала `a=F/m`. Для массы 10 кг и силы 20 Н ускорение 2 м/с². При импульсе 20 Н·с изменение скорости равно `J/m=2 м/с`. Сила действует в течение времени, импульс уже содержит время. Этим объясняется, почему взрывной толчок лучше задавать импульсом, если он должен произойти мгновенно.

3. Алгоритм маршрута: граф, стоимость и A*

Комнаты корабля можно представить вершинами графа, двери — рёбрами. Пусть стартовая комната A связана с B стоимостью 2 м, B с целью C стоимостью 2 м, а A с C через длинный проход стоимостью 10 м. Минимальный путь — A→B→C стоимостью 4, даже если на экране прямая до C кажется короткой. Если дверь A–B заперта, ребро отсутствует или получает очень высокую стоимость; решение меняется. Для NavMesh один waypoint не является фактическим движением: после планирования капсула ещё должна пройти через физический мир.

Алгоритм Дейкстры хранит лучшую известную цену `g(v)` от старта и выбирает вершину с наименьшей текущей ценой. A* добавляет оценку оставшегося пути `h(v)` и выбирает минимум `f(v)=g(v)+h(v)`. Эвристика `h` не должна завышать реальную остаточную стоимость, если нужна гарантия кратчайшего пути. Для движения по плоскости с ценой не меньше геометрической длины прямая евклидова дистанция до цели подходит. Если за закрытую дверь назначен дополнительный штраф, прямая всё ещё нижняя граница. Если в игре есть телепорт с нулевой ценой, обычная дистанция уже может завышать реальную стоимость и гарантия нарушится.

Пройди выбор вручную. Из A доступны B с `g=2,h=2,f=4` и D с `g=1,h=8,f=9`. Сначала раскрывается B. Если из B в C ребро 2, новая цель получает `g=4,h=0,f=4`. Алгоритм находит маршрут стоимостью 4. Для NPC после двери важно не только построить путь, но и зафиксировать причину его устаревания: дверь изменила состояние, NavMesh перестроен, цель сместилась. Кэшируй путь вместе с revision данных, от которых он зависит. Бесконечный repath каждый кадр может быть дороже самого движения и не решит проблему запертой двери.

4. Выбор работы экипажа как задача оптимизации

Один NPC может выбрать работу по максимальному score, но распределение двух NPC между двумя работами требует учитывать конкуренцию. Пусть инженер A даёт полезность 9 для пробоины X и 8 для Y, инженер B — 7 для X и 1 для Y. Жадный выбор A→X оставит B→Y, сумма 10. Другой вариант A→Y, B→X даёт сумму 15. Значит локально лучший первый выбор не обязательно максимизирует пользу группы. Для маленького экипажа можно перебрать допустимые назначения или использовать алгоритм оптимального паросочетания. Для большой сцены можно оставить детерминированный жадный вариант, если скорость и понятность важнее оптимальности, но это осознанный компромисс.

Формула score должна иметь понятную шкалу. Если severity в [0,1], а расстояние в метрах от 0 до 100, сырое выражение `severity-distance` практически всегда определяется дистанцией. Нормализуй: например `distanceScore=1/(1+d/10м)`. При d=0 он равен 1, при d=10м — 0.5, при d=30м — 0.25. `score=4*severity+2*distanceScore+roleBonus` теперь имеет сравнимые слагаемые. Но score не должен отменять жёсткие запреты: отсутствует путь, нет нужного ресурса, цель уже занята, опасная среда без скафандра. Сначала фильтруй допустимость, затем ранжируй.

Инвариант claim: на эксклюзивную задачу назначен максимум один живой исполнитель. После смерти, ручного приказа, уничтожения цели или истечения срока claim должен освободиться. Если цель удаляется во время ремонта, хранить указатель на компонент через удаление опасно. Сохрани Entity ID, после структурного изменения перечитай Registry. В текущем движке цепь Utility/Coordinator/StateSystem уже существует; задача оптимального паросочетания здесь предлагается как развитие, а не заявляется готовой функцией.

5. Вероятность, разброс оружия и воспроизводимость

Случайное число из [0,1) может выбирать событие с вероятностью p: `u<p`. При `p=0.2` из 100 независимых испытаний ожидается около 20 успехов, но не ровно 20. Вероятность нуля попаданий за n независимых попыток равна `(1-p)^n`: при p=0.2 и n=5 это `0.8^5≈0.32768`. Нельзя проверять случайную стрельбу тестом «из пяти выстрелов обязаны попасть ровно один раз». Проверяй распределение на большом числе запусков, а конкретный баг воспроизводи с сохранённым seed.

Равномерный выбор угла не всегда даёт равномерную площадь на диске. Если радиус r выбирать просто равномерно от 0 до R, слишком много точек окажется близко к центру относительно площади. Для равномерной площади диска возьми `r=R*sqrt(u)`, угол `ϕ=2πv`, где u и v независимы и равномерны. Тогда `x=r*cosϕ`, `y=r*sinϕ`. Проверка: доля попаданий внутри половины радиуса должна быть площадью маленького круга относительно большого, `(R/2)²/R²=1/4`, а не 1/2. Для конуса направления нужна отдельная геометрия и нормировка; у текущего `WeaponSystem` есть собственное правило разброса, поэтому перед заменой сначала измерь его и не меняй баланс по учебной формуле вслепую.

Для NPC различай наблюдение и случайность. Пусть противник не видит игрока за стеной; случайный разброс не должен выдавать точную скрытую позицию. Он меняет направление луча вокруг уже допустимой точки прицеливания, а не создаёт знание. Запиши seed, shooter Entity, target Entity, исходный aim и фактический ray direction. Тогда один промах можно воспроизвести до пикселя и отличить ошибку геометрии дула от случайного отклонения.

6. Пространственные алгоритмы: AABB и отсев

Ось-ориентированный ограничивающий параллелепипед AABB хранит минимум и максимум по каждой оси. Точка `(x;y;z)` внутри, если `minX≤x≤maxX`, `minY≤y≤maxY`, `minZ≤z≤maxZ`. Для двух AABB пересечение есть, если интервалы перекрываются по всем трём осям. Если хотя бы по одной оси `A.maxX < B.minX`, коробки разделены, и дорогую проверку треугольников можно не делать. Это широкая фаза физики и полезный предварительный отсев видимости/взаимодействия.

Пример: A по X `[0,2]`, B по X `[3,4]`. Разделение уже доказано, Y и Z можно не читать. Если B по X `[1,3]`, X совместима, но ещё нужно проверить Y и Z. Для вращающегося объекта локальный AABB нельзя переносить только двумя диагональными углами; после вращения экстремальная вершина может быть другой. Либо трансформируй все восемь углов, либо используй центр и абсолютные компоненты матрицы для вычисления новой половины размера. Ошибочный world bounds может скрыть видимый mesh на краю камеры или пропустить raycast предварительной фазы.

Для множества объектов наивное сравнение каждого с каждым стоит `n(n-1)/2`. При n=1000 это 499500 пар. Разбиение пространства на клетки, дерево BVH или broadphase Bullet сокращает число реальных кандидатов, особенно если объекты разбросаны. Но структура ускоряет запрос ценой обновления: если каждый объект постоянно движется, её надо поддерживать. Не оптимизируй наугад; сначала измерь число проверяемых пар, стоимость обновления и время кадра.

7. Численная устойчивость и тесты свойств

При вычислениях с `float` не проверяй геометрию через абсолютное `==`. Для длины в метрах выбери допустимую ошибку в метрах; для нормированного dot — безразмерную ошибку. `epsilon=1e-6` не является универсальным ответом: для корабля километрового масштаба и для миллиметрового muzzle одна и та же абсолютная ошибка имеет разный смысл. Проверь конечность входов через `isfinite`, прежде чем NaN попадёт в Transform, Bullet или shader buffer. Сравнение `NaN < 0` ложно, поэтому обычный `clamp` не всегда спасёт испорченное значение.

Тест свойства проверяет закон, а не копирует реализацию. Для случайных совместимых матриц `transpose(transpose(A))` должно вернуть A в пределах погрешности. Для ненулевого вектора `length(normalize(v))≈1`. Для пары преобразований `p*(M*M^-1)≈p`, если M обратима. Для распределения

энергии сумма выданной мощности не превосходит доступную. Для двух комнат без утечек общий запас вещества в физической модели сохраняется. Эти проверки полезны после рефакторинга: они находят ошибку даже если конкретные числа в примере изменились.

Закончи курс мини-проектом: возьми одну пробоину, двух инженеров и две двери. На бумаге вычисли расстояния, score и допустимые назначения; нарисуй граф комнат и лучший маршрут; затем запусти сцену и сравни debug числа. Разница между расчётом и игрой должна иметь имя: физическое препятствие, изменившееся состояние двери, другой `dt`, конкурирующий claim или ошибка формулы. Так математика становится инструментом исследования собственного кода.

8. Луч, плоскость и сфера: вычисляем попадание до Bullet

Луч описывается началом O, единичным направлением D и параметром `t≥0`: `P(t)=O+tD`. Если D нормирован, t измеряется в метрах; если нет, t просто параметр и его нельзя сравнивать с дальностью оружия без пересчёта. Возьмём O=`(0;1;0)`, D=`(0;0;1)`, плоскость стены `z=5`. У луча z-координата `0+t*1=t`, значит t=5 и точка попадания `(0;1;5)`. Для направления `(0;0;2)` тот же контакт имеет t=2.5, но расстояние всё ещё 5 м. Вот почему оружейный `ValidRay` и нормировка направления важны до проверки range.

Плоскость в общем виде задаётся точкой Q и единичной нормалью N: точка P лежит на ней, когда `dot(P-Q,N)=0`. Подставим луч: `dot(O+tD-Q,N)=0`, раскроем линейность, получим `t=dot(Q-O,N)/dot(D,N)`. Знаменатель около нуля означает, что луч почти параллелен плоскости. Если одновременно числитель около нуля, луч лежит в плоскости, и «единственная точка попадания» не определена. Если t отрицателен, пересечение находится позади начала луча. Для оружия дополнительно требуется `t≤range`. Для picking редактора t может ограничиваться ближней и дальней плоскостью камеры.

Сфера с центром C и радиусом r задаётся `|P-C|²=r²`. Подстановка `P=O+tD` даёт квадратное уравнение. При нормированном D обозначь `m=O-C`; тогда `t²+2dot(m,D)t+dot(m,m)-r²=0`. Дискриминант меньше нуля — нет пересечения; равен нулю — касание; положителен — два пересечения, вход и выход. Выбирай наименьший неотрицательный t. Если начало луча внутри сферы, первый корень отрицателен, а второй положителен: это точка выхода. В Bullet тебе не нужно повторять эту математику для каждого объекта, но она помогает понять, почему отладочный луч попал в ближний collider, а не в выбранную NPC цель.

9. Столкновение прямоугольников и разделяющая ось

Для AABB проверка интервалов по X, Y, Z достаточна. Для повернутых коробок оси этих объектов различаются, поэтому простое сравнение мировых min/max может давать ложное пересечение. Теорема разделяющей оси говорит: две выпуклые фигуры не пересекаются, если существует ось, на которой их проекции не пересекаются. Для двумерных прямоугольников кандидаты — нормали к сторонам обоих. Проекция вершины P на единичную ось n — `dot(P,n)`. Для каждого многоугольника найди минимум и максимум этой проекции. Если интервалы разошлись хотя бы на одной оси, столкновения нет. Если не разошлись на всех необходимых осях, фигуры пересекаются.

Пример на оси X: проекция первого прямоугольника `[0,2]`, второго `[3,5]`. Условие `2<3` сразу доказывает отсутствие контакта. На другой оси проверять не нужно. Если интервалы `[0,2]` и `[1,3]` перекрываются, это ещё не доказывает столкновение: по Y фигуры могут быть разделены. Для трёхмерных OBB нужны оси граней обоих объектов и оси попарных векторных произведений их базисных осей; это уже более дорогая проверка. Bullet выполняет специализированную collision работу, но понимание SAT полезно для editor gizmo, UI геометрии и предварительной диагностики collider.

Есть тонкий крайний случай: «касаются» или «пересекаются»? Для интервалов `[0,2]` и `[2,4]` общая точка 2. Если условие разделения строгое `<`, касание считается контактом; если `≤`, нет. В физике обычно нужен допуск и контактный зазор, поэтому символ сравнения — часть игрового правила. На

высоких скоростях дискретная проверка положения до и после шага может пропустить тонкую стену: объект окажется по одну сторону на t=0 и по другую на t=dt. Для такого случая используют swept collision/continuous detection или ограничивают шаг, а не просто увеличивают радиус объекта.

10. Управление дверью или камерой: ошибка, скорость и устойчивость

Пусть привод двери должен довести её до позиции 1 м, сейчас она в 0.6 м. Ошибка `e=target-current=0.4 м`. Простейшее пропорциональное управление задаёт скорость `v=Kp*e`. Чтобы результат был в м/с, `Kp` измеряется в 1/с. При `Kp=2/c`, v=0.8 м/с; за `dt=0.1c` дверь сместится на 0.08 м до 0.68 м. Следующая ошибка станет 0.32 м, новая скорость 0.64 м/с. Движение постепенно замедляется и асимптотически приближается к цели. Чтобы не ждать бесконечно, введи допуск, например 1 мм: внутри него установи конечную позицию и закончи переход.

Если применить `current += Kp*e*dt`, устойчивость простого дискретного правила зависит от `Kp*dt`. При значении 1 результат достигается за шаг, при значении 2 прыгает через цель на противоположную сторону, при ещё больших значениях может расходиться. Поэтому изменение dt без проверки может превратить плавную дверь в дрожащую. Для критичного движения используй фиксированный шаг, ограничение скорости или аналитическое сглаживание из главы 1. Для физического тела не записывай Transform поверх Bullet каждый кадр; применяй согласованные силы/импульсы или кинематический контроллер.

PID добавляет интегральный член для накопленной ошибки и дифференциальный член для реакции на скорость её изменения. Но он не является обязательной «улучшенной версией» для каждой двери. В игре простая интерполяция с известной длительностью часто понятнее и легче тестируется. PID полезнее там, где есть инерция и возмущения, например стабилизация корабля. Перед его добавлением измерь реальное поведение, выбери единицы коэффициентов, зажми интегратор при насыщении привода и проверь устойчивость при фиксированном dt. Три буквы без числового эксперимента только скрывают проблему.

Практический курс: от первого кадра до своей сцены

Цель курса — довести читателя от пустой тестовой сцены до законченного вертикального среза космического симулятора. В нём есть последовательная история: корабль получает повреждение, экипаж замечает аварию и распределяет работу, противник проникает на палубу, игрок принимает решение, а мир продолжает работать за пределами камеры. Этот сценарий станет проверкой каждой системы. Если новая функция не меняет наблюдаемое поведение, её рано считать готовой.

Курс опирается на текущий проект. Путь к классу означает место в исходниках, а не обещание, что все описанные улучшения уже реализованы. Я буду явно отделять действующий код от учебных заданий. Не пропускай расчёты: движок трудно развивать, когда число «0,2» у скорости, давления или полезности не имеет единиц и проверяемого смысла.

Урок 1. От запуска приложения до одного игрового кадра

1.1. Зачем нужен главный цикл

Игра отличается от обычной программы тем, что не завершает работу после одного ответа. Она постоянно принимает ввод, обновляет состояние и рисует результат. В этом проекте путь начинается в Win32, проходит через `SystemClass` и попадает в `ApplicationClass`. Последний является composition root: он знает о SceneManager, ECS, физике, рендере, вводе и игровых системах, а потому обязан задавать порядок их работы. Если система ремонта меняет пробоину после того, как атмосфера уже рассчитана, новый поток воздуха попадёт в расчёт только на следующем кадре. Это может быть допустимо, если задержка осознанная; случайный порядок создаёт трудноуловимые ошибки.

Разбей кадр на четыре вопроса. «Что произошло?» — сообщения окна, нажатия, завершившиеся фоновые задачи. «Что должно произойти в мире?» — команды игрока, решения NPC, двери, питание, атмосфера, физика, урон. «Что теперь видит камера?» — обновлённые Transform, анимация, отбор видимых объектов. «Как это показать?» — render passes, интерфейс и Present. Такую схему удобно держать рядом с `ApplicationClass::Frame`, потому что один большой метод легко воспринимать как случайный список вызовов.

В действующем Play цикле weapon timers обновляются до попыток стрельбы. Затем работают `NpcUtilitySystem`, `CrewTaskCoordinator`, `NpcCombatSystem`, `NpcCoverSystem`, `StateSystem`, поворот персонажа, Bullet controller, анимация и урон. Позже, когда поза известна, `NpcRangedCombatSystem` определяет дуло и вызывает общий `WeaponSystem`. Это не формальная диаграмма: поменяй местами animation и muzzle sampling — пуля выйдет из положения прошлой позы. Поменяй местами решение и движение — персонаж начнёт выполнять приказ на кадр позже. Иногда задержка в один кадр нормальна, но она должна быть названа и проверена.

1.2. Три вида времени

`real delta time` показывает, сколько секунд прошло по часам. `simulation delta time` может быть равен нулю во время паузы или масштабироваться при ускорении мира. `fixed step` дробит накопленное время на равные шаги для физики. Например, кадры пришли через 0,010, 0,020 и 0,020 с. Всего прошло 0,050 с. При шаге 1/60 с физика должна выполнить три шага по ≈0,01667 с, оставив в аккумуляторе небольшой остаток. Если передать Bullet все 0,050 с одним непредсказуемым шагом, контакты и контроллер сильнее зависят от FPS.

Не умножай скорость дважды на dt. Если `movement.desiredVelocity` измеряется в м/с, контроллер сам применяет временной шаг. Запись `desiredVelocity = direction * speed * dt`, после которой контроллер ещё раз умножает на dt, даёт смещение пропорционально dt^2. При удвоении FPS скорость заметно изменится. Простой диагностический опыт: пройди одним NPC одну и ту же прямую при 30 и 120 FPS за 10 секунд реального времени. Разница в положении сверх ожидаемой физической погрешности указывает на ошибку времени или порогов.

1.3. Editor и Play являются разными режимами, но одной сценой

Руководство редактора прямо предупреждает: F5 переключает режим без автоматического восстановления снимка ECS. Если в Play подобрать оружие, сломать дверь и затем вернуться в Editor, runtime изменения могут остаться в Registry. Это не мелкая особенность интерфейса, а правило работы с контентом. Перед опытами сохрани сцену. После опыта загрузи авторскую версию заново, если не хочешь сохранить последствия. В будущей production версии полезно добавить Play snapshot/restore, но до его реализации нельзя описывать F5 как безопасный откат.

Практика. Открой `src/Core/SystemClass.cpp` и `src/Core/ApplicationClass.cpp`. Найди, где вычисляется dt, где проверяется Play, где запускаются NPC и где начинается рендер. Нарисуй на бумаге стрелки от одного результата к следующему. Затем поставь в сцену дверь и NPC и запиши, в каком моменте кадра дверь успевает открыться относительно маршрута NPC. Результат урока — не запомненный список функций, а объяснение, почему каждый потребитель получает уже подготовленные данные.

Урок 2. ECS как модель игрового мира

2.1. Entity не является объектом C++

В `src/ECS/ECSTypes.h` Entity — `uint32_t`, а `INVALID_ENTITY` — максимальное значение этого типа. Идентификатор сам по себе не хранит имя, Transform или здоровье. `ECSRegistry` содержит множество живых ID и отдельные pool по типам компонентов. Компонент — данные. Система — проход по подходящим компонентам и изменение мира. Например, пробоина существует как Entity с HullBreach и Repairable, а `RepairSystem` решает, что значит увеличить её прогресс и завершить ремонт.

Эта модель позволяет сочетать признаки без дерева наследования. Генератор может иметь Transform, Damageable, Power, Interaction и WorkZone. Его не нужно объявлять `class RepairableInteractivePoweredGenerator : ...`. Но композиция требует дисциплины: система должна явно описывать минимальный набор компонентов и поведение при их отсутствии. Если NPC имеет модель, но нет `NpcBrainComponent`, дальний бой не знает, можно ли ему стрелять. Если у стрелка нет FactionComponent, нынешний код возвращает статус `Shooter has no faction`.

2.2. Как работает ForEach

В текущем `ECSRegistry::ForEach<T1,T2>` система перебирает первый pool и проверяет наличие сущности во втором. Поэтому порядок типов влияет на стоимость: если активных NPC десятки, а Transform тысячи, выгоднее начинать с NPC pool. Это не повод менять сигнатуру без измерения, но полезно понимать при профилировании. `GetComponent<T>` возвращает указатель или `nullptr`; проверка обязательна, когда компонент опционален. `HasComponent` отвечает только на наличие, а не на то, что Entity жива и активна. Для цели боя или ремонта проверяй `IsAlive`, `ActiveComponent` и предметные условия.

У текущего Registry есть важное ограничение: ID увеличивается монотонно, generation index в Entity пока нет. Поэтому описание в старых учебных разделах о generation handle относится к предлагаемому усилению, а не к уже действующему API. После уничтожения Entity все её компоненты удаляются, но сохранённый в другом компоненте ID может указывать на несуществующую цель. Перед использованием такого ID нужен `IsAlive`. При загрузке сцены новый ID может отличаться от сохранённого, поэтому SceneManager выполняет remap ссылок.

2.3. Структурные изменения во время обхода

`ForEach` и `GetComponents` перебирают контейнеры pool. Добавление или удаление компонентов того же типа во время обхода может инвалидировать итератор. Даже если один конкретный запуск «случайно работает», это не контракт. Собери команды изменения в отдельный vector и примени после прохода либо используй отложенную очередь. `DamageSystem` и `RepairSystem` особенно чувствительны к этому, потому что создание пробоины или завершение ремонта может менять

набор сущностей. Запомни правило: после функции, которая может уничтожить цель, не держи ссылку на её старый компонент и получи её заново, если она ещё существует.

Пример модели данных для будущего пожара:

```
struct FireComponent {
    float intensity = 0.0f;       // 0..1, runtime
    float spreadRatePerSecond = 0.1f;
    Entity room = INVALID_ENTITY;
    bool enabled = true;
};
```

Поля не должны содержать `ID3D11ShaderResourceView*` или `btRigidBody*`, если их задача — описывать пожар. Эти ресурсы принадлежат другим слоям. Компонент хранит то, что необходимо для правила мира; рендер получает снимок интенсивности и показывает огонь.

2.4. Сохранение компонента — отдельный контракт

Добавив C++ структуру, ты ещё не научил сцену её сохранять. Найди в `SceneManager` путь `SerializeObject` и соответствующее чтение. Раздели authoring данные (например, скорость распространения) и runtime данные (текущая интенсивность). Scene save может хранить первое, save game — оба. Сохраняй версию формата и безопасные значения по умолчанию: старые сцены не должны ломаться после добавления поля. Ссылки на другие Entity сериализуй через устойчивые ID файла и remap, а не через адрес памяти. После загрузки проверь, что дверь по-прежнему соединяет те же комнаты, а рабочая зона — нужное оборудование.

Практика. Создай в тестовой ветке один компонент данных, например `TemperatureComponent`, и пройди весь путь: создание через редактор, наличие в Registry, один системный update, отображение в Inspector, scene round trip. Сначала продемонстрируй один объект, затем два независимых. Если второму объекту достаётся состояние первого, вероятно, ты спрятал mutable состояние в static переменную или singleton.

Урок 3. Сцена, редактор и первый игровой уровень

3.1. Уровень начинается с проверяемого сценария

Построй палубу из двух комнат, соединённых одной дверью. В первой находятся игрок и генератор, во второй — пробоина и инженер. Эта простая геометрия заставляет проверить сразу четыре связи: принадлежность объектов комнатам, путь через дверь, питание двери и выбор ремонтной задачи. Если сразу строить большой корабль, ошибка в одной связи затеряется среди десятков помещений. Сначала создай сцену, где каждое изменение видно.

В редакторе используй Scene Hierarchy для выбора сущности, Inspector для данных компонентов и gizmo для Transform. Quick Create удобен как начальный шаблон, но проверяй состав созданной сущности: `Crew NPC (logic)` может быть полноценной логикой без подходящей модели, клипов и физического контура. Визуальная модель не делает NPC способным ремонтировать, а наличие NPCComponent не создаёт автоматически правильно настроенный Bullet controller.

Сохрани авторскую сцену до Play. После входа в Play открой дверь, задай ручную команду, нанеси урон корпусу и выйди. Повтори загрузку сохранённой сцены: авторское состояние должно вернуться. Для игрового save нужен другой вопрос: какие изменения мира должны сохраниться как прогресс игрока? Не смешивай scene authoring и save game только потому, что оба записываются в JSON.

3.2. Набор компонентов для учебного сценария

Для комнаты нужны геометрия, принадлежность и данные среды. Для двери — геометрия, состояние, связь комнат, при необходимости питание и collider. Для пробоины — `HullBreachComponent` и `RepairableComponent`; пробоина может появиться от DamageSystem, поэтому ручной шаблон должен имитировать те же обязательные данные. Для инженера — NPC, Brain, Movement, CrewMember,

Transform, Active, character controller и анимация/модель, если нужна видимая фигура. Для противника дополнительно нужны `FactionComponent`, `NPCCombatComponent`, оружие или melee настройки. Это минимальные контракты, а конкретные фабрики в проекте могут добавлять ещё компоненты.

При настройке объекта задавай смысл числа. `repairDistance = 1,5` — метры от центра NPC до рабочей точки, а не «примерная близость». Размер capsule и расположение WorkZone должны допускать стоять в этой зоне без входа в collider оборудования. `detectionRange` — геометрический фильтр выбора задач, а не доказательство достижимости по NavMesh. `maxPressure` — предел материала корпуса в единицах системы давления. Если Inspector показывает число без единиц, запиши единицы в описание или имя поля.

3.3. Триггеры и игровой поток без нового C++

`TriggerZoneSystem` обнаруживает вход в область, `TriggerActionSystem` переводит событие в действие. Для первой игровой цели можно создать зону возле капитанского терминала: вход показывает сообщение «Восстановите питание двери», а включение генератора переводит сценарий к аварии во втором отсеке. Триггер не должен напрямую менять десяток несвязанных компонентов в обход систем. Он выдаёт событие или команду, а соответствующая gameplay система меняет своё состояние. Такой подход позволяет повторить сценарий после загрузки и понять, кто совершил переход.

Проверка первого уровня: новая игра запускается в правильной комнате; игрок может получить подсказку; дверь закрыта по понятной причине; генератор включает нужную цепь; инженер получает одну аварийную работу; при повторном входе в зону событие не дублируется бесконечно; после сохранения и загрузки текущий этап явно восстанавливается или по правилам начинается заново. Это уже кусочек игры, а не только демонстрация движка.

Урок 4. Математика как инструмент, а не список формул

4.1. Один численный пример проходит через весь движок

Пусть NPC находится в A = (2, 0, 3), а пробоина в B = (8, 0, 11). Разность B − A = (6, 0, 8) — направление и расстояние до цели в ненормированном виде. Горизонтальная длина равна $\sqrt{(6^2+8^2)}=10$ м. При рабочем радиусе 1,5 м NPC должен пройти приблизительно 8,5 м, если между ним и целью нет препятствий. Нормированный вектор (0,6, 0, 0,8) говорит лишь направление; он не содержит скорость. При скорости 2 м/с и dt=0,1 с желаемое смещение (0,12, 0, 0,16) м. Если трасса NavMesh обходит стену и имеет длину 15 м, время без остановок ближе к (15−1,5)/2=6,75 с, а не к 4,25 с по прямой.

Ту же пару точек можно использовать для FOV. Пусть NPC смотрит в +Z: forward=(0,0,1). Dot с нормированным направлением до пробоины равен 0,8; угол arccos(0,8)≈36,87°. При полном конусе зрения 90° половинный угол 45°, точка внутри. Но для пробоины зрение может вообще не быть условием назначения: экипаж получает данные аварии от корабельной системы. Здесь различаются два типа знания. Геометрический dot проверяет направление взгляда, а игровое правило решает, должен ли конкретный NPC знать о событии.

Пусть после перемещения модель находится в world space, камера смотрит на неё, а shader получает вершину p. В текущем PBR vertex shader путь — `p_world = mul(p, W)`, затем `mul(p_world, V)`, затем `mul(p_view, P)`. Если вместо этого перемножить P и W в обратном порядке, перенос объекта повлияет на проекцию неверно. На CPU `PBRShaderClass.cpp` транспонирует матрицы перед загрузкой в constant buffer. Это не «исправление порядка»; это согласование хранения с HLSL. Проверяй геометрический результат на простом объекте: точка у origin, перенос +5 по X, затем наблюдение камерой.

4.2. Почему численные инварианты сильнее картинки

Картинка может выглядеть правдоподобно при неправильных единицах: корабль тонет слишком быстро, но игрок не знает нормальную скорость. Инвариант можно проверить. Скорость умножается на секунды и даёт метры. Мощность в ваттах умножается на секунды и даёт джоули. Поток воды в м³/с умножается на секунды и даёт м³. При перетоке воздуха между двумя закрытыми комнатами сумма массы остаётся постоянной. При повороте единичного направления чистым вращением длина остаётся 1. При преобразовании нормали и касательной после неравномерного масштаба dot остаётся близким к 0, если применена правильная normal matrix.

Задание. Измени скорость персонажа и частоту кадров, затем вручную предскажи расстояние за 5 секунд. Отдельно вычисли, какой score координатор даст инженеру и медику для одной пробоины, используя коэффициенты из `CrewTaskCoordinator`. Затем сравни с debug полями. Если формула и наблюдение расходятся, не подгоняй коэффициент сразу: проверь clamp, расстояние, текущую роль, уже занятый claim и порядок вызовов.

Урок 5. Bullet Physics от формы до игрового события

5.1. Физический мир и игровой мир отвечают на разные вопросы

Bullet знает тела, collision shapes, контакты, ограничения, массы, скорости и лучи. ECS знает, что конкретное тело является дверью, персонажем, ящиком или частью корпуса. Один слой не заменяет другой. Когда луч пересекает `btCollisionObject`, это геометрический факт. Чтобы решить, можно ли нанести урон, нужно сопоставить объект с Entity, проверить фракцию, здоровье, броню и тип поверхности. Поэтому физика не должна напрямую открывать дверь или выбирать ремонтную задачу.

В проекте `PhysicsWorld` создаёт и обслуживает Bullet world, а `RigidBodyComponent`, character controller и ECS системы связывают тела с сущностями. При чтении класса начни с жизненного цикла: кто создаёт shape, кто добавляет body в world, кто удаляет constraints и actions, кто освобождает память при смене сцены. Ошибка порядка уничтожения может не проявиться в первом запуске, но даст падение после второй загрузки. Комментарий в `ApplicationClass` указывает важный контракт: ECS владеет Bullet actions, constraints, bodies и shapes; их надо освободить до разрушения PhysicsWorld.

5.2. Выбор тела: static, dynamic, kinematic

Стена палубы — static: она не интегрирует скорость, но участвует в контактах и raycast. Ящик — dynamic: масса больше нуля, столкновения и импульсы меняют скорость. Дверь, которую двигает анимация или логика открытия, может быть kinematic: положение задаёт система, Bullet использует его как препятствие. Для каждой Entity запиши владельца Transform. Если это dynamic, после физического шага мир Bullet является источником положения для рендера. Если kinematic, gameplay задаёт физике новую позу. Если static, положение меняется редко, и физическое представление нужно обновить вместе с Transform.

Типичная ошибка — перед каждой симуляцией ставить dynamic body в координаты Transform, а после шага читать позицию обратно. Получается борьба двух источников: тело отскакивает от стены, но Transform снова переносит его внутрь. Второй симптом — дрожащая модель, когда character controller и код AI одновременно пишут position. Исправление начинается не с коэффициента сглаживания, а с выбора владельца и направления синхронизации.

5.3. Collision shape не равен видимой модели

Для стен часто достаточно коробок или статической вогнутой сетки; для персонажа нужна капсула; для подвижного ящика — box или convex hull. Точная треугольная сетка для каждого динамического предмета повышает стоимость контактов и может вести себя хуже, чем простая форма. При этом слишком грубая форма создаёт игровые противоречия. Если в двери нарисовано отверстие, а collision shape остался сплошным, NPC не пройдёт и луч попадёт в невидимую стену.

Если после разрезания корпуса визуальная пробоина появилась, а физическая форма не обновилась, игрок увидит проход, который физика не признаёт.

Отладка начинается с debug draw Bullet shapes одновременно с render mesh. Проверяй положение, масштаб, ориентацию, высоту капсулы, толщину порога и маску столкновения. Особенно опасен non-uniform scale: некоторые формы и алгоритмы не должны получать произвольное масштабирование после создания. Лучше пересоздать shape или использовать явно поддерживаемую операцию. Не делай вывод «NavMesh сломан», пока не увидел реальные физические препятствия.

5.4. Шаг симуляции и импульс

При fixed step 1/60 с контроллер получает одинаковые интервалы даже при неравномерном рендере. Накопитель времени не должен бесконечно догонять после паузы в отладчике: лимит подшагов предотвращает spiral of death. При ограничении нужно знать политику: потерянное время отбрасывается или симуляция временно отстаёт? Это влияет на таймеры повреждений и NPC, поэтому physics time и gameplay time должны быть согласованы.

Импульс меняет количество движения. Для тела массой 10 кг импульс 30 Н·с по X даёт $\Delta v=3$ м/с. Сила 30 Н за один шаг 1/60 с даёт $J\approx0{,}5$ Н·с и $\Delta v\approx0{,}05$ м/с. Если взрыв кажется слабым после замены impulse на force, причина может быть именно в единицах. Приложенный не по центру масс импульс создаёт ещё и момент вращения. Для взрыва вычисли радиальный вектор, обработай нулевую длину, ограничь радиус и не создавай бесконечную силу при расстоянии 0.

5.5. Raycast в трёх разных механиках

Для выбора объекта редактором луч идёт от камеры через курсор. Для зрения NPC он идёт от точки глаза к допустимым точкам цели. Для оружия он начинается у дула после анимации. Все три луча используют геометрию, но правила после попадания разные. Редактору нужна Entity под курсором, NPC — доказательство видимости, оружию — первое препятствие и применимость урона. Если один общий helper сразу возвращает «враг виден» для любого луча, границы ответственности потеряны.

Пример. Глаз NPC находится над низким укрытием, дуло ниже его верхнего края. Луч глаза не встречает стену, а луч дула встречает. Правильное поведение — NPC видит игрока, но не стреляет до выхода в точку выглядывания. В текущем коде зрение проверяет `NpcCombatSystem`, а выстрел и линию огня — `NpcRangedCombatSystem`. Для разбора ошибки нарисуй оба луча разными цветами и подпиши hit Entity и расстояние.

Практика. Поставь дверь между NPC и игроком. Проверь закрытую, открытую и сломанную дверь: видимость, путь и физический контакт должны совпадать с намерением дизайна. Повтори с пробоиной корпуса и ремонтной операцией. Зафиксируй, какая система обновляет collision после изменения геометрии и на каком кадре результат становится доступен raycast.

Урок 6. Рендер DirectX 11 и смысл каждого шейдера

6.1. CPU готовит команду, GPU выполняет её

Renderer не решает, сколько у персонажа здоровья и какую аварию выбрать. Он читает состояние сцены и строит команды рисования. CPU выбирает mesh, material, shader, transform, ресурсы и состояния. GPU читает vertex/index buffers и выполняет стадии графического конвейера. `Renderer::RenderFrame` — точка входа кадра в рендер. Он готовит тени, при необходимости reflection/refraction, затем основную сцену, прозрачность и интерфейс. Наличие условных off-screen passes означает, что изменение материала воды может добавить целый дополнительный проход, а не только несколько операций pixel shader.

Чтобы понять стоимость кадра, раздели её на количество draw calls, количество геометрии, стоимость pixel shader и число проходов. Увеличение размера экрана почти не меняет число вершин, но может резко повысить стоимость пиксельного освещения и прозрачности. Добавление

shadow pass повторно рисует геометрию с точки света. Сначала измерь, где узкое место: CPU подготовка draw list или GPU пиксели. «Оптимизировать рендер» без такого разделения слишком расплывчато.

6.2. Что делает вершинный шейдер

Открой `src/Render/Shader/Vs/pbr.vs`. Vertex input содержит позицию, UV, normal, tangent, binormal и до четырёх весов костей. Для skinned модели сначала применяется bone transform каждого валидного влияния, результаты взвешиваются и нормируются по сумме весов. Статичный mesh не должен использовать оставшуюся в b8 матрицу кости от предыдущего draw: текущий код специально оставляет исходную позицию, когда валидных влияний нет. После этого вершина переводится через world, view и projection. Выход `SV_POSITION` задаёт положение для растеризации; worldPos, UV и базис TBN передаются пиксельному шейдеру.

Почему vertex shader не вычисляет весь PBR цвет? Свет может меняться внутри одного большого треугольника. Если осветить только три вершины и интерполировать готовый цвет, блик внутри треугольника может исчезнуть. Передача worldPos и нормали позволяет PS вычислять освещение для фрагмента. Но чем больше интерполяторов, тем выше стоимость и риск превышения лимитов. В проекте векторы до четырёх источников передаются через явные TEXCOORD слоты; при изменении структуры нужно синхронно менять VS output и PS input.

6.3. Что делает растеризатор и пиксельный шейдер

Растеризатор определяет, какие пиксели покрывает треугольник, и интерполирует данные вершин. Pixel shader в `src/Render/Shader/Ps/pbr.ps` получает UV, normal, tangent, worldPos и параметры освещения. Он читает albedo, normal, metalness, roughness, shadow map, environment и маску повреждений. Результат проходит тест глубины и, возможно, blending. Поэтому прозрачность — не просто `alpha < 1`: порядок draw, depth write и blend state определяют правильность картинки. Стекло, нарисованное раньше непрозрачной стены и записавшее depth, может скрыть стену позади.

PBR требует согласованного цветового пространства. Albedo, записанный в sRGB, надо декодировать в линейное пространство до вычисления света. Значение 0,5 sRGB соответствует примерно 0,214 linear. Normal map — векторные данные, их не следует декодировать как цвет. После линейного освещения HDR результат проходит exposure, tone mapping и кодирование для вывода. Если одновременно применять gamma в шейдере и использовать sRGB render target, сцена окажется слишком светлой или тёмной. Для каждой текстуры и render target веди явную таблицу «что хранит и в каком пространстве».

6.4. Constant buffer — контракт байтов

В HLSL `register(b0)` относится к конкретной стадии. В текущем PBR VS b0 — MatrixBuffer, а в PS b0 — LightColorBuffer. Это не конфликт: VS и PS привязывают разные наборы буферов. C++ структура должна совпадать с HLSL по полям и упаковке; размер D3D11 constant buffer кратен 16 байтам. При добавлении нового параметра проверь HLSL declaration, C++ struct, заполнение перед draw и правильный bind slot. Иначе эффект может читать старые байты предыдущего объекта, а «исправление» цвета константой лишь замаскирует ошибку.

Матрицы в `PBRShaderClass.cpp` транспонируются перед загрузкой. Не делай двойное транспонирование ради визуально похожего результата в одной сцене. Проверь model point (1,0,0) с переносом +5 по X и поворотом 90° на тестовой модели, а потом проверь skinned mesh и normal. Такой тест разграничивает ошибку матрицы, ошибки в весах костей и ошибку исходной оси модели.

6.5. Добавление шейдерного эффекта через gameplay

Допустим, горячий реактор должен светиться красным. Сначала gameplay считает температуру `T` и предел `Tmax`; render snapshot получает `heat01 = clamp((T-Tstart)/(Tmax-Tstart), 0, 1)`. PS использует heat01 для emissive цвета. Привязка не должна менять температуру: shader не возвращает gameplay результат. Добавь параметр в C++ и HLSL, выставляй его для каждого draw, предусмотрев 0 для

объектов без HeatComponent. Если параметр остаётся от предыдущего материала, новый объект неожиданно светится. После изменения проверь основной проход, тени, reflection и прозрачность, потому что разные passes могут использовать разные shaders и состояние контекста.

Практика. Возьми один куб с PBR материалом. Проверь albedo без света, один источник, roughness 0,1 и 0,9, normal map, shadow, emissive и non-uniform scale. Для каждого опыта ожидаемый результат сформулируй заранее. Если свет меняется при перемещении камеры как будто приклеен к ней, worldPos или light direction оказались в разных пространствах.

Урок 7. Корабль как связанная система, а не набор HUD чисел

7.1. Причинная цепь аварии

Космический симулятор становится игрой, когда событие имеет последствия. В текущем коде попадание обрабатывает `DamageSystem`: оно может повредить корпус или создать пробоину, а водный `FloodingSystem` рассчитывает приток в отсек. Повреждённый генератор меняет доступную мощность, и питаемая дверь может изменить путь экипажа. Отток воздуха в вакуум и связь питания насоса с `ShipPowerGridSystem` ещё требуют отдельной реализации; это важные задачи развития, а не готовая цепочка. Игрок должен видеть причину аварии через HUD и принимать решение по состоянию мира, а не по одному визуальному эффекту.

Удобно составить таблицу для каждого ресурса: владелец, единицы, источник, потребитель, период обновления и правило сохранения. Вода: м³ в отсеке, м³/с через отверстие или насос. Энергия: текущая мощность МВт, запас аккумулятора — энергия, а не МВт. Воздух: масса, давление, температура и объём. Повреждение: health и максимальное значение. Смешивание единиц заставляет «балансировать» систему случайными коэффициентами. Сначала добейся правильной размерности, потом выбирай игровые масштабы.

7.2. Давление и затопление на числах

`PressureSystem.cpp` рассчитывает внешнее давление как атмосферное плюс `rho·g·depth`. При rho=1000 кг/м³, g=9,81 м/с² и depth=10 м добавка равна 98 100 Па. Это приблизительно ещё одна атмосфера к атмосферному давлению поверхности. Если maxPressure корпуса ниже этого значения, код начисляет урон пропорционально превышению. Для космического корабля водная модель может быть использована только в специфической сцене; не переноси её автоматически на вакуум. Для разгерметизации в космосе нужен поток газа и разность внутреннего/внешнего давления, а не гидростатическое давление воды.

`FloodingSystem.cpp` суммирует приток пробоин и отток насоса, затем меняет объём на `(inflow-outflow)·dt`. При притоке 0,3 м³/с и насосе 0,1 м³/с за 5 с добавится 1 м³ воды. Работа насоса зависит от `CompartmentComponent::hasPower`, но это поле сейчас не синхронизируется автоматически с `PowerConsumerComponent::hasPower`. Прежде чем связывать насос с энергосетью, определи владельца поля и поставь обновление питания до затопления в нужном тике. Если отток больше текущего объёма, clamp предотвращает отрицательную воду. Если пробоину запечатали, поток должен стать 0, а не продолжать использовать прежний cache.

7.3. Сеть питания и brownout

`ShipPowerGridSystem.h` собирает источники и потребителей, обнуляет результат прошлого кадра, подключает аварийный аккумулятор и распределяет доступную мощность по приоритетам. `PowerConsumerComponent` хранит `requiredMw`, `suppliedMw`, `powerRatio` и `hasPower`. Сейчас `hasPower` означает почти полное обеспечение (`powerRatio >= 0,999`), поэтому устройству с 70% мощности не обязательно разрешено работать. Если будущий насос должен иметь плавную производительность, вычисляй `effectivePumpRate = nominalRate · powerRatio` в его системе и явно определи нижний порог. Не меняй смысл hasPower тайно для одного устройства.

Пример: генератор даёт 5 МВт, жизненно важные потребители просят 3 МВт, остальные 4 МВт. Первые получают 3, остаток 2 делится согласно текущей политике между потребителями следующего приоритета. Это не означает, что каждый из четырёх одинаково нуждающихся объектов получит полный запрос. После включения аккумулятора available может временно вырасти. В HUD показывай generation, demand, battery output, supplied и причину отказа конкретного устройства. Число «энергия 50%» без объяснения недостаточно для решения игрока.

7.4. Связь симуляции и игрового решения

Создай ситуацию: во втором отсеке падает давление, генератор повреждён, дверь требует питание. Инженер получает ремонтную задачу, но не может пройти. Что должно произойти? В текущем коде путь может стать Blocked. Для полной игры нужны правила реакции: запросить другую доступную аварию, идти к ручному переключателю, дождаться восстановления питания или сообщить игроку. Не добавляй все варианты сразу. Сначала обеспечь наблюдаемость: «задача назначена», «путь через дверь N», «дверь без питания», «назначение освобождено». После этого добавляй новую работу.

Практика. В тестовой сцене отключи генератор. Запиши, какие потребители потеряли питание и почему. Добавь одну пробоину и проверь, что скорость утечки, HUD и ремонт связаны. Повтори после загрузки сцены и при другой частоте кадров. Нарушенная размерность, неправильный порядок системы или несохранённая связь Entity должны давать разные диагностические признаки.

Урок 8. Дружественный NPC от аварии до завершённого ремонта

8.1. Раздели обнаружение, назначение, путь и работу

Слова «NPC чинит пробоину» скрывают четыре разных алгоритма. Пробоина сначала должна существовать в ECS и быть незапечатанной. Координатор затем должен назначить работу подходящему члену экипажа. Исполнитель должен привести персонажа к достижимой рабочей точке. Наконец, RepairSystem должен накопить прогресс и изменить мир. Если объединить все четыре операции в `NPCComponent::Update`, сложно будет поддержать ручной приказ, несколько работников, закрытые двери и другой тип задачи.

В текущем коде `DamageSystem` может породить Entity пробоины с `HullBreachComponent` и `RepairableComponent`. `NpcUtilitySystem` вычисляет полезность ремонта. `CrewTaskCoordinator` выдаёт claim. `StateSystem::UpdateRepairTask` занимается перемещением и взаимодействием. `RepairSystem::ApplyRepairProgress` и `CompleteRepair` меняют состояние цели. Такая цепочка важна ещё и для отладки: каждый слой может ответить, почему остановился. «Пробоины нет» отличается от «никто не назначен», «дверь недоступна» и «асинхронное восстановление меша ещё не завершилось».

8.2. Выбор работника на численном примере

Координатор рассматривает доступные цели с Repairable. Для пробоины серьёзность берётся из `leakRate/0,15`, ограниченного интервалом 0,2–1; для оборудования — из доли потерянного здоровья. Близость равна `1−distance/detectionRange` с ограничением 0–1. Дальше используется смесь `0,65·severity + 0,35·distanceScore`, коэффициент роли и `repairSkill`, а итог ограничивается 0–2. Инженер имеет больший коэффициент, чем медик. Это не магический «AI»: каждый множитель имеет игровое объяснение.

Пусть пробоина имеет leakRate=0,12, а инженер находится на половине своего detectionRange. Серьёзность 0,8, близость 0,5. База 0,8·0,65 + 0,5·0,35 = 0,695. Инженер с roleAffinity=1,25 и skill=1,5 получает около 1,303. Медик с 0,62 и skill=1 получает около 0,431. Если score инженера неожиданно нулевой, проверь предел detectionRange, отсутствие Transform, активность NPC и то, не занята ли задача. Если score высок, но назначение не произошло, проверь активную ручную команду и claim другого работника.

У текущего алгоритма геометрическая близость не тождественна длине маршрута. В двух комнатах через запертую дверь цель может быть в 2 м по прямой и недостижимой физически. Для надёжного экипажа добавь проверку достижимости до окончательного назначения либо быстро освобождай claim после понятного отказа `Blocked`. Иначе один NPC будет навсегда владеть задачей, которую не способен выполнить, а остальные её не возьмут.

8.3. Почему claim находится на цели

`CrewTaskClaimComponent` висит на ремонтируемой Entity и хранит assignee, тип, score и время назначения. Это делает занятость видимой для любой системы и Inspector. `CrewMemberComponent` одновременно хранит assignedTask — удобную обратную ссылку. Две записи требуют согласованного обновления: если одна очищена, а другая нет, мир думает, что задача и занята, и свободна. Координатор начинает проход с очистки недействительных claims, затем сохраняет существующее назначение, если оно ещё корректно, и только после этого ищет новое. Сохранение назначения предотвращает переключение цели от малых колебаний score.

При ручной команде `NpcCommandSystem` меняет приказ и сбрасывает маршрут; в следующем проходе `CrewTaskCoordinator::Update` освобождает автоматическое назначение, а `StateSystem` отдаёт ручной команде приоритет. Когда команда завершается или вызывается `ResumeAutomatic`, маршрут старой команды сбрасывается. Это нужно, даже если конечная точка кажется похожей: старый путь мог идти через другую дверь. При смерти NPC, удалении цели, смене сцены и завершении ремонта claim должен исчезнуть. Для save game обычно сохраняют мировое состояние задачи, но runtime claim безопаснее восстановить заново из живых NPC и доступных задач.

8.4. Исполнение маршрута и рабочая зона

`StateSystem::UpdateRepairTask` перечитывает цель каждый кадр. Для HullBreach берёт `worldPosition` и room. Для оборудования — Transform, RoomMembership и при наличии `WorkZoneComponent` подходящую точку работы. Почему нельзя просто идти к центру модели? У генератора origin может быть в центре большого корпуса, куда капсула физически не войдёт. WorkZone задаёт свободное место перед панелью. Её нужно проверить на достижимость NavMesh и расстояние до самого объекта.

До прибытия NPC находится в `MoveToRepair`. Навигация может переводить его в `WaitingForDoor` и затем обратно, а при невозможности продолжить — в `Blocked`. `StateSystem` задаёт CharacterMovement, но не телепортирует Transform. `CharacterControllerSystem` применяет движение к Bullet capsule. Анимация читает итоговые параметры движения и выбирает Idle/Walk/Run. При достижении горизонтального радиуса `repairDistance` NPC останавливается. Порог должен учитывать капсулу, WorkZone и положение пробоины; слишком маленький порог заставит его вечно толкаться в стену.

8.5. Прогресс не зависит от FPS

В NPC ветке текущего `StateSystem` прирост вычисляется из `requiredProgress`, skill, dt и repairDuration. При requiredProgress=1, duration=8 с, skill=1,5 за dt=0,1 с прирост 0,01875. За 50 таких шагов получится 0,9375; после примерно 54 шагов ремонт завершится. Если за один кадр 0,2 с прогресс возрастёт вдвое относительно 0,1 с, это ожидаемо. Если время ремонта в 120 FPS вдвое меньше, где-то прирост применяется дважды или dt перепутан.

`RepairSystem::ApplyRepairProgress` — общая точка изменения цели. Когда прогресс достигает порога, завершение может удалить Entity пробоины или ждать async восстановления геометрии. Поэтому после вызова запрещено обращаться к ранее полученной ссылке на Repairable как будто она гарантированно жива. Повторно запроси компонент через Registry, если нужна дальнейшая проверка. Сейчас `StateSystem::UpdateRepairTask` передаёт `jobs`, но не передаёт `WorldEventLog`, время и источник ремонта: NPC ремонт не записывает событие завершения через этот вызов. Для журнала, HUD и звука проведи эти аргументы по цепочке; тогда они смогут реагировать на событие, а не угадывать завершение по исчезновению mesh.

8.6. Отладочная таблица причин

Если NPC стоит на месте, проверь по порядку: `ActiveComponent`, `CrewMember.onDuty`, `NPCComponent::isHostile` и Faction/Combat, активную ручную команду, доступные Repairable цели, score, claim, repairTarget, состояние NavMesh, room, дверь, WorkZone, distance, `progress`, async job. Порядок важен. Начинать с настройки анимации бессмысленно, если claim вообще не создан. Если State=Repairing, но прогресс не меняется, смотри dt, skill, requiredProgress и вызов ApplyRepairProgress. Если прогресс дошёл до порога, но пробоина видна, смотри завершение runtime mesh operation и состояние collider.

Практика. Создай три варианта одной сцены: открытая дверь, закрытая дверь с питанием, закрытая дверь без питания. Укажи ожидаемый State и status на каждом этапе. Затем вручную прикажи инженеру чинить другое оборудование и вызови ResumeAutomatic. Проверка считается успешной, когда NPC меняет работу без старого маршрута, прежний claim освобождается, а пробоины не получают второго исполнителя без явного правила совместной работы.

Урок 9. Вражеский NPC от зрения и поиска до попадания

9.1. Цель должна быть допустимой

Прежде чем считать расстояние, определи отношения фракций. `FactionComponent` хранит сторону; `AreEnemies` сейчас считает разные ненейтральные стороны враждебными, одинаковые дружественными, Neutral не атакует и не становится целью по этому правилу. Старый `isHostile` ещё встречается для совместимости сцен и разграничения crew utility, но новый боевой NPC должен иметь FactionComponent. Боевая система должна проверять живую, активную и допустимую цель. Иначе мёртвый объект останется целью или NPC начнёт стрелять в союзника, потому что тот ближе.

Для каждой попытки выбора цели записывай причины: не враждебна, слишком далеко, вне FOV, закрыта, неактивна, мертва. Один флаг `targetVisible=false` не объясняет, почему враг не реагирует. Если несколько врагов, порядок обхода контейнера не должен неявно определять цель при одинаковом score: используй явное правило, например расстояние, угрозу и устойчивую развязку по Entity ID.

9.2. Геометрия обзора и реальный raycast

Видимость состоит из быстрого фильтра и дорогой проверки. Сначала расстояние: квадрат длины между NPC и целью сравнивается с квадратом дальности. Затем горизонтальный FOV: dot единичного forward и единичного направления к цели сравнивается с cos половины полного угла. Для FOV 90° порог около 0,707. Цель под 60° к forward даёт dot=0,5 и не проходит. Дешёвые фильтры сокращают число лучей, но не доказывают видимость: стена между NPC и игроком проходит FOV и distance, а raycast должен её обнаружить.

`NpcCombatSystem` проверяет лучи к разным частям цели. Точка глаза NPC может браться из кости головы по уже доступной позе; в текущем порядке кадра `AnimationController::Update` вызывается позже. Точка цели может быть выше или ниже центра Entity. Это уменьшает ошибку, когда персонаж стоит за укрытием и видна только голова. Если кость недоступна, используется фиксированная высота глаза. Для каждой проверки исключай собственного NPC из raycast и проверяй первое столкновение.

Личное зрение обновляет `targetVisible` и `visibleAimPoint`. При потере контакта NPC хранит lastKnownPosition и время наблюдения. Если когда-нибудь введёшь слух, он должен создавать отдельное свидетельство с меньшей точностью: «звук в области комнаты B», а не «точный Transform игрока». Сообщение товарища может передать последнюю увиденную позицию, но не флаг личной видимости. Это защищает игру от всеведущего AI и делает укрытие игрока осмысленным.

9.3. Состояния и тактика не должны драться за одну запись

После подтверждённого наблюдения `NpcCombatSystem` обычно переводит мозг в Chasing или Attacking, но сохраняет исполняемый путь к нераскрытому укрытию. При потере контакта переход в Searching выполняет `StateSystem`: например, после достижения последней наблюдавшейся позиции, блокировки пути к ней или при оборонительном стиле. `NpcCoverSystem` выбирает индивидуальное укрытие и может удерживать уже выбранный маршрут, чтобы новая проверка видимости не отменяла его каждый кадр. `StateSystem` исполняет итоговое состояние: строит путь, выставляет CharacterMovement и останавливает NPC в нужной точке. Тактический компонент хранит память о входящем огне, укрытии, очереди, поиске. Если две системы каждый кадр без приоритета перезаписывают `brain.state`, NPC начнёт дёргаться между укрытием и атакой.

Изучи текущий комментарий в `ApplicationClass.cpp`: `NpcSquadCombatSystem::Update` закомментирован. Поэтому роли прикрывающего и фланкера в старых учебных практикумах являются проектом для будущего, а не гарантированной работой сегодняшней сцены. Индивидуальные короткие очереди, перезарядка, укрытие и поиск не равны групповому подавлению. В текущем `NpcRangedCombatSystem` после первой очереди на ходу ставится `movingBurstPauseTimer`, но ни одна система его не уменьшает: пока `movingFire` остаётся активным, следующий выстрел блокируется; переход к обычной стрельбе сбрасывает таймер. Это подтверждённый дефект, который нужно исправить перед балансировкой очередей. Не балансируй уровни исходя из того, что отряд уже координируется.

9.4. Выстрел — отдельное разрешение

Наличие состояния Attacking ещё не создаёт выстрел. `NpcRangedCombatSystem` проверяет активность, мозг, фракцию, здоровье, ручной приказ, экипированное оружие, патроны/перезарядку, личную видимость, задержку прицеливания, допустимую цель, ориентацию, выход из дула и союзника на центральной линии. Только затем вызывает `WeaponSystem::TryFire`. Последний запускает оружейный cooldown, применяет разброс, выпускает hitscan луч через Bullet и добавляет событие урона в очередь при физическом попадании. Отдельной летящей Entity пули текущий путь не создаёт. Разделение сохраняет одно правило оружия для игрока и NPC.

Пример. Враг видит игрока через низкое укрытие, но дуло находится за стенкой. `NpcCombatSystem` оставляет `targetVisible=true`, а `NpcRangedCombatSystem` должен отказать в выстреле, если короткий проверочный луч от груди к дулу первым пересёк препятствие. Если заменить всё одним флагом «виден», NPC стрелял бы сквозь укрытие. Другой пример: между стрелком и игроком проходит живой союзник. Центральный луч должен удержать огонь. Из-за разброса это не математическая гарантия отсутствия случайного friendly fire, так что game design может дополнительно расширить безопасный коридор.

9.5. Попадание, реакция и смерть

`WeaponSystem::TryFire` добавляет событие попадания в очередь `DamageSystem`. В текущем порядке кадра NPC стреляет после вызова `DamageSystem::Update`, поэтому здоровье и реакция на попадание обычно обновятся на следующем шаге симуляции. После обработки урона NPC может получить сигнал «под огнём» и при следующем тактическом решении выбрать укрытие. Не позволяй pixel shader решать величину урона: он может вообще не выполнить fragment из-за culling или depth, а gameplay должен быть независим от камеры. После смерти проверь прекращение движения и огня, освобождение cover reservation и crew claim, очистку цели у товарищей и корректное повторное создание сцены.

Практика. Построй пять сцен: открытое поле; стена; низкое укрытие; союзник на линии огня; дверь без collider. Для каждой заранее напиши ожидаемые visible, brain.state и rangedStatus. Запусти при одинаковом начальном положении и сравни с наблюдением. Не исправляй AI, пока не исключил ошибку collision shape или muzzle socket.

Урок 10. NavMesh, двери и движение персонажа

10.1. Маршрут не является движением

Recast строит область, по которой может пройти агент с заданными радиусом, высотой и шагом. Detour отвечает на запрос пути. `NpcNavMeshSystem` находит маршрут, а `StateSystem` превращает его в ближайшую цель движения. `CharacterControllerSystem` передаёт желание движения Bullet капсуле. Это три разных результата: «маршрут существует», «NPC хочет идти» и «NPC реально продвинулся». Первое не гарантирует третье. Физический ящик может перекрыть коридор после построения NavMesh, дверь может измениться, а радиус агента может не совпасть с капсулой.

Если NavMesh строится для радиуса 0,3 м, а фактическая капсула имеет радиус 0,5 м, путь вдоль стены может формально существовать, но персонаж будет тереться о неё. Если высота двери на один сантиметр ниже допустимой высоты агента, проектирование target на NavMesh может провалиться. Параметры сборки надо брать из реального character controller и проверять на минимальной сцене с известной шириной прохода.

10.2. Дверь — графовая и физическая связь

Маршрут между комнатами может использовать граф соединений дверей. Само открытие двери — не мгновенное освобождение прохода: створка ещё движется и collider может оставаться на траектории. Поэтому есть `WaitingForDoor`. Если дверь требует питание, `TryOpen` может отказать. В этом случае StateSystem должен записать понятную причину, а задача NPC — получить альтернативный путь или быть освобождённой. Бесконечный цикл «попросить открыть → не удалось → повторить каждый кадр» перегружает и AI, и debug.

Дверь влияет сразу на несколько систем. Collider блокирует движение и raycast; граф NavMesh влияет на маршрут; room connection влияет на атмосферу; power state влияет на возможность открыть; анимация показывает створку. Изменение одной подсистемы без остальных создаст парадокс: NPC проходит через закрытую дверь или воздух течёт через визуально герметичную. При новой двери выпиши все эти контракты и проверь их отдельно.

10.3. Застревание и перепланирование

Сравни фактическую позицию NPC через интервал. Если он хотел идти, но продвинулся меньше порога, увеличивай stuck timer. После тайм-аута сбрось или перестрой маршрут, но ограничь количество попыток и дай причину отказа. Не перестраивай путь каждый кадр: это дорого и может заставить NPC менять ближайший waypoint до того, как он к нему приблизился. Перепланирование требуется при изменении цели, комнаты, двери, проходимости или долгом отсутствии прогресса.

Практика. Поставь NPC за дверью, а целевую точку внутри комнаты; добавь второй путь в обход. Сначала дверь открыта, затем выключи питание, затем заблокируй проход физическим ящиком. Для каждого случая объясни отдельно результат графового поиска, намерение StateSystem и реальное движение Bullet. Отладочные линии маршрута и collision shapes должны показывать разные сущности задачи.

Урок 11. Как добавлять системы без разрушения архитектуры

11.1. Сначала опиши игровое правило

Возьмём будущую систему пожара. Слишком общий запрос «добавить FireSystem» не говорит, когда огонь появляется, что он расходует, кто узнаёт о нём и как он исчезает. Сформулируй поведение: повреждённый реактор создаёт очаг в своей комнате; интенсивность растёт при наличии топлива и кислорода; огонь повышает температуру, повреждает оборудование и создаёт видимый дым; инженер или игрок может тушить его, расходуя ресурс. Если кислорода нет, интенсивность должна падать. Уже здесь появляются проверяемые предельные случаи и единицы.

Запиши инварианты. Интенсивность не отрицательна и ограничена выбранным максимумом. Один источник не создаёт бесконечные дубликаты FireComponent. Урон применяется не чаще и не сильнее, чем задают dt и коэффициент. Выключенная сцена не продолжает фоновые jobs. Завершённый огонь не оставляет claim экипажа. Сохранение и загрузка восстанавливают ровно один очаг с ожидаемой интенсивностью. Инварианты задают границы реализации лучше, чем длинный список красивых эффектов.

11.2. Раздели данные, решение и исполнение

`FireComponent` хранит состояние очага. `FireSystem` обновляет физическую/игровую интенсивность, читая room environment и источник. `CrewTaskCoordinator` решает, кто тушит. `StateSystem` ведёт NPC к точке работы, но сам не вычисляет химическое горение. `ExtinguishSystem` или метод FireSystem принимает действие тушения и уменьшает интенсивность. Renderer рисует огонь по уже вычисленному heat/intensity. Звук и тревога читают WorldEvent. Система, которая одновременно вычисляет температуру, строит NavMesh, создаёт particle texture и записывает Scene JSON, станет хрупкой после первой новой задачи.

Для нового типа работы экипажа расширь `CrewTaskType`, сбор кандидатов, score, claim, StateSystem исполнение, имена состояний, Inspector, сериализацию authoring полей и диагностику. Для ручного приказа расширь `NpcCommandType`, публичный метод `NpcCommandSystem` и обработку StateSystem. Не полагайся на прямую запись `brain.command.type` из каждой кнопки UI: общий метод сбрасывает старый маршрут и увеличивает revision. Для врага новый retreat можно сначала выразить существующим состоянием TakingCover и тактической точкой; отдельный enum нужен лишь когда исполнение действительно другое.

11.3. Вставка в кадр и зависимость по данным

Назови производителя и потребителя каждого поля. Если PowerGrid выставляет `hasPower`, а насос в FloodingSystem читает его, PowerGrid должен отработать раньше насоса для эффекта в том же тике. Если FireSystem меняет health оборудования через DamageSystem, порядок событий определяет, когда координатор увидит новую работу. Избегай скрытых циклов: «Power зависит от Damage, Damage зависит от Heat, Heat зависит от Power». Для такой модели выбери явные фазы и состояние прошлого тика. Тогда задержка в один тик становится договором системы, а не ошибкой порядка.

Псевдокод шага новой системы показывает только распределение ответственности:

```
void FireSystem::Update(ECSRegistry& ecs, float dt, EventQueue& out) {
    for (auto& [entity, fire] : ecs.GetComponents<FireComponent>()) {
        if (!ecs.IsAlive(entity) || !fire.enabled) continue;
        const float oxygen = ReadRoomOxygen(ecs, fire.room);
        const float growth = ComputeGrowth(fire, oxygen) * dt;
        fire.intensity = std::clamp(fire.intensity + growth, 0.0f, 1.0f);
        if (CrossedAlarmThreshold(fire)) out.Push(FireAlarm{entity});
    }
}
```

Это схема, не готовый код проекта: `EventQueue`, `ReadRoomOxygen` и тип FireAlarm надо связать с реальными `WorldEvent` и данными комнаты. Важны проверка Entity, dt в секундах, clamp и событие порога, которое не должно отправляться заново каждый кадр. После реализации сравни 10 секунд симуляции при 30 и 120 FPS.

11.4. Узкий вертикальный срез

Первый рабочий вариант включает один очаг, одну комнату, одно устройство, одного NPC и одну кнопку ручного тушения. Не добавляй распространение по всем отсекам, частицы, звук, обугливание меша и сложную тактику в один коммит. Когда минимальный путь работает, добавляй второй очаг и второго NPC: именно тогда проявятся конфликты claim и приоритетов. Потом — закрытую дверь, отсутствие ресурса, паузу, save/load и объект вне камеры. Каждый следующий случай проверяет новый контракт, а не просто делает сцену красивее.

Урок 12. Анимация, оружие и согласование позы

12.1. Кто двигает персонажа

У игрового персонажа есть несколько источников движения: намерение AI или input, физическая капсула, root motion клипа, поворот корпуса и визуальная поза костей. Нужно выбрать владельца перемещения в мире. В текущем NPC пути `StateSystem` выставляет CharacterMovement, `CharacterControllerSystem` двигает капсулу, `AnimationController` выбирает визуальный клип. Если импортированная анимация Walk одновременно переносит root bone по миру, NPC может двигаться вдвое быстрее или шагать назад относительно collider. Для этого проекта root motion надо либо исключить из локомоции, либо явно интегрировать в контроллер как единственный источник смещения.

Скорость клипа и скорость капсулы должны согласовываться. Если шаг визуально соответствует 1,5 м/с, а контроллер движется 4 м/с, ноги будут скользить. Можно менять playback speed пропорционально реальной горизонтальной скорости, но сначала измерь длину шага и длительность цикла клипа. При резком переходе из Idle в Run полезен blend, однако blend не исправит неверный facing. Модель может иметь собственную «ось вперёд», отличную от +Z мира; это должно быть одной настройкой модели, а не серией дополнительных π в разных системах.

12.2. Поза до дула

Вооружённый NPC должен стрелять из muzzle socket актуальной позы. Порядок: AI выбирает огневое состояние, facing разворачивает персонажа, animation обновляет кости, weapon IK или socket расчёт получает мировой transform дула, затем `NpcRangedCombatSystem` формирует луч. Если muzzle рассчитывается до анимации, он на кадр отстаёт при развороте. Если кость в локальном пространстве умножена не на те родительские transforms, пуля стартует у ног или в центре сцены.

Проверка: нарисуй в debug мировую точку дула, его forward и луч на 2 м. Сравни Idle, Walk, Run, Aim и Reload, а также first-person и third-person модель. Сначала проверь отсутствие клипа: fallback socket должен иметь объяснимое положение. Потом включай анимацию и IK. Если прицел визуально наведён, а луч направлен иначе, изолируй model forward, socket rotation, bone global transform и направление камеры/цели.

12.3. Оружейные числа имеют физический и игровой смысл

`range` — длина физического луча. `fireRate` или cooldown определяет интервал попыток выстрела. `spread` определяет распределение направления, а не гарантию попадания. `damage` применяется после геометрического hit с учётом допустимой цели. `rangedAimDelay` описывает время подготовки NPC. Для коротких очередей тактический компонент хранит число оставшихся выстрелов и паузу. Если оружие пусто, `WeaponSystem::RequestReload` запускает перезарядку; визуальная анимация должна соответствовать этому runtime состоянию, но не быть единственным владельцем патронов.

Для баланса сравни expected time to kill на бумаге с наблюдением. Если у цели 100 health, выстрел наносит 20, фактический интервал между попаданиями 0,5 с, то для пяти точных попаданий от первого до пятого проходит минимум 2 с плюс aim delay. Разброс, укрытие и перезарядка увеличивают время. Если игра ощущается нечестной, не обязательно уменьшать damage: возможно, враг узнаёт скрытую позицию или стреляет без видимого окна прицеливания. Сначала покажи игроку причину угрозы.

Урок 13. Сохранение, фоновые задачи и устойчивость сцены

13.1. Два типа сохранения

Scene JSON хранит авторское устройство уровня: объекты, Transform, настройки компонентов, связи и редакторские значения. Save game хранит последствия игры: повреждения, состояние дверей, запасы, положение персонажей, текущий этап сценария и мировое время. Эти форматы могут

использовать общую сериализацию полей, но их назначение различно. Если scene save записывает текущие runtime повреждения после Play, автор может случайно перезаписать исходную палубу аварией. Если save game хранит только исходную сцену, игрок потеряет прогресс.

При загрузке не доверяй сохранённому числовому Entity ID как runtime ID. SceneManager создаёт новые Entity и remap ссылок. Сначала загрузи сущности и их данные, затем восстанови связи, затем пересоздай runtime ресурсы: Bullet bodies, GPU buffers, path caches, claims и временные указатели. Для сохранения на диск используй временный файл и атомарную замену, чтобы сбой питания не оставил обрезанный основной save. Перед принятием новой версии формата проверь старый save и значения по умолчанию новых полей.

13.2. Что не должно попадать в JSON

Адрес `btRigidBody*`, COM interface pointer, указатель на компонент в unordered_map, ID GPU buffer и `std::future` не являются долгосрочным состоянием игры. После перезапуска они недействительны. Сохраняй данные, из которых их можно восстановить: transform, shape type/параметры, путь asset, настройки материала, логическую связь. Runtime claim экипажа лучше пересчитать из существующих задач, чем восстанавливать старый указатель на assignee. Для асинхронной операции сохрани устойчивое игровое состояние, а работу пересоздай или заверши по правилам загрузки.

13.3. JobSystem и граница главного потока

Фоновый worker подходит для чтения/разбора asset, расчёта геометрии или чистого path query на неизменяемом снимке. Он не должен напрямую создавать D3D11 ресурсы на устройстве без установленного контракта потоков, менять ECS Registry во время обхода главного потока или обращаться к сцене после её уничтожения. Для операции mesh repair типичный путь: main thread делает снимок входных данных и revision; worker вычисляет результат; main thread проверяет, что сцена и Entity ещё актуальны; только затем применяет ECS изменение и GPU upload.

Представь: игрок запускает восстановление меша, сразу загружает другую сцену, а job заканчивает на старом указателе SceneManager. Если completion не проверяет generation сцены или cancel token, получится use-after-free. На shutdown сначала перестань принимать новые задания, дождись или отмени существующие согласно контракту, обработай оставшиеся main-thread completions безопасно, затем освободи ECS, Bullet и D3D. Порядок важнее одного дополнительного mutex.

Практика. Сохрани и загрузи сцену с дверью, пробоиной, инженером и оружием. Сравни числа объектов и связи до/после. Затем начни асинхронный импорт модели и смени сцену до завершения. Ожидаемый результат — отсутствие падения, применения к новой сцене и утечки. Если такой сценарий пока не проходит, оставь его явным release blocker, а не скрытой заметкой.

Урок 14. Интерфейс, звук и понятная обратная связь

14.1. HUD должен объяснять состояние мира

Игроку не нужна отладочная таблица всех ECS компонентов, но ему необходима информация для решения. Если дверь не открылась, покажи «нет питания» или «заблокирована», а не молча оставь кнопку без эффекта. Если экипаж пошёл к аварии, покажи назначенного работника и примерное местоположение. Если давление падает, показывай отсек, направление изменения и срочность. Debug UI может быть гораздо подробнее: точные численные значения, Entity ID, score, last transition и текст отказа. Раздели presentation layer для игрока и diagnostics для разработчика, оставив общим источник истины.

Интерфейс должен читать события и snapshot состояния после симуляции. Не делай кнопку HUD единственной точкой, которая физически чинит корпус: ту же команду могут вызвать NPC, скрипт и контроллер игрока. UI инициирует действие, игровая система валидирует его и публикует

результат. Так ошибка «нажали кнопку, но ремонт не произошёл» имеет проверяемое место: input, command, validation, execution или отображение.

14.2. Звук как информация, а не только украшение

В текущем дереве исходников не видно полноценного отдельного аудиомодуля, поэтому этот подраздел описывает проектируемый контракт. Звук пробоины, выстрела и тревоги должен получать позицию, категорию, радиус и время события. Микшер решает громкость, пространственное ослабление и приоритет. AI слышит не PCM данные и не текущую позицию скрытого игрока, а gameplay событие «шум в такой области» с вероятной точностью и давностью. Это позволяет реализовать поиск по звуку без телепатии.

Не создавай новые звуковые голоса каждый кадр, пока горит пожар. Событие начала воспроизводит loop, изменение интенсивности обновляет параметр, завершение останавливает loop. При выгрузке сцены все принадлежащие ей голоса останавливаются. В меню и при паузе явно реши, какие звуки продолжаются. Для игрока звук должен подтверждать решение: щелчок двери, неудача из-за питания, начало ремонта, завершение, попадание, аварийный сигнал.

14.3. Доступность и читаемость

Не кодируй важное состояние только цветом: добавь иконку, текст или форму. Сообщение о пробоине должно оставаться достаточно долго, чтобы его прочитать, и иметь историю в журнале событий. Раздели настройку громкости по категориям, сделай читаемый размер интерфейса на разных разрешениях, поддержи переназначение клавиш или хотя бы полный список управления. Это не финальная косметика: если игрок не понимает, почему NPC бросил ремонт или почему оружие не стреляет, симуляция кажется сломанной даже при правильном коде.

Урок 15. Контент, производительность и граница почти готовой игры

15.1. Вертикальный срез как проверка всего проекта

Минимальный цельный эпизод: игрок приходит в машинный отсек, получает тревогу о пробоине, видит снижение давления, отдаёт инженеру приказ или берёт ремонт на себя, сталкивается с врагом, восстанавливает питание, закрывает дверь, побеждает угрозу и сохраняет игру. После загрузки все важные последствия остаются согласованными. В эпизоде должны быть начало, понятная цель, несколько путей решения, риск, обратная связь и окончание. Редакторские окна и debug hotkeys не должны быть обязательными для прохождения.

Для каждого объекта уровня составь контракт. Дверь: геометрия, collider, graph link, атмосфера, power, взаимодействие, звук. Генератор: внешний вид, health, power output, fuel или правило бесконечной работы, рабочая зона, HUD, save. NPC: фракция, controller, brain, навигация, анимация, оружие/инструмент, реакция на смерть и загрузку. Такой список предотвращает «красивые» объекты, которые не участвуют в мире. Контент производи повторяемыми шаблонами, чтобы вторая палуба не требовала ручного исправления десятков компонентных связей.

15.2. Бюджет кадра

При 60 FPS один кадр занимает около 16,67 мс. Если CPU simulation тратит 8 мс, render submission 5 мс и GPU 20 мс, итог ограничен GPU, а сокращение AI на 1 мс не даст 60 FPS. Измеряй CPU и GPU отдельно; записывай сцену, число NPC, комнат, источников света, particles и draw calls. Не отключай AI вне камеры ради FPS: экипаж должен продолжать чинить аварию, а пожар распространяться. Используй более редкое принятие решений для далёких NPC, но движение и мировые события должны оставаться согласованными по игровому времени.

Ищите дорогие места по профилю. Для AI это могут быть массовые raycasts и повторные NavMesh queries. Для рендера — дополнительные reflection/refraction passes, число теневых объектов, overdraw прозрачности и тяжёлый PBR PS. Для физики — слишком сложные collision meshes,

избыток динамических тел и частые пересоздания shape. Меняй один фактор за раз и сравни результат на той же сцене. Оптимизация, нарушившая правильность ремонта или save, не является улучшением.

15.3. Проверки качества перед выпуском

Создай набор проверяемых сценариев. Пустая сцена и отсутствующий asset не должны ронять приложение. Новый и старый save открываются согласно версии формата. После многократной смены сцен число Bullet objects и D3D resources возвращается к ожидаемому уровню. NPC не стреляет без фракции и оружия, не чинит исчезнувшую цель, освобождает claim после смерти и не видит игрока сквозь стену. Пауза останавливает simulation time, а UI остаётся управляемым. Долгий кадр не заставляет physics бесконечно догонять. Все эти проверки нужны потому, что каждая охватывает границу между системами.

Проект содержит `scripts/verify_engine.ps1`, который проверяет shader entry points, собирает x64 Debug/Release и запускает core contract tests. Это полезный технический gate, но он не заменяет игровой прогон. Сборка показывает, что код компилируется; ручной сценарий показывает, что игрок понимает происходящее. Перед кандидатом к выпуску создай отдельный build package и запусти его из каталога, отличного от каталога исходников. Иначе относительные пути к assets и shaders могут случайно работать только у разработчика.

15.4. Конкретное определение почти готовой игры

Игровой цикл проходится без редактора. Контент выдерживает несколько часов прохождения без обязательного debug вмешательства. Вход, сохранение, загрузка, смерть, перезапуск и выход из игры определены. У NPC есть объяснимые причины действий; у корабля — последовательные последствия аварий; у игрока — достаточная информация для решения. Производительность измерена на целевой машине, а критические утечки и падения устранены. Для каждого используемого asset понятны источник, условия использования и способ повторной сборки. Это не список «идеальных возможностей», а практическая граница между демонстрацией механик и игрой, которую можно дать внешнему человеку.

Урок 16. Выпускная работа на одном корабле

16.1. Сценарий и карта ответственности

Собери уровень из трёх отсеков: мостик, машинный, ремонтный. Между ними две двери, один генератор, один аккумулятор, одна пробоина, один повреждаемый кислородный прибор, два дружественных NPC и два врага. Игрок начинает с доступом к мосту, но без полного знания аварии. Тревога сообщает комнату, инженер получает работу, оператор может восстановить устройство, враги реагируют на видимый контакт и звук. Структура достаточно мала, чтобы её проверить вручную, и достаточно сложна, чтобы выявить ошибки порядка систем.

Перед реализацией нарисуй граф зависимостей: Damage → Breach → Atmosphere; Generator/Battery → PowerGrid → Door/Oxygen; авария → CrewTaskCoordinator → StateSystem → Bullet controller → RepairSystem; контакт врага → Perception → State/Cover → Animation/Weapon → Damage. На каждой стрелке подпиши тип данных и время обновления. Если стрелка пересекает worker thread, добавь снимок входов и проверку актуальности сцены на выходе. Если пересекает save/load, добавь сериализуемый ключ и remap.

16.2. Последовательность реализации без второй дорожной карты

Сначала добейся сценария без красивых моделей: примитивные комнаты, debug линии, текстовые статусы. Затем подключай визуальные модели, анимации, частицы и звук к уже работающим событиям. Это принцип построения одного вертикального среза, а не дополнительный план по дням. После каждого подключения сохраняй способность объяснить, где истинное состояние: в ECS,

Bullet, WorldEvent или render snapshot. Никогда не заставляй визуальный эффект быть единственным доказательством геймплейного события.

Наконец дай сборку человеку, который не читал код. Наблюдай, где он не понимает следующую цель, почему дверь не открывается, куда пошёл NPC и что означает тревога. Не подсказывай через debug UI. Запиши каждый момент непонимания и исправь интерфейс, звук, анимацию или правило мира. Именно этот прогон покажет, стала ли система игрой.

16.3. Критерий сдачи

Сценарий можно пройти от начала до конца, проиграть и повторить. Вражеский NPC не получает скрытые координаты, дружественный NPC не удерживает недостижимую работу навсегда, разрушение и ремонт согласованы с коллизией и атмосферой, сохранение восстанавливает состояние, а релизная сборка запускается вне рабочего каталога. У каждой аварии есть причина, численный эффект и ясная обратная связь. У каждого NPC решения есть объяснение в debug monitor. Когда эти условия выполнены, справочные главы дальше по книге помогут расширять проект без потери архитектуры.

Системы DX Framework: чтение реального кода от кадра до пикселя

Эта часть учебника учит читать существующий движок как цепь причин и следствий. Если объект переместился, надо выяснить, кто записал `TransformComponent`; если погас свет, пройти от генератора через сеть питания к `LightComponent`; если пропал предмет после загрузки, проверить перечисление сущностей в `SaveScene`, поле JSON, обратное чтение и remap ссылок. Здесь описано **состояние кода**, проверенное по файлам проекта, а изменения, которых в нём пока нет, помечены **«Проектное предложение»**. Пути вроде `src/Core/ApplicationClass.cpp` отсчитываются от `DX_FrameWork/` внутри репозитория. Номера строк помогают найти блок в текущей версии, но после правок будут сдвигаться.

Ключевой принцип для переучивания: у каждого значения есть владелец, единицы измерения, время жизни и момент записи. `Entity` — числовой ключ, `TransformComponent` — авторское или итоговое положение, Bullet хранит физическое тело, `PowerConsumerComponent::hasPower` — вычисленное за кадр состояние, JSON — снимок части компонентов, а видеопамять и потоковые задания — временные ресурсы. Когда эти категории смешиваются, появляются ошибки, которые выглядят как «физика дрожит», «шейдер не работает» или «сохранение иногда ломается».

1. Один кадр целиком: как читать `ApplicationClass::Frame`

1.1. Входные данные, режим и два времени

Точка входа кадра — `ApplicationClass::Frame(float dt)` (`src/Core/ApplicationClass.cpp:1023`). `dt` выражает прошедшее время кадра в секундах. В начале есть специальная ветка проверки оружия: при `m_weaponQA` значение принудительно становится `1/60`. Затем `DebugHud` обновляется даже при паузе игрового времени. Вызовы `m_inputManager.Update(m_input)` и `m_events.Clear()` (`:1033–1034`) означают соответственно новый снимок ввода и пустой список событий на новый кадр. Событие, оставшееся только в прошлой очереди, здесь исчезнет; производитель и потребитель события должны работать в ожидаемом порядке в одном кадре либо хранить долгоживущее состояние отдельно.

F5 обрабатывается по фронту нажатия (`f5Down && !m_wasF5Down`, `:1078`). При входе в Play вызывается `m_gameplayController.EnterPlayMode`, затем сбрасываются таймер, журнал событий мира, trigger-состояние, взаимодействие и камера. Для игрока создаётся инвентарь и тестовая винтовка; сцена для наблюдения за AI без игрока отдельно защищена проверкой `player != INVALID_ENTITY`. При выходе из Play очищаются команды движения у `CharacterMovement` и контроллеров. Важная привычка: читать **условие вызова** вместе с вызываемым методом. Комментарий возле выхода из Play о том, что физика продолжает шагать в Editor, устарел относительно фактического вызова `UpdatePhysics(isPlayMode ? simulationDeltaTime : 0.0f)` и раннего возврата при `dt <= 0` (`:1570`, `:2276`). По текущему вызову Editor не шагает Bullet и не выполняет затопление/давление.

На строках `1185–1186` вводятся `simulationDeltaTime` и `simulationRunning`. В обычном Play это время симуляции, в NPC Studio его может изменить пауза или одиночный шаг (`:1279–1283`); `simulationRunning = simulationDeltaTime > 0` (`:1352`). Поэтому `dt` не всегда равно времени игрового мира. UI, камера и диагностические сообщения могут обновляться по кадру, а оружие, AI и физика — по симуляционному времени. Когда добавляешь таймер перезарядки, выбери время осознанно: при паузе Studio перезарядка должна остановиться, значит ей нужен `simulationDeltaTime`. Когда обновляешь анимацию воды в окне редактора, используется `dt`, чтобы визуал оставался живым (`:1571`). Если перепутать их, ошибка обнаружится лишь в паузе или при нестандартной частоте кадров.

1.2. Порядок стадий и зависимость от предыдущей записи

После ввода главный поток выполняет `m_jobSystem.ExecuteMainThreadJobs()`, `m_assetLoadSystem.Update(device)` и `UpdateAsyncShipImport(...)` (`:1228–1234`). Первый вызов переносит готовые фоновые результаты на главный поток; второй может создать буферы Direct3D из подготовленных CPU-данных. Затем идут редакторский ввод, взаимодействие, подбор оружия, двери и ремонт. Камера до движения считывает новый yaw мыши (`:1405–1442`), чтобы направление намерения игрока и взгляда было согласовано. Блок `isPlayMode && simulationRunning` (`:1448`) формирует намерения управления и последовательность NPC. Таймеры оружия обновляются **до** попыток стрелять (`:1467`); AI принимает решение, движение и контроллеры получают команду, анимация выбирает клипы; `DamageSystem` потребляет события урона (`:1549–1551`). Отдельная глава книги разбирает NPC, здесь важно лишь место их записи в общей цепи.

Далее идут `SyncCompartmentPositions`, `UpdatePhysics`, `UpdateWaterAnimation` (`:1566–1571`). В `UpdatePhysics` строго задана последовательность: затопление, создание соединений, обновление корня подлодки, физика Bullet, проверка сломанных соединений, давление (`:2274–2302`). Это не косметический порядок. Затопление изменяет объём воды до расчёта выталкивания; соединение создаётся до физического шага; разрушение соединения проверяется после нагрузки; давление читает уже новую позицию тела. Значит новый ущерб от давления станет виден следующей фазе, которая его читает, а не «одновременно» со старым состоянием.

После физики камера следует итоговому `TransformComponent` игрока (`:1583–1613`). Далее `SceneManager::UpdateAll` получает ноль игрового времени в Editor (`:1628–1631`), обновляет анимационные и корабельные системы, а `UpdateLights` формирует положение светильников. Лишь затем камера строит view-матрицу (`:1634–1635`). Отдельный блок позднее вычисляет луч оружия: прицел берётся из камеры, но фактический выстрел начинается от дула. `WeaponSystem::Update` и `NpcRangedCombatSystem::Update` идут после позы и физики (`:1757–1769`), поэтому положение дула соответствует уже актуальной руке. Наконец `m_renderer->RenderFrame` (`:2010`) читает подготовленную сцену. Если новую систему помещают до той, чьи данные она читает, она увидит вчерашнее состояние; если после рендера, эффект станет заметен лишь через кадр.

Практика чтения кадра: выпиши для новой системы четыре пункта — вход, источник времени, изменяемые компоненты, первый читатель результата. Например «электроснабжение»: вход — мощности генераторов и потребностей, время — `deltaTime` из `SceneManager::UpdateAll`, запись — `hasPower`, читатели — свет, кислородный генератор, насос. Затем найди все вызовы этих читателей, а не только название класса. Так выявляется реальный pipeline.

1.3. Почему один кадр не равен одному шагу физики

`PhysicsWorld::Update` (`src/Core/PhysicsWorld.h:86–113`) складывает `deltaTime` в аккумулятор, ограничивает его `0.25` секунды и, пока хватает времени, шагает Bullet фиксированным `1/60` секунды. В кадре длительностью `1/120` секунды физический шаг произойдёт через два визуальных кадра; при резком лаге может произойти несколько шагов подряд, но не бесконечная «догонялка». `m_interpolationAlpha = accumulator / timestep` предназначен для сглаживания визуала между фиксированными состояниями. Это причина, почему нельзя рассчитывать игровую скорость как «два метра за вызов Update»: нужно умножать скорость в м/с на секунды. И наоборот, нельзя в коде силы умножить на `dt`, если затем эту величину как силу передать в `applyCentralForce` и Bullet уже интегрирует её по шагу: тогда время фактически учитывается дважды. Для конкретной силы проверяй единицы и API тела.

2. ECS: сущность, компонент, система и владение состоянием

2.1. `Entity` — ключ, а не объект

В `src/ECS/ECSRegistry.h:15–21` метод `CreateEntity` проверяет исчерпание пространства ID, выдаёт `nextEntity++` и помещает ID в `m_liveEntities`. У сущности нет встроенной позиции, здоровья или

меша. Это числовая связь между отдельными хранилищами компонентов. `DestroyEntity` (`:24–28`) убирает ключ из множества живых и вызывает `Remove` во всех пулах. `IsAlive` проверяет именно множество живых (`:30`). В этой реализации не видно generation counter: после полного `Clear` реестр создаётся заново и ID опять начинаются с нуля. Поэтому долгоживущую ссылку на `Entity` нельзя переносить через `LoadScene` или `Clear`, даже если после загрузки цифра случайно совпала. До использования сохраняемой ссылки обычно нужны `IsAlive`, наличие нужного компонента и, если это сериализованная ссылка, корректный remap.

`AddComponent<T>` делегирует вставку пулу (`:33–43`). У метода нет проверки `IsAlive(e)`: вызывающая сторона обязана создать сущность и не добавлять компонент к уничтоженному ID. Более того, `ComponentManager<T>::Add` сначала удаляет прежнюю запись, затем делает `emplace` (`src/ECS/Components/ComponentManager.h:29–42`). Это **замена всего компонента**, а не изменение одного поля. Если код сохранил `T*` и затем повторно вызвал `AddComponent<T>` для того же Entity, старый указатель уже недействителен. Для обычного изменения состояния бери существующий компонент через `GetComponent` и меняй конкретное поле; `AddComponent` используй при создании или осознанной полной замене.

`GetComponent<T>` возвращает указатель или `nullptr` (`ECSRegistry.h:61–65`), `HasComponent<T>` даёт булево (`:67–71`), `RemoveComponent<T>` удаляет запись при наличии пула (`:73–77`). `GetComponents<T>` вызывает `GetOrCreatePool<T>` даже при намерении только прочитать пул (`:79–82`). `ForEach<T>` не создаёт пустое хранилище, если тип ещё нигде не использовался (`:83–92`). Двухкомпонентный `ForEach<T1,T2>` (`:45–59`) перебирает **первый** пул и ищет такой же Entity во втором. Поэтому выбери более редкий тип первым: если `PressureHullComponent` имеют 20 сущностей, а `TransformComponent` — 3000, `ForEach<PressureHullComponent,TransformComponent>` делает существенно меньше итераций. Это вывод из структуры обхода, а не гарантия порядка: внутри `unordered_map` порядок сущностей не задан.

`ActiveComponent` хранится особым образом как множество ID (`ComponentManager.h:69–96`). У него нет индивидуальных полей на каждую сущность; `Get` возвращает указатель на один `m_dummy`. Для проверки активности используй `HasComponent<ActiveComponent>`. Нельзя записывать в найденный через `Get` экземпляр и ожидать уникального состояния объекта. И важное отличие: живость и активность независимы. Неактивный объект остаётся сущностью, может сохраняться в сцену и иметь все компоненты; системы сами решают, пропустить его или нет. Например физика отключает контактную реакцию и движение неактивного RigidBody (`PhysicsSystem.cpp:1007–1027`), а `PressureSystem::Update` явной проверки `ActiveComponent` не делает. Если правило нового эффекта требует паузы для неактивных объектов, добавь проверку в этом эффекте.

2.2. Где живёт положение объекта

`TransformComponent` содержит `position`, `rotation`, `scale` и вычисляет мировую матрицу как `scaleMatrix * rotationMatrix * translationMatrix` (`src/ECS/Components/Components.h:27–40`). Углы вращения применяются в порядке X, затем Y, затем Z. Для соглашения `mul(position, matrix)` в шейдере позиция вершины сначала масштабируется, потом вращается, потом переносится. `SceneManager::UpdateAll` прямо не поддерживает отдельную копию трансформации в старом `GameObject`; `RenderAllECS` читает `transform->GetWorldMatrix()` (`SceneManager.cpp:2457`). Значит обычный путь изменения позиции для отображения — компонент, а не ручной сдвиг матрицы только перед одним draw call. Исключение — объекты с Bullet: там надо понимать, кто хозяин на данном этапе.

Кинематическое тело получает положение из ECS каждый кадр; динамическое обычно получает положение из Bullet, а внешнее редактирование `TransformComponent` распознаётся и отправляется обратно в Bullet. Статическое тело не интегрируется динамикой, но может быть явно пересинхронизировано при изменении трансформации. В `PhysicsSystem.cpp:965–982` сравниваются `lastSyncedPosition`, `lastSyncedRotation`, `lastSyncedScale` с текущим `Transform`. Это не «почти равно», а прямое сравнение записанных значений. Для внешней телепортации динамического тела (`:1080–1113`) обновляются world transform, motion state, interpolation transform и AABB, затем сбрасываются скорости и силы. Поэтому нельзя использовать прямую запись `Transform.position` как способ плавного толчка: это будет воспринято как телепорт. Для толчка применяй силу/импульс к Bullet-

телу в подходящей фазе. После шага динамическое тело пишет позицию и вращение обратно в ECS (`:1353–1394`). Именно это означает «владелец Transform» в этой системе: постоянное хранилище отображения — ECS, но авторитет движения динамического объекта во время симуляции — Bullet.

2.3. Как безопасно итерировать и удалять

Обычные пулы — `std::unordered_map<Entity,T>` (`ComponentManager.h:15–56`). `ForEach` держит итератор пула, пока выполняет callback. Удаление текущей записи внутри callback инвалидирует текущий итератор; добавление того же типа может вызвать rehash и сломать обход. Поэтому для массового удаления сначала собери ID в отдельный `std::vector<Entity>`, закончи обход, затем удали. Если создаёшь компоненты другого типа, проверь, не держит ли параллельный обход указатели в этом пуле. Здесь не требуется универсальный запрет на любую запись, но структурную модификацию **перебираемого** пула нужно считать потенциально опасной.

Удаление сущности сцены — больше чем `registry.DestroyEntity`. `SceneManager::RemoveObject` (`src/Scene/SceneManager.cpp:2092–2122`) сначала убирает соединения Bullet для обоих видов joint, удаляет `ActiveComponent`, освобождает уникальный материал, уничтожает runtime character controller, затем удаляет компоненты из реестра. Если обойти `RemoveObject` и удалить только ECS ID, constraints могут сохранить указатель на уже уничтоженное тело. Для новой системы заранее реши, есть ли у компонента внешний ресурс: Bullet body, GPU buffer, открытый дескриптор, очередь job или кэш NavMesh. Если есть, у удаления нужен явный этап освобождения. `Clear` (`:2128–2161`) сбрасывает асинхронный импорт, инвалидирует NavMesh, вызывает `ReleasePhysicsRuntime`, удаляет материалы и кэш моделей, после чего целиком заменяет `m_registry = ECSRegistry{}`. `ReleasePhysicsRuntime` (`:2163–2178`) сначала отсоединяет constraints, затем контроллеры персонажей — порядок обусловлен зависимостями Bullet. Это полезный образец для любого ресурса, который ссылается на другой.

3. Сцена и сохранение: путь от Entity к JSON и обратно

3.1. Что именно сохраняет `SaveScene`

`SceneManager::SaveScene` (`src/Scene/SceneManager.cpp:3270`) создаёт JSON с `scene_name`, `version = 2` и массивом `objects`. Лямбда `saveEntity` проверяет `IsAlive`, пропускает `NpcWeaponVisualComponent` (визуал экипировки восстанавливается позже) и `ShipPartComponent` (импортированные части восстанавливаются из корня GLB), а множество `savedEntities` защищает от дублирования одной сущности, найденной в нескольких пулах (`:3277–3289`). Затем метод перечисляет корни кораблей, объекты с мешем, отсеки, комнаты, поверхности NavMesh, физические тела, пробоины, NPC без меша, оружие, области мира, взаимодействия, триггеры, инвентарь, предметы и именованные логические сущности (`:3292–3415`). Это список **владельцев включения** в файл. Если ты создашь новый «чисто логический» компонент без меша, имени или любого уже перечисляемого признака, он может не попасть в JSON, даже если `SerializeObject` умеет записать его поля. Нужно проверить и перечисление, и сериализацию.

На `:3417–3420` файл открывается и записывается `scene.dump(4)`. Возврат `true` означает, что `ofstream` удалось открыть и запись дошла до конца функции; явной проверки ошибки после `file <<` здесь нет. `SerializeObject(entity,id)` (`:3428–3433`) пишет два разных идентификатора: `id` — последовательный индекс объекта в документе, `entity_id` — реальный runtime ID на момент сохранения. Ссылки других компонентов ремапятся по второму полю. Не путай индекс в массиве с ключом ECS. Хранить в ручном JSON ссылку на `id` вместо `entity_id` — значит потерять цель при загрузке.

В коде чтения есть блок `scene["world"]` для fog и skybox (`LoadScene:4314–4326`), но в текущем `SaveScene` нет присваивания `scene["world"]`. Следствие: новые файлы, записанные этим методом, не несут мировые настройки этим путём, хотя старые или внешне подготовленные файлы могут их восстановить. Это факт конкретной реализации, а не обещание, что какой-то другой редакторский механизм обязательно исправляет пробел. **Проектное предложение:** включить `WorldSettings` в `SaveScene` симметрично `LoadScene`, затем проверить round-trip тумана и пути skybox на новой сцене.

3.2. Почему загрузка не может просто восстановить старые числа ID

`LoadScene` (`:4301–4455`) сначала открывает файл и парсит JSON, после этого очищает старую сцену (`Clear`). Затем создаёт таблицу `old entity_id → new Entity` и набор `restoredEntities`. Для корня корабля с `has_ship_root` вызывается `ImportShipFromGlb`, после чего возвращённому корню применяются позиция, вращение, масштаб, состояние аккумулятора и сохранённые данные генераторов; части корабля создаются импортом заново. Обычный объект идёт через `DeserializeObject`. На `:4476–4490` десериализатор вызывает `CreateEntity`, создаёт `TransformComponent` и по полям JSON добавляет `ActiveComponent`, `InvulnerableComponent`, `NameComponent`. Далее большой метод восстанавливает остальные поддержанные типы. Важно: даже если прежний ID был `41`, новый может оказаться `76`: импорт корня GLB и его частей расходует ID в другом порядке. Поэтому ссылка «дверь ведёт в комнату 41» не готова к употреблению до remap.

Когда все объекты созданы, `RemapEntityReferences(remap,restoredEntities)` (`:4433–4437`, определение `:5890`) проходит по поддержанным ссылочным полям. Обрабатываются только объекты из сериализованного набора, чтобы случайно не переназначить новые ID импортированных деталей корабля, которые уже указывают друг на друга правильно. Затем физические тела, сохранённые лишь как данные компонента, получают новые Bullet runtime bodies (`:4439–4448`), разрешаются зоны ремонта и соединения комнат. Это двухфазный протокол: сначала создание, потом связи и ресурсы. Если добавить компонент, хранящий `Entity target`, но забыть `RemapEntityReferences`, после загрузки цель может стать другой сущностью или не существовать. Для новых ссылок удобно требовать `INVALID_ENTITY` при отсутствующей цели.

В `RemapEntityReferences` есть два варианта: `remapEntity` при отсутствии записи печатает диагностику и возвращает старый ID (`:5894–5905`), `remapEntityOrInvalid` возвращает `INVALID_ENTITY` (`:5906–5912`). Первый вариант допускает, что старое число случайно совпадёт с новым живым Entity; для связи, которая должна быть гарантированно сериализована, это риск. **Проектное предложение:** для каждой ссылки явно определить политику «цель отсутствует» и использовать её, а не оставлять старый ID по умолчанию; проверить на сцене, где целевой объект специально удалён из JSON. Парсер `file >> scene` не обёрнут локальной обработкой исключений и `LoadScene` не проверяет схему до `Clear`. Битый JSON бросит исключение; неверное поле может привести к частично восстановленной сцене. Это не повод дописывать «магический» catch вслепую: сначала нужны тестовые файлы для отсутствующего `objects`, неверного типа координат и отсутствующего asset, затем явная политика ошибки и безопасная точка замены сцены.

3.3. Упражнение: добавляем сохраняемый «маяк тревоги»

Проектное предложение, не существующая функция. Допустим, `AlarmBeaconComponent` содержит `float radiusMeters` и `Entity owner`. Путь реализации: создать компонент; решить, кто создаёт его Entity (редактор, импорт корабля или gameplay); обеспечить перечисление сущности в `SaveScene`, если она может быть без меша и имени; записать `has_alarm_beacon`, радиус и старый `owner` в `SerializeObject`; прочитать поля в `DeserializeObject`; добавить `owner` в remap; после загрузки пересоздать любые runtime-ресурсы (например, свет) и проверить `IsAlive(owner)`. На тестовой сцене разместить два маяка, один выключенный, сохранить, очистить и загрузить. Сравни не только количество компонентов, но и радиус, активность, связь с правильным кораблём и видимость в рендере. Это цепочка проверки, а не один тест функции сериализации.

4. Bullet и ECS: два представления одного физического объекта

4.1. Создание мира и тела

`PhysicsWorld::Initialize` (`src/Core/PhysicsWorld.h:20–84`) создаёт collision configuration, dispatcher, broadphase, constraint solver и `btDiscreteDynamicsWorld`; включает гравитацию `(0,-9.81,0)`, ghost callback для персонажей и GImpact для динамических вогнутых мешей. Настроены 50 итераций солвера, split impulse и ERP `0.2`. Эти значения — параметры текущей реализации, а не универсальные константы физики. Они влияют на устойчивость соединений тяжёлых отсеков и

стоимость шага. Метод сначала создаёт зависимости во временных `unique_ptr` и переносит их в поля лишь после успешной настройки. Это уменьшает риск мира, который наполовину инициализирован. Повторная инициализация при уже существующем мире возвращает успех без пересоздания: нельзя ожидать, что вызов `Initialize` «обнулит» тела.

`PhysicsSystem::CreateRigidBody` (`src/ECS/System/PhysicsSystem.cpp:554-960`) требует `RigidBodyComponent` и `TransformComponent`; дальше выбирает форму по `shapeType`, строит Bullet shape, local inertia для массы, motion state, `btRigidBody`, collision group/mask и добавляет тело в мир. Внутри есть ветки Sphere, Box, Cylinder, Capsule, Plane, ConvexHull, TriangleMesh, GImpactMesh, составные элементы и особые внутренние коллизии корабля. При чтении этого длинного метода не пытайся запомнить каждую строку. Сначала выдели инвариант: на входе у сущности есть авторские параметры формы/массы и world transform; на выходе `rb.body` существует в Bullet и `lastSynced...` соответствует `Transform`. Если редактируется форма, старое тело нужно снять из мира и создать заново; в `RefreshRigidBody` (`:491-554`) сохранён старый transform, удаляются body и motion state, затем вызывается создание. Поэтому изменение масштаба нельзя считать обычным сдвигом матрицы: collision shape часто зависит от масштаба и требует перестройки.

В `PhysicsWorld` fixed step `1/60` секунд, но `PhysicsSystem::Update` получает кадровый `deltaTime` и до физического шага применяет силы плавучести. Если рендер идёт быстрее или медленнее 60 FPS, здесь стоит отдельно проверять, сколько раз силы ставятся относительно реальных Bullet шагов. Код уже ограничивает накопление времени, но сила задаётся на вызов `PhysicsSystem::Update`, а шаги выполняются внутри `PhysicsWorld::Update`. **Проектное предложение:** для воспроизводимой симуляции перенести расчёт сил, зависящих от физического шага, в цикл fixed step или явно проверить текущую схему на 30/60/144 FPS. На тесте одинаковый корабль должен иметь близкую траекторию через десять секунд игрового времени при разных частотах отображения.

4.2. Разбор `PhysicsSystem::Update` по фазам

На `PhysicsSystem.cpp:984-988` берутся пулы плавучести, тел и трансформаций. Первая фаза (`:1003-1145`) идёт по `RigidBodyComponent`. Если `rb.body == nullptr`, объект не участвует в Bullet и пропускается. Если нет `ActiveComponent`, тело получает флаг `CF_NO_CONTACT_RESPONSE`, обнуляет скорости и силы, переводится в `DISABLE_SIMULATION`; прежние collision flags и activation state сохранены для восстановления при включении (`:1007-1040`). Это runtime-остановка, не удаление авторского компонента. При повторной активации флаги и состояние возвращаются, тело будится.

Далее требуется `TransformComponent`. Для кинематического тела (`:1045-1076`) ECS всегда авторитетен: позиция и Euler-вращение преобразуются в `btTransform`, записываются в body и motion state, обновляются interpolation transform и AABB. Если изменился scale, вызывается `RefreshRigidBody` и текущая итерация заканчивается, поскольку форма зависит от размера. Для динамического тела (`mass > 0`, `:1077-1116`) обычное движение живёт в Bullet. Только если `TransformChangedExternally` обнаружил отличия от последней обратной синхронизации, ECS принудительно отправляется в Bullet: это телепорт/редакторское изменение, со сбросом скоростей и сил. Для тела с нулевой массой есть отдельная ветка внешнего изменения (`:1117-1144`) без динамической интеграции. Следовательно, при добавлении новой системы движения сначала реши: объект физически движется из-за сил либо следует программному пути? Для первого варианта не перезаписывай `Transform` каждый кадр; для второго явно выбирай кинематику.

Вторая фаза (`:1149-1344`) рассчитывает плавучесть по `BuoyancyComponent`. Третья синхронизирует ghost-персонажей из ECS до шага, вызывает `PhysicsWorld::Get().Update(deltaTime)` и читает ghost-позиции после шага (`:1345-1348`). `SyncCharacter` (`:32-217`) учитывает смещение центра капсулы относительно визуальных ног, отслеживает внешнюю правку позиции, защищает от NaN и сглаживает визуальную позицию при простое. Комментарий `:87-93` объясняет, почему визуальный поворот персонажа не следует каждый кадр вталкивать в симметричную Y-up капсулу: лишний warp может потянуть контроллер назад из-за интерполяции. Это пример разделения «где смотрит меш» и «как столкновения видят капсулу».

Четвёртая фаза (`:1353–1395`) берёт динамические активные тела, пропускает кинематические и статические, получает Bullet world transform и записывает позицию/вращение в ECS. Перед записью есть защита от NaN координат и от `abs(y)>10000`, в проблемных случаях тело останавливается. После успешной записи обновляются `lastSynced...`: на следующем кадре `TransformChangedExternally` не примет собственный результат Bullet за внешнюю телепортацию. Проверка NaN здесь касается только позиции, а не всех полей quaternion или масштабов; это граница защиты, о которой важно помнить при поиске неисправной физики.

4.3. Пример: импульс, телепорт и статическая преграда

Представь коробку массой 10 кг с `RigidBodyComponent::Box` на координатах `(0,2,0)`. Если нужно «толкнуть» её взрывом, игровая система получает `rb.body` и применяет Bullet импульс в выбранном направлении. Затем шаг Bullet меняет скорость и координаты, а обратная синхронизация обновляет ECS. Если вместо этого каждый кадр писать `transform.position.x += speed*dt`, `TransformChangedExternally` будет запускать ветку телепорта и обнулять скорости; объект может проходить сквозь динамические ожидания и разрушать естественную реакцию столкновений. Телепорт же при респавне как раз должен обновить world transform, motion state, AABB и сбросить старую скорость: в таком сценарии ветка текущего кода соответствует смыслу действия. Статическая преграда с массой ноль не падает под гравитацией; если дизайнер подвинул её в редакторе, Bullet должен получить новый transform, иначе луч попадания и визуальная стена разойдутся.

Для отладки ставь рядом три наблюдения: `TransformComponent::position`, `rb.body->getWorldTransform().getOrigin()` и `rb.lastSyncedPosition`. Если первое меняется до шага, а второе нет, ищи проверку активности, отсутствие body или неверную ветку sync. Если второе меняется, а первое нет, ищи пропуск обратной фазы, статическое/кинематическое тело или защиту NaN. Если визуал меняется, а луч Bullet попадает в старое место, проверь AABB и перерождение формы после scale. Такое сравнение локализует проблему лучше, чем перестановка `Update` наугад.

5. Вода, глубина, пробоины и атмосферное давление: не смешивать среды

5.1. Текущая физическая среда проекта

Хотя проект развивается в сторону космического симулятора, конкретные `PressureSystem`, `FloodingSystem`, `BuoyancyComponent`, `SubmarineComponent` и часть renderer описывают **подводную** среду. У них есть горизонтальный `waterLevel`, глубина `waterLevel - worldY`, плотность воды и выталкивающая сила. В вакууме формула давления от глубины неприменима: наружное давление близко к нулю, для повреждения от разгерметизации важна разность между внутренним давлением отсека и внешним. Эти модели можно использовать как отдельный «водный режим» корабля, но учебник не должен выдавать их за уже реализованную космическую разгерметизацию. Внутренние комнаты `RoomEnvironmentSystem` и кислород — другая подсистема (`SceneManager.cpp:2287–2291`), её состояние нужно соединять с пробоинами отдельным контрактом.

`PressureSystem::Update` (`src/ECS/System/PressureSystem.cpp:5–79`) объявляет `P0=101325 Pa`, `ρ=1000 kg/m³`, `g=9.81 m/s²`. Перебирает `PressureHullComponent`. Неуязвимость убирает флаги течи и пропускает урон, но наличие/отсутствие `ActiveComponent` в этом методе не проверяется. Объект без `RigidBodyComponent` или `rb.body` пропускается; глубина берётся из **Bullet**, а не из ECS. `depth=max(0,waterLevel-bodyY)`. Давление записывается в `hull.lastPressure` и считается как `P=P0+ρgh`. При превышении `hull.maxPressure` отношение `pressure/maxPressure` формирует `damagePerSecond=(ratio-1)*20`, здоровье убывает на `dps*deltaTime`, ниже нуля не идёт, утечка включается при здоровье меньше 100. Иначе здоровье медленно растёт на `repairRate*deltaTime*0.1` до 100. Это игровая кривая повреждения, а не формула прочности реального корпуса. `maxPressure` по умолчанию `202000 Pa` (`Components.h:778–786`); при 10 м глубины давление `101325+1000·9.81·10=199425 Pa`, ещё ниже порога, а около 10.3 м порог будет превышен. Если `maxPressure<=0`, код делит на него без валидации — проектная задача для конфигурационного контроля.

5.2. Почти построчный путь воды через отсек

`FloodingSystem::Update` (`src/ECS/System/FloodingSystem.cpp:5–143`) берёт отсеки, transform и связи с подлодкой. Отсек без `SubmarineComponent`, без валидного корня, сам корень или без `TransformComponent` пропускается. Это важно при создании нового корабля: одной пробоины в списке `CompartmentComponent::breaches` недостаточно; нужна правильная привязка к корню. Для каждой незапечатанной `BreachData` позиция хранится локально в отсеке. `XMVector3TransformCoord(localPos, compartmentWorldMatrix)` превращает её в world position (`:29–38`). Затем глубина пробоины `globalWaterLevel - breachWorldPos.y`. Над поверхностью расход ноль, под поверхностью вычисляется `pressure=breachDepth*9.81f`, затем `CalculateFlowRate`. В `CalculateFlowRate` (`:227–237`) площадь считается как `π·size²`, скорость как `sqrt(2·waterPressure/1000)`, расход как `area·velocity`. Именно здесь видно несогласование единиц: если `waterPressure` должен быть в паскалях, для глубины `h` нужно `ρgh`, а сейчас передаётся лишь `gh`. Получаемый расход примерно в `sqrt(1000)` раз меньше физически согласованного с такой интерпретацией. Возможно, это намеренно ослабленная игровая скорость, но тогда переменная `pressure` и комментарий о Па вводят в заблуждение; коэффициент баланса надо выделить явно. Не меняй это число автоматически в готовой сцене: течение может резко ускориться.

Например, пробоина радиуса `0.1 м` на глубине `10 м`: площадь `π·0.1²≈0.0314 м²`. Текущий код передаст `98.1`, даст `v≈0.443 м/с` и `Q≈0.0139 м³/с`. При гидростатическом перепаде `98100 Па` и той же формуле `v≈14.0 м/с`, `Q≈0.439 м³/с` до коэффициента расхода и иных ограничений. Для игры может быть разумным расход между ними, но его надо выбирать как параметр дизайна в единицах, а не получать из пропущенной плотности. **Проектное предложение:** ввести `pressureDifferencePa`, `dischargeCoefficient`, ограничение по доступному объёму/размеру шага и тесты на глубинах 0, 1, 10, 100 м.

Насос на `:73–77` откачивает `comp.pumpRate`, только если `comp.hasPower` и воды больше нуля. Чистый объём за кадр равен `(totalInflow-totalOutflow)*deltaTime`; `waterVolume` ограничивается `[0,comp.volume]` (`:84–86`). Высота воды выводится как доля заполнения, умноженная на геометрическую высоту отсека (`:91–99`). Для этого нужны `comp.volume>0` и положительная высота; в коде проверена высота, но в ветке `waterVolume/comp.volume` нет отдельной проверки ненулевого объёма. Потом создаётся `DamageDecalComponent`, если его не было, и синхронизируется с пробоинами (`:101–106`). Это перевод логического повреждения в визуальные данные для шейдера. `CreateBreachAtWorldPosition` (`:175–193`) использует обратную мировую матрицу: координата попадания мира превращается в координату отсека и сохраняется локально, поэтому маркер движется вместе с корпусом. `SealBreach` (`:198–204`) только выставляет `isSealed`; накопленная вода не исчезает и должна быть откачана отдельно.

Текущая связь насоса с сетью питания неполная. `CompartmentComponent::hasPower` по умолчанию `true` (`Components.h:692`) и сохраняется/читается как `compartment_has_power` (`SceneManager.cpp:4221`, `:4963`). По поиску в текущем `src/` нет записи из `PowerConsumerComponent::hasPower` обратно в `CompartmentComponent::hasPower`; `ShipPowerGridSystem` меняет свои consumer-поля, а не это поле. Поэтому нельзя утверждать, что отключение реактора автоматически остановит откачку каждого отсека. **Проектное предложение:** назначить насос отдельным consumer, определить поведение при частичном `powerRatio`, а вычисленное питание использовать до `FloodingSystem`. Сейчас `SceneManager::UpdateAll` с расчётом сети вызывается **после** физической стадии `FloodingSystem`; при исправлении зависимости надо осознанно перестроить порядок, иначе насос увидит состояние прошлого кадра.

5.3. Плавучесть и два противоречащих действия кода

`PhysicsSystem::Update` (`:1149–1344`) вычисляет погружение по вертикальной области тела `[objectY-height/2, objectY+height/2]` относительно поверхности. Если объект внутри отсека, вместо внешней поверхности берётся уровень воды отсека (`:1175–1202`). Для корня подлодки суммируется вода во всех отсеках. Сглаженный коэффициент погружения — смесь старого значения и нового с коэффициентом `0.3`; это уменьшает скачок силы, но вводит временную задержку и зависит от количества кадров. Выталкивающая сила считается как `fluidDensity * g * buoyancy.volume * submersionRatio * buoyancyMultiplier` (`:1271–1272`), добавочный вес воды — `totalWaterInCompartments *`

`1000 * g`, итоговая вертикальная сила — разность (`:1274–1286`). Затем есть сопротивление по скорости, ослабление вертикального drag при затоплении, пружинная стабилизация при малом затоплении и угловое демпфирование. Всё это смешивает физическое приближение с игровыми стабилизаторами: формулу можно объяснять, но коэффициенты `50`, `5`, `0.3` нельзя называть законом природы.

Важная несогласованность: комментарий и формула в `PhysicsSystem.cpp:1253–1279` говорят, что внешний вытесняемый объём корпуса при заливании воды **не уменьшается**, а вода добавляет вес. Но `FloodingSystem.cpp:116–140` при `lastCalculatedVolume>0` прямо записывает `buoy->volume = max(0,lastCalculatedVolume-totalWaterVolume)`. Следовательно, в таком состоянии вода одновременно уменьшает выталкивающую силу и добавляет вес — двойной штраф. Пока `lastCalculatedVolume<=0`, уменьшения объёма не будет, поэтому проявление зависит от того, кто и когда вычислил это поле. **Проектное предложение:** выбрать единую модель: внешний объём остаётся постоянным, а масса внутренней воды добавляется один раз; либо явно описать и проверить альтернативу для открытого корпуса. Создать диагностическую сцену с известной массой и объёмом, по шагам прибавлять 0, 1, 2 м³ воды, записывать `buoy.volume`, `waterWeight`, `netUpForce` и высоту через 10 секунд при разных FPS. Только после этого править формулы.

Ещё одна деталь: `FloodingSystem::CreateBreach` (`:148–169`) проверяет, выходит ли локальная точка за границы `localMin/localMax`, но тело ветки пустое — фактического запрета или clamp нет. Если учебный пример «стреляем в стену» создаёт дырку вне отсека, функция всё равно добавит её в `breaches`; мировой уровень воды будет вычисляться из этой точки. Следует либо явно разрешать внешнюю точку с объяснением, либо возвращать ошибку/подрезать координату. Такое исправление относится к проектному предложению, а не к текущему поведению.

5.4. Космический режим как отдельный контракт

Проектное предложение. Для космического симулятора добавь тип среды `EnvironmentMode` или отдельную конфигурацию мира. В режиме Space наружное давление задаётся в Па, гравитация берётся из локального источника, нет глобальной `waterLevel`, `FloodingSystem` и плавучесть не запускаются. Комната хранит объём в м³, количество газа/давление, температуру, соединения дверей и пробоины в вакуум. Поток газа определяется разностью давлений и геометрией отверстия, а здоровье экипажа — парциальным давлением кислорода и временем воздействия. `PressureHullComponent` в таком режиме сравнивает **разность** внутреннего и наружного давления с прочностью, а не глубину. Один и тот же `HullBreachComponent` может быть входом двум моделям, но алгоритмы и единицы должны быть разделены. Начни с простого теста: отсек 100 м³ при 101325 Па, закрытые двери, пробоина в вакуум — давление должно монотонно падать, масса газа не возрастать, а без пробоины оставаться постоянной. Реалистичные скорость звука и истечение можно добавить после проверки сохранения массы; пока это проект, не заявляй, что он уже реализован.

6. Электросеть, генератор и зависимые системы корабля

6.1. Генератор производит мощность, сеть её распределяет

В `SceneManager::UpdateAll` (`src/Scene/SceneManager.cpp:2237–2250`) сначала выключаются генераторы у уничтоженных объектов, затем вызывается `GeneratorSystem::Update` и только после него `ShipPowerGridSystem::Update`. `GeneratorSystem` (`src/Gameplay/Power/GeneratorSystem.h:13–113`) пропускает неактивную сущность, обнулив текущую выходную мощность и расход топлива, но сохранив настройки. При `deltaTime<=0` не расходует топливо и не меняет тепловое состояние. Для активного генератора он ограничивает `maxOutputMw`, `targetOutputMw` и остатки дейтерия/трития допустимыми диапазонами; проверяет наличие обоих видов топлива; при превышении `scramTemperature` ставит `isScrammed=true`, `isOnline=false`, а повторный запуск разрешается только после охлаждения до `restartTemperature`. `currentOutputMw` приближается к эффективной цели максимум на `outputRampMwPerSecond*deltaTime`, то есть не прыгает мгновенно. Затем из отношения текущей

мощности к максимуму выводятся расход топлива в кг/с и изменение температуры. Формула температурного сглаживания `1-exp(-response*dt)` даёт более устойчивый результат при разном `dt`, чем постоянное «добавь 10% каждый кадр».

`ShipPowerGridSystem::Update` (`src/Gameplay/Power/ShipPowerGridSystem.h:18-62`) собирает временные `WorkingGrid` отдельно по корню корабля. `FindGridRoot` (`:74-84`) возвращает собственный Entity, если на нём `PowerGridComponent`, иначе `ShipPartComponent::root`, иначе `INVALID_ENTITY` для локальной сети вне корабля. Это защищает от питания одного корабля генератором другого, но все потребители без корня попадают в один условный `INVALID_ENTITY` grid: эту семантику стоит учесть при добавлении независимых станций. Активный генератор добавляет `max(0,currentOutputMw)` только если `isOnline && !isScrammed`. Каждый consumer перед сбором сбрасывает `suppliedMw`, `powerRatio`, `hasPower`; неактивный остаётся обесточенным. Его `requiredMw` добавляется к demand без отрицательных значений. Если ошибочно считать `hasPower` сохранённым авторским переключателем, сброс каждый кадр будет выглядеть багом. Это вычисленная телеметрия, которую нужно заново получить от сети.

`UpdateGrid` (`:86-154`) начинает с мощности генераторов. Если `demandMw` больше `availableMw` и есть `PowerGridComponent`, вызывается разряд аварийной батареи на дефицит. Затем приоритеты `0..3` обходятся по порядку; в каждом собираются потребители с положительной потребностью и вызывается `SupplyEvenly`. Значит низшее число получит ресурс раньше. `SupplyEvenly` (`:156-189`) делит имеющееся поровну между потребителями данного приоритета; потребитель с малой нуждой насыщается, остаток перераспределяется на следующем круге. После распределения `powerRatio=clamp(supplied/required,0,1)`, `hasPower` становится true только от `0.999`. У потребителя с `requiredMw<=0` считается полное питание (`:119-123`). Телеметрия grid записывает generation, demand, supplied, output батареи, резерв, load ratio и brownout. Система электричества не знает о насосе или лампе по имени — она выставляет общий контракт consumer.

Пример. Генератор дал 8 MW, батарея недоступна. Приоритет 0: устройства требуют 3 и 2 MW — оба получают полностью, остаётся 3 MW. Приоритет 1: устройства требуют 4 и 1 MW. Первая равная доля 1.5 MW; маленькое устройство берёт только 1 MW, большое 1.5 MW, остаётся 0.5 MW. Во втором круге оно идёт большому: итог 2 MW из 4, `powerRatio=0.5`, `hasPower=false`. `suppliedMw` сети равна 8 MW, `demandMw=10 MW`, `brownout=true`. Если добавить аккумулятор, который выдаёт 2 MW, дефицит может исчезнуть на этот шаг — пока хватает энергии и не превышен предел мощности. Именно поэтому для части устройств полезно использовать `powerRatio`, а для бинарных устройств — `hasPower`. Текущий код лампы пользуется бинарным признаком (`SceneManager.cpp:2253-2267`), так что лампа погаснет и при 99% номинала.

6.2. MW, MWh и правильное время батареи

`EmergencyBatterySystem::Discharge` (`src/Gameplay/Power/EmergencyBatterySystem.h:9-23`) возвращает минимум из требуемой мощности, аппаратного лимита разряда и ограничения по запасённой энергии `batteryChargeMwh*3600/deltaTime`. Затем вычитает `suppliedMw*deltaTime/3600` MWh. Это размерно корректно: MW — скорость передачи энергии, MWh — запас; час содержит 3600 секунд. Например, батарея 1 MWh при отдаче 2 MW теоретически выдержит 0.5 часа, то есть 1800 секунд, если нет других ограничений. При `dt=1 c` расход `2/3600≈0.000556 MWh`. `Charge` симметрично ограничивает мощность surplus, лимитом зарядки и свободной ёмкостью (`:25-46`). Если в новом коде вычесть `MW*dt` прямо из MWh, ресурс сгорит в 3600 раз быстрее. Если применить этот же расход при `deltaTime=0`, батарея не должна измениться — методы уже имеют ранние проверки.

После сети `SceneManager::UpdateAll` связывает питание с отдельными приборами: свет с `PowerConsumerComponent` включается лишь при живом объекте и полном питании (`:2253-2267`); кислородному генератору выставляются `hasPower` и `broken`, затем обновляются `RoomEnvironmentSystem`, `OxygenGeneratorSystem`, `PlayerVitalsSystem`, `ShipAlertSystem` (`:2269-2292`). Обрати внимание на временную причинность: физическое затопление сейчас уже прошло до `UpdateAll`, а питание сети рассчитывается здесь. Если насос станет настоящим consumer, последовательность надо перестроить. Также `UpdateAll(0)` в Editor вызывает `ShipPowerGridSystem::Update`: её телеметрия может измениться без расхода топлива, поскольку сеть пересчитывает текущие мощности, а генератор при

нулевом времени просто возвращается. Это допустимо как preview, но при расширении системы следи, чтобы новый consumer не расходовал ресурсы в Editor.

6.3. Добавление новой системы жизнеобеспечения

Проектное предложение. Пусть нужен фильтр CO_2. Компонент содержит заданный расход воздуха и энергоёмкость; `PowerConsumerComponent.requiredMw` показывает запрос. `FilterSystem::Update` читает `powerRatio`, состояние повреждения, текущую концентрацию CO_2 и `deltaTime`, рассчитывает удалённую массу и записывает её в комнату. Помести фильтр после `ShipPowerGridSystem`, но до расчёта витальных эффектов игрока; иначе здоровье получит концентрацию до очистки. Питание фильтра сделай либо непрерывным (`effectiveFlow = nominalFlow * powerRatio`), либо пороговым с документированным порогом; для двоичного `hasPower` при небольшом brownout фильтр полностью выключится. При нулевом времени функция не должна менять массу газа. На интеграционном тесте сравни комнату с фильтром при 100%, 50% и 0% питания, затем сохранение/загрузку значения авторского расхода и отдельно вычисляемого runtime-состояния. Всё это решение дизайна; текущего `FilterSystem` в коде нет.

7. Renderer и шейдеры: от Transform до цвета пикселя

7.1. Что делает CPU перед GPU

`Renderer::RenderFrame` (`src/Render/Renderer.cpp:203–514`) получает Direct3D context, view и projection, выбранный Entity и параметры отладки. В начале обновляется контекст вида от первого лица, создаётся snapshot кандидатов видимости для фоновой frustum-проверки (`:213–222`), собирается статистика активных мешей и обнаруживаются материалы, которым нужны reflection/refraction. Метод `InspectSceneRenderFeatures` (`:34–59`) считает reflection нужным для Water/Reflection, refraction — для Glass/Water. Это означает, что дорогие дополнительные проходы запускаются по **реально активным материалам**, а не всегда. Однако сама проверка проходит по пулу мешей, значит при большом числе объектов это тоже измеряемая работа на CPU.

Далее `RenderFrame` обновляет направленный свет и environment map, рисует shadow map, при нужде reflection и refraction в отдельные render textures, при наличии editor callback — preview, потом основную сцену в backbuffer (`:263–448`). Shadow pass использует выбранные видимые комнаты. Reflection pass строит отражённую view-матрицу для плоскости `y=0`; это конкретное соглашение текущей водной сцены, поэтому для произвольной наклонной отражающей поверхности его не хватает. Refraction pass пропускает Glass/Water, чтобы текстура преломления содержала фон за ними (`:316–382`). После каждого render texture вызываются восстановление backbuffer/viewport и `ClearSRVSlots`: в D3D11 один ресурс нельзя бездумно держать одновременно как вход SRV и выход RTV. Main pass очищает кадр цветом fog или чёрным, рисует skybox с view-матрицей без переноса камеры (`:415–445`), обычные 3D объекты, sprites, частицы, отладочные объёмы и в конце first-person view model (`:447–510`). Для оружия от первого лица очищается depth buffer и задаётся отдельная near/far projection (`:516–550`), чтобы руки не исчезали за мировой стеной из-за глубины. Это визуальный приём, а не изменение попаданий: луч выстрела отдельно вычисляется из камеры/дула в мире.

`Render3DObjects` (`:556–606`) включает Z-buffer, выключает alpha blending, передаёт fog в шейдеры, привязывает shadow map, затем просит `SceneManager::RenderAllECS`. Тот (`src/Scene/SceneManager.cpp:2393–2550`) собирает свет, строит frustum, список видимых комнат и проходит по `MeshRendererComponent`. Для непрозрачного прохода он пропускает неактивные объекты, исключённые из мирового вида, неподходящие кандидаты, editor gizmo, отсутствие Transform/model/material, невидимые комнаты и объекты вне frustum (`:2434–2462`). Лишь после этих фильтров `GetWorldMatrix()` становится матрицей конкретного draw. Далее `PreparePbrDrawState` привязывает decal-данные и вызывается renderer меша. Если объект не виден, искать причину нужно последовательно: `ActiveComponent`, видовой слой, готовность модели, комната, frustum, затем шейдер. Замена текстуры не поможет, если draw вообще не дошёл до GPU.

7.2. Три пространства координат и вершинный этап

Для одного треугольника CPU берёт локальные вершины меша, `TransformComponent` даёт world matrix, камера даёт view и projection. В `PBRShaderClass::Render` (`src/Render/Shader/PBRShaderClass.cpp:143-176`) C++ транспонирует `world`, `view`, `projection` перед отправкой в VS constant buffer `b0`; `SetMatrixBuffers` делает то же (`:127-140`). Это контракт C++/HLSL о памяти матриц, а не лишнее вращение объекта. Если убрать transpose только в одном месте, модель может исчезнуть или вести себя зависимо от камеры. В `pbr.vs` (`src/Render/Shader/Vs/pbr.vs:7-12`) тот же `b0` объявляет три `matrix` в том же порядке. Затем код использует row-vector умножение:

```
float4 worldPos = mul(finalPos, worldMatrix);
float4 viewPos  = mul(worldPos, viewMatrix);
output.position = mul(viewPos, projectionMatrix);
```

Здесь `finalPos` — координата в пространстве модели, `worldPos` — в мире, `viewPos` — относительно камеры, `SV_POSITION` — координата для растеризации. Позиция `(1,0,0)` в меше сначала масштабируется и поворачивается вокруг локального начала, затем переносится в мир. Если свет имеет позицию в мире, расстояние до него надо вычитать из `worldPos`, а не из `SV_POSITION`: последняя уже зависит от камеры и перспективы. Pixel shader получает `worldPos` отдельно через `TEXCOORD5` (`pbr.vs:50-64`, `pbr.ps:80-94`). Между VS и PS rasterizer интерполирует атрибуты по поверхности треугольника; это не новый вызов C++ для каждого пикселя.

У анимированной модели до world/view/projection выполняется skinning (`pbr.vs:71-116`). Каждый входной vertex хранит до четырёх индексов костей и весов. Шейдер пропускает индекс вне `[0,128)`, неположительный вес, суммирует преобразованные позиции и направления, затем делит на общую сумму весов. При отсутствии валидных весов он использует исходную вершину, а не случайную `boneTransforms[0]`, которая могла остаться после предыдущего skinned draw (`:98-114`). Если модель «разъехалась», проверяй `boneIDs`, веса, bind pose и тот факт, что статический меш действительно пришёл без валидных весов. Нельзя тестировать skinning только на одной позе: ошибки иногда скрываются в bind pose и проявляются при повороте руки.

Нормаль — направление поверхности, не точка. При неравномерном scale простое `mul(normal,world)` даёт неверную ориентацию. `normal_transform.hlsli` строит normal matrix через кофакторы, соответствующую обратной транспонированной линейной части, нормализует нормаль и переортогонализует tangent/binormal. Вершинный шейдер вызывает `BuildWorldTangentBasis` (`pbr.vs:127-136`). На CPU тот же принцип используется в `PBRShaderClass::SetDamageDecals` для нормали трещины (`:327-335`): позиция `XMVector3TransformCoord` учитывает перенос, нормаль `XMVector3TransformNormal` — нет. Тест: поверхность с scale `(2,1,0.5)` и нормалью не по оси. Если освещение сохраняется при вращении и scale, преобразование согласовано; если блик внезапно перескакивает, сначала проверь normal/TBN, а не roughness.

7.3. Пиксельный этап: текстуры, материал и свет

`pbr.ps` (`src/Render/Shader/Ps/pbr.ps`) объявляет albedo `t0`, normal `t1`, metalness `t2`, roughness `t3`, crack `t6`, shadow `t7`, environment `t8`. C++ в `PBRShaderClass::Render` выставляет `t0..t3`, crack и environment, вызывает `DrawIndexed`, затем отвязывает `t0..t8` (`PBRShaderClass.cpp:210-245`). Это точное соответствие слотов: если HLSL читает `t8`, а C++ привязал новый ресурс к `t5`, пиксель не «догадается» о другом месте. `m_defaultNormalSRV`, `m_defaultMetalnessSRV`, `m_defaultRoughnessSRV`, `m_defaultCrackSRV` создаются как 1×1 fallback (`:91-95`) для отсутствующих карт. Albedo при `nullptr` получает предупреждение после draw (`:247-249`); это не полноценная гарантия корректного цвета, поэтому путь создания материала надо проверять отдельно.

PS берёт albedo, metalness из синего канала, roughness из зелёного (`pbr.ps:220-255`); это соответствует metallic-roughness упаковке glTF и отдельным grayscale картам, если все каналы одинаковы. Нормаль из карты переводится из `[0,1]` в `[-1,1]`, затем через TBN — в мир (`:257-268`). Вектор к наблюдателю `V=normalize(cameraPos-worldPos)` и нормаль `N` должны быть в одном пространстве. `F0` смешивает `0.04` для диэлектрика с albedo для металла (`:270-274`). Направленный свет и до четырёх локальных источников используют GGX distribution, Smith geometry и Schlick Fresnel (`:188-218`, `:282-342`). Физический смысл: NDF отвечает за распределение микрофасеток, geometry term — за

взаимное затенение, Fresnel — за долю отражения при данном угле. Диффузная часть уменьшается для металла через `(1-metalness)`, чтобы поверхность не получала одновременно весь металлический блик и полный диффузный цвет. Точечный свет затухает с нормализованным расстоянием до радиуса, spot дополнительно проверяет угол конуса (`:304–340`). Это художественная модель затухания проекта; её нельзя автоматически считать точной обратноквадратичной физикой.

Ambient и отражение окружения добавляются на `:344–381`. Sky visibility приближённо выводится из shadowFactor, так что освещённость закрытых комнат уменьшается без полноценной системы light probes. Затем идут tone mapping и gamma (`:383–386`), alpha стекла (`:391–408`), трещины (`:416–473`), подсветка выделенного объекта и fog (`:476–489`). Последовательность имеет последствия: трещина в текущем коде затемняет **уже посчитанный color** после освещения. Комментарий о «повышении roughness» есть, но вычисленная ранее переменная `roughness` после трещины не меняется и BRDF повторно не считается (`:466–472` прямо это признают). **Проектное предложение:** если нужен физически изменённый блик на повреждении, перенеси смешивание карты трещин до расчёта света, отдельно проверь влияние на metalness/roughness и сохранение визуального совпадения с пробоиной.

7.4. Как менять контракт CPU/GPU без случайной поломки

Для нового параметра сначала выбери, где он хранится: авторская настройка материала, вычисленное состояние компонента или глобальное состояние кадра. Затем выбери HLSL `register(bN/tN/sN)` и зеркальную привязку в C++. В `PBRShaderClass.h:60–93` структуры C++ соответствуют `MatrixBuffer`, `PBRParamsBuffer`, `DamageBuffer` HLSL; порядок `float3` + `float` или массивы `float4` нужен для 16-байтной упаковки constant buffer. Проверяй `sizeof` и offsets, особенно при добавлении `bool` или структуры с другим выравниванием. Если параметр per-object, обновляй буфер перед **каждым** draw; иначе второй объект унаследует состояние первого. `PreparePbrDrawState` (`SceneManager.cpp:2295–2391`) именно перед draw отправляет трещины выбранного объекта, в том числе переводит локальные decal координаты панелей в координаты batch-меша. Тестируй сперва постоянным ярким цветом без освещения, потом одной текстурой, затем комбинацией света, тени и анимации. Наконец проверь выключенный объект и следующий draw: старый ресурс в слоте может создать «призрачный» эффект даже при правильном шейдере.

8. Выстрел, Bullet raycast и `DamageEvent`: что происходит с «пулей»

8.1. Текущая модель — мгновенный луч

Слово «Bullet» в проекте имеет два возможных значения: библиотека физики Bullet и пуля оружия. Текущая ветка `WeaponSystem::TryFire` (`src/Gameplay/Combat/WeaponSystem.h:75–155`) реализует **hitscan**: в момент выстрела бросает луч в `btDynamicsWorld`, находит первое столкновение и создаёт событие урона. Отдельной ECS-сущности летящего снаряда, его позиции/скорости и физического времени полёта в этой ветке нет. Видимый muzzle flash — частица, а отладочная линия — диагностическое изображение луча; ни одно из них не определяет попадание. Это критично для будущего космического оружия: лазер или сверхскоростной выстрел может оставаться hitscan, ракета и медленный плазменный шар потребуют отдельной реализации.

Игрок и NPC используют общий механический метод `TryFire`. Владелец и оружие — разные сущности: `GetEquippedWeaponEntity` читает `EquippedWeaponComponent`, с совместимостью для старой сцены, где `WeaponComponent` лежит на самом персонаже (`WeaponSystem.h:20–29`). `UpdateTimers` один раз за симуляционный кадр сбрасывает `triggerHeld` и продвигает cooldown/reload у всех оружий (`:63–71`), поэтому нельзя дополнительно вызывать `AdvanceTimers` в каждом NPC: ускорится стрельба и перезарядка. `WeaponFiringRules::AdvanceTimers` (`src/Gameplay/Combat/WeaponFiringRules.h:9–33`) уменьшает cooldown, recoil и reload timer на `dt`; по окончании перезарядки переносит из резерва только недостающие патроны. `CanFire` (`:59–66`) проверяет конечные положительные fireRate/range, готовый cooldown, наличие патрона и отсутствие перезарядки. `CommitShot` (`:80–90`) выставляет

`1/fireRate` секунд до следующего выстрела, вычитает патрон, увеличивает счётчик, flash timer и recoil. При `fireRate=4` это `0.25 с` между разрешёнными выстрелами, не четыре вызова кадра.

8.2. Проверка стрелка, разброс и первый контакт

Вход `TryFire` — registry, стрелок, world-координата начала, направление, указатель на Bullet world, DamageSystem, debug, device и jobs. Ранние проверки `WeaponSystem.h:82–93` отклоняют отсутствующий мир, мёртвого/неактивного стрелка, некорректный луч, отсутствующее оружие, пустой магазин и неготовое оружие. Пустой магазин пытается начать механическую перезарядку. Затем `direction` нормализуется, читается `CharacterMovement::isMoving`, вычисляется множитель разброса. Две случайные выборки дают направление внутри конуса через `WeaponFiringRules::ConeDirection` (`:93–110`): радиальная выборка задаёт cos угла между центральным лучом и выстрелом, азимут — поворот вокруг оси. Код строит ортонормированный базис `right/up/forward` в мировых координатах и возвращает нормализованный результат. Подвижность и накопленная отдача увеличивают полуугол конуса. Случайность берётся **один раз на состоявшийся выстрел**, и этот же луч используется для debug, урона и разрезания корпуса (`WeaponSystem.h:102–110`); иначе линия трассера могла бы показывать одно, а пробоина появлялась бы в другом месте.

`CommitShot` вызывается **до** raycast (`:110`), поэтому и промах тратит патрон и запускает cooldown; это ожидаемая игровая семантика. После необязательной вспышки `PhysicsRaycast::CastRay` проверяет мир до `weapon->range` (`:119–131`). Сам `CastRay` (`src/Physics/PhysicsRaycast.h:72–123`) обнуляет `outHit`, требует положительную дистанцию, нормализует направление, проверяет конечность его компонент, строит точки начала и конца `origin + normalized*distance`, вызывает `world->rayTest`. Специальный callback (`:20–53`) игнорирует собственный ghost/body стрелка, объекты с тем же `userIndex` Entity и тела в `DISABLE_SIMULATION`. Результат содержит первый collision object, Entity из Bullet `userIndex`, мировую точку и нормаль попадания. Возврат false при объекте с `INVALID_ENTITY` означает: геометрический контакт мог быть, но игровой Entity не разрешён; debug при этом может показывать контакт, а урона не будет. При диагностике такого случая проверяй установку `setUserIndex` при создании RigidBody, а не сразу множитель damage.

Стена перед NPC остановит луч: `ClosestRayResultCallback` отдаёт ближайший допустимый контакт, `TryFire` не переносит урон на выбранную AI цель за стеной. Если `CastRay` вернул false, `TryFire` всё равно возвращает true — выстрел состоялся, только не было игровой цели. Если есть hit, создаётся `DamageEvent`: source = Entity оружия, target = hit Entity, damageAmount, physical type, world-space hit position/normal и breachRadius (`WeaponSystem.h:133–154`). Для панели корпуса `CanDamageHull` true только у `WeaponType::ShipCannon` (`src/Gameplay/Combat/WeaponComponent.h:79–82`). `CalculateHullDamage` оценивает, дойдёт ли здоровье панели до порога; при разрешённом D3D device и возможности пробоины вызывается `MeshCuttingTest::CutByRayAsync`. Его возвращаемое bool означает принятую/запущенную операцию разрезания, не то, что новый меш уже готов. `meshCutOperationId` связывает runtime-операцию с будущей ремонтной логикой. Обычный ручной выстрел по броне может показать искру, но не снимать structural HP.

8.3. Когда урон реально применяется

`DamageSystem::AddDamageEvent` (`src/Gameplay/Combat/DamageSystem.h:15–18`) лишь добавляет копию в `std::deque`. `DamageSystem::Update` (`src/Gameplay/Combat/DamageSystem.cpp:104–371`) проходит очередь, проверяет валидную цель, положительный урон, активность. Тактическая реакция NPC на попадание может зарегистрироваться до проверки `InvulnerableComponent`; затем неуязвимость отменяет уменьшение здоровья и пробоины (`:123–194`). Для игрока меняется `PlayerComponent::health`, для NPC без общего `DamageableComponent` — `NPCComponent::health`, для панели корпуса — её собственное здоровье/пробоины, для остальных — `DamageableComponent`. По окончании очередь очищается (`:371`). Если событие поставлено **до** вызова `m_damageSystem.Update` в `ApplicationClass::Frame:1549`, оно будет применено в этом кадре. Выстрел игрока и NPC через `WeaponSystem::Update` вызывается позже, на `:1757–1769`, значит его событие обычно дождётся обработки следующего кадра. Это небольшой, но реальный сдвиг: UI, который читает здоровье сразу после `TryFire`, увидит прежнее число. Если

нужна обработка в тот же кадр, меняй порядок или добавляй вторую строго определённую фазу обработки, а не вызывай `DamageSystem::Update` случайно из оружия.

Для `HullPanelComponent` (`DamageSystem.cpp:215–328`) capability «может повредить броню» и факт созданной пробоины различны. Если capability false, рождается только metal impact. Иначе здоровье уменьшается на эффективный урон. Если `canBreachHull` true, создаётся новый `HullBreachComponent` с привязкой к панели и комнате, мировым hit, радиусом, локальной точкой и нормалью; добавляются `DecompressionSourceComponent`, `RepairableComponent`, ссылка на mesh cut operation, `ActiveComponent`; пишется `WorldEventType::BreachOpened`, создаются decal и частицы. Если порог пробоины ещё не достигнут, система только отмечает визуальные стадии повреждения при пересечении 20%/50%. Это не тот же список `CompartmentComponent::breaches`, который использует водный `FloodingSystem`; для подводной течи нужен явный мост между типами повреждений. Иначе можно увидеть красивую дырку в панели без роста `waterVolume`. В космической модели `DecompressionSourceComponent` — подходящий вход для атмосферы, но проверяй конкретный код потребителя перед утверждением о результате.

8.4. Как добавить настоящий летящий снаряд

Проектное предложение. Создай `ProjectileComponent` с owner/weapon Entity, скоростью в м/с, направлением, временем жизни, радиусом столкновения, типом урона и уникальным shot ID. При выстреле выполняй всё то же `CanFire`/`CommitShot`, но вместо немедленного raycast создавай projectile Entity. На каждом **фиксированном физическом шаге** перемещай снаряд от `previousPosition` к `nextPosition=previousPosition+velocity*fixedDt`; делай swept ray/sphere cast вдоль **всего сегмента**, чтобы быстрая пуля не проскочила тонкую стену между кадрами. Игнорируй owner и собственный collider, как делает `IgnoreEntityRayCallback`. При первом контакте сформируй тот же `DamageEvent` и уничтожь/деактивируй снаряд после обработки. Жизненный цикл должен учитывать попадание, максимальную дальность, истёкшее время, выгрузку сцены и save/load политику: сохранять ли пули на лету или стирать их при загрузке. Для ракеты добавь поворот к цели и взрывной радиус; для энергетического шарика — расход энергии и визуальный след. Тестируй одну стену между стрелком и целью при 30/60/144 FPS и скорость, за шаг проходящую больше толщины стены. Только после такого теста можно говорить о реальных «снарядах» в коде.

9. Фоновые задачи и ассеты: когда результат готов к использованию

9.1. Две очереди `JobSystem`

`JobSystem` (`src/Core/JobSystem.h`) содержит очередь заданий worker threads и отдельную очередь завершений для главного потока. `Initialize(workerCount)` (`:31–59`) требует положительное число работников, останавливает прежние потоки, очищает флаг остановки и создаёт worker threads. В `ApplicationClass` вызывается `m_jobSystem.Initialize(4)` (`src/Core/ApplicationClass.cpp:558`). `SubmitJob` (`JobSystem.h:61–72`) под mutex отклоняет работу, если система остановлена, иначе копирует `std::function` в очередь и будит один поток. Начальное значение `stop=true` (`:158–161`) не даёт тихо поставить задачу до инициализации, когда некому её исполнить.

`WorkerLoop` (`:124–150`) ждёт задание или остановку, извлекает job, увеличивает `activeJobs`, отпускает mutex и исполняет callback. После выполнения уменьшает `activeJobs` и, если очередь пуста, будит ожидающий `Wait`. Важно: `job()` не обёрнут в `try/catch`; `try/catch` в `Initialize` защищает создание потоков, а не любое пользовательское задание. Необработанное исключение внутри worker не превращается в `AssetState::Error` автоматически и может завершить процесс. **Проектное предложение:** ловить исключение на границе worker, передавать диагностическое состояние на главный поток и гарантировать уменьшение счётчика активных заданий. Но при проектировании конкретной задачи лучше возвращать структурированный результат с `success/error`, как делает загрузчик ассетов, и не бросать из обычной ошибки файла.

`SubmitMainThread` (`:74–78`) складывает callback в другую очередь под отдельным mutex. `ExecuteMainThreadJobs` (`:80–94`) быстро меняет очередь на локальную, отпускает блокировку и

выполняет jobs уже в вызывающем потоке; `ApplicationClass::Frame` делает это на `:1228`. Поэтому worker может читать и готовить CPU-данные, а создание D3D buffers и изменение ECS, зависящее от текущей сцены, нужно делать в главном потоке после передачи результата. `Wait` (`:96–102`) ждёт только worker-очередь и `activeJobs==0`; он **не выполняет** `ExecuteMainThreadJobs`. Если worker завершился и положил callback в главную очередь, `Wait()` может вернуться до видимого результата. Это частый источник ошибки «загрузка закончилась, но меша нет».

`Shutdown` (`:104–118`) выставляет `stop=true`, будит всех, присоединяет workers. Worker выходит лишь когда `stop && jobs.empty`, поэтому уже поставленные задания будут доработаны. В `ApplicationClass::Shutdown` вызывается `m_jobSystem.Shutdown()`, затем `ExecuteMainThreadJobs()`, а позже `m_assetLoadSystem.Shutdown()` (`ApplicationClass.cpp:598–639`): готовые callback успевают примениться, пока asset system жива. Если поменять порядок удаления объектов, lambda, захватившая `this` загрузчика, может обратиться к уже уничтоженному экземпляру. Для нового async-сервиса заранее запиши, кто владеет объектом, сколько живёт callback и как отменяется работа при `SceneManager::Clear`. `Clear` отдельно сбрасывает `m_asyncShipImport` и использует shared state, чтобы поздний результат не вошёл в новую сцену (`SceneManager.cpp:2128–2132`).

9.2. `AssetLoadSystem`: четыре перехода состояния

`AssetState` (`src/Assets/AssetsLoadSystem.h:15–22`) задаёт `Unloaded`, `Loading`, `CpuLoaded`, `Ready`, `Error`. `RequestMeshLoad(path)` (`:103–157`) первым делом ищет запись в `m_meshes`. Если она есть и не в Error, повторный запрос ничего не запускает; если была Error, старые GPU-буферы освобождаются, запись стирается и тот же путь можно попробовать снова после исправления файла. Новая запись получает `Loading` и tracker прогресса. Worker вызывает `LoadMeshFromFile`, а его результат передаёт в main-thread callback: при ошибке ставятся `Error` и текст, при успехе CPU-меш переносится в asset и состояние становится `CpuLoaded` (`:125–147`). Если `SubmitJob` отклонён, метод сразу ставит Error (`:150–155`). Такой автомат состояний лучше не сокращать до одного bool `loaded`: различаются чтение файла, подготовка геометрии, создание GPU и ошибка.

`LoadMeshFromFile` (`:411–...`) использует Assimp с triangulation, generated normals, tangent calculation, объединением одинаковых вершин и `aiProcess_PreTransformVertices` (`:420–429`). Это CPU-этап: он читает формат модели и формирует массивы, которые можно безопасно подготовить вне D3D context. Флаг pretransform означает, что данный быстрый путь превращает узловые преобразования в вершины; для skinned иерархий нужен отдельный проверенный путь импорта, не следует переносить сюда анимацию без анализа костей. `AssetLoadSystem::Update(device)` (`:158–178`) на главном потоке ищет `CpuLoaded` и вызывает `CreateGpuMesh`. Тот требует device и непустые vertices/indices, делает D3D vertex и index buffers, а после успеха очищает CPU-массивы (`:354–409`). Если index buffer не удалось создать, освобождает уже созданный vertex buffer (`:396–400`), чтобы не оставить половину GPU-ресурса. Статус становится `Ready` или `Error`.

`BindMeshAsset` (`:255–...`) отказывается от ресурса, пока нет `Ready` и обоих GPU buffers. Это граница между «файл прочитан» и «можно рисовать». При медленной загрузке renderer не должен пытаться читать `cpuData.vertices` как D3D buffer. В `Renderer.cpp:705` этот путь привязывает готовый async mesh для shadow pass; у `SceneManager::RenderAllECS` есть проверка особого пути асинхронного меша, которая при наличии spot light возвращается к полному renderer, поскольку быстрый путь передаёт лишь point-light данные (`SceneManager.cpp:2467–2470`). Значит «готовый GPU buffer» ещё не означает «полностью функциональный материал во всех проходах»; проверяй каждый render pass.

Практический тест загрузчика: запросить существующий меш дважды и убедиться, что загрузка не дублируется; запросить отсутствующий путь, увидеть Error и текст; создать файл на том же пути и повторить запрос; получить Loading → CpuLoaded → Ready, где `Ready` наступает только после main-thread pump и GPU Update. Затем закрыть приложение при активной загрузке: порядок Shutdown должен завершить work и освободить ресурсы без позднего обращения к уничтоженным объектам. Поскольку результат асинхронен, проверка `Ready` сразу после `RequestMeshLoad` не является тестом ошибки — это нормальная задержка.

10. Как добавлять систему, чтобы она стала частью игры

10.1. Контракт системы до первой строки C++

Для новой механики составь короткую карточку: «какая сущность и компонент владеют настройкой», «какие данные вычисляются каждый кадр», «в каких единицах поля», «кто обновляет и когда», «кто читает результат», «что сохраняется», «что уничтожается», «что происходит при паузе и загрузке». Например для аварийной сирены авторские `volume` и `enabled` можно хранить в компоненте, а вычисляемое `isAudible` зависит от питания, повреждения, расстояния до игрока и активного режима тревоги. Если просто добавить `SirenSystem::Update` в конец `Frame`, она может прочесть старое питание, а звук будет обновляться после UI или после освобождения сцены. Правильное место определяется входами и потребителями, а не алфавитным названием функции.

Сначала найди ближайший существующий путь. Для оборудования это цепочка `GeneratorSystem` → `ShipPowerGridSystem` → конкретный consumer → `RoomEnvironmentSystem`/HUD. Для повреждения — `WeaponSystem::TryFire` → `DamageEvent` → `DamageSystem` → `HullBreachComponent`/decal/particle → ремонт. Для визуала — ECS `Transform` + mesh + material → culling → shader buffers → VS/PS. Для ассета — запрос → worker CPU → main thread → GPU Ready → render pass. Использование готового потока данных сокращает число новых соглашений. Но нельзя насильно использовать `PressureSystem` для вакуума: среда и единицы там другие.

10.2. Минимальный полный вертикальный срез

Проектное предложение. Возьми небольшую функцию «аварийная лампа у пробоины». На сцене должны быть корабль, панель, пробоина, лампа и источник питания. `DamageSystem` открывает пробоину; система тревоги читает активные `HullBreachComponent` и ставит сигнал; consumer лампы получает мощность; визуальный компонент меняет цвет/эмиссию; HUD показывает аварийную комнату; сохранение/загрузка возвращает авторское расположение и состояние пробоины, а runtime телеметрия пересчитывается заново. Тест из одного события проверяет всю цепь. Если лампа не горит, измерь на каждом звене: создан ли breach Entity, какой у него `room`, есть ли `ActiveComponent`, какой `requiredMw`, `suppliedMw`, `hasPower`, дошёл ли объект до `RenderAllECS`, что лежит в emissive constant buffer. Так можно объяснить конкретную причину, а не переписать систему целиком.

При добавлении компонента с `Entity`-ссылками обнови SaveScene и remap, при добавлении GPU ресурса — инициализацию, освобождение и состояние после потери/пересоздания устройства, при добавлении Bullet тела — порядок удаления constraints/body, при добавлении фоновой задачи — lifetime capture и main-thread commit. Если поле влияет на геймплей и рендер, не позволяй шейдеру быть единственным хранилищем факта: он должен показывать состояние, которое есть в ECS. Если поле выводится из других каждый кадр, обычно не надо сохранять его как авторскую истину; при загрузке сначала восстанови входы, затем пересчитай выход. Если порядок зависит от предыдущего результата, зафиксируй его рядом с вызовом в `Frame` или `UpdateAll`, чтобы будущий рефакторинг не переставил его «для красоты».

10.3. Набор проверок перед большим рефакторингом

Перед переписыванием системы сделай маленькую сцену с известными числами. Для матриц — одна вершина и один объект с scale/rotation/translation; для физики — одно тело с заданной массой и импульсом; для давления — глубина 0/10/20 м; для сети — две ступени приоритета и батарея с известными MW/MWh; для рендера — один меш без normal map, один с ней и один с неравномерным scale; для сохранения — объект без меша с ссылкой на другой Entity. На каждом опыте запиши вход и ожидаемый выход в единицах. Это не «тест ради теста», а способ зафиксировать смысл кода, который ты собираешься переучить. Особенно полезно сравнивать один и тот же сценарий при 30/60/144 FPS, Editor/Play и до/после Save/Load: эти три оси обнаруживают неверное время, скрытую зависимость от порядка и недосериализованные данные.

Для работы в перерывах достаточно читать один поток за раз: `Frame` показывает **когда**, компонент — **где лежит состояние**, система — **как меняется**, serializer — **что переживает загрузку**, renderer — **как это видно**. Пройдя этот путь для одного объекта от создания до удаления, ты узнаешь больше о движке, чем после заучивания названий всех классов. В следующем шаге выбери конкретную фичу космического режима и заполни эту же цепочку без пропусков.

11. Атмосфера комнат: уже работающая игровая модель и её границы

`RoomEnvironmentSystem::Update` (`src/Gameplay/Ship/RoomEnvironmentSystem.h:15–45`) уже реализует часть эффекта разгерметизации. Это важно отделить от проектного предложения о более физической модели газа. При `deltaTime<=0` метод ничего не меняет; положительное время ограничивает `0.10 с` за вызов. Для каждой активной `RoomComponent` он сбрасывает текущую `leakRate`, считает персонажей по `objectsInRoom` с `CharacterController` и уменьшает `room.oxygen` на `count*0.01*dt`, ограничивая результат `[0,1]`. После этого вызываются `ApplyBreaches` и `EqualizeConnectedRooms`. `RoomComponent::pressure` и `oxygen` здесь — **нормализованные доли 0..1**, а `temperature` по умолчанию `20` (`src/ECS/Components/Components.h:1012–1021`); не подставляй `room.pressure=1` как один паскаль в формулу `pV=nRT`.

`ApplyBreaches` (`RoomEnvironmentSystem.h:57–105`) перебирает `HullBreachComponent`, пропускает запечатанную пробоину или пробоину без комнаты. Для активной комнаты обеспечивает наличие `DecompressionSourceComponent`, в том числе у старого сохранения. Отношение радиуса к `0.25` ограничивается `[0.25,4]` и умножает `breach.leakRate`; затем `lostFraction=clamp(effectiveLeak*dt,0,0.95)`. Давление и кислород умножаются на `(1-lostFraction)`, температура приближается к `outsideTemperature`, а `room.leakRate` хранит текущую интенсивность. По прежнему давлению, радиусу действия и расстоянию `ApplyPlayerSuction` (`:107–160`) добавляет игроку внешнюю скорость в направлении пробоины через `CharacterExternalVelocitySystem`. В текущем коде этот suction применяется только к `PlayerGameTag` из `objectsInRoom`, предметы с `RigidBodyComponent` пока не затрагиваются; этот ограниченный эффект не следует описывать как физическое истечение газа, воздействующее на любые тела.

`EqualizeConnectedRooms` (`:162–255`) проходит пары из `connectedRooms` только один раз (`roomEntity >= connectedRoomEntity` пропускается), ищет дверь между комнатами и берёт её `GetEffectiveFlowRate`. При отсутствии двери для уже объявленной связи множитель равен 1. Для кислорода, давления и температуры используются собственные скорости `0.35`, `0.55`, `0.14`; `LimitedFlow` берёт разность значений, умножает на `rate*doorFlow*dt` и ограничивает половиной текущей разности. Сначала собираются `RoomTransfer`, потом изменения применяются к обеим комнатам симметрично. Это устраняет зависимость расчёта потока от порядка записей в `unordered_map`, хотя последующий clamp может влиять на точное сохранение суммы в крайних случаях. Объём комнаты в этих формулах не учитывается: маленькая и большая комнаты с одной и той же разностью `pressure` обмениваются одинаковой долей. Поэтому это игровая модель выравнивания нормализованных показателей, не сохранение массы газа по м³. **Проектное предложение:** при переходе к физической атмосфере хранить количество газа/массу, объём и температуру, выводить давление из состояния, переносить массу между комнатами равными и противоположными приращениями, а наружный отток считать отдельно. На первой итерации сохраняй существующий интерфейс HUD и NPC как нормализованные производные величины, но источник правды сделай размерным.

Связь с выстрелом такова: `DamageSystem` создаёт `HullBreachComponent` и `DecompressionSourceComponent` (`src/Gameplay/Combat/DamageSystem.cpp:241–276`), затем в `SceneManager::UpdateAll` атмосфера видит пробоину и уменьшает показатели. Водный `FloodingSystem` читает другой тип — `CompartmentComponent::breaches` — и потому не получает её автоматически. Ремонт, который выставит `breach.sealed`, останавливает атмосферную утечку на следующем `ApplyBreaches`, но воду в отсеке откачивает отдельный насос. Это две разные цепочки, и именно на их границе удобнее всего строить осознанное переключение «космос / вода».

12. Секции корабля, масса и разрушаемые соединения

`SubmarineSystem::AddCompartmentToSubmarine` (`src/Gameplay/Ship/SubmarineSystem.h:13–79`) проверяет у корня `TransformComponent` и `SubmarineComponent`, создаёт Entity отсека, задаёт его Transform, геометрические `localMin/localMax`, `CompartmentComponent`, ссылку `SubmarineComponent::submarineRoot`, `ActiveComponent`, добавляет ID в `rootSub->allCompartments` и выставляет `needsPhysicsUpdate`. Здесь список отсеков — часть владения корня: если компонент есть, но ID не добавлен, `FloodingSystem` ещё может найти отсек через его связь, но суммарная масса/объём и сила корня будут неполными. У метода есть параметры `localOffset` и `compartmentMass`, но текущий код ставит `Transform.position` равной позиции корня, `localOffset` кладёт в `comp.localCenter`, а `compartmentMass` не использует. Не предполагай, что переданная масса и смещение сами превращаются в отдельное физическое тело отсека: это следует проверять визуально и в Bullet для конкретной сцены. **Проектное предложение:** если API обещает смещение и массу, либо задействовать их последовательно, либо переименовать/убрать неработающие параметры и покрыть сценой с двумя отсеками на разных позициях.

`RecalculateSubmarineMass` (`:82–133`) суммирует `comp.volume` по `allCompartments`, добавляет `15 м³` условного корпуса, затем считает нейтральную массу как `fluidDensity*totalVolume` и выбирает `0.95` от неё для небольшого запаса плавучести. Результат пишет в `rootRb->mass`, `rootBuoy->volume`, `rootBuoy->lastCalculatedVolume` и кэши `rootSub`. Это **водная** модель и один из источников `lastCalculatedVolume`, из-за которого описанное выше двойное снижение плавучести при затоплении реально может активироваться. `RefreshSubmarinePhysics` (`:136–175`) сохраняет линейную и угловую скорости, удаляет прежнее тело из Bullet, пересчитывает массу, создаёт новое через `PhysicsSystem::CreateRigidBody`, возвращает скорости. Для динамического корабля восстановление скорости избегает остановки после редакторского изменения отсека, но момент такого пересоздания всё равно может влиять на constraints и контакты — проверяй их после перестройки.

`UpdateSubmarine` (`:201–223`) не пересчитывает всё каждый кадр. `NeedsPhysicsUpdate` проверяет явный флаг, изменение количества отсеков или их размеров через кэш (`:177–199`). После refresh флаг снимается, число отсеков и размеры кэшируются. Если новая система меняет физическую геометрию, она обязана выставить `needsPhysicsUpdate`; если меняется только уровень воды, геометрию пересоздавать каждый кадр не нужно. В `ApplicationClass::UpdatePhysics` метод вызывается только для корневых `SubmarineComponent` до `PhysicsSystem::Update` (`src/Core/ApplicationClass.cpp:2286–2295`). Такая позиция позволяет Bullet увидеть новую форму и массу при следующем шаге.

Есть два вида соединений. `JointConnectionSystem::Update` (`src/ECS/System/JointConnectionSystem.cpp:5–40`) находит `JointConnectionComponent`: если target выбран, joint отсутствует и он не сломан, создаёт связь; если target сброшен — удаляет; если Bullet отключил существующий joint — отмечает `isBroken` и снимает его. `CreateJoint` (`:42–82`) требует оба `RigidBodyComponent` и живые Bullet bodies, создаёт `btFixedConstraint` в локальных pivot A/B, задаёт порог импульса `breakForce`, 100 итераций solver, регистрирует constraint в мире с выбранной настройкой столкновений между связанными телами. Если world отсутствует, выделенный joint удаляется. Это пример полного пути «авторская ссылка → runtime constraint»; только сериализовать указатель `joint` нельзя.

`SubmarineJointSystem::CreateJoint` (`src/ECS/System/SubmarineJointSystem.cpp:5–85`) похож, но хранит constraint на отдельной ECS-сущности `SubmarineJointComponent`, передаёт владение компоненту до регистрации в Bullet, чтобы исключение при создании Entity не оставило неуправляемый constraint. Его `Update` вызывается **после** Bullet шага (`ApplicationClass.cpp:2297–2298`) и читает `joint->getAppliedImpulse()` как `currentStress`; если `!joint->isEnabled()` или импульс больше `breakForce`, вызывает `ResetJoint`, отмечает `isBroken` и `isDetached` у обоих отсеков (`SubmarineJointSystem.cpp:87–123`). `RemoveJointsForEntity` сначала собирает ID связей в vector, затем удаляет их вне обхода (`:135–148`) — ровно тот безопасный способ изменения ECS-пула, который нужен при удалении сцены. Когда пробоина или взрыв должны разрывать секцию, не удаляй тело прямо из callback контакта: создай событие, дай физическому шагу закончиться, затем удали constraint и только после этого тело.

Дополнительные системы DX Framework: от нажатия клавиши до события и эффекта

Предыдущие главы подробно разобрали кадр, ECS, NPC, физику и основные сети корабля. Здесь мы пройдём ещё шесть подсистем текущего репозитория. Для каждой важно одно и то же: где хранятся постоянные данные, где живёт временное состояние, в какой момент кадра оно меняется и какой результат увидит игрок. Путь к методу указан для поиска в исходнике; точный номер строки после правок может сдвинуться.

1. Ввод игрока: от физической клавиши к игровому действию

1.1. Почему ввод разделён на два слоя

`InputClass` знает про клавиши и кнопки мыши. `GameplayInputSystem` в `src/Gameplay/Player/GameplayInputSystem.cpp` знает, что клавиша W означает `MoveForward`, R — `Reload`, E — `Interact`, а V — `ToggleView`. Это различие даёт возможность поменять привязку без изменения кода движения. Движение не должно спрашивать `IsKeyDown('W')` напрямую: иначе новая настройка клавиш подействует только на часть механик. В конструкторе `GameplayInputSystem` вызывается `SetBindings` для каждой команды. У `Fire` сейчас две привязки: левая кнопка мыши и H; у `PrimaryMouse` тоже левая кнопка. Это не ошибка само по себе, но контекст Editor/Play должен решить, кому достанется нажатие.

`Sample(input, playMode, gameplayEnabled)` проходит все `GameAction`. Для каждого действия вычисляет `enabled = playMode && (gameplayEnabled || IsSessionAction(action))`. Значит обычные действия заблокированы вне Play и при отключённом gameplay, а команды сессии вроде паузы и вида могут продолжать работать. Затем цикл перебирает привязки и складывает их логическим ИЛИ: если нажата хотя бы одна, `down=true`. Когда добавишь переназначение клавиш, не забудь проверять конфликт двух действий на одной физической кнопке в одном контексте. Сейчас такая общая кнопка существует намеренно, поэтому простое «запретить любой дубль» было бы неверной валидацией.

1.2. `pressed`, `down`, `released` на трёх кадрах

Ключевые строки `state.pressed = down && !state.down; state.released = !down && state.down; state.down = down` сравнивают новое наблюдение со старым. На первом кадре без клавиши получаем `(false,false,false)`. При нажатии: новое `down=true`, старое `state.down=false`, значит `pressed=true`, `released=false`. Пока клавиша удерживается, старое `down=true`, новое тоже true: `pressed=false`, `down=true`. В кадре отпускания новое false и старое true: `released=true`. Это маленький конечный автомат, а не три независимых сигнала.

Для движения нужна непрерывная проверка `Down(MoveForward)`: персонаж идёт всё время удержания. Для переключения камеры нужен `Pressed(ToggleView)`: иначе камера будет менять режим каждый кадр и почти сразу вернётся назад. `UpdateSession` именно так обрабатывает `ToggleView`, `TogglePause`, ускорение и замедление времени. `UpdateInventory` по `Pressed(Slot1..Slot3)` выбирает слот и по `Pressed(DropWeapon)` передаёт позицию и направление камеры в `WeaponPickupSystem`. `static_assert` там связан с тремя слотами инвентаря: если расширишь его до четырёх, компилятор напомнит обновить список команд. Это пример полезной проверки архитектурной связи во время сборки.

Упражнение. Нарисуй таблицу для клавиши R на четырёх последовательных кадрах: не нажата, нажата, удерживается, отпущена. Затем проследи, почему `RequestReload` должен запускаться на `Pressed`, а автоматический огонь — использовать `Down` либо отдельный флаг спуска. При потере фокуса окна проверь, что `InputClass` сбрасывает состояния: иначе старое `down=true` может оставить персонажа движущимся без руки на клавиатуре.

2. Триггерные зоны: геометрия, память пересечения и действие

2.1. Зона не является Bullet телом

`TriggerZoneSystem.h` прямо описывает point-based вычисление. `Update(registry, deltaTime, events)` сначала накапливает собственное время, затем собирает `TriggerZoneComponent + TransformComponent` в список зон. Вторым проходом собирает кандидатов актёров по Transform и классификации типа. Оба списка сортируются по `Entity`. Это даёт устойчивый порядок событий даже если внутренний порядок pools меняется. Проверка пересечения идёт по координате актёра и форме зоны, а не по контакту Bullet ghost object. Поэтому видимый меш, физический collider и зона могут не совпадать; debug отображение формы зоны обязательно для авторской сцены.

На бумаге возьми круг радиуса 2 м в точке `(5;0;5)` и игрока `(6;0;5)`. Разница `(1;0;0)`, квадрат расстояния 1, квадрат радиуса 4: игрок внутри. Для игрока `(8;0;5)` квадрат расстояния 9, он снаружи. Проверка квадрата избавляет от `sqrt` и сохраняет тот же ответ при положительном радиусе. Текущий `ContainsPoint` поддерживает условия формы из компонента; при изменении способа расчёта сравни граничные точки ровно на поверхности. Флаг «внутри» относится к одному актёру и одной зоне, а не ко всей сцене.

`TriggerZoneSystem` хранит прошлые пересечения, cooldown и one-shot состояние в своём `m_runtime`, а не в авторском компоненте. Поэтому `Enter` означает переход «в прошлом кадре снаружи, сейчас внутри», `Stay` — «внутри снова», `Exit` — «был внутри, теперь снаружи». `Reset` очищает эту память при начале нового эпизода. Если удалить зону, система удалит её временную запись без выдуманного Exit: исходной зоны уже нет. Память состояния важна: без неё каждое нахождение внутри выглядело бы новым входом, и дверь либо HUD срабатывали бы десятки раз за секунду.

2.2. От события к команде

`TriggerActionSystem.h` получает `EventQueue`, ищет в `GameEvent` вариант `TriggerZoneEvent` через `std::get_if` и читает `TriggerActionComponent` у зоны. Флаги `executeOnEnter`, `executeOnStay`, `executeOnExit` решают, на какой переход реагировать. Затем `Execute` выбирает цель: явный `action.target`, иначе `event.linkedEntity`. При выходе и `invertOnExit` логическое значение меняется на противоположное. Например, зона входа активирует свет, зона выхода деактивирует тот же свет без второго компонента.

Доступные действия в текущем коде: сообщение HUD, включение/выключение Entity через `ActiveComponent`, переключение активности и установка `DoorComponent::isOpen`. При `SetDoorOpen` эта система меняет желаемое состояние двери, но физический проход зависит от дальнейшего обновления DoorSystem, а питание и анимация могут препятствовать фактическому открытию. Событие не должно обещать, что collider уже освобождён. `TriggerActionFeedback` держит текст и оставшееся время для HUD; `Update` уменьшает таймер по `deltaTime`, а `GetActivationsThisFrame` помогает отладке.

Практика: сделай зону перед дверью, `SetDoorOpen=true` на Enter и `invertOnExit=true` на Exit. Выведи по кадрам положение игрока, состояние inside, тип события, `door.isOpen`, питание двери и положение collider. Пройди границу медленно, затем телепортируй игрока через неё, поставь паузу и перезагрузи сцену. Так обнаруживаются повторный Enter, неверный cooldown и оставшаяся после reload runtime память.

3. Инвентарь: одна Entity оружия и три слота

`InventorySystem.h` отделяет данные `InventoryComponent` от операций `PickupWeapon`, `SelectSlot`, `DropSlot`, `SwapSlots`, `Sanitize` и `SyncEquippedWeapon`. Слот хранит ссылку на Entity предмета; ссылка должна указывать на живой объект и на допустимого владельца. `EnsureInventory` создаёт компонент, если его ещё нет. `PickupWeapon` не должен просто нарисовать винтовку в руке: он должен занять слот, назначить владельца, убрать или изменить представление предмета в мире и при необходимости

экипировать его. Если операция неуспешна из-за полного инвентаря, состояние владельца и world pickup должно остаться прежним.

`WeaponPickupSystem.h` связывает взаимодействие и инвентарь: `Update` обрабатывает запросы контакта/подбора, `DropActive` возвращает выбранное оружие в мир, при настройке может создать физику для брошенного предмета. `ConsumeInventoryChanged` возвращает и сбрасывает флаг изменения — это сигнал для HUD или другого потребителя, чтобы он не перечитывал всё без причины. `InventoryOperation` и статусы нужны не только программе: игроку полезно объяснить «слоты заняты» или «предмет уже у другого владельца», а тесту — сравнить конкретный отказ.

Инвариант: одна Entity оружия не должна одновременно быть предметом двух инвентарей. На сцене с игроком A и NPC B помести оружие W в слот A, попытайся поднять его B, затем выброс A и подними B. На каждом шаге запиши владельца W, слот, активный предмет и world представление. Если после reload W встречается дважды, ищи ошибку сериализации ссылок или восстановления производного визуала. `SanitizeSlots` проверяет живость и валидность ссылок — полезная защита после удаления Entity, но она не заменяет правильный lifecycle в операции подбора.

4. Анимация: игровое время, поза и вид от первого лица

`AnimationSystem::Update` в `src/Animation/AnimationSystem.h` делает два прохода по `AnimationComponent`. Сначала обновляет мировых персонажей. Синхронизируемые first-person копии пропускаются, чтобы их не опередил произвольный порядок Entity в pool. Если `isPlaying=false`, нет меша, модели или клипов, текущая Entity пропускается. Для персонажа `AssimpModelClass::UpdateAnimation(animComp, deltaTime)` является единственным местом накопления времени клипа; двойное прибавление `dt` в системе ускорило бы анимацию. В редакторском socket preview пауза даёт `UpdateAnimation(...,0)`: поза сэмплируется в установленное время без хода часов.

Вторым проходом first-person представление получает `owner`, читает его анимацию и копирует нужные настройки скорости и фаз. Так руки в камере и мировой персонаж используют согласованную позу. Когда мир уже обновлён, fp-копия может брать его текущее время; если сделать наоборот, оружие в руках и ствол на мировой модели разойдутся на один кадр. Для NPC это критично: `NpcRangedCombatSystem` строит мировую позицию дула из кости и Transform. Если вызвать стрельбу до обновления костей, математически правильный `muzzlePosition` будет принадлежать прошлой позе. Случай «дуло у ног» часто является ошибкой пространства или порядка матриц, а не логики AI.

Анимация показывает действие, но не обязана быть владельцем игрового результата. `RepairSystem` хранит прогресс работы, а не кадр клипа ремонта. WeaponSystem списывает патрон и создаёт луч, а не цвет пикселя muzzle flash. Если добавить animation event, определи, что именно оно запускает и как исключить повтор события при зацикливании или пересэмплировании клипа в редакторе. Проверь: пауза игры останавливает gameplay действие; пауза preview меняет только редакторскую позу; изменение FPS не меняет длительность шага атаки.

5. Атмосфера комнат: простая игровая модель и физические пределы

Текущий `RoomEnvironmentSystem::Update` в `src/Gameplay/Ship/RoomEnvironmentSystem.h` уже уменьшает кислород, давление и температуру при пробоине. Это важно назвать точно: `PressureSystem` из другого каталога описывает внешнюю водную глубину, а `RoomEnvironmentSystem` — упрощённую атмосферу комнат. В начале update неположительный `dt` прекращает работу, положительный ограничивается 0,10 с. Затем `CharacterExternalVelocitySystem::BeginAtmosphereFrame` начинает новый набор внешних скоростей. Для активной комнаты считается число `CharacterController` среди `objectsInRoom`, после чего `oxygen -= count*0.01*dt` и ограничивается [0,1]. Это игровая доля, не масса кислорода в килограммах.

`ApplyBreaches` обходит незапечатанные `HullBreachComponent`, проверяет комнату и добавляет недостающий `DecompressionSourceComponent` для старых сохранений. `effectiveLeak =`

`max(0,leakRate)*clamp(radius/0.25,0.25,4)`. За шаг `lostFraction=clamp(effectiveLeak*dt,0,0.95)`; давление и кислород умножаются на `1-lostFraction`. Если `effectiveLeak=0.2/c` и `dt=0.1c`, то доля потери 0.02, давление 1.0 станет 0.98; на следующем таком шаге 0.98 станет 0.9604, а не 0.96. Поэтому кривая похожа на экспоненциальный спад. За 10 шагов получится `0.98^10≈0.817`, если другие системы ничего не добавляют. Это численное правило текущей игры, не уравнение газодинамики через сечение отверстия в паскалях.

Температура сдвигается к `outsideTemperature` по `thermalStep`; давление, кислород и температура ограничиваются. `ApplyPlayerSuction` находит игрока в комнате, нормирует направление к пробоине, вычисляет спад `(1-distance/radius)^2` и передаёт внешнюю скорость через `CharacterExternalVelocitySystem`. При нулевой длине направления деление запрещено. Тяга сейчас применяется к игроку, не ко всем предметам. После пробоин `EqualizeConnectedRooms` считает обмен между связанными помещениями. Текст учебника может использовать закон сохранения массы как цель дальнейшей физической модели, но эти поля [0,1] и коэффициенты обмена сейчас являются игровым приближением. Для добавления реалистичной модели пришлось бы хранить объём комнаты, количество газа и температуру в единицах, затем проверять баланс вещества при каждой связи.

Проверка на одной сцене: две одинаковые комнаты, дверь закрыта, в первой давление 1, во второй 0.5. Зафиксируй показатели до и после 1 с, затем открой дверь и повтори. С пробоиной отдельно сравни 30/60/120 FPS по одинаковому игровому времени. Если разница растёт, проверь ограничение `dt=0.10`, порядок открытия двери и вызовы EnvironmentSystem. Не смешивай этот тест с `FloodingSystem`: тот считает воду и плавучесть.

6. Частицы: рождение, жизнь и отрисовка

`ParticleSystem.cpp` хранит emitters, частицы и render items в заранее зарезервированных векторах. `CreateEmitter` ограничивает число источников `MAX_EMITTERS`, выдаёт ненулевой ID, сохраняет описание и может сразу подготовить первое рождение через `spawnAccumulator`. Обычно скорость эмиссии выражается в частицах/с: если она 12/с и `dt=0.25c`, аккумулятор увеличится на 3 и можно создать три частицы. При 60 FPS каждый шаг даст 0.2, но после пяти шагов сумма снова равна 1. Без дробного аккумулятора эмиттер с малой скоростью на высоком FPS вообще ничего бы не порождал.

В `Render` есть защита от двойного обновления: Play уже вызывает update из `ApplicationClass`, а Editor может сделать запасной шаг для preview. Флаг `m_updatedSinceLastRender` решает, нужен ли этот шаг; после рендера он сбрасывается. `ParticleRenderer` создаётся при необходимости и только один раз пробует `Initialize(device)`; это граница CPU симуляции и GPU ресурсов. Если GPU инициализация не удалась, gameplay событие взрыва не должно превращаться в отсутствие урона. Текстуры и shader particles — способ показать событие, но его игровая причина хранится вне render items.

Для одной частицы с начальной позицией `(0;0;0)`, скоростью `(2;4;0)` м/с и временем жизни 1 с простое движение без сил даёт через 0.25 с позицию `(0.5;1;0)`. Если подключить гравитацию, положение зависит от выбранного интегратора. Цвет можно считать `lerp(startColor,endColor,t/lifetime)`; при половине жизни результат находится между цветами по каждой компоненте. `Lerp` и `LerpColor` есть в этом файле. Следи, чтобы визуальная альфа и физический урон не управляли друг другом: исчезновение дыма не должно «отменять» уже полученное повреждение.

7. Взрыв: видимый эффект и импульс для физических тел

`ExplosionSystem::Emit` в `src/Gameplay/Effects/ExplosionSystem.cpp` принимает позицию и `ExplosionSettings`. Сначала ограничивает radius снизу 0.05 м, impulse — нулём, и вызывает `particleSystem.EmitExplosion`. Это визуальная часть. Затем проходит `RigidBodyComponent`. Тела без Bullet pointer, статические и кинематические исключаются; масса через `getInvMass()` должна быть

положительной. Это важно: от взрыва не должен двигаться неподвижный интерьер лишь потому, что у него есть видимый меш.

Для оставшегося тела система берёт центр масс и вектор от эпицентра. Если расстояние больше радиуса, ничего не происходит. Если почти равно нулю, вместо деления на ноль выбирается направление вверх. После добавления `upwardBias` направление нормируется. Сила толчка задаётся `impulseStrength*(1-distance/radius)^2`. При radius=10 м, impulseStrength=100 Н·с и distance=5 м получаем долю 0.5, квадрат 0.25, импульс 25 Н·с. Для тела массой 5 кг это могло бы дать изменение скорости 5 м/с в направлении импульса до влияния других контактов. У границы radius импульс ноль; рядом с центром почти полный. Применение `applyCentralImpulse` будит тело, а небольшой `applyTorqueImpulse` добавляет вращение. Возвращаемое число — количество затронутых физических тел, а не число получивших урон персонажей.

У взрыва здесь нет обещания прямого уменьшения здоровья: этот метод занимается частицами и физическим импульсом. Если нужен игровой урон по радиусу, создай отдельный путь через `DamageEvent` и `DamageSystem`, с проверкой стен и фракций по выбранным правилам. Не используй число затронутых Bullet тел как суррогат количества раненых: статический игрок или персонаж с отдельным character controller может быть опасно близко, но не попасть в этот список.

8. Тревога корабля: выбор самого серьёзного сообщения

`ShipAlertSystem::Update` в `src/Gameplay/Ship/ShipAlertSystem.h` каждый раз очищает `s_current`, находит комнату игрока по `objectsInRoom` и затем последовательно проверяет кислород, пробоину, power grid и генераторы. При кислороде ≤0.20 выставляется Critical, при ≤0.45 — Warning. Открытая пробоина именно в комнате игрока также даёт Critical. У энергосети brownout — Critical; работа от батареи без генерации — Warning; запас батареи ≤15% — Advisory. У генератора `isScrammed` даёт Critical, высокая температура — Warning, нехватка топлива — Advisory.

`SetAlert` заменяет сообщение только если новый `level` строго выше текущего. Если два события имеют уровень Critical, остаётся первое, встреченное в порядке обхода. Для UI это простой приоритет, но он скрывает остальные аварии. Например, пробоина и brownout одновременно покажут лишь одну надпись. Если игра должна показывать очередь тревог, потребуется список событий с устойчивым порядком и сроком действия, а не одно поле `s_current`. Обрати внимание и на область видимости: найденный `shipRoot` ограничивает проверку генераторов, если корень известен. Если комната игрока не найдена, логика работает с fallback `INVALID_ENTITY`; этот случай стоит испытать отдельно в сцене без разметки комнат.

Для проверки создай комнату с кислородом 0.44 и генератором без топлива: ожидается Warning, потому что оно выше Advisory. Добавь пробоину: ожидается Critical. Закрой пробоину и подними oxygen до 0.8 — останется Advisory, если именно низкий запас топлива всё ещё виден системе. Это не «переключение текста» в HUD: оно показывает результат чтения нескольких ECS компонентов в определённом порядке.

9. Room culling: какие комнаты видит камера

`RoomCullingSystem.cpp` уменьшает число объектов, передаваемых рендеру. `FindRoomContainingPoint` проходит комнаты, берёт Transform и переводит мировую точку камеры в локальное пространство комнаты через обратную world matrix. Затем сравнивает её с `localMin/localMax` с небольшими допусками границ. Почему нужен inverse? Если отсек переместили или повернули, сравнение мировой координаты камеры с локальными min/max ошибочно. Обратная матрица возвращает точку в систему координат отсека, где прямоугольные границы снова имеют простой смысл.

Кэш `s_roomTransformCache` хранит position, rotation, scale и worldToLocal. При неизменном Transform повторно вычислять inverse не нужно. Это экономит работу при множестве объектов, но требует корректного сброса при загрузке сцены и изменении geometry bounds. `BuildVisibleRooms` начинает с комнаты камеры и использует связи дверей, чтобы сформировать список видимых помещений.

Дверь может быть видна из любой из двух соседних комнат. Это уровень отсечения кандидатов, не доказательство фактической видимости каждого треугольника: от камеры до предмета ещё может стоять колонна.

Сложность. Если в корабле 100 комнат и 10 тысяч объектов, отправлять все объекты в каждый draw pass дорого. Отсечение по комнатам сокращает число кандидатов, но сама проверка принадлежности комнаты тоже имеет цену. Поэтому в коде есть кэши Entity→Room и debug статистика: число запросов, проверок и текущая комната камеры. При подозрении на исчезающий объект сначала сравни найденную комнату камеры, комнату объекта, состояние двери и итоговый visibleRooms. Не лечи culling отключением depth или изменением PBR shader — объект, который не попал в список draw candidates, GPU никогда не увидит.

10. Показатели игрока: перевод комнаты в здоровье и выносливость

`PlayerVitalsSystem::Update` в `src/Gameplay/Player/PlayerVitalsSystem.h` ищет `PlayerComponent` и комнату, содержащую Entity игрока. Доля `room.oxygen` в [0,1] превращается в целевой процент игрока умножением на 100. Текущее `player.oxygen` приближается к цели со скоростью 25 процентных пунктов в секунду. Если комната внезапно упала с 1.0 до 0.5, а у игрока пока 100, при `dt=0.1` показатель уменьшится до 97.5, а не мгновенно до 50. Такой лаг является правилом текущей системы; он может представлять запас дыхания/снаряжения, но код прямо не моделирует объём лёгких.

При нуле кислорода здоровье убывает на 10 единиц/с, если нет god mode или `InvulnerableComponent`. Значения ограничиваются [0,100]. `StaminaUpdate` уменьшает stamina на 20/с при `isRunning`, восстанавливает на 10/с, если `wantsRun=false`. Различие двух флагов существенно: игрок может хотеть бег, но фактически не бежать из-за иных условий; тогда обычное восстановление по этой ветке не происходит. Проверь границы: нулевой `dt`, потерю комнаты, паузу, смерть, переход между комнатами и смену флага бега. Система считывает атмосферу; она не должна самостоятельно менять давление комнаты, иначе двое игроков или NPC нарушат единый источник состояния среды.

Рендер и физика: объяснение от данных до кадра

Рендер нужен игре, чтобы показать состояние мира. Он читает Transform, mesh, материал, свет и камеру, но не должен определять игровые правила. Вершинный шейдер отвечает за положение вершин и передаваемые атрибуты; пиксельный — за цвет видимого фрагмента. Между ними GPU растеризует треугольники и интерполирует атрибуты. Эта часть даёт маршрут одного пикселя и способ отладки, когда он неверен.

18. CPU, GPU и жизнь одного draw call

18.1. Что подготавливает CPU

Renderer получает геометрию, выбирает pipeline state, связывает vertex/index buffers, input layout, shaders, constant buffers, textures и samplers, затем отправляет draw call. GPU не знает, что Entity называется «генератор» или что ему нужен ремонт: ему переданы массив вершин, индексов и числовые параметры. Если объект отсутствует на экране, начни с CPU границы: был ли он отобран на рендер, есть ли валидные buffers, какая world matrix отправлена, какая camera active и какие ресурсы привязаны к slots. Попытка лечить отсутствие draw call изменением освещения в pixel shader ничего не даст.

Input layout связывает структуру vertex buffer с семантиками HLSL: position, normal, texcoord, tangent, bone indices и weights. Если layout ожидает float3 normal, а CPU записал другое смещение, vertex shader получит мусор даже при успешной компиляции. Введи контрольный mesh из одного треугольника с известными координатами и цветом. С ним проще отделить ошибку загрузки модели от ошибки pipeline. После него проверь куб, затем skinned NPC. Сложность должна расти по одному измерению за раз.

18.2. Вершинный шейдер и пространства координат

В `src/Render/Shader/Vs/pbr.vs` позиция после возможного skinning проходит три операции: `mul(finalPos, worldMatrix)`, `mul(worldPos, viewMatrix)`, `mul(viewPos, projectionMatrix)`. World position также нужен pixel shader для освещения. `SV_POSITION` — clip position до растеризации; viewport и perspective divide определяют пиксель на экране. Не передавай `SV_POSITION` как будто это world position для расчёта расстояния до источника света. Если освещение «плывёт» при перемещении камеры, первым делом проверь, в каком пространстве находятся light position, surface position и normal.

Для skinned mesh вершинный шейдер умножает позицию на bone transforms с весами. Веса должны быть неотрицательны и суммироваться примерно в 1; если часть костей отсутствует или индекс выходит за массив, вершина улетит в неожидаемую точку. Отдельно проверь normal/tangent: они не должны получать перенос. Bind pose, inverse bind matrices и текущие transforms костей должны использовать совместимое соглашение. Начни с одного жёсткого веса 1,0 для одной кости, затем добавь две кости и смесь, чтобы видно было, на каком этапе появляется ошибка.

18.3. Растеризация, depth и culling

После vertex shader треугольник попадает в rasterizer. Winding определяет лицевую сторону для back-face culling. Depth test решает, виден ли фрагмент за уже нарисованной поверхностью. Поэтому «модель не отображается» может означать невидимость из-за камеры, неправильное winding, clipping по near/far, depth state или прозрачный материал. Проверяй по порядку: wireframe/culling off на тестовой сцене, отладочный одноцветный shader, depth buffer и затем материал. Не отключай depth навсегда только потому, что объект появился: это подтвердило слой проблемы, но не исправило его.

19. Пиксельный шейдер и физический смысл материала

19.1. Albedo, normal, metallic и roughness

`pbr.ps` проекта читает albedo, normal, metalness и roughness карты. Albedo задаёт базовое отражение поверхности; normal map меняет локальное направление микрогеометрии для освещения, но не форму collider и силуэт. Metallic определяет, ведёт ли материал себя как металл; roughness задаёт ширину и характер блика. Если normal map хранит значения 0…1, их переводят в диапазон −1…1 и преобразуют из tangent space через TBN basis в world space. Если перепутать цветовое пространство или каналы, металл станет пластиком, а углубления будут выглядеть выпуклыми.

Не все текстуры представляют цвет. Albedo обычно требует учёта sRGB при чтении; roughness, metallic, normal и masks являются линейными данными. Применить к roughness sRGB преобразование означает изменить числовой параметр материала. При импорте GLB проверь не только путь к текстуре, но и семантику канала. В `pbr.ps` есть отдельная логика чтения roughness и обработка вариантов glTF/Assimp; при подозрительном блеске сначала выведи сырой sample и final roughness, а не подбирай интенсивность света на глаз.

19.2. От Lambert к микрофасеткам

Для идеальной матовой поверхности вклад света зависит от `max(dot(N,L),0)`: поверхность, обращённая к свету, получает больше энергии, а развернутая от света — ноль. Это простая модель диффузного отражения. Блик требует учёта направления к камере V и распределения микроскопических нормалей. PBR шейдер использует функции GGX distribution, geometry term и Fresnel. Их смысл: сколько микрофасеток правильно ориентировано, какая часть не закрыта собственной поверхностью и какая доля энергии отражается на границе материала. Числа нельзя интерпретировать как независимые «эффекты красоты»; вместе они ограничивают характер отражения.

Для проверки отключай слои по очереди. Сначала выведи world normal цветом `(N*0,5+0,5)`, затем NdotL, затем roughness и metallic, потом итоговую diffuse/specular сумму. Если на кубе нормали неправильны уже на первом шаге, сложная BRDF не исправит результат. Если normal верна, а блик вращается вместе с камерой некорректно, сравни пространства N, V и L. Если материал слишком тёмный только при включении normal map, проверь TBN и направление Y канала карты.

19.3. Свет, тени и эффекты повреждения

Directional light задаёт одно направление для всей сцены; point/spot lights зависят от расстояния и геометрии конуса. Shadow map хранит глубину от источника света; сравнение глубины текущего фрагмента с сохранённой решает, закрыт ли он для света. Типичная ошибка тени — acne или отрыв от объекта из-за bias. Подбирай bias в единицах глубины и проверяй на нескольких масштабах, не скрывай проблему огромным смещением, от которого NPC начинает «парить» над собственной тенью.

Трещина, мокрое пятно и подсветка выбора — визуальное следствие gameplay события. В проекте `pbr.ps` содержит crack map и дополнительные параметры повреждения. Источником правды о пробоине остаётся gameplay/physics, а shader получает отображаемые параметры. Если shader рисует пробоину без `HullBreachComponent` или collider, игрок увидит дырку, через которую не течёт вода или воздух. Для новой визуальной механики опиши событие, которое устанавливает параметр материала, и момент, когда он снимается после ремонта.

20. Как добавить новый шейдер безопасно

Начни с одного видимого результата и входных данных. Например, аварийная панель должна пульсировать красным при падении давления. Gameplay вычисляет severity в диапазоне 0…1; renderer передаёт её в constant buffer; pixel shader смешивает исходный цвет с аварийным по функции времени. Время для пульса может быть render time, потому что эффект декоративный.

Состояние «давление низкое» должно оставаться в CPU системе: иначе его нельзя сохранить, проверить тестом или использовать NPC без GPU.

Опиши slots до компиляции: какой `register(bN)` занимает новый constant buffer и какие `register(tN)` уже использованы PBR шейдером. Структуры C++ и HLSL должны согласовывать порядок полей и выравнивание. Для constant buffer в D3D11 размер кратен 16 байтам; `float3` рядом с `float` требует внимания к упаковке. Выведи один контролируемый параметр как сплошной цвет, чтобы проверить передачу данных, затем добавляй текстуру и освещение. Сохрани компиляционные ошибки и проверяй shaders в Debug/Release.

Практика. На одном объекте отобрази `severity=0`, `0,5`, `1` тремя кадрами; для каждого заранее предскажи смешанный цвет. Перенеси камеру и объект: цвет от severity не должен меняться, если он не зависит от освещения. Затем отключи текстуру: fallback материал должен остаться видимым. После этого подключи эффект к реальной пробоине через событие gameplay и проверь, что ремонт убирает визуальный след в нужный момент.

Bullet Physics и симуляция мира: углублённый разбор

Bullet отвечает за столкновения и движение физических тел, но не знает игровых понятий «враг», «герметичная дверь» и «пробоина». Движок соединяет Bullet object с Entity и решает, какая система владеет позицией. Физика не должна заменять AI, а AI не должен телепортировать тело сквозь collision shape ради достижения waypoint.

21. Тело, форма и мир физики

21.1. Static, dynamic и kinematic

Static body имеет нулевую массу и служит неподвижной геометрией. Dynamic body имеет положительную массу, скорость и откликается на силы и столкновения. Kinematic body перемещается по заданной траектории и влияет на другие тела, но не ведёт себя как обычное динамическое тело. Дверь на анимации может потребовать kinematic подхода; ящик, который отбрасывает взрыв, — dynamic; палуба — static. Выбирай тип по тому, кто является владельцем transform. Если вручную каждый кадр ставить dynamic body в Transform, а Bullet в том же кадре решает столкновения, два владельца будут бороться за позицию.

Collision shape не обязан совпадать с render mesh. Box, sphere и capsule дешевле и устойчивее для большинства предметов. Для сложной статичной палубы возможен треугольный mesh collider, но движущийся сложный mesh требует другого подхода. При создании dynamic body вычисляют локальную инерцию по форме и массе; одна лишь масса без корректной shape/inertia меняет вращение. В debug режиме одновременно показывай render bounds, collision shape и origin тела. Если NPC застрял у визуально широкого проёма, возможно, фактическая капсула шире доступного прохода.

21.2. Шаг и синхронизация

`btDiscreteDynamicsWorld::stepSimulation` продвигает физическое состояние. Для плавной и устойчивой игры важно согласовать его с `dt` и внутренним fixed step. Обновление, которое каждый кадр вызывает physics с `dt=1/60`, даже когда реально прошло 1/30 с, замедлит мир вдвое. Напротив, один огромный шаг после зависания увеличивает риск пропущенного столкновения и неустойчивости constraint. Выбери лимит substeps и показывай диагностическую метрику «сколько шагов выполнено, сколько времени потеряно».

После физического шага прочитай world transform тела для рендера и gameplay, если body является владельцем движения. Для kinematic персонажа порядок может быть обратным: controller задаёт желаемое перемещение, Bullet разрешает его с учётом контактов, затем фактическая позиция записывается в ECS. Факт «StateSystem хочет идти» не гарантирует «NPC сдвинулся». Логируй желаемую скорость и реальное смещение раздельно. Если они расходятся, переходи к коллизии и NavMesh, а не только к AI.

21.3. Контакты, raycast и фильтры

Broadphase быстро отбрасывает пары, чьи bounds не пересекаются; narrowphase вычисляет конкретный контакт shapes. Collision group/mask решают, какие пары и лучи вообще проверяются. Если луч оружия проходит сквозь дверь, выясни, отсутствует ли collider, неверна ли mask, исключён ли объект фильтром или ray начинается за дверью. Не предполагай сразу ошибку Bullet. Для выстрела физический hit возвращает тело/форму и точку; игровая система преобразует это в Entity, затем определяет материал, урон и событие.

Собственное тело стрелка обычно исключают из raycast. Но исключение не должно случайно удалить из проверки союзника или дверь. Раздели луч зрения, muzzle clearance и игровой damage ray: у них могут быть разные начала и допустимые collision filters. Для взрыва используй overlap query в радиусе, затем рассчитай направление и импульс для каждого подходящего тела. В центре

взрыва длина вектора к телу может быть нулевой; выбери безопасное fallback направление или не применяй импульс, а не дели на ноль.

22. Персонаж, дверь и разрушение

22.1. Контроллер персонажа и NavMesh

NavMesh отвечает, где можно пройти, а character controller — прошёл ли персонаж фактически. Параметры Recast agent radius/height должны соответствовать Bullet капсуле, иначе путь проходит слишком близко к стене. Если дверь ещё закрывает проём collider, даже корректный путь через открытую графовую связь не позволит пройти. Три диагностических изображения — NavMesh corridor, желаемое направление контроллера и реальная capsule position — быстро указывают, на каком слое ошибка.

При подходе к ступени controller должен согласовать step height и физическую геометрию. Слишком большой step height позволяет подниматься на непредусмотренные препятствия; слишком малый делает даже маленький порог двери непроходимым. Проверяй одну ступень известной высоты и один проход ширины, равной диаметру капсулы плюс запас. Если NPC пробегает сквозь визуальную стену, проверь collision registration и owner transform. Если стена есть в Bullet, но отсутствует в NavMesh, путь строится через неё и NPC будет застревать.

22.2. Разрушение и обновление нескольких миров

Разрушение корпуса или двери меняет render mesh, collision geometry, NavMesh, room/pressure connections и игровой список задач. Эти изменения нельзя считать одной операцией «удалить треугольники». Пока новый mesh строится асинхронно, должен существовать безопасный промежуточный статус: текущая collision shape ещё действует или уже заменена; NPC получает определённую версию проходимости; атмосфера знает, открыта ли связь. При завершении worker job изменения Bullet и ECS применяются на согласованной границе main thread. Старые shape и body удаляются после того, как мир перестал их использовать.

Построй конечный автомат `Intact → Damaged → GeometryPending → Breached → RepairPending → Repaired`. Для каждого состояния запиши collision, визуальное представление, airflow и доступность ремонта. Так можно обнаружить неочевидную ошибку: трещина нарисована, но воздух ещё не течёт; или пробоина уже закрыта gameplay, но collider ещё открыт. Не требуй идеально атомарного обновления GPU и Bullet в одном такте, но обязательно определи, что видит игрок в промежуточный момент.

23. Физические проверки, которые нужны перед игрой

Создай тестовую комнату с палубой, ящиком, дверью и NPC. Проверь: ящик падает на палубу, не проваливается и не «взрывается» от ошибочной инерции; NPC останавливается перед закрытой дверью; после открытия проём физически свободен; raycast от глаза и от дула возвращают разные результаты при низком укрытии; удаление ящика убирает Bullet body; reload сцены не оставляет невидимую копию collider. Каждому случаю соответствует одна ожидаемая геометрия и один владелец transform.

После функциональной проверки меняй частоту рендера и длительность кадров. Измеряй число physics substeps и фактическое положение тела через одинаковое simulation time. Малые численные различия допустимы, но зависание ящика в воздухе при другом FPS говорит о неверном времени или синхронизации. Не расширяй сцену до десятков динамических обломков, пока базовый цикл создания, разрушения и повторной загрузки не стабилен.

Шейдеры с нуля: от одного треугольника до материала DX Framework

Шейдер — небольшая программа, которую GPU выполняет для элементов рисования. Начинающему полезно представить фабрику: CPU подготавливает геометрию, матрицы и ресурсы; vertex shader получает одну вершину и сообщает, где она окажется на экране и какие данные передать дальше; rasterizer определяет покрытые треугольником экранные фрагменты и интерполирует данные; pixel shader рассчитывает выходной цвет. Это не отдельный мир, который сам знает про Entity, здоровье NPC или двери. Он видит только переданные числа, buffers и textures. Официальное описание стадий и семантик есть в документации Microsoft (https://learn.microsoft.com/en-us/windows/win32/direct3dgetstarted/work-with-shaders-and-shader-resources) и справочнике HLSL semantics (https://learn.microsoft.com/en-us/windows/win32/direct3dhlsl/dx-graphics-hlsl-semantics).

1. Один треугольник до первого HLSL слова

Пусть в локальной плоскости есть вершины A=`(-1;-1;0)`, B=`(1;-1;0)`, C=`(0;1;0)`. Vertex buffer содержит их координаты, возможно цвет и UV; index buffer говорит, что треугольник составлен из индексов 0,1,2. Input layout сообщает D3D, в каких байтах каждой вершины находятся `POSITION`, `COLOR`, `TEXCOORD`. Ошибка смещения в layout означает, что HLSL прочтёт цвет как позицию или мусор, даже если текст шейдера безупречен. Для первого опыта используй один треугольник без освещения и текстуры: меньше стадий могут сломаться одновременно.

После vertex shader вершины проходят через world, view, projection. Для принятой в проекте конвенции с вектором-строкой: `clip = local * world * view * projection`. `world` ставит объект в мир, `view` описывает мир относительно камеры, `projection` создаёт перспективу. Результат содержит четыре координаты `(x;y;z;w)`. После деления на w получается нормализованная экранная позиция; если w некорректен или вершина вне видимого объёма, часть геометрии отсечётся. Не путай переменную `SV_POSITION` в pixel shader с мировой позицией: там уже экранная информация. Если нужен свет в мировом пространстве, передай отдельный `worldPos` и нормаль из VS.

2. Читаем простейший vertex shader почти по строкам

Файл `src/Render/Shader/Vs/color.vs` объявляет `cbuffer MatrixBuffer : register(b0)` с тремя матрицами. `b0` — слот буфера констант vertex stage, который CPU должен заполнить и привязать. `VertexInputType` получает `position : POSITION` и `color : COLOR`; `PixelInputType` отдаёт `position : SV_POSITION` и цвет. В `ColorVertexShader` сначала `input.position.w=1.0f`: это точка, на которую должен действовать перенос. Затем идут три `mul` в порядке world, view, projection. Последняя строка `output.color=input.color` переносит значение к следующей стадии. Если убрать эту запись, pixel shader, который ожидает COLOR, получит неопределённые для твоего алгоритма данные; если текущий pixel shader их не читает, картинка может случайно остаться прежней.

Минимальная учебная версия того же смысла:

```
cbuffer MatrixBuffer : register(b0) {
    float4x4 worldMatrix;
    float4x4 viewMatrix;
    float4x4 projectionMatrix;
};
struct VSIn { float4 position : POSITION; float4 color : COLOR; };
struct VSOut { float4 position : SV_POSITION; float4 color : COLOR; };
VSOut MainVS(VSIn input) {
    VSOut output;
    input.position.w = 1.0f;
    output.position = mul(input.position, worldMatrix);
    output.position = mul(output.position, viewMatrix);
    output.position = mul(output.position, projectionMatrix);
```

```
    output.color = input.color;
    return output;
}
```

Это учебный фрагмент для объяснения. Реальные entry point и buffer layout бери из файла проекта и соответствующего C++ класса. На CPU `ColorShaderClass::SetShaderParameters` в `src/Render/Shader/colorshaderclass.cpp` получает матрицы, делает `Map` dynamic buffer, записывает значения, `Unmap` и привязывает буфер через `VSSetConstantBuffers`. `InitializeShader` компилирует VS и PS через `D3DCompileFromFile`, создаёт shader objects и input layout. Если текст HLSL правильный, но CPU не привязал `b0`, шейдер не получит нужные матрицы. Если размер/упаковка C++ структуры не совпали с HLSL, одна из матриц или параметров будет считана неверно. Сначала сравни поля и их порядок, затем слоты и стадию привязки.

3. Что делает rasterizer между VS и PS

Vertex shader выполняется для вершин, но экран состоит из пикселей. Rasterizer превращает треугольник в покрытые им фрагменты. Для внутренней точки он вычисляет веса относительно трёх вершин. Если веса равны `(0.2;0.3;0.5)` и вершинные цвета красный `(1;0;0)`, зелёный `(0;1;0)`, синий `(0;0;1)`, интерполированный цвет равен `(0.2;0.3;0.5)`. Веса суммируются до 1. Для точек внутри треугольника они неотрицательны; если один вес отрицателен, точка лежит вне него в этой простой геометрической проверке. Для UV применяется схожая идея, но при перспективе GPU учитывает глубину, чтобы текстура не выглядела растянутой неправильно.

Rasterizer также учитывает лицевую сторону треугольника, режим culling, viewport и depth. Если порядок индексов поменять с (0,1,2) на (0,2,1), треугольник может стать обратной стороной и исчезнуть при back-face culling. Если он за другой поверхностью, depth test не позволит ему перекрасить ближний пиксель. Поэтому «pixel shader не работает» сначала проверяют одноцветным материалом, culling и depth, затем текстуру и свет. В документации Microsoft описаны pipeline stages и rasterization (https://learn.microsoft.com/en-us/windows/win32/direct3dgetstarted/work-with-shaders-and-shader-resources); конкретные states выбирает renderer твоего движка.

4. Простейший pixel shader и ловушка текущего файла

`src/Render/Shader/Ps/color.ps` объявляет `ColorBuffer : register(b0)` и `ColorPixelShader(...):SV_TARGET`. `SV_TARGET` означает цвет, записываемый в текущий render target. Хотя структура входа содержит `color : COLOR`, текущая реализация возвращает `uniformColor`, полученный из constant buffer. Следовательно, разноцветные вершины из `color.vs` **не создадут градиент** с этим pixel shader. Это важный урок чтения пары VS/PS: надо проследить, что PS действительно использует из вывода VS. Для учебного эксперимента можно заменить возврат на `input.color`, но тогда потребуется согласовать C++ параметры и ожидаемое назначение `ColorShaderClass`; это изменение поведения проекта, а не описание текущего результата.

`register(b0)` в VS и `register(b0)` в PS не означают один общий буфер: это слоты двух разных стадий. C++ привязывает matrix buffer к VS и color buffer к PS. Если ошибочно привязать оба через `VSSetConstantBuffers`, PS не увидит цвет. Шейдер ничего не «ищет» по имени C++ переменной — совпадают слот, стадия и байтовая форма данных.

5. Текстура: UV, sampler и прозрачный вырез

В `src/Render/Shader/Vs/texture.vs` к позиции добавляется `float2 tex : TEXCOORD0`. Vertex shader вычисляет clip position, затем копирует UV в выход. `texture.ps` объявляет `Texture2D shaderTexture : register(t0)` и `SamplerState SampleType : register(s0)`. Выражение `shaderTexture.Sample(SampleType,input.tex)` возвращает цвет текстуры в интерполированной UV точке. Texture resource и sampler — разные вещи: первый хранит данные, второй задаёт способ выборки за границами, фильтрацию и другие правила. `TextureShaderClass::SetShaderParameters`

передаёт texture SRV в pixel stage через `PSSetShaderResources`; без этой привязки `t0` не содержит нужную картинку.

Текущий `texture.ps` выполняет `clip(textureColor.a-0.01f)`. Если alpha меньше 0.01, выражение отрицательно и фрагмент отбрасывается. Это пороговый вырез, полезный для решётки или листвы: там нет полупрозрачного смешения у края. Если нужен дым или стекло с alpha 0.5, одного `clip` недостаточно; нужны соответствующие blend и depth правила. Пиксель с RGBA `(0.8;0.2;0.1;0.005)` отбрасывается, с alpha `0.02` остаётся. Не путай alpha cutout с «прозрачностью всего объекта».

Для проверки UV создай квадрат из двух треугольников и текстуру 2×2: красный, зелёный, синий и белый углы. Если цвета переставлены, проверь U/V и ориентацию изображения. Если виден один размазанный цвет, проверь UV в vertex buffer, input layout, передачу TEXCOORD из VS, sampler и привязку t0. Последовательная проверка каждого интерфейса быстрее случайной замены файла изображения.

6. Нормаль, Lambert и первая формула освещения

Нормаль N — единичное направление, перпендикулярное поверхности. Направление L от поверхности к источнику тоже нормируем. Простой диффузный свет пропорционален `max(dot(N,L),0)`. Если N и L совпадают, dot=1 и поверхность освещена сильнее. Если угол 60°, dot=cos60°=0.5. Если свет за поверхностью, dot отрицателен, `max` даёт 0. Зачем нормировать? При N длиной 2 и L длиной 3 dot может быть 6, хотя косинус никогда не больше 1. Тогда яркость ошибочно зависит от длины вектора.

Нормаль не является точкой: перенос world matrix не должен на неё влиять. При неравномерном масштабе простое умножение на world 3×3 может испортить перпендикулярность; нужна матрица, обратная транспонированной линейной части, затем нормировка. В проекте есть `normal_transform.hlsli`; сравни его с `pbr.vs` при диагностике растянутого меша. Проверка на кубе с scale `(2;1;1)`: выведи normal как RGB `(N*0.5+0.5)` и вращай камеру. Если цвет меняется от камеры, нормаль, вероятно, оказалась в другом пространстве, чем ожидалось. Если нормаль верна, переходи к light direction и shadow map.

7. От обычного света к PBR без заклинаний

`pbr.vs` и `pbr.ps` уже сложнее учебной пары. Вершинный этап может учитывать кости персонажа, преобразует позицию world/view/projection и передаёт сведения для освещения. Пиксельный этап читает material textures и вычисляет цвет на основе normal, направления света и камеры, roughness и metallic. Сначала покажи на debug material каждую входную величину отдельно: albedo; normal; roughness; metallic; shadow factor. Затем объединяй их. Если итоговая картинка чёрная, один из входов может быть неверным; общая формула не объяснит, какой.

Albedo обычно представляет цвет и проходит через sRGB преобразование при чтении/выводе; normal, roughness и metallic — числовые данные, которые нельзя бездумно декодировать как цвет. Roughness влияет на ширину блика, metallic меняет баланс отражения и диффузной составляющей. PBR микрофасеточная модель использует распределение микронормалей, маскирование и Fresnel. Для новичка сначала достаточно понять энергетический вопрос: сколько света приходит, в какую сторону отражается и почему отражение меняется с углом взгляда. Параметры материала — входы модели, не независимые «эффекты красоты».

Тень обычно начинается отдельным проходом. `ShadowMap` хранит глубину от источника; основной pixel shader сравнивает текущую глубину с записанной и ослабляет прямой свет, если на пути была другая поверхность. Если тень оторвана от объекта, проверь bias, матрицу света и единицы глубины. Если поверхность исчезает лишь в теневом проходе, проверь culling и skinning обоих проходов. Световое PBR и теневая карта должны смотреть на одну и ту же геометрию текущего кадра.

8. Как добавить свой маленький шейдерный эффект в движок

Поставь конкретную задачу: аварийная панель плавно мигает, когда уровень опасности комнаты больше нуля. Gameplay система вычисляет `severity` в [0,1]. C++ передаёт её и время визуального эффекта в pixel constant buffer; PS рассчитывает `pulse=0.5+0.5*sin(time*frequency)` и смешивает базовый цвет с красным по `severity*pulse`. При `severity=0` панель остаётся исходной; при `severity=1` и `pulse=1` становится красной. Время можно брать из render clock, если мигание декоративное. Причина тревоги живёт в ECS, чтобы AI, сохранение и HUD видели то же состояние без GPU.

Сделай этот эффект по шагам. Сначала верни постоянный красный `float4(1,0,0,1)` и убедись, что нужный объект получает новый PS. Потом передай один `severity` через buffer, проверь значения 0, 0.5, 1. Затем добавь time и синус. После этого проверь обновление constant buffer (`Map`, запись, `Unmap`, `PSSetConstantBuffers`) и отсутствие конфликта слотов с другим материалом. Наконец проверь, что renderer сбрасывает ресурсы между draw calls: иначе следующая обычная панель может наследовать аварийный buffer.

Выполни четыре диагностические сцены: один треугольник с uniformColor; треугольник с vertex colors после явного изменения PS; квадрат с текстурой 2×2; куб с normal visualizer и одним источником Lambert. Каждый шаг добавляет только одну неизвестную. Когда они работают, читай `PBRShaderClass.cpp` как перечень CPU→GPU договоров: какие constant buffers созданы, в какие слоты привязаны, какие SRV поступают в PS, какой input layout ожидается и когда вызывается DrawIndexed. Эта цепь делает сложный шейдер проверяемым.

9. Буфер констант — точное соглашение о байтах

HLSL `cbuffer` не является C++ объектом, который GPU «находит» по имени. Это область байтов с правилами упаковки и привязкой к стадии и номеру регистра. Если в HLSL есть `float4 color; float severity;`, на стороне CPU нельзя без проверки положить подряд только 20 байт и предположить, что следующий buffer начнётся сразу после `severity`. Для D3D11 размер constant buffer должен соответствовать требованиям выравнивания, а поля внутри упаковываются по правилам HLSL. Проще начинать с группы из `float4` и матриц, сверять `sizeof` C++ структуры и отладочным шейдером выводить каждое поле как цвет. Официальное объяснение constant buffers и их привязки есть в документации Microsoft (https://learn.microsoft.com/en-us/windows/win32/direct3dgetstarted/work-with-shaders-and-shader-resources).

Путь обновления выглядит так: `Map(buffer,WRITE_DISCARD)` даёт CPU доступ к новой памяти, код копирует структуру, `Unmap` завершает запись, `VSSetConstantBuffers` или `PSSetConstantBuffers` привязывает buffer к нужному stage. Для карты, которая меняется каждый кадр, такой путь должен соответствовать флагам её создания. Если `Map` не удался, не вызывай Draw со случайным старым содержимым, если игра ожидает новую матрицу. Если изменился HLSL layout, но C++ структура осталась прежней, ошибка может выглядеть как «всё тёмное» или «свет прыгает», хотя компиляция обеих сторон успешна: C++ компилятор не проверяет контракт с HLSL автоматически.

Матрицы требуют ещё одной осторожности. Транспонирование перед записью в shader buffer связано с соглашением размещения и выбранной формой `mul`, а не с желанием поменять геометрический порядок. В конкретном `PBRShaderClass.cpp` CPU использует `XMMatrixTranspose`; в `pbr.vs` HLSL умножает позицию слева на world, затем view, projection. При отладке запиши одну тестовую точку на бумаге: локальная `(1;0;0;1)`, перенос по X на 5, камера без поворота. После world ожидается X=6. Если шейдер выдаёт другое число, сначала проверь upload и соглашение вектора/матрицы, а не подбирай camera FOV.

10. Какие ещё бывают шейдеры и нужны ли они здесь

В этом курсе подробно рассматриваются vertex и pixel shader, потому что они составляют основной путь видимого mesh в текущем DirectX 11 рендере. В графическом pipeline могут быть и другие

стадии: geometry shader работает с примитивами, hull/domain используются для tessellation, compute shader запускает общее параллельное вычисление вне обычного draw треугольника. Наличие этих стадий в API не означает, что проект уже применяет их в главном рендере. Добавляй стадию только под конкретную задачу с измерением стоимости и данных. Для обычной подсветки аварийной панели новый compute pass не нужен: достаточно параметра pixel shader.

Один и тот же pixel shader может запускаться для очень многих фрагментов. Если экран 1920×1080, это около 2.07 млн пикселей до учёта перекрытия объектов и нескольких render passes. Дополнительная операция на каждом пикселе может стоить гораздо больше одной операции на каждом из нескольких тысяч объектов CPU. Но нельзя оценивать производительность только числом строк HLSL: выборки текстур, ветвления, память, overdraw и кэш GPU имеют разные цены. Измеряй GPU время конкретного прохода, а при диагностике сначала сделай одноцветную версию, затем по одному возвращай normal map, shadow, IBL и эффекты повреждения.

11. Цвет, яркость и тональная коррекция

Цвет текстуры, число света и цвет монитора относятся к разным пространствам значений. Большинство привычных цветных изображений предназначены для просмотра в sRGB; расчёт освещения удобнее делать в линейном пространстве, где удвоение числа примерно соответствует удвоению светового вклада. Если просто сложить два закодированных sRGB цвета и ограничить каждый канал единицей, результат может быть неверно ярким или тёмным. Карты roughness, normal и metallic хранят данные; применять к ним такое же цветовое преобразование нельзя. В отладке отдельно показывай сырое значение sample и значение после необходимого декодирования.

HDR позволяет временно хранить значения яркости больше 1.0. Затем tone mapping переводит диапазон сцены к диапазону экрана, а output conversion готовит цвет для отображения. Если светильник в HDR buffer даёт `(4;2;1)`, простой clamp сделал бы `(1;1;1)` и потерял цвет яркого источника; tone mapping может сохранить различие каналов. Но выбранная кривая — художественное и техническое решение, а не физический закон для всей игры. Не пытайся исправить отсутствующую normal map увеличением exposure: сначала отладь входы материала.

12. Задачи для самостоятельной проверки шейдеров

Задача A. VS получил локальную точку `(2;0;0;1)`, world переносит по X на 3, view и projection условно единичные. Какой X уйдёт из VS? Ответ: 5. Если `w=0`, перенос не должен подействовать; это уже направление, а не точка. Найди в `color.vs` строку, которая принудительно ставит `w=1`.

Задача B. В вершинах треугольника цвета `(1;0;0)`, `(0;1;0)`, `(0;0;1)` и веса внутренней точки `(0.25;0.25;0.5)`. Интерполированный цвет `(0.25;0.25;0.5)`. Почему текущая пара `color.vs` + `color.ps` не покажет этот цвет? Pixel shader возвращает `uniformColor`, а не `input.color`.

Задача C. Normal N=`(0;1;0)`, направление света L=`(0.6;0.8;0)`, оба единичные. `dot=0.8`, Lambert diffuse factor 0.8. Если L=`(0;-1;0)`, фактор `max(-1,0)=0`. Проверь, что N и L находятся в одном пространстве: world normal и view light direction нельзя складывать без преобразования.

Задача D. Texture sample вернул alpha 0.009. `texture.ps` считает `clip(0.009-0.01)=clip(-0.001)`, поэтому фрагмент отбрасывается. При 0.011 выражение положительно, фрагмент остаётся. Почему это не равнозначно полупрозрачности 1%? Потому что `clip` принимает двоичное решение оставить/отбросить, а смешение с фоном выполняется другой стадией при подходящем blend state.

Задача E. Добавь свой параметр `severity` в учебный pixel shader. Сначала выведи `float4(severity,severity,severity,1)` для значений 0, 0.5, 1. Если экран остаётся одинаковым, проверь C++ buffer, `PSSetConstantBuffers`, номер `register(b#)` и то, что Draw действительно использует этот PS. Только после прохода такого простого теста добавляй цветовое смешение и синусоидальный pulse.

Корабельные сети и среда: углублённый разбор

Разбор 24. Корабельные сети как связанные правила

Космический симулятор интересен, когда авария меняет не только число HP. Повреждение корпуса может влиять на среду, потеря питания — на двери и другие потребители, а экипаж — менять приоритет работы. Связь отключения насоса с результатом `ShipPowerGridSystem` здесь предлагается как развитие: текущий `FloodingSystem` читает отдельное поле `CompartmentComponent::hasPower`, которое не обновляется из `PowerConsumerComponent::hasPower`. Чтобы причинная цепочка стала рабочей, каждому звену нужен владелец данных, единица измерения, порядок обновления и видимый для игрока результат. Отдельная анимация без этих связей не создаёт симуляцию.

24.1. Энергосеть на конкретном примере

`ShipPowerGridSystem` собирает сети по root Entity. Генераторы добавляют `currentOutputMw`, потребители требуют `requiredMw`, затем сеть распределяет доступное по приоритетам от 0 до 3. Внутри одного уровня приоритета `SupplyEvenly` делит мощность между ещё неудовлетворёнными потребителями и повторяет деление после насыщения тех, кому нужно меньше. Это важно: «сначала по приоритету» не означает «первому Entity всё, остальным ничего». После расчёта у каждого потребителя есть `suppliedMw`, `powerRatio` и `hasPower`; текущий код считает `hasPower=true` при ratio≥0,999.

Пример. Генератор выдаёт 8 МВт. Критические устройства просят 3 и 2 МВт; обычные — 4 и 1 МВт. Первые получают полные 5, остаётся 3. Обычные сначала получают по 1,5, но второму нужен только 1, поэтому остаётся 0,5, которое достаётся первому: он получает 2 МВт из 4. Итого 3+2+2+1=8. Для первого обычного `powerRatio=0,5`, значит `hasPower=false` по текущему контракту, хотя часть мощности выделена. Если нужна частичная работа, например тусклый свет при 50%, потребитель должен явно использовать ratio, а не bool.

Батарея покрывает нехватку через `EmergencyBatterySystem::Discharge`, затем избыток генерации может её заряжать. Не смешивай мощность и энергию: мощность измеряется в МВт, количество энергии — в МВт·с, МДж или MWh после пересчёта. При 2 МВт на протяжении 3 с потрачено 6 МДж. Для каждой сети проверь инвариант: выделенная мощность ≤ доступной, запас батареи не выходит за [0,capacity], устройства разных root не питают друг друга. Отладочный экран должен показать генерацию, спрос, выдачу батареи, brownout и список потребителей по приоритету.

24.2. Дверь как потребитель и проход

Если дверь требует мощность, её логика не должна смотреть только на наличие GeneratorComponent где-то в сцене. Она читает именно свой `PowerConsumerComponent` после обновления сети. Важно выбрать семантику: мощность нужна только для запуска привода или для удержания открытого состояния? Что происходит при отключении во время движения — дверь замирает, закрывается пружиной или остаётся открытой? Ответ влияет на физический collider, NavMesh, атмосферу и бой. Запиши конечный автомат и переходы при потере питания. Без этого разные системы начнут независимо придумывать поведение.

Проверка: с двумя кораблями и разными root отключи генератор первого. Потребители второго должны остаться запитанными. Затем на первом восстанови питание и открой дверь. NPC должен пройти только после фактического освобождения проёма, а не в момент изменения команды `Open`. Если атмосфера начинает смешиваться в момент начала анимации, это тоже выбранное правило, которое надо показать игроку и проверить на сохранении.

Разбор 25. Давление, затопление и пространство

25.1. Текущая модель давления — водная

В исходном `PressureSystem.cpp` внешнее давление вычисляется как `101325 + ρgh`, где `depth=max(0,waterLevel−bodyY)`, ρ=1000 кг/м³ и g=9,81 м/с². Это модель погружения в воду. Она не описывает вакуум вокруг космического корабля. В `FloodingSystem` также есть water volume, pump rate и buoyancy. Если цель игры — именно космос, эти системы можно использовать как отдельный режим планеты/подводного участка, но нельзя выдавать их за готовую симуляцию разгерметизации в вакууме. Для космоса нужна отдельная модель внутренних газов, наружного давления, связи комнат и потоков через отверстия.

Математика водной модели: на глубине 10 м добавка `1000·9,81·10=98 100 Па`; полное давление около 199 425 Па. Код сравнивает его с `maxPressure` корпуса и наносит урон за секунду при превышении. Проверь границы: maxPressure должен быть положительным до деления, здоровье ограничено [0,100] или другим документированным диапазоном, `dt` не отрицателен. При разрыве `FloodingSystem` переводит локальную позицию пробоины в world, определяет глубину и поток. После притока и откачки обновляет `waterVolume += (inflow−outflow)dt` и ограничивает объём комнаты.

Заметь, что в разных местах текущего кода давление и поток могут использовать разные игровые коэффициенты. Не предполагай, что все они уже образуют физически точную модель. Перед балансом выпиши единицы каждого поля и прогони численный опыт: одна пробоина, известная глубина, нулевая откачка, объём комнаты 10 м³, шаг 1 с. Проверь, соответствует ли прирост воды заявленному flowRate. Если нет, исправь соглашение о единицах или назови коэффициент игровым, не скрывая его под физическим именем.

25.2. Проектируем разгерметизацию космического отсека

Для нового космического режима начни с простого состояния комнаты: объём V, количество газа n или масса m, температура T, давление p. Идеальный газ даёт `pV=nRT`, но только если единицы согласованы и температура в кельвинах. Для учебного шага можно считать T постоянной; тогда при уменьшении n давление пропорционально количеству газа. Связь двух комнат или отверстие наружу переносит газ от высокого давления к низкому. Точную газодинамику с критическим течением добавляй только если она нужна геймплею: для первой версии ограниченная функция потока с сохранением массы и ясными коэффициентами лучше сложной формулы без проверки.

Предположим две равные комнаты по 100 м³ при одинаковой температуре: A имеет условно 100 единиц газа, B — 50. После открытия двери и обмена 10 единицами A станет 90, B — 60; сумма 150 сохраняется. Если B соединена с вакуумом, потерянная масса должна появиться в отдельном счётчике `ventedToSpace`, иначе инвариант не проверить. Давление после шага выводи из нового количества газа, не меняй массу и давление независимо. Для NPC атмосферная опасность должна быть игровым входом: доступен ли скафандр, сколько времени можно работать, какую дверь следует закрыть прежде ремонта. Так математика становится причиной решений.

25.3. Порядок обновления среды

Выбери единый порядок: команды двери и её питание → фактическое состояние проходов → обмен воздухом/водой → урон и состояние оборудования → уведомление экипажа → выбор работы. Если NPC принимает решение до обновления среды, допустима задержка на один кадр, но UI и debug должны позволить её объяснить. При разных частотах обновления, например среда 10 Гц и AI 5 Гц, сохраняй метки времени и не применяй один и тот же поток дважды. При загрузке сцены восстанови связи комнат до первого тика обмена, иначе один кадр может мгновенно переместить газ через неинициализированную дверь.

Разбор 26. Авария, ремонт и наблюдаемый результат

Пробоина начинается как событие повреждения с мировыми координатами и связью с комнатой или корпусным сегментом. Создай `RepairableComponent`, подходящий WorkZone и визуальный след; затем сообщи координатору экипажа. Ремонтник выбирается по severity, расстоянию, роли, skill и доступности. Работа наращивает progress только в пределах рабочей зоны, при достаточном ресурсе и допустимой среде. Завершение закрывает leak, удаляет или переводит цель в Repaired, обновляет геометрию и collider, затем публикует событие для HUD/звука. Если одна из этих операций асинхронная, введи состояние `RepairPending`, чтобы не показать герметичность раньше физического завершения.

На одном тесте измерь цепочку: пробоина появляется на отметке t=0; давление комнаты начинает падать; инженер получает claim и route; дверь открывается; инженер входит в WorkZone; progress растёт с ожидаемой скоростью; по окончании утечка прекращается и давление стабилизируется. Каждое звено должно иметь время, Entity и причину перехода. Если пробоина исчезла визуально, но давление продолжает падать, ищи не «баг NPC», а несогласованный статус утечки. Если pressure исправилось, но Collider остался отверстием, ищи этап rebuild геометрии. Этот метод проверки используется дальше для любой новой механики корабля.

NPC в DX Framework: от данных ECS до ремонта, поиска и выстрела

Эта глава написана по исходникам проекта на 29 октября 2026 года. Её цель — дать возможность восстановить в голове причинно следственную цепочку: **какой проход читает компонент, какое решение принимает, что записывает и какой следующий проход использует эту запись**. Здесь нет абстрактного «обычного игрового AI», выданного за уже реализованный код. Формулировка «сейчас» означает поведение проверенных исходников. Формулировка «предложение» означает изменение, которое ещё нужно реализовать.

Соглашения для чтения: `Entity` — идентификатор, а не адрес объекта; `INVALID_ENTITY` — отсутствие выбранной сущности; `dt` или `deltaTime` — секунды одного шага симуляции; `TransformComponent::position` — мировая позиция; координаты `x,z` задают горизонтальную палубу, `y` — высоту. Ссылки вида StateSystem.h:1629 (../StateSystem.h#L1629) ведут к проверяемым строкам. После изменения кода номера строк могут сдвинуться, поэтому ищите также названия функций.

1. Сначала поймём один игровой кадр

NPC не живёт внутри одного класса `Npc`. Его поведение составлено из ECS компонентов и систем, проходящих по ним. Это ключ к переучиванию кода: **компонент хранит факт или намерение; система превращает входы в новые факты или намерения**. Например, `NpcUtilitySystem` может записать `brain.repairTarget`, но не передвигает капсулу и не чинит объект. `StateSystem` превращает эту цель в направление движения или прогресс ремонта. `CharacterControllerSystem` передаёт направление в Bullet. `RepairSystem` завершает изменение игрового мира.

В ApplicationClass.cpp:1448–1551 (../src/Core/ApplicationClass.cpp#L1448) при `isPlayMode && simulationRunning` цепочка начинается с ввода игрока. Дальше порядок таков:

26. `WeaponSystem::UpdateTimers` один раз уменьшает таймеры всех `WeaponComponent` и сбрасывает `triggerHeld`.
27. `m_npcUtilitySystem.Update` считает локальную пользу ремонта для каждого подходящего мирного NPC.
28. `CrewTaskCoordinator::Update` очищает старые заявки, назначает ремонтные цели и ставит `CrewTaskClaimComponent` на саму цель.
29. `NpcCombatSystem::Update` выбирает возможного врага, проверяет сектор и физический луч, обновляет личную память наблюдения.
30. `NpcCoverSystem::Update` выбирает или проверяет укрытие, выставляет тактический `moveTarget` и состояние `TakingCover`, когда это требуется.
31. `StateSystem::Update` исполняет ручную команду, ремонт, погоню, поиск или тактическое движение. Он пишет намерение в `CharacterMovement`.
32. `CharacterFacingSystem`, `CharacterControllerSystem`, `AnimationController` обрабатывают разворот, физическое движение и позу. `DamageSystem::Update` в районе 1549 (../src/Core/ApplicationClass.cpp#L1549) применяет **накопленную до этого момента** очередь событий урона.
33. После обновления позы и физики ApplicationClass.cpp:1757–1769 (../src/Core/ApplicationClass.cpp#L1757) выполняются выстрел игрока и `NpcRangedCombatSystem::Update`. Их `WeaponSystem::TryFire` добавляет событие в очередь `DamageSystem`.

Следствие пункта 8: событие от выстрела в конце данного кадра обычно будет применено следующим вызовом `DamageSystem::Update`, то есть на следующем шаге симуляции. Это **очередь событий**, а не немедленное `target.health -= damage`. В том же конце кадра уже виден результат трассировки луча и оружейный кулдаун. Реакция здоровья и `underFireTime` наступает при обработке очереди. WeaponSystem.h:119–155 (../src/Gameplay/Combat/WeaponSystem.h#L119), DamageSystem.cpp:104–113 (../src/Gameplay/Combat/DamageSystem.cpp#L104).

В CrewTaskCoordinator.h:107–109 (../src/Gameplay/AI/CrewTaskCoordinator.h#L107) сохранился комментарий «вызывать до Utility/State». Реальный вызов в `ApplicationClass` ставит координатор **после Utility и до State**. Читая код, доверяйте месту вызова, а комментарий считайте устаревшей памяткой. Важная практическая причина: `Utility` даёт предварительный лучший `repairTarget`, но координатор может заменить его окончательным назначением, и `StateSystem` в том же кадре пойдёт уже к нему.

В ApplicationClass.cpp:1494–1504 (../src/Core/ApplicationClass.cpp#L1494) вызов `NpcSquadCombatSystem::Update` закомментирован. Поля `squadId`, `NpcSquadRole` и связанные типы существуют, но текущий запущенный боевой цикл **не назначает роли прикрывающего и обходящего через этот проход**. Индивидуальные укрытия, короткие очереди и поиск работают через другие системы. Не приписывайте им координацию отряда только потому, что поля отряда есть в компоненте.

1.1. Как читать ECS вызов `ForEach`

В ECSRegistry.h:45–59 (../src/ECS/ECSRegistry.h#L45) `ForEach<T1,T2>` итерирует пул первого типа и для каждой сущности проверяет наличие второго. Он не гарантирует порядок ID. `GetComponent<T>(entity)` возвращает указатель либо `nullptr`; `HasComponent` отвечает только на наличие; `IsAlive` проверяет, что ID ещё зарегистрирован как живой. `ActiveComponent` в этом проекте служит отдельным игровым разрешением участвовать в симуляции. Живая `Entity` может быть неактивной. Поэтому типичная охрана имеет несколько проверок: ID существует, компоненты есть, Entity активна и здоровье больше нуля.

Ссылки из `GetComponent` и параметров `ForEach` привязаны к хранилищу. После `RemoveComponent<T>` или `DestroyEntity` соответствующая ссылка недействительна. Добавление компонента в **тот же** пул во время его обхода тоже опасно: `ComponentManager` может перераспределить контейнер. Координатор сначала собирает ID экипажа в `vector`, и уже затем в отдельном цикле добавляет отсутствующий `CrewMemberComponent`: это правильная граница между чтением и структурной мутацией. CrewTaskCoordinator.h:130–151 (../src/Gameplay/AI/CrewTaskCoordinator.h#L130). В ремонте пробоина уничтожается при завершении, поэтому `StateSystem` после `ApplyRepairProgress` не продолжает читать старый `RepairableComponent*`, а обрабатывает результат и очищает свои ID. StateSystem.h:1909–1932 (../StateSystem.h#L1909).

1.2. Два вида времени

`deltaTime` участвует в движении, прицеливании, таймерах и прогрессе ремонта. Скорость 2 единицы/с при `dt=0.1` означает желаемый путь порядка 0.2 единицы за кадр до коллизий. `worldTime` в `CrewTaskCoordinator` используется как временная метка назначения, а не как множитель скорости. Поэтому `assignedAtSeconds` не надо увеличивать каждый кадр. CrewTaskCoordinator.h:104–110 (../src/Gameplay/AI/CrewTaskCoordinator.h#L104). `StateSystem::Update` сразу возвращается при `deltaTime <= 0`, так что пауза не увеличивает ремонт. StateSystem.h:212–217 (../StateSystem.h#L212).

2. Карта данных: кому принадлежит какое состояние

2.1. Сущность, здоровье, фракция

`NPCComponent` хранит имя, описание, `health`, `maxHealth` и старый флаг `isHostile`. `NpcBrainComponent` хранит решение и память маршрута. Разделение видно в NPCComponents.h:601–650 (../NPCComponents.h#L601): здоровье — свойство персонажа, а состояние маршрута — рабочее состояние AI. Если заменить `NpcBrainComponent`, не следует терять здоровье. Если NPC получил урон, не нужно менять маршрут там же, если только тактическая система не получила специальный сигнал.

Боевой выбор фракций находится в NPCComponents.h:51–89 (../NPCComponents.h#L51). `GetFactionRelation(a,b)` даёт `Friendly` для одинаковых `FactionId`, `Neutral`, если хотя бы одна сторона `Neutral`, и `Hostile` для прочих разных фракций. `AreEnemies` — короткая проверка именно `Hostile`.

Старый `isHostile` ещё исключает NPC из мирной repair utility и координатора, но боевой поиск и стрельба проверяют `FactionComponent` пары стрелок–цель. Поэтому для нового противника нужно настроить фракцию, а не надеяться только на `isHostile=true`.

Есть две схемы здоровья цели. Игрок имеет `PlayerComponent::health`, обычный NPC — `NPCComponent::health`; устройство — `DamageableComponent`. `DamageSystem` отдельно обрабатывает игрока, NPC без `DamageableComponent`, затем устройство. Если на NPC одновременно есть `NPCComponent` и `DamageableComponent`, в конце обработка общего `Damageable` синхронизирует `npc.health` с ним. DamageSystem.cpp:196–211 (../src/Gameplay/Combat/DamageSystem.cpp#L196), DamageSystem.cpp:331–339 (../src/Gameplay/Combat/DamageSystem.cpp#L331). При добавлении нового типа персонажа это место придётся осознанно расширять: иначе восприятие не сочтёт его живой целью, даже если на нём есть фракция и Transform.

2.2. `NpcState` — итог текущего шага; `NpcCommand` — внешний приоритет

Перечисление NPCComponents.h:92–109 (../NPCComponents.h#L92) содержит `Idle`, `Wandering`, `MoveToRepair`, `Repairing`, `WaitingForDoor`, `Chasing`, `Searching`, `Attacking`, `TakingCover`, `InCover` и другие состояния. Это не очередь работ. `state` означает, какую ветку `StateSystem::UpdateNpc` выполнит сейчас; несколько систем вправе переписать его за один кадр по описанному порядку. Например, `Utility` пишет `MoveToRepair`, но `NpcCombatSystem` при подтверждённом контакте может записать `Chasing`. Для задач экипажа важно понимать, что координатор успевает поставить `repairTarget` до восприятия боя, но текущий боевой NPC обычно вообще исключён из мирной utility по фракционному признаку.

`NpcCommand::type` — другой слой. Значение `None` означает автономию; `Stop`, `Wander`, `MoveToPosition`, `MoveToRoom`, `FollowEntity`, `RepairEntity` задают ручной приказ. NPCComponents.h:117–126 (../NPCComponents.h#L117). `IsActive()` проверяет `type != None`; `Clear()` стирает целевые ID и повышает `revision`. NPCComponents.h:167–200 (../NPCComponents.h#L167). `revision` передаётся в NavMesh кеш как часть ключа пути. Если меняется приказ, старый маршрут не обязан оставаться пригодным.

Почти все автономные системы проверяют активную команду. `StateSystem::UpdateNpc` делает это первым условием и немедленно `return` после `UpdateManualCommand`. StateSystem.h:395–413 (../StateSystem.h#L395). Это правило сильнее текущего `state`: даже если другой проход случайно поставил `Repairing`, активная команда `Stop` всё равно остановит движение. `Stop`, `Wander`, `FollowEntity` остаются до отмены; `MoveToPosition`, `MoveToRoom`, `RepairEntity` сами очищаются после выполнения. Если цель Follow потеряла Transform, система показывает `Blocked`, но команду не удаляет: сцену можно исправить и продолжить. StateSystem.h:1345–1458 (../StateSystem.h#L1345).

2.3. Движение, двери, рабочая зона

`NPCMovementComponent` задаёт авторские настройки ходьбы 2, бега 4.5 единицы/с, порог бега 8, обычный допуск прибытия 0.45, дистанции открытия и ожидания двери 2.25 и 1.25, глубину выхода за дверь 1.5. NPCComponents.h:616–641 (../NPCComponents.h#L616). `StateSystem` переносит эти настройки в `CharacterMovement` и пишет `walkDirection`, `currentSpeed`, `isMoving`, `isRunning`, `locomotionState`. Поле `canFly` сейчас не влияет на `MoveTowards`: вертикальная компонента направления обнуляется.

В `NpcBrainComponent` лежат `currentRoom`, `navigationTargetRoom`, `doorRoute`, `doorRouteIndex`, `activeDoor`, `routeDirty`, `repathTimer`, параметры ремонта и диагностические строки. Это не просто «большой компонент»; эти поля отвечают на конкретные вопросы: где я сейчас, куда хочу, какие двери запланированы, какую прохожу и нужно ли перестроить путь. NPCComponents.h:646 и далее (../NPCComponents.h#L646). `WorkZoneComponent` на отдельной Entity указывает `owner` оборудования. `StateSystem` ищет её и подбирает точку на поверхности, достижимую по NavMesh, если он доступен. RepairComponent.h:17–20 (../src/Gameplay/Repair/RepairComponent.h#L17), StateSystem.h:135–203 (../StateSystem.h#L135).

2.4. Бой, память, тактика и оружие

`NPCCombatComponent` в NPCComponents.h:475–599 (../NPCComponents.h#L475) хранит `targetEntity`, `targetVisible`, `visibleAimPoint`, радиус зрения и атаки, таймеры, последнее **лично увиденное** положение, наблюдавшуюся скорость и план поиска. `visibleAimPoint` актуальна только при `targetVisible=true`; не используйте старую точку без проверки. `NpcTacticalComponent` хранит авторский и активный стиль, позицию укрытия, позицию выглядывания, таймеры фаз, короткую очередь стрельбы на ходу, счётчик неудачных выглядываний и диагностический статус. NPCComponents.h:305–395 (../NPCComponents.h#L305). `WeaponComponent` находится на Entity оружия; `WeaponSystem::GetEquippedWeaponEntity(registry, owner)` разрешает владельца в оружие. `WeaponOwnerComponent` в событии урона позволяет обратное разрешение оружия в стрелка.

Не смешивайте память наблюдения и текущий Transform цели. Сам `NpcCombatSystem` смотрит на текущую геометрию кандидата, чтобы **проверить**, может ли NPC его увидеть. Но после потери прямой видимости `StateSystem` идёт к `lastKnownPosition`, а `NpcCoverSystem` оценивает укрытие относительно неё. Новую координату скрытой цели эти слои не записывают. NpcCombatSystem.h:373–379 (../NpcCombatSystem.h#L373), NpcCoverSystem.h:199–229 (../NpcCoverSystem.h#L199).

3. Мирный NPC: как задача ремонта превращается в назначение

3.1. `NpcUtilitySystem::Update`: допуск к автоматике

Сигнатура `void Update(ECSRegistry& registry, float deltaTime)` в NpcUtilitySystem.h:54–104 (../NpcUtilitySystem.h#L54) показывает два входа, но `deltaTime` пока намеренно не используется: score строится из текущего мира. `ForEach<NPCComponent,NpcBrainComponent>` получает каждую пару данных. Внутри дополнительно читаются `CrewMemberComponent`, `NPCCombatComponent`, `FactionComponent`. Комбинация боевого компонента с ненейтральной фракцией считается боевым NPC и выводится из старой crew utility. Неактивная сущность, `isHostile` и член экипажа `onDuty=false` также не получают автоматический ремонт. Для них система очищает диагностический score и при отсутствии ручного приказа приводит автоматическое ремонтное состояние к бездействию. Это защита от случайного «дёргания» боевого персонажа между погоней и пробоиной.

Вызов `EvaluateNpc(registry,npc,brain)` происходит только для допущенного NPC. Его выход — изменения `brain.idleScore`, `brain.repairScore`, `brain.repairTarget`, `brain.routeDirty`, `brain.state`, `brain.status`. Он **не добавляет заявку** и не изменяет `RepairableComponent`. Это важно для трассировки: увидев `brain.repairScore=0.8`, нельзя заключать, что цель уже принадлежит этому NPC.

3.2. `EvaluateNpc`: два прохода и арбитраж

В NpcUtilitySystem.h:115–301 (../NpcUtilitySystem.h#L115) базовый `idleScore` принудительно ставится в 0.10. Затем код перебирает все `HullBreachComponent`, считает оценку, запоминает максимум; отдельно перебирает `DamageableComponent`, пропуская объекты, которые одновременно являются пробоинами. Перед оценкой обоих видов проверяется `CrewTaskCoordinator::IsClaimedByOther`, чтобы предварительная utility не отправляла второго NPC на чужую заявку. Внутренний цикл не создаёт маршрут и не вызывает Bullet. Его результат — ID одного лучшего кандидата и численная оценка.

`CalculateBreachRepairScore` в NpcUtilitySystem.h:406–523 (../NpcUtilitySystem.h#L406) сначала проверяет наличие пробоины, `RepairableComponent`, Transform NPC и Brain; запечатанная или помеченная `repaired` пробоина получает 0. Далее считается трёхмерное расстояние `sqrt(dx²+dy²+dz²)`. Если `detectionRange>0` и дистанция больше диапазона, оценка 0. В пределах диапазона `distanceScore = 1 - clamp(distance/detectionRange,0,1)`, `severity = clamp(leakRate/0.15,0.15,1)`, итог `clamp(0.75*severity + 0.25*distanceScore,0,1)`. Коэффициенты выражают политику: интенсивность утечки весит втрое больше близости. При расстоянии 6 из 30 и `leakRate=0.12` получаются `distanceScore=0.8`, `severity=0.8`, итог `0.75*0.8+0.25*0.8=0.8`.

`CalculateEquipmentRepairScore` в NpcUtilitySystem.h:532–702 (../NpcUtilitySystem.h#L532) требует `Damageable`, `Repairable`, оба Transform и Brain; полное здоровье либо `repaired=true` даёт 0. `damageScore` — доля потерянного здоровья `1 - clamp(health/maxHealth,0,1)`. Тип устройства задаёт `importance`: обычное 0.40, генератор 1.0, кислородный генератор 0.90, свет 0.15. Итог `clamp(0.45*damageScore + 0.40*importance + 0.15*distanceScore,0,1)`. Для генератора с 50/100 HP на расстоянии 6 из 30 получаем `0.45*0.5+0.40*1+0.15*0.8=0.745`. Эта таблица зашита в utility; если добавить новый критичный тип оборудования, он получит 0.40, пока вы не расширите правило. Формула координатора другая, поэтому debug `brain.repairScore` не обязан равняться `CrewTaskClaimComponent::priority`.

После сканирования `brain.repairScore` сохраняется для диагностики. Активная ручная команда останавливает дальнейшую запись состояния: score всё равно виден в UI, но команду нельзя перехватить. Если NPC уже `Repairing` и цель ещё пригодна, цель сохраняется. Иначе при новом лучшем `repairTarget` ставится `routeDirty=true`; score больше `idleScore` переводит в `MoveToRepair`, иначе `Idle`. NpcUtilitySystem.h:219–301 (../NpcUtilitySystem.h#L219). Нулевой score не даёт автоматическую работу. Важная особенность: utility рассматривает местную выгодность, но окончательное распределение происходит следующим проходом.

3.3. `CrewTaskCoordinator::Update`: одна цель — один исполнитель

Вход `Update(ECSRegistry& registry,double worldTime)` не получает `dt`. В начале CrewTaskCoordinator.h:110–138 (../src/Gameplay/AI/CrewTaskCoordinator.h#L110) сбрасывается снимок телеметрии, удаляются негодные заявки, собираются незавершённые цели и уже занятые ID. Вектор экипажа строится через `ForEach<NPCComponent,NpcBrainComponent>` и сортируется по Entity ID. Сортировка делает назначение повторяемым при одинаковых входах, несмотря на порядок unordered хранилища. Внутри цикла при отсутствии `CrewMemberComponent` он добавляется с настройками по умолчанию. Это значит, что созданный в редакторе мирный NPC без явной роли всё равно может быть назначен на ремонт.

В CrewTaskCoordinator.h:139–170 (../src/Gameplay/AI/CrewTaskCoordinator.h#L139) неактивный, `isHostile`, `onDuty=false` или управляемый вручную персонаж теряет старую заявку. При ручной команде автоматическое состояние не перезаписывается. Если назначение уже существует и валидно, `KeepExistingAssignment` оставляет его. Эта «липкость» важна: без неё два ремонтника могли бы каждый кадр менять цели при малом изменении расстояния. При смене ID `brain.repairTarget` ставится `routeDirty`, а не перестраивается путь прямо в координаторе.

Здесь есть точная граница между двумя проходами, которую легко пропустить: `NpcUtilitySystem` исключает также сочетание `NPCCombatComponent` с ненейтральной фракцией, а `CrewTaskCoordinator` **не повторяет этот `factionCombatant` фильтр**; он проверяет старый `isHostile`. Значит, боевой персонаж с ненейтральной фракцией, `NPCCombatComponent`, но `isHostile=false` может не участвовать в utility и всё же получить ремонтную заявку от координатора, если он активен и на дежурстве. Для нового вражеского NPC явно задавайте согласованную конфигурацию; для развития кода лучше вынести общий предикат «может работать в экипаже» в одну функцию и вызывать из обоих проходов. Это предложение, а не уже существующая единая политика.

Для свободного NPC цикл по незанятым кандидатам вычисляет `CalculateAssignmentScore` и выбирает только результат выше `brain.idleScore`. При равном score используется меньший ID цели. На выбранную цель ставится `CrewTaskClaimComponent{assignee,type,priority,claimedAtSeconds}`; на NPC пишутся `member.assignedTask`, время, `brain.repairTarget`, `routeDirty`, `MoveToRepair` и статус. `claimedTargets.insert` сразу не даёт следующему NPC этого же кадра занять цель. CrewTaskCoordinator.h:172–220 (../src/Gameplay/AI/CrewTaskCoordinator.h#L172). Инвариант в нормальном проходе: для одной ремонтной цели один `CrewTaskClaimComponent`, и назначенный NPC указывает на неё через `assignedTask`.

Формула координатора CrewTaskCoordinator.h:382–446 (../src/Gameplay/AI/CrewTaskCoordinator.h#L382) **отличается** от формулы utility. Для пробоины серьёзность `clamp(leakRate/0.15,0.2,1)`, для оборудования — доля утраченного HP. Дистанция снова

трёхмерная; за `detectionRange` оценка 0. Профессия меняет множитель: инженер 1.25, Operations 1.05, Security 0.72, Medic 0.62, Generalist 0.90. Operations получает ещё 0.15 для генератора или кислородного генератора. `repairSkill` ограничен диапазоном 0.25–2. Итог:

```
score = clamp((0.65 * severity + 0.35 * distanceScore)
              * roleAffinity * repairSkill, 0, 2)
```

Например, инженер с `repairSkill=1.5`, в 6 единицах от пробоины `leakRate=0.12`, с `detectionRange=30`: `severity=0.8`, `distanceScore=0.8`, базовый score 0.8, итог `0.8*1.25*1.5=1.5`. Он превосходит `idleScore=0.10`. Второй такой же NPC уже увидит цель в `claimedTargets` и должен искать другую. Обратите внимание: оценка использует **геометрическое расстояние**, не длину реального маршрута через двери. Поэтому близкая через стену цель может оказаться назначенной раньше доступной, но более далёкой. Это текущий компромисс алгоритма, а не доказательство достижимости.

`CleanupInvalidClaims` в CrewTaskCoordinator.h:281–308 (../src/Gameplay/AI/CrewTaskCoordinator.h#L281) удаляет заявку, если цель больше не нуждается в ремонте либо исполнитель исчез, выключен, снят с дежурства или получил ручную команду. Удаление делается во втором цикле после сбора ID, чтобы не менять пул во время его перебора. `completedTasks` здесь увеличивается при невалидной цели с совпадающим `assignedTask`; отдельно `RepairSystem` увеличивает его **до уничтожения** успешно закрытой пробоины. Поэтому при аналитике не следует считать `completedTasks` безусловно точным числом только успешных окончаний для всех способов удаления целей. RepairSystem.h:660–690 (../src/Gameplay/Repair/RepairSystem.h#L660).

4. Исполнение ремонта почти по строкам

4.1. `StateSystem::Update`: подготовка физического исполнителя

`StateSystem::Update(ECSRegistry&,float,JobSystem&)` в StateSystem.h:212–357 (../StateSystem.h#L212) проходит по `NpcBrainComponent` и `TransformComponent`. Это шире, чем список `NPCComponent`: любая Entity с мозгом и Transform может дойти до этого кода. Поэтому при добавлении компонентов в редакторе следите за целостностью сущности, а не только за именем класса.

Первые строки защищают паузу и неактивность. Для неактивного NPC система обнуляет `CharacterMovement` и, если есть готовый Bullet контроллер, его walk direction и velocity interval. Неактивность сама по себе не удаляет Entity. Далее проверяется конфликт `RigidBodyComponent` с `btKinematicCharacterController`: оба владельца физики не должны одновременно переписывать один Transform. Код удаляет `RigidBodyComponent` и пишет диагностику. Это изменение состава Entity происходит до получения нижележащих указателей на движение и контроллер.

`EnsureComponent<CharacterMovement>`, `EnsureComponent<CharacterController>`, `EnsureComponent<NPCMovementComponent>` добавляют недостающие runtime компоненты. Контроллер бывает полностью готовым (`controller`, `ghostObj`, `capsuleShape` все есть), полностью пустым либо частично построенным. Пустой спавнится `CharacterSpawnSystem::Spawn`, частичный не пересоздаётся поверх старых Bullet объектов. После этого настройки NPC нормализуются: скорость ходьбы не ниже нуля, бег не ниже ходьбы. Если модель умеет анимации, недостающий `AnimationComponent` добавляется автоматически. Эти шаги важны при странном поведении созданного через редактор NPC: «мозг» сам по себе не создаёт модель и корректный скелет, но часть инфраструктуры движения система действительно создаёт.

Текущая комната определяется каждый кадр `FindContainingRoom` с предпочтением прежней комнаты в перекрывающихся bounds двери. Затем `UpdateNpc` заполняет намерение. После него отдельное разведение капсул добавляет локальное steering направление, если два живых активных NPC слишком близко. Оно не телепортирует Transform, а меняет `walkDirection`. Если в текущем кадре не требовался временный обход коллизии дверного портала, он отключается. Последним вызывается `UpdateStuckStatus`, который сравнивает физический сдвиг Transform после прошлых шагов, а не просто факт желаемого движения. StateSystem.h:297–355 (../StateSystem.h#L297).

4.2. `UpdateNpc`: диспетчер состояний

Сигнатура StateSystem.h:395–403 (../StateSystem.h#L395) принимает Registry, ID, ссылки на Brain, Transform и `CharacterMovement`, константные настройки NPC движения, `deltaTime` и `JobSystem`. Она **не возвращает значение**: выходом являются изменённые компоненты и, для ремонта, вызов `RepairSystem`. Первым проверяется ручная команда. Ниже `switch(brain.state)` направляет `MoveToRepair`, `Repairing` и `WaitingForDoor` в одну функцию `UpdateRepairTask`; это позволяет после открытия двери продолжить ту же работу без нового назначения. `TakingCover` идёт к `tactical->moveTarget`; `Chasing` идёт к последней увиденной точке; `Searching` исполняет план поиска; `Attacking` останавливает нижнюю часть тела, разворачивает корпус и готовит атаку. StateSystem.h:453–538 (../StateSystem.h#L453).

Почему `WaitingForDoor` не самостоятельная «задача»? Потому что дверь — промежуточная фаза **любого** маршрута, а не новая цель экипажа. При каждом кадре `UpdateRepairTask` сначала вновь проверит, что цель всё ещё нужна, и затем снова вызовет `NavigateTo`. Если дверь теперь открыта, маршрут продолжится. Так ремонтник не может бесконечно «ждать старую дверь», когда утечку уже закрыли другим способом.

4.3. `UpdateRepairTask`: вход и две формы цели

Сигнатура StateSystem.h:1629–1639 (../StateSystem.h#L1629) включает `Entity target` и `bool manualCommand`. Один и тот же механизм обслуживает автоматическое назначение и ручной `RepairEntity`; различается только способ завершить работу. На входе по `target` берётся `RepairableComponent*`. Отсутствие компонента либо `repaired=true` немедленно завершает ручную команду через `CompleteManualCommand` или очищает автоматический `brain.repairTarget`, переводит в `Idle`, помечает маршрут грязным и останавливает ходьбу. Никакой прогресс в таком кадре не начисляется.

Дальше функция локально создаёт `repairPosition`, `repairRoom`, `validTarget`, `targetType`. Для пробоины она требует `HullBreachComponent` и `!sealed`, берёт `worldPosition` и `room`. Для оборудования берёт `DamageableComponent`, требует `health < maxHealth`, затем `TransformComponent`. В качестве запасной точки служит Transform самого устройства. `RoomMembershipComponent` даёт комнату, а `FindRepairZoneForOwner` может найти отдельный `WorkZoneComponent`; тогда `TryPickReachableWorkZonePoint` выберет точку на рабочей зоне. StateSystem.h:1671–1776 (../StateSystem.h#L1671). Порядок `if breach ... else if damageable ...` означает, что сущность с обоими компонентами трактуется как пробоина. Это согласовано с пропуском двойного учёта в utility.

`TryPickReachableWorkZonePoint` в StateSystem.h:135–203 (../StateSystem.h#L135) сначала пробует ближайшую точку в зоне, центр и восемь локальных образцов, переведённых в мир. Если известна комната, каждый кандидат должен спроецироваться через `NpcNavMeshSystem::FindReachablePoint` на связный остров сетки. Из успешных берётся ближайший к NPC по горизонтали. Если комната неизвестна, допускаются исходные точки. Если ни один кандидат не подходит при известной комнате, функция возвращает `false`, и остаётся fallback к Transform оборудования. Поэтому отсутствие рабочей зоны не блокирует ремонт, но может привести NPC слишком близко к коллайдеру оборудования. Это место нужно проверить при авторинге сложных аппаратов.

После распознавания типа `!validTarget` снова приводит к ручному завершению или автоматическому `Idle`. Затем StateSystem.h:1813–1862 (../StateSystem.h#L1813) вычисляет **горизонтальное** расстояние до рабочей позиции. Здесь Y специально исключён: ремонтник ходит по палубе, а точка пробоины может быть на стене выше центра капсулы. Это отличается от трёхмерной дистанции в оценке utility и координатора. Если `distance > brain.repairDistance`, ставится `MoveToRepair` и вызывается `NavigateTo` с комнатой, допуском и `movingState`. Результат `Blocked` даёт конкретную строку для пробоины или оборудования; в любом случае функция возвращается до начисления прогресса. `Waiting` тоже означает отсутствие прогресса. Если расстояние уже в допуске, маршрут в этом кадре не вызывается: ходьба останавливается, состояние становится `Repairing`.

4.4. От секунд к прогрессу и завершению

`brain.repairDuration` — время ремонта с нуля при `repairSkill=1`, нижний предел 0.05 с. `CrewMemberComponent::repairSkill` ограничивается диапазоном 0.25–2. В StateSystem.h:1875–1915 (../StateSystem.h#L1875) вычисляется:

```
progressDelta = requiredProgress * dt * repairSkill / max(repairDuration, 0.05)
```

В исходнике эта формула записана так:

```
const float progressDelta =
    repairable->requiredProgress *
    deltaTime *
    repairSkill /
    duration;
```

Здесь множитель `requiredProgress` переводит долю работы за кадр в единицы шкалы компонента, `deltaTime` делает начисление пропорциональным прошедшему времени, а деление на `duration` задаёт полное время при навыке 1. Нельзя заменить `deltaTime` константой «за кадр»: частота 30 и 120 кадров/с тогда дала бы разные скорости ремонта.

Если `requiredProgress=1`, `repairDuration=3`, `dt=0.1`, skill=1.5, то `progressDelta=0.05`. Из нуля потребуется примерно 20 таких шагов, то есть 2 секунды симуляции. Причина деления на duration проста: за duration секунд при skill=1 сумма `requiredProgress * dt / duration` достигает `requiredProgress`. При skill=1.5 скорость в полтора раза выше, полное время — duration/1.5. Если `requiredProgress=5`, за тот же шаг добавится 0.25, а нормированная доля всё равно 0.05. Это делает авторскую шкалу прогресса независимой от скорости труда.

Вызов `RepairSystem::ApplyRepairProgress(registry,target,progressDelta,&jobs)` — общий путь ремонта, не отдельная NPC симуляция. RepairSystem.h:189–234 (../src/Gameplay/Repair/RepairSystem.h#L189) он отвергает отсутствующий, уже законченный компонент и `progressDelta<=0`, увеличивает `progress` с ограничением `requiredProgress`, а до порога возвращает `false`. Достижение численного порога запускает `CompleteRepair`. Для пробоины с `HullBreachRepairLinkComponent` там сначала восстанавливается runtime mesh и collision; если асинхронная операция ещё не готова, результат `false`, хотя численный прогресс уже 100%. На следующих кадрах `ApplyRepairProgress` снова вызывает завершение, пока меш не готов. Поэтому статус `Repair: waiting for runtime operation` означает **не** недостаточную скорость NPC, а ожидание структурного восстановления корпуса. RepairSystem.h:526–556 (../src/Gameplay/Repair/RepairSystem.h#L526).

Если `completed=false`, `StateSystem` заново получает указатель `RepairableComponent*` и лишь затем читает `Progress01()`, ставит `brain.repairProgress` и статус `Repairing hull breach`, `Repairing equipment` либо ожидание runtime операции. StateSystem.h:1921–1967 (../StateSystem.h#L1921). Повторное получение важно даже если сегодняшний конкретный путь при `false` обычно сохраняет Entity: это безопасная привычка после функции, которая может структурно менять ECS. При `completed=true` код **не разыменовывает прежний `repairable`**. Он выставляет debug прогресс 1, очищает `repairTarget`, `doorRoute`, `doorDestinationRooms`, индекс и активную дверь; затем либо снимает ручную команду, либо переводит автоматический NPC в `Idle`. StateSystem.h:1970–2024 (../StateSystem.h#L1970).

При реальном закрытии пробоины `RepairSystem::CompleteRepair` помечает `sealed`, обнуляет `leakRate`, корректирует связанную панель, помечает `repairable.repaired`, учитывает завершённую заявку экипажа, затем вызывает `registry.DestroyEntity(repairEntity)` и возвращает `true`. RepairSystem.h:570–690 (../src/Gameplay/Repair/RepairSystem.h#L570). Для оборудования Entity остаётся: `Damageable.health` становится `maxHealth`, `isDestroyed=false`, генератор включается, у кислородного генератора снимается `broken`, `repairable.repaired=true`. RepairSystem.h:698–765 (../src/Gameplay/Repair/RepairSystem.h#L698). В текущем NPC вызове не переданы `eventLog`, `worldTime`, `repairSource`; необязательные параметры остаются значениями по умолчанию. Поэтому не обещайте, что именно автоматический ремонт запишет `BreachSealed` или `EquipmentRepaired` в `WorldEventLog`, пока не проведёте эти аргументы через вызов. Это конкретный пример полезного улучшения для аналитики и миссий.

4.5. Ручной ремонт и отмена

`UpdateManualCommand` в StateSystem.h:1345–1459 (../StateSystem.h#L1345) при `RepairEntity` передаёт `brain.command.targetEntity` в ту же `UpdateRepairTask` с `manualCommand=true`. Он не обязан владеть `CrewTaskClaimComponent`: ручной приказ имеет другой источник приоритета. При завершении `CompleteManualCommand` вызывает `brain.command.Clear()`, возвращает `Idle`, помечает маршрут грязным, очищает дверной путь и останавливает движение. StateSystem.h:3071–3085 (../StateSystem.h#L3071). На следующем кадре utility снова имеет право назначить автоматическую работу. Для ручного `Stop` команда намеренно не очищается; восстановление автономии требует внешнего снятия команды.

Отказы в ремонте нельзя сводить к одному «не работает». Если NPC вообще не получает цель — смотрите `Active`, `CrewMember.onDuty`, фракцию, `isHostile`, `Repairable`, `Damageable`/`HullBreach`, `repaired`, диапазон и score. Если цель есть, но NPC не идёт — смотрите `brain.state`, `routeDirty`, `currentRoom`, дверной граф, NavMesh и `CharacterController`. Если стоит рядом, но не ремонтирует — сравните **горизонтальное** расстояние с `repairDistance` и проверьте, не выбрана ли недоступная fallback позиция устройства. Если `progress=100%`, но цель не исчезла — смотрите завершение async mesh операции.

5. Навигация: граф комнат, локальная сетка и Bullet капсула

5.1. `NavigateTo` — общий контракт для разных задач

Сигнатура StateSystem.h:2184–2196 (../StateSystem.h#L2184) принимает `finalTarget`, `targetRoom`, `arrivalDistance`, `movingState` и `dt`. Возвращаемый внутренний `NavigationResult` может означать `Moving`, `Arrived`, `Waiting` или `Blocked`; вызвавшая ветка решает, что это значит для своей задачи. На входе стоит **мировая** цель. На выходе `NavigateTo` не меняет Transform напрямую: он заполняет `CharacterMovement`. Поэтому если при отладке `walkDirection` ненулевой, а позиция стоит, проблема уже ниже AI — в физике, коллизии или контроллере.

В начале `repathTimer` уменьшается на `dt`, debug поля дверного шага сбрасываются. Если `currentRoom` и `targetRoom` известны и различаются, нужен маршрут по комнатам. Если переход через дверь уже начат, он сохраняется, даже если `currentRoom` на пороге временно сменился: иначе путь обрывался бы посередине проёма. Новый target room либо `routeDirty` после истечения `repathTimer` вызывает `BuildDoorRoute`. Неудача ставит `Blocked` с `No traversable door route to target room`; ручная команда при этом сохраняется. Если маршрут грязный, но кулдаун перестроения ещё не истёк, NPC безопасно стоит. Если обе точки уже в одной комнате, промежуточный дверной путь очищается. StateSystem.h:2201–2270 (../StateSystem.h#L2201).

`BuildDoorRoute` в StateSystem.h:2503–2582 (../StateSystem.h#L2503) строит граф: вершина — `RoomComponent` Entity, ребро — `DoorComponent(roomA,roomB)`. BFS выбирает путь с минимальным **числом дверей**, так как каждая дверь имеет одинаковую стоимость. Дверь с требованием питания исключается, если она не открыта, не сломана и питания нет; сломанная дверь считается проходимой по контракту `DoorSystem`. `visited` хранит предыдущую комнату и дверь, затем путь разворачивается в порядок прохождения. `doorRoute` и `doorDestinationRooms` — параллельные векторы одинаковых шагов. Если позже появится «опасная дверь», «запертая дверь для определённой роли» или стоимость перехода по времени, BFS уже не выразит это корректно; понадобится весовой алгоритм и политика доступа.

`FindContainingRoom` в StateSystem.h:2584–2632 (../StateSystem.h#L2584) преобразует мировую позицию обратной мировой матрицей комнаты в локальное пространство и сравнивает с локальными bounds. Это принципиально для повернутой или масштабированной комнаты: сравнивать мировую точку прямо с `localMin/max` нельзя. Предпочтение прежней комнаты предотвращает прыжки ID в перекрывающихся объёмах у двери. Возврат `INVALID_ENTITY` допустим; тогда верхний граф не строится и остаётся локальное прямое направление. На сложной геометрии это может довести NPC до стены, поэтому пустой `currentRoom` в диагностике — значимый симптом, а не всегда безобидная мелочь.

5.2. Дверной переход как последовательность фаз

В StateSystem.h:2272–2499 (../StateSystem.h#L2272) цикл берёт очередную дверь, её `DoorComponent` и мировую позицию. Отсутствие двери или Transform помечает маршрут грязным и возвращает `Blocked` без чтения несуществующих данных. Затем вычисляется точка выхода **внутри следующей комнаты** на `doorExitDistance`; точка подхода смещена назад от оси выхода. Эти две точки предотвращают типичную ошибку: кратчайший NavMesh путь к центру двери приводит капсулу по диагонали в косяк.

Когда дистанция до двери не больше `doorOpenDistance`, вызывается `DoorComponent::TryOpen(hasPower)`. NPC не анимирует створки; он задаёт намерение, а `DoorSystem` продвигает `openProgress`. Если открыть нельзя, `Blocked` показывает причину. Когда NPC подошёл ближе `doorWaitDistance`, а `openProgress < 0.65`, он останавливается с `WaitingForDoor`. После открытия сначала выравнивается к approach point; затем короткий участок через портал проходит напрямую к exit point, потому что NavMesh исходной комнаты не может проектировать точку в соседней. В этой фазе код временно игнорирует коллизию **конкретной открытой двери** для капсулы и восстанавливает её вне перехода. Дверной шаг считается законченным после достаточного смещения за плоскость двери или близости к exit point. Только после этого начинается движение к следующей двери или финальной цели.

Для понимания проекции у двери: `progressBeyondDoor` — скалярное произведение вектора от двери к NPC с осью выхода, делённое на длину оси. Положительное значение означает продвижение в сторону следующей комнаты. Сравнивать только `currentRoom` недостаточно, потому что bounds комнат могут пересекаться прямо в проёме. Поэтому дополнительно требуется `progressBeyondDoor >= max(0.35,0.75*doorExitDistance)` или фактическая близость к точке выхода. Это локальный пример того, зачем в игре нужна векторная математика, а не только «расстояние до цели».

5.3. `MoveTowardsNavigated`, NavMesh и движение

`MoveTowardsNavigated` в StateSystem.h:2950–3020 (../StateSystem.h#L2950) сначала запрашивает `NpcNavMeshSystem::FindNextWaypoint`. Если запрос успешен, текущей целью становится ближайший waypoint; иначе используется исходная цель. Поверх него `FindDynamicAvoidanceWaypoint` может выдать короткий объезд подвижного Bullet AABB. Для подхода к двери `requireForwardProgress` дополнительно проверяет положительное скалярное произведение направления кандидата с направлением к двери: боковой обход не должен отправить NPC назад от единственного прохода. Это не решение всей задачи планирования пути; это локальная защита конкретного портального участка.

В конфигурации проекта DX_FrameWork.vcxproj:336–338 (../DX_FrameWork.vcxproj#L336) свойство `NpcNavMeshEnabled` по умолчанию 1, а `NpcNavMeshSystem.cpp` включён в компиляцию. Заголовок NpcNavMeshSystem.h:85–180 (../src/Gameplay/AI/NpcNavMeshSystem.h#L85) имеет условные заглушки при `NPC_NAVMESH_ENABLED=0`. Тогда запросы возвращают `false`, `StateSystem` идёт напрямую, а не падает. При включённом слое NpcNavMeshSystem.cpp:1495–1541 (../src/Gameplay/AI/NpcNavMeshSystem.cpp#L1495) кеширует путь по NPC, комнате, `command.revision` и изменению цели больше 0.60. Неудачный запрос не повторяется каждый кадр до изменения ключа или явной инвалидизации. `FindReachablePoint` проектирует желаемую точку на связный остров. Значит, в отладке «NavMesh не нашёл путь» и «StateSystem не пытался идти» — разные случаи: при первом может действовать прямой fallback.

`MoveTowards` в StateSystem.h:3024–3056 (../StateSystem.h#L3024) берёт `to-from`, обнуляет Y, вычисляет длину, делит X и Z на неё и записывает единичное направление. Без нормализации NPC ускорялся бы с расстоянием до цели: вектор `(10,0,0)` сильнее `(1,0,0)` при одной и той же настройке скорости. Нулевая длина вызывает `StopMovement`, чтобы не делить на ноль. Выбор Walk/Run идёт по `allowRunning && distance >= runWhenFartherThan`; это решение по локальной текущей цели движения. Физический сдвиг происходит позднее через `CharacterControllerSystem` и Bullet, а анимация читает `locomotionState`. ApplicationClass.cpp:1537–1548 (../src/Core/ApplicationClass.cpp#L1537).

`UpdateStuckStatus` в StateSystem.h:3108–3180 (../StateSystem.h#L3108) при неподвижном намерении обнуляет таймер; при движении сравнивает новую позицию с `lastProgressPosition`. Порог в коде `0.08`

единицы, несмотря на комментарий «3 см» рядом с ним; ориентируйтесь на число в условии. Если прогресса нет в течение `stuckTimeout` (в Brain по умолчанию 2 с), собирается подробный snapshot двери, позиции, фаз и направления. Не пытайтесь чинить это простым бесконечным `repath` каждый кадр: если геометрия запирает капсулу, новые одинаковые пути не сдвинут физический объект.

6. Противник: выбор, зрение, память и состояния боя

6.1. `NpcCombatSystem::Update`: выбор кандидата

Сигнатура NpcCombatSystem.h:42–47 (../NpcCombatSystem.h#L42) принимает Registry, `deltaTime`, Bullet `dynamicsWorld` и настройки отладочных лучей. Главный `ForEach<NPCCombatComponent>` означает: участие в **этом** восприятии определяется наличием боевого компонента. Внутри получаются `NPCComponent`, `NpcBrainComponent`, `TransformComponent`; без одного из них NPC пропускается. Активная ручная команда сбрасывает время стационарного прицеливания и не даёт автоматическому бою принять решение. Отсутствие `FactionComponent` сбрасывает `targetVisible` и прицеливание. NpcCombatSystem.h:57–97 (../NpcCombatSystem.h#L57).

Далее внутренний `ForEach<FactionComponent,TransformComponent>` перебирает возможные цели. Кандидат не может быть самим наблюдателем, должен быть активным, иметь `NPCComponent` или `PlayerComponent`, иметь здоровье больше нуля и быть враждебным по `AreEnemies`. Сравнивается `candidateDistanceSquared = dx²+dy²+dz²`, выбирается ближайшая по трёхмерной дистанции Entity. Квадратный корень не нужен, так как для неотрицательных чисел `a<b` тогда и только тогда, когда `sqrt(a)<sqrt(b)`. Это небольшая, но правильная экономия внутри вложенного цикла. NpcCombatSystem.h:101–169 (../NpcCombatSystem.h#L101).

Важное ограничение текущей реализации: **сначала** выбирается ближайший живой враг, и **только затем** проверяются FOV и заслонение. Если близкий враг находится за стеной, а другой дальше, но виден, второй не проверяется как запасная визуальная цель в этом проходе. Это не сознательная система приоритета угроз, а следствие порядка фильтров. Для улучшения позже выбирайте ближайшего среди **видимых** кандидатов либо отдельно храните ближайшего и лучшего видимого; при этом не ломайте личную память цели. При полном отсутствии живых врагов `targetEntity` очищается, `targetVisible=false`, `rangedAimTime=0`.

Ещё одно точное различие guard условий: внутри `NpcCombatSystem::Update` для **наблюдателя** проверяется наличие `NPCComponent`, Brain, Transform, команды и фракции, но нет отдельного раннего `ActiveComponent` или `npcData.health>0` guard. Поздний `NpcRangedCombatSystem` и `NpcCoverSystem` такие ограничения ставят. Поэтому выключенный или мёртвый NPC теоретически может ещё обновить свои поля восприятия, хотя стрелять и выполнять движение не должен. При рефакторинге добавьте единый предикат жизнеспособного наблюдателя и проверьте обратную совместимость с отключением Entity. Это предложение; сегодняшнюю трассу выстраивайте по существующим ранним выходам.

Указатель `candidateTransform` из внутренней лямбды не сохраняется наружу. После выбора ID код заново получает `targetTransform` — правильный способ избежать ссылки на временный контекст выбора. `stanceTimeRemaining` уменьшается на `dt`. Прежде чем считать дистанцию атаки, система отдельно готовит геометрию наблюдения. NpcCombatSystem.h:171–205 (../NpcCombatSystem.h#L171).

6.2. Сектор зрения и три луча

Глаза NPC по умолчанию находятся на `position.y+1.6`. При доступной анимации и кости `mixamorig:Head` точка берётся из актуальной матрицы головы, умноженной на мировую матрицу персонажа. NpcCombatSystem.h:208–221 (../NpcCombatSystem.h#L208). Это объясняет, почему один и тот же NPC может видеть поверх низкой стены при подходящей позе, а константная точка на высоте корпуса дала бы ложное заслонение. Для цели пробуются высоты груди 1.0, головы игрока по `PlayCameraComponent::firstPersonEyeOffset.y` или 1.6 для NPC, затем нижней части корпуса 0.55.

До дорогих лучей вычисляются расстояние от глаз к груди и горизонтальный сектор. Текущий forward для yaw модели:

```
XMFLOAT3 forward = { -sin(rotation.y), 0.0f, -cos(rotation.y) };
bool inCone = dot >= cos(radians(fieldOfViewDegrees * 0.5f));
```

При `fieldOfViewDegrees=120` половина сектора 60°, `cos(60°)=0.5`. Цель точно впереди даёт dot около 1 и проходит, под углом 75° даёт примерно 0.259 и отбрасывается. Вектор к цели нормализуется **по XZ**, потому что сектор должен быть горизонтальным; высота цели сама по себе не меняет yaw обзора. Если горизонтальные координаты почти совпали, сектор считается пройденным, чтобы избежать деления на почти ноль. NpcCombatSystem.h:258–298 (../NpcCombatSystem.h#L258).

После теста диапазона и сектора цикл пробует три высоты. Для каждой требуется ненулевой `dynamicsWorld`, длина луча больше 0.001 и не больше `seeRange`. `PhysicsRaycast::CastRay` идёт от глаз к точке, исключая самого NPC; цель видна, если луч никого не задел или первым задел именно целевую Entity. При первой успешной высоте `visibleAimPoint` запоминает **эту** открытую точку, debug renderer может нарисовать луч. NpcCombatSystem.h:300–352 (../NpcCombatSystem.h#L300). Отсутствие Bullet мира здесь не означает «видно всё»: цикл продолжает без успешного результата. Это полезная безопасная семантика при загрузке/остановке мира.

6.3. Память не равна всеведению

После лучей `previouslyVisible` истинно, только если в прошлом кадре была видна **эта же** цель. Затем записывается новый `targetVisible`. Если видимость подтверждена, `combat.targetEntity=bestTargetEntity`, и `ObserveVisibleTarget(targetTransform->position,dt,previouslyVisible)` обновляет факт наблюдения. NpcCombatSystem.h:354–379 (../NpcCombatSystem.h#L354). В NPCComponents.h:535–551 (../NPCComponents.h#L535) `ObserveVisibleTarget` сначала обнуляет `lastSeenVelocity`, а при непрерывной видимости вычисляет `Δx/dt` и `Δz/dt` по двум наблюдениям; Y не предсказывается. После повторного выхода из за стены скорость начинается заново, чтобы прыжок скрытой цели не превратился в «измеренную» скорость. Функция обнуляет таймер забывания и старый поисковый план.

При потере видимости новая текущая позиция цели не попадает в память. `EstimateSearchPosition()` в NPCComponents.h:555–570 (../NPCComponents.h#L555) берёт `lastKnownPosition + lastSeenVelocity*predictionSeconds` по горизонтали, ограничивая длину прогноза `maxPredictionDistance` (по умолчанию 3). Например, последняя лично увиденная точка `(10,0,5)`, скорость `(4,0,0)` единиц/с и горизонт 0.8 с дают сырое упреждение 3.2; ограничение до 3 даёт план `(13,0,5)`. Это гипотеза, не новое наблюдение. Пока цель скрыта, её реальный Transform не должен «таскать» план вслед за ней.

Когда цель видна, вооружённый NPC входит в `Attacking` при расстоянии не больше `min(combat.rangedAttackRange,weapon.range)`; иначе `Chasing`. Без оружия используется `attackRange` и гистерезис `ShouldHoldMeleeStance`. При входе в стойку `EnterMeleeStance` устанавливает минимальную паузу до удара, не сокращая уже более длинный cooldown. Система не сбрасывает активный путь к укрытию, если NPC сейчас исполняет неоткрытую cover фазу: `executingCoverOrder` удерживает прежнее состояние. NpcCombatSystem.h:407–435 (../NpcCombatSystem.h#L407). Это пример арбитража между сенсором и тактическим исполнителем: видимость — факт, но не любой факт немедленно отменяет уже выданную фазу движения.

Если ранее выбранная цель пропала из зрения, но Entity остаётся, NPC продолжает поиск. `timeSinceTargetSeen` растёт только в `Searching`, когда план уже готов и NPC больше не идёт к предполагаемой точке. `loseTargetDelay` по умолчанию 3 с; до забывания завершается круговой осмотр `searchTurnRemaining` и при соответствующем стиле назначенные сектора. Только затем `targetEntity` и личная память очищаются, `state=Idle`. NpcCombatSystem.h:436–467 (../NpcCombatSystem.h#L436). Не путайте этот таймер с «прошло 3 секунды с первого кадра невидимости»: на пути к последней позиции время ожидания не расходуется.

6.4. `StateSystem` в погоне, поиске и атаке

В `Chasing` StateSystem.h:642–817 (../StateSystem.h#L642) проверяется цель и память. Для маршрута используется `combat.lastKnownPosition`, не скрытый текущий Transform, хотя Transform цели первоначально читается для валидации существования. У безоружного NPC включается бег уже на коротком расстоянии; вооружённый учитывает стиль, входящий огонь и желаемую дистанцию. `Defensive` удерживает позицию, прекращает наступательные поисковые приказы и при потере контакта переходит в `Searching`, где не бежит вслед за предположением. `Assault` и `Balanced` могут исследовать продолжение наблюдаемой скорости и ограниченное число секторов. При достижении последней точки без видимой цели состояние становится `Searching`, а не автоматически `Attacking`.

В `Searching` StateSystem.h:819–915 (../StateSystem.h#L819) сначала останавливается ходьба и проверяется существование наблюдения. План строится один раз на эпизод потери цели: `searchPlanReady=true`, `BeginSearchTurn()` задаёт полный оборот `2π`, таймеры сбрасываются. `AdvanceSearchTurn(yaw,rotationSpeed,dt)` ограничивает шаг остатком угла и возвращает нормализованный угол через `atan2(sin,cos)`. При скорости 90°/с и `dt=0.1` один шаг 9°, полный поворот требует примерно 40 шагов. Если скорость настройки равна нулю, используется безопасный запас 90°/с. NPCComponents.h:511–530 (../NPCComponents.h#L511). Важна конечность этого действия: угловой остаток уменьшается, и NPC не вращается вечно, если дальше должен забыть цель.

В `Attacking` StateSystem.h:1156–1335 (../StateSystem.h#L1156) сначала проверяются живой ID, личная видимость, Transform цели и дистанция. При выходе за дальность NPC снова `Chasing`; при потере прямого зрения стоящий стрелок останавливается и тоже переходит к погоне/поиску. В состоянии атаки `StopMovement` сразу делает нижнюю анимацию Idle, корпус разворачивается к цели ограниченным угловым шагом `rotationSpeed*dt`. Допуск 8° проверяется **после** поворота. Для вооружённого NPC этот проход только накапливает `rangedAimTime`; фактический луч из дула будет построен после обновления скелета. Для безоружного проход отсчитывает `attackTimer`, уважает паузу стойки и активный Punch клип, выбирает взвешенный вариант анимации и запрашивает `AnimationState::Punch`. Сейчас эта ветка, согласно коду и его собственному комментарию, **создаёт анимацию, но сама не наносит melee урон**. Не объявляйте рукопашный бой готовым для релиза без отдельной проверки hit window, дальности и события урона.

7. Укрытие, оружие и очередь урона

7.1. `NpcCoverSystem::Update`: тактика одного NPC

Сигнатура NpcCoverSystem.h:36–46 (../NpcCoverSystem.h#L36) требует Bullet мир и положительное конечное `dt`. `ForEach<NPCCombatComponent>` дополнительно требует `NpcBrainComponent`, `NpcTacticalComponent`, `FactionComponent`, `TransformComponent`, `NPCComponent`. Нет одного — нет решения укрытия. Система уменьшает `underFireTime`, `decisionTimer`, `phaseTimer`, таймер отклонённой точки и другие индивидуальные интервалы. `damageReactionPending` будит новое решение, но сам сигнал не сообщает точную позицию стрелявшего: для геометрии используется лично наблюдавшаяся `lastKnownPosition`. Неактивный, мёртвый, управляемый вручную NPC или персонаж без подтверждённой вражеской цели и памяти выходит до поиска позиции. NpcCoverSystem.h:90–229 (../NpcCoverSystem.h#L90).

Выбранный `reservedCover` — авторская Entity с `NpcCoverPointComponent`; динамическое укрытие Entity не имеет и отмечено `dynamicCoverActive`. При существующем укрытии `NpcCoverQuery::IsProtectedFromTarget` проверяет защиту относительно последней наблюдавшейся позиции. Один неудачный raycast увеличивает `coverValidationFailureTime`, но позиция сохраняется до `coverValidationGraceDuration`; это гасит однокадровое дрожание у края коллайдера. NpcCoverSystem.h:230–285 (../NpcCoverSystem.h#L230). Если NPC открыт в peek фазе, истёк таймер или он получил попадание, цель движения возвращается к `coverPosition`, состояние становится `TakingCover`. Попадания во время peek увеличивают `failedPeekCount`; после порога запрашивается смена укрытия, но она ждёт окончания `coverHoldTimer`. Когда скрытая фаза заканчивается,

`moveTarget=peekPosition`, NPC идёт выглянуть. NpcCoverSystem.h:287–375 (../NpcCoverSystem.h#L287).

Когда укрытие нужно выбрать, `NpcTacticalDecisionSystem::EvaluateCoverRequest` учитывает стиль и угрозу. `NpcCoverQuery::FindBest` ищет авторскую или динамическую точку по текущей комнате, позиции NPC, памяти о цели и параметрам тактики. Если не найдено — ставится `noSuitableCover` и cooldown, чтобы не делать дорогой поиск каждый кадр. Если найдено — сохраняются `coverPosition`, `peekPosition`, `moveTarget`, тип укрытия, таймер удержания и `brain.state=TakingCover`; старый NavMesh путь инвалидируется. NpcCoverSystem.h:390–479 (../NpcCoverSystem.h#L390). `NpcCoverSystem` не перемещает NPC. На следующем проходе **того же кадра** `StateSystem` увидит `TakingCover` и построит движение. При прибытии к скрытой точке он ставит `InCover` и таймер ожидания, при прибытии к peek — `exposed=true`, `Attacking` и окно огня не короче `rangedAimDelay+1.2`. StateSystem.h:539–600 (../StateSystem.h#L539).

`ReleaseCurrentCover` NpcCoverSystem.h:490–506 (../NpcCoverSystem.h#L490) временно запрещает немедленное повторное избрание старой авторской точки на 4 с и очищает runtime флаги. Это лучше, чем бесконечный цикл «эта же точка снова лучшая — она же снова не защищает». Параметры `hideDuration`, `exposeDuration`, `minimumCoverHoldDuration`, `failedPeeksBeforeRelocate` из `NpcTacticalComponent` дают четыре разных рычага. Не подменяйте их одним общим «агрессивность»: один управляет ритмом выглядывания, другой устойчивостью к смене позиции.

7.2. `NpcRangedCombatSystem::Update`: серия обязательных ворот

Сигнатура NpcRangedCombatSystem.h:30–37 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L30) получает Registry, Bullet мир, `DamageSystem`, D3D устройство, `JobSystem` и debug параметры. `ForEach<NPCComponent,NPCCombatComponent>` начинает с `rangedStatus="Not shooting"`. Потом последовательно отсекаются отсутствующий Brain, мир, `Active`, `PlayerGameTag`, фракция, нулевое здоровье, активная команда, отсутствие экипированного оружия. NpcRangedCombatSystem.h:39–118 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L39). Это полезная отладочная лестница: если ранний guard вернул управление без специальной строки, новый статус останется `Not shooting`.

При `weapon.isReloading` стрельба заканчивается статусом `Reloading`. При пустом магазине запрашивается `WeaponSystem::RequestReload`; статус сообщает, началась ли перезарядка или боеприпасы исчерпаны. Затем определяется `movingFire`: требуется тактический компонент, подходящее состояние `Chasing` или `TakingCover` и личная видимость. На ходу используются собственные `movingAimTime`, размер и пауза короткой очереди. Стационарный NPC обязан быть в `Attacking`, и **в обоих режимах** нужен `combat.targetVisible` именно в текущем кадре. Пока соответствующий таймер меньше `rangedAimDelay`, выстрел не создаётся. NpcRangedCombatSystem.h:120–189 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L120).

После этого цель вновь проверяется: живая Entity, активна, не мертва, имеет фракцию и остаётся враждебной. Повторная фракционная проверка непосредственно перед огнём защищает от устаревшего `targetEntity`, если отношения изменились между выбором и выстрелом. NpcRangedCombatSystem.h:192–245 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L192). Для вычисления мирового дула нужны Transform, `CharacterMovement`, модель и анимация. Кость руки берётся по `weapon.rightHandBone`; если она отсутствует, статус `Missing right-hand bone`. `muzzlePosition` — локальная точка оружия; цепочка `GripMatrix(worldGrip) * hand * transform.GetWorldMatrix()` переводит её через хват, кость и мир. Порядок умножения связан с принятой в DirectXMath конвенцией проекта. Если заменить порядок на `world * hand * grip`, дуло окажется не там, где рука. NpcRangedCombatSystem.h:247–307 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L247).

Направление выстрела — `visibleAimPoint - origin`. `WeaponFiringRules::ValidRay` отбрасывает некорректный или нулевой вектор. Дальность сверяется с `weapon.range`, затем корпус должен смотреть на цель с ошибкой не более 8°. Ошибка yaw нормализуется через `atan2(sin(Δ),cos(Δ))`, иначе при переходе через ±π обычное вычитание выбрало бы длинный оборот. NpcRangedCombatSystem.h:309–363 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L309).

Наконец короткий луч от груди к дулу проверяет, не оказался ли длинный ствол за укрытием, а отдельный луч от дула к видимой точке блокирует огонь, если первым на линии стоит живой NPC дружественной фракции. NpcRangedCombatSystem.h:365–451 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L365).

Только после всех ворот ставится `weapon.triggerHeld=true` и вызывается `WeaponSystem::TryFire`. Результат записывается как `Fired`, `Reloading`, `Weapon cooling down` либо `Weapon unavailable`. Если это успешный выстрел на ходу при свободной линии, `firedOnRoute` и счётчик очереди обновляются; по исчерпании очереди выставляется `movingBurstPauseTimer`. NpcRangedCombatSystem.h:453–500 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L453). **Подтверждённый дефект текущего кода:** `NpcRangedCombatSystem` проверяет положительный `movingBurstPauseTimer` перед следующей очередью, но ни он, ни другой проход его не уменьшает. Пока `movingFire` остаётся активным, стрельба на ходу может зависнуть после первой очереди со статусом `Moving: pause between bursts`; переход к обычной стрельбе сбрасывает таймер. NpcRangedCombatSystem.h:155–174 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L155). `triggerHeld` — признак попытки спуска в этом шаге, а не доказательство попадания. Доказательство состоявшегося выстрела — возврат `TryFire=true`; доказательство попадания — результат физического луча внутри оружейной системы.

Короткая цитата из NpcRangedCombatSystem.h:176–181 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L176) фиксирует главный запрет:

```
if ((!movingFire && brain.state != NpcState::Attacking) || !combat.targetVisible)
{
    combat.rangedStatus = "No visible target in firing stance";
    return;
}
```

Внешние скобки важны: при разрешённом `movingFire` стационарная стойка не нужна, но `!combat.targetVisible` запрещает **оба** режима. Это место стоит перечитать перед добавлением подавляющего огня: простой новый state не превратится в возможность стрельбы по невидимой цели.

7.3. `WeaponSystem::TryFire` и `DamageSystem::Update`

`WeaponSystem::TryFire` в WeaponSystem.h:75–155 (../src/Gameplay/Combat/WeaponSystem.h#L75) — общая точка игрока и NPC. Он проверяет мир, живого и активного стрелка, корректный луч, оружие, патроны и `WeaponFiringRules::CanFire`. При пустом магазине пытается начать механическую перезарядку. Затем нормализует направление и рассчитывает множитель разброса от движения. Две случайные выборки дают направление внутри конуса с учётом `spreadDegrees` и текущей отдачи; `CommitShot` расходует патрон и запускает cooldown. После этого Bullet луч длиной `weapon.range` останавливается на **первом** объекте. Даже промах считается состоявшимся выстрелом и возвращает `true`. Если луч задел стену перед выбранным AI врагом, урон цели за стеной не переносится: `DamageEvent.target` — Entity физического первого попадания.

Две точные строки из WeaponSystem.h:121–131 (../src/Gameplay/Combat/WeaponSystem.h#L121) показывают различие между фактом выстрела и фактом попадания:

```
const bool hit = PhysicsRaycast::CastRay(origin, normalized, world,
    weapon->range, shooterEntity, shooterObject, rayHit);
```

После необязательной отладочной отрисовки стоит отдельная строка:

```
if (!hit) return true;
```

Здесь `true` у `TryFire` означает, что патрон уже израсходован и луч создан. Логический `hit=false` означает отсутствие первого пересечения, поэтому события урона нет. В этом движке «пуля» обычного оружия — **hitscan луч**, не отдельная летящая Entity с траекторией и временем полёта. `Bullet` в названиях физического мира — библиотека коллизий; она проверяет пересечение луча и управляет капсулой NPC. Это два разных смысла слова «bullet». Если позже понадобится медленный плазменный снаряд, ему потребуется собственный компонент движения, время жизни и обработка столкновения; текущий `TryFire` сам такой снаряд не создаёт.

`DamageEvent` содержит Entity оружия как `source`, Entity попадания как `target`, число урона, тип, точку, нормаль и параметры возможной пробоины. Для корпуса дополнительно вычисляются разрешение

и порог резки меша. Затем событие добавляется в очередь `DamageSystem`. WeaponSystem.h:133–155 (../src/Gameplay/Combat/WeaponSystem.h#L133). `DamageSystem::Update` позже обрабатывает очередь: проверяет валидную активную цель и положительный урон; ставит `NpcTacticalComponent::underFireTime=3`, считает попадания за окно 4 с и может поставить `damageReactionPending`. Событие оружия разрешается через `WeaponOwnerComponent` во владельца, но боевой `targetEntity` жертвы не заполняется магически — направление входящего огня лишь короткая ориентировка. DamageSystem.cpp:104–189 (../src/Gameplay/Combat/DamageSystem.cpp#L104). `InvulnerableComponent` и god mode не уменьшают здоровье; обычная тактическая реакция на попадание при этом может всё же появиться, кроме специально тихого события `silentKill`.

Для NPC без общего `Damageable` здоровье уменьшается с ограничением от 0 до max; при нуле снимается `ActiveComponent`. Для устройства `Damageable.health` уменьшается, `Normalize` приводит состояние к допустимому диапазону, при переходе в уничтоженное состояние создаётся взрыв и событие уничтожения, а `Repairable.progress` сбрасывается в 0, `repaired=false`. DamageSystem.cpp:204–211 (../src/Gameplay/Combat/DamageSystem.cpp#L204), DamageSystem.cpp:331–369 (../src/Gameplay/Combat/DamageSystem.cpp#L331). Последнее связывает бой с работой экипажа: после попадания повреждённое оборудование снова становится кандидатом ремонта на следующем AI проходе.

8. Прогоним данные руками: два проверяемых сценария

8.1. Один кадр назначения и один кадр фактического ремонта

Зададим **учебную сцену** с активным мирным NPC `Entity=10` в `(0,0,0)` и активной незапечатанной пробоиной `Entity=40` в `(6,0,0)`. У NPC есть `NPCComponent{isHostile=false}`, `NpcBrainComponent` с `detectionRange=30`, `idleScore` после utility 0.10, `repairDistance=1`, `repairDuration=3`, `CrewMemberComponent{role=Engineer,onDuty=true,repairSkill=1.5}`. У пробоины есть `HullBreachComponent{leakRate=0.12,room=5}`, `RepairableComponent{progress=0,requiredProgress=1,repaired=false}`. Путь по комнатам и Bullet капсула считаются корректными. Пусть `dt=0.1` с. Это выбранные числа для расчёта, а не обещание о конкретной сохранённой сцене.

На первом проходе `NpcUtilitySystem` видит `Entity=40`, заявок другого NPC нет. `distance=6`, `distanceScore=1-6/30=0.8`, `severity=clamp(0.12/0.15,0.15,1)=0.8`, `repairScore=0.8`. Так как `0.8>0.10`, он пишет `brain.repairTarget=40`, `state=MoveToRepair`, `routeDirty=true`. Если рядом есть второй мирный NPC, на этом предварительном этапе **оба** могут посчитать одну пробоину лучшей; это пока только заявка в голове каждого.

Затем координатор заново собирает кандидатов. Для инженера базовая смесь `0.65*0.8+0.35*0.8=0.8`, роль 1.25, skill 1.5, итог `1.5`. Он создаёт `CrewTaskClaimComponent` на пробоине: `assignee=10`, `type=RepairBreach`, `priority=1.5`, время текущего мира. У NPC `assignedTask=40`. Если следующий NPC в том же проходе увидит `claimedTargets` с 40, он уже не возьмёт эту пробоину. Обратите внимание: `brain.repairScore=0.8` и `claim.priority=1.5` одновременно корректны, потому что это разные формулы и разные задачи проходов.

В том же кадре `StateSystem` до физики видит расстояние 6>1. `UpdateRepairTask` ставит `MoveToRepair`, вызывает `NavigateTo` к `(6,0,0)`. Допустим комната та же и первый waypoint совпадает с финальной точкой. `MoveTowards` пишет `walkDirection=(1,0,0)`, `currentSpeed=2` при обычной ходьбе; `Repairable.progress` всё ещё 0. Позднее `CharacterControllerSystem` передаст движение Bullet. Если нет коллизий, ожидаемый шаг порядка `2*0.1=0.2` единицы; фактическая позиция может отличаться из за физики. Нельзя требовать увеличения прогресса уже в кадре назначения при дистанции 6.

Предположим, после нескольких шагов Transform NPC на **начале** следующего AI кадра находится в `(5.1,0,0)`, то есть горизонтальная дистанция до пробоины 0.9. `Utility` сохраняет или повторно выбирает цель, `KeepExistingAssignment` сохраняет заявку без скачка к другой аварии.

`StateSystem::UpdateRepairTask` теперь не вызывает `NavigateTo`, а останавливает ходьбу, ставит `Repairing`. За `dt=0.1` вычисляется `progressDelta=1*0.1*1.5/3=0.05`; `RepairSystem` записывает `progress=0.05` и возвращает `false`, потому что до `requiredProgress=1` ещё далеко. `brain.repairProgress=0.05`. Через 20 таких рабочих шагов сумма достигнет 1.0, если авария не была повторно повреждена и dt не менялся. При пробоине с async mesh восстановлением число уже может показывать 1.0, а `completed=false` продлится до готовности операции. После фактического `DestroyEntity(40)` нельзя читать прежний `repairable*`; следующий координатор увидит отсутствие цели и не должен сохранять старую заявку.

Этот прогон проверяет два инварианта. Во первых, оценка полезности не равна физическому ремонту; между ними есть арбитраж и маршрут. Во вторых, сумма `progressDelta` зависит от **симуляционного** времени, поэтому пауза и `dt=0` не чинят корпус. Если в реальной сцене прогресс идёт у NPC за стеной, измерьте `repairPosition`, `currentRoom`, выбранный work zone point и горизонтальный `repairDistance`: формула сама по себе не проверяет физическую видимость рабочей поверхности.

8.2. Один кадр подтверждённого выстрела и следующий кадр урона

Зададим другой учебный снимок. Вооружённый активный NPC в `(0,0,0)`, yaw=0, его модель смотрит в направлении `(0,0,-1)` согласно формуле forward. Единственный живой вражеский игрок в `(0,0,-8)`, оба имеют разные ненейтральные `FactionId`. У NPC `seeRange=10`, `fieldOfViewDegrees=120`, `rangedAttackRange=12`, `rangedAimDelay=0.35`. Экипированное оружие имеет `range=20`, 5 патронов, `currentCooldown=0`, `spreadDegrees=0` и нулевую текущую отдачу; анимация содержит нужную кость руки. Предположим реальный Bullet мир содержит коллайдер игрока, но не содержит стены или союзника на луче. `dt=0.1` с; перед кадром `rangedAimTime=0.30` от уже видимой цели.

`WeaponSystem::UpdateTimers` сначала сбрасывает `triggerHeld=false` и обновляет оружейный таймер. `NpcCombatSystem` в поиске кандидатов вычисляет `distanceSquared=64`; игрок единственный вражеский персонаж, поэтому выбирается его ID. Вектор до груди примерно `(0,-0.6,-8)` от фиксированной точки глаз `(0,1.6,0)`; горизонтальное направление `(0,0,-1)`, скалярное произведение с forward равно 1, порог `cos(60°)=0.5`, сектор пройден. Луч к груди (или к следующей открытой высоте, если грудь закрыта) подтверждает видимость. Система записывает `targetVisible=true`, `visibleAimPoint` и наблюдаемую позицию `(0,0,-8)`; дальность центр–центр 8 меньше `min(12,20)=12`, значит `brain.state=Attacking`.

Пусть `NpcCoverSystem` в этом кадре не выдал приказа уйти в укрытие. `StateSystem` останавливает ходьбу, проверяет дальность и оставшуюся ошибку yaw. Поскольку корпус уже повернут на цель, ошибка 0<8°, `rangedAimTime` становится 0.40. Далее контроллер, анимация и физика обновляют позу. `NpcRangedCombatSystem` в конце кадра видит готовое оружие, `Attacking`, личную видимость и 0.40≥0.35. Из матрицы хвата, руки и мира он получает **фактический** origin дула; для примера пусть он равен `(0,1.4,-0.4)`. Точка груди игрока `(0,1.0,-8)` даёт ненормализованный луч `(0,-0.4,-7.6)` с длиной около 7.61, меньше 20. Короткий луч грудь NPC–дуло свободен, полный луч до цели не встречает союзника.

`WeaponSystem::TryFire` нормализует луч; при нулевом разбросе он остаётся направлен к выбранной точке. `CommitShot` расходует один патрон и устанавливает кулдаун согласно параметрам оружия. Bullet луч от дула первым пересекает коллайдер игрока. `TryFire` создаёт `DamageEvent{source=weaponEntity,target=playerEntity,damageAmount=weapon.damage,...}`, добавляет его в очередь и возвращает `true`; `combat.rangedStatus="Fired"`. Если бы физический луч промахнулся из за разброса или движения цели после проверки зрения, возврат всё равно был бы `true`, но `DamageEvent` бы не появился. Если бы первым попал в стену, событие адресовалось бы стене, не игроку.

В этом же кадре `DamageSystem::Update` уже был вызван **раньше**, перед поздним выстрелом. При обычном цикле новый `DamageEvent` ждёт следующего шага. Допустим `weapon.damage=20`, здоровье игрока перед очередью 100 и нет god mode. Следующий `DamageSystem::Update` уменьшит его до 80. Если целью вместо игрока был NPC, его `NpcTacticalComponent` получит `underFireTime=3` и сигнал реакции; при нуле HP обычный NPC потеряет `ActiveComponent`. Эта задержка на границе очереди

важна при тестах: проверка HP сразу после `NpcRangedCombatSystem::Update` ожидает ещё старое здоровье, проверка после следующего `DamageSystem::Update` — новое.

9. Как добавлять поведение, не разрывая цепочку

9.1. Сначала определите место нового факта

Предложение: добавить задачу «проверить аварийный клапан», которая доступна мирному экипажу, требует подхода к устройству и занимает 2 секунды. Не начинайте с нового `case` в `StateSystem`, пока не ответили на четыре вопроса: **что делает цель пригодной, кто может её взять, как подтвердить достижение и что меняется в мире после выполнения**. Для этой задачи потребовался бы компонент цели, например `ValveFaultComponent{faulted,room,interactionPosition}`, авторский параметр длительности и runtime прогресс. ID исполнителя можно хранить через обобщённую заявку либо расширить нынешний `CrewTaskClaimComponent`. Пока текущий `CrewTaskCoordinator` собирает исключительно `RepairableComponent` цели через `IsRepairTargetValid`, новый компонент сам по себе не попадёт в очередь. CrewTaskCoordinator.h:225–276 (../src/Gameplay/AI/CrewTaskCoordinator.h#L225).

Для небольшого расширения **существующего** ремонта, например нового типа генератора, путь короче. Если устройство уже имеет `DamageableComponent`, `RepairableComponent`, Transform и нужный WorkZone, оно попадёт в кандидаты. Нужно решить, какую `importance` оно получит в utility, нужна ли поправка `roleAffinity` в координаторе и что `RepairSystem::CompleteRepair` должен делать с его собственным состоянием после восстановления HP. Иначе `Damageable.health` станет полным, но функциональный флаг нового устройства может остаться `broken=true` и игра увидит противоречие. Добавьте сохранение его авторских полей в `SceneManager` и редактирование в Editor, если настройка должна жить между запусками.

Для **нового вида задачи** сначала опишите состояние данных, затем интерфейс оценки, затем резервирование и лишь после этого исполнение. Оценщик не должен менять мир; координатор должен выпускать ровно одного исполнителя; исполнитель должен быть идемпотентным по отношению к повторным кадрам и безопасным к удалению цели. Если действие занимает время, храните прогресс на цели, если несколько исполнителей могут способствовать одной работе, либо на назначении, если прогресс принадлежит одному исполнителю и должен обнуляться при его уходе. Нынешний ремонт хранит прогресс на `RepairableComponent` цели, поэтому при смене ремонтника работа продолжается. Это осознанное поведение, которое стоит явно выбрать для новой системы.

9.2. Практический маршрут изменения проекта

34. **Данные.** Добавьте минимальный компонент цели и авторские параметры. Разделите сохраняемые настройки и runtime память. Пример разделения уже есть у `CrewMemberComponent`: `role/onDuty/repairSkill` сохраняются, `assignedTask` и счётчик текущего запуска не записываются. SceneManager.cpp:3552–3558 (../src/Scene/SceneManager.cpp#L3552), SceneManager.cpp:4629–4640 (../src/Scene/SceneManager.cpp#L4629).
35. **Валидация цели.** В одном месте проверьте `IsAlive`, `Active`, обязательные компоненты и незавершённость. Нельзя считать «есть Transform» эквивалентом «клапан сломан». Выберите политику при удалении цели: освободить claim, поставить `Idle`, прекратить старый путь.
36. **Оценка.** Задайте численную формулу в понятных единицах и проверьте её на ручных примерах: близкая мелкая авария, дальняя критическая, два NPC разных ролей, порог `idleScore`. Если доступность зависит от маршрута, геометрическую дистанцию можно использовать как дешёвый предварительный фильтр, а достижимость проверять до закрепления назначения.
37. **Резервирование.** Запишите, кто владеет задачей, и решите, удерживается ли назначение при колебаниях score. Для работ вроде ремонта липкость полезна; для тушения быстро растущего

пожара может понадобиться прерывание по аварийному приоритету. Не переназначайте каждый кадр без cooldown или гистерезиса.

38. **Исполнитель.** Добавьте состояние и ветку в `StateSystem::UpdateNpc` или отдельную исполнительную систему в конкретной точке `ApplicationClass`. Для перемещения используйте общий `NavigateTo` и `CharacterMovement`, для изменения мира — специализированную систему действия. Границу после потенциального `DestroyEntity` обрабатывайте по ID и результату, не через сохранённый указатель.
39. **Авторинг.** Добавьте поля сохранения и загрузки в SceneManager.cpp (../src/Scene/SceneManager.cpp#L3951), UI создания и редактирования в EditorUI.cpp (../src/Editor/EditorUI.cpp#L6737), диагностику в NPC Studio или Object Debug. Не сохранённый авторский параметр будет возвращаться к значению по умолчанию при перезагрузке сцены.
40. **Проверка.** Пройдите по сценарию «один NPC–одна задача», затем «два NPC–одна задача», удаление цели в пути, ручной Stop посередине, потеря питания двери, pause и повторная загрузка. В каждой проверке фиксируйте ID, состояние, статус, маршрут и фактическое изменение мира.

Если понадобится новый **боевой** тип поведения, например точечный подавляющий огонь, отделите его решение от факта выстрела. Решение должно выставить намерение, таймер и допустимую точку, но не вычитать HP непосредственно. Общий `WeaponSystem::TryFire` уже отвечает за патрон, разброс, физический первый hit, эффекты и очередь `DamageEvent`. Новый режим обязан сохранить проверки фракции, личной видимости либо явно описанного способа подавления, блокировки дула, союзника в линии и разрешения оружия. Если он должен стрелять в точку без визуального контакта, это **новое правило дизайна**, которого текущий `NpcRangedCombatSystem` не допускает. Его нельзя получить простым изменением `brain.state=Attacking`.

9.3. Инварианты, которые стоит проверять в коде ревью

Для ремонта: у пригодной цели есть `RepairableComponent` и конкретный тип (`HullBreach` или `Damageable`); у незавершённой автоматической задачи один claim; `member.assignedTask`, `brain.repairTarget` и claim указывают на согласованные ID; ручной приказ имеет приоритет; до достижения `repairDistance` прогресс не меняется; после уничтожения пробоины нет чтения старых ссылок. Для боя: цель живой активный враг; `targetVisible` соответствует лучу текущего кадра; `visibleAimPoint` читается только при этой видимости; скрытая цель не обновляет `lastKnownPosition`; патрон расходуется только при успешном `TryFire`; попадание адресовано первому Bullet коллайдеру; урон проходит через очередь. Для движения: Brain хранит цель и маршрут, `CharacterMovement` хранит намерение, Bullet владеет фактической позицией; при неактивности намерение обнулено.

Эти правила полезнее огромного списка «возможных тестов» без причин: каждый инвариант можно привязать к конкретной границе системы. Если он нарушен, ищите **первый проход, после которого данные стали неправильными**, а не последний визуальный симптом. NPC, который не стреляет, может быть исправен как оружие, но не иметь `targetVisible`; NPC, который не чинит, может правильно выбрать цель, но не иметь достижимой рабочей точки.

10. Диагностика по наблюдаемым данным

10.1. Ремонтник не начинает работу

В Object Debug и NPC Studio сначала найдите NPC Entity и ремонтную Entity. Для NPC проверьте `ActiveComponent`, `NPCComponent`, Brain, `isHostile=false`, `CrewMember.onDuty=true`, отсутствие активной `NpcCommand`. Для цели проверьте `Active`, `Repairable`, `!repaired` и либо `HullBreach&&!sealed`, либо `Damageable.health<maxHealth`. Эти условия прямо соответствуют `IsRepairTargetValid`, CrewTaskCoordinator.h:225–243 (../src/Gameplay/AI/CrewTaskCoordinator.h#L225). Затем сравните `brain.repairScore` с `brain.idleScore=0.10`, `CrewTaskCoordinatorSnapshot.openTasks`, `assignmentsThisFrame`, и `CrewTaskClaimComponent.assignee` на цели. Наличие score без claim означает, что локальная utility видит работу, но координатор мог назначить её другому или предпочесть другую пару NPC–задача.

Если claim есть и `brain.state=MoveToRepair`, но персонаж стоит, проверьте `brain.currentRoom`, `navigationTargetRoom`, `doorRouteIndex`, `activeDoor`, `debugDoorPhase`, `blockedReason`, `routeDirty` и `repathTimer`. `No traversable door route` означает проблему графа комнат/питания двери; `WAITING_FOR_OPEN` означает нормальное ожидание `openProgress`; `CROSSING_DIRECT` при нулевом физическом прогрессе указывает на коллизию портала или капсулы. StateSystem.h:2201–2499 (../StateSystem.h#L2201). `NpcNavMeshSystem::GetDebugInfo(npc)` сообщает `compiledIn`, `roomMeshBuilt`, `polygonCount`, путь, динамический обход и статус. NpcNavMeshSystem.h:29–60 (../src/Gameplay/AI/NpcNavMeshSystem.h#L29). При `compiledIn=false` прямой fallback ожидаем, но на сложном корабле он может вести в стену.

Если `brain.state=Repairing`, посмотрите фактическую `Repairable.progress`, `requiredProgress`, `brain.repairProgress`, `repairDuration`, `repairSkill`, `dt`. При `progress=requiredProgress` и статусе ожидания runtime операции исследуйте `HullBreachRepairLinkComponent` и завершение mesh repair. При полном HP оборудования, но `repaired=false`, проверьте, не было ли нового попадания, сбросившего прогресс; если функциональный флаг устройства остаётся сломанным после полного ремонта, расширьте `RepairSystem::CompleteRepair` именно для этого типа.

10.2. Противник не видит или не стреляет

Для восприятия последовательно проверьте `NPCCombatComponent`, `FactionComponent` на обоих участниках, здоровье и Active цели, расстояние до **ближайшего выбранного кандидата**, yaw наблюдателя, `fieldOfViewDegrees`, `seeRange`, Bullet мир и debug sight rays. Если видимый дальний враг игнорируется, посмотрите, не выбран ли более близкий враг за стеной: текущая система не ищет второго после отказа луча. `combat.targetVisible` должен быть истинным именно в нужный кадр, а `visibleAimPoint` — одна из проверенных высот цели. Если луч начинается внутри стены, проверьте кость головы и мировую матрицу модели.

Для стрельбы идите по NpcRangedCombatSystem.h:30–500 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L30) в порядке guard: Active, `PlayerGameTag` отсутствует у NPC, Brain, фракция, здоровье, ручная команда, экипированное оружие, патроны, перезарядка, `Attacking` или разрешённый movingFire, `targetVisible`, время прицеливания, живой вражеский target, Transform и анимация, правильное имя кости руки, дальность от **дула**, ошибка yaw не более 8°, свободный короткий участок грудь–дуло, отсутствие союзника на линии. `rangedStatus` часто прямо говорит, где остановился проход: `Unarmed`, `Aiming`, `Turning toward target`, `Missing right-hand bone`, `Muzzle blocked by cover`, `Holding fire: ally in firing lane`, `Weapon cooling down`.

При статусе `Fired` и отсутствии потери HP проверьте shot ray: промах всё равно возвращает `true`. Если физический луч задел другое тело первым, `DamageEvent.target` не равен выбранной AI цели. Если событие есть, проверьте следующий шаг `DamageSystem::Update`, `InvulnerableComponent`, `PlayerComponent::godMode` и фактический тип здоровья цели. Для NPC без оружия `AnimationState::Punch` пока нельзя считать доказательством урона: этот код запускает клип, а не `DamageEvent`.

10.3. Короткая таблица симптомов

Наблюдение	Первое поле для проверки	Вероятная граница системы
`repairScore > 0`, **назначения нет**	`CrewTaskClaimComponent`, `assignedTask`, `onDuty`	Utility → Coordinator
Назначение есть, `progress=0`	`state`, `blockedReason`, дистанция до work zone	Coordinator → State/NavMesh
`progress=1`, **пробоина открыта**	статус ожидания runtime операции	State → RepairSystem/MeshCutting

NPC идёт на месте	`walkDirection`, фактический Transform, `stuckTimer`	State → Bullet controller
Враг близко, `targetVisible=false`	yaw, FOV, три sight ray, ближайший кандидат	Combat perception → Bullet raycast
`targetVisible=true`, выстрела нет	`rangedStatus`, aim time, кость, дуло, союзник	State/Animation → Ranged combat
`Fired`, HP не изменилось	weapon shot ray, `DamageEvent.target`, следующий кадр	Weapon → Damage queue

11. Что сейчас является пределом реализации

Эта глава различает подтверждённый путь и идеи развития. **Подтверждено кодом:** мирные NPC сами оценивают и получают ремонтные работы, маршрутизируются через комнаты и двери, чинят пробоины и оборудование через общий `RepairSystem`; боевые NPC выбирают враждебную цель по фракциям, проверяют личную видимость, помнят последнее наблюдение, ищут, занимают индивидуальное укрытие, могут стрелять через общий hitscan `WeaponSystem` с Bullet raycast и очередью урона. **Ограничения текущего пути:** групповой `NpcSquadCombatSystem::Update` отключён; безоружная NPC атака в `StateSystem` сейчас анимационная; выбор ближайшего врага предшествует проверке видимости; оценка ремонта использует прямое расстояние, а не фактическую стоимость маршрута; NPC вызов `ApplyRepairProgress` не передаёт `WorldEventLog` и автора действия.

Предложения к развитию, не объявленные готовым функционалом: объединить или документировать расхождение двух ремонтных scoring формул; выбирать лучшую **видимую** угрозу; добавить честное окно melee попадания и `DamageEvent`; подключить стоимость/доступность маршрута в назначение; провести источник NPC ремонта в WorldEventLog; вводить squad слой только после фиксации контрактов индивидуального боя. Каждое такое изменение должно сопровождаться конкретным сценарием, в котором проверяются входные компоненты, выходные поля, физический результат и поведение при отмене/удалении цели. Так учебник остаётся точной картой движка, а новые идеи не маскируются под уже существующий код.

Редактор, сцена и самостоятельная работа с кодом

Математика и анализ методов полезны, когда после чтения можно открыть свой проект и повторить результат. Эта часть ведёт от создания одного Entity в редакторе до сохранённого эпизода. Здесь разобраны настоящие границы `EditorUI`, `SceneManager` и `ApplicationClass`. Мы не будем считать, что сохранённая сцена автоматически равна сохранению игрового прогресса: это два разных договора.

1. Как объект проходит путь от кнопки редактора до мира

1.1. Начало: шаблон в `EditorUI`

В `src/Editor/EditorUI.cpp` метод `CreateEditorTemplate` находится около строки 1729. Он получает `SceneManager&` и `QuickCreateKind`. Это означает, что UI не владеет отдельным Registry: он просит существующий SceneManager создать или наполнить Entity. `switch (kind)` выбирает профиль — пустой объект, точку взаимодействия, свет, `CrewNpc`, cover point, patrol point, pickup, trigger area и другие. Прочитай одну ветку, например `CrewNpc`, и выпиши каждое добавление компонента. Затем найди `RenderQuickCreateMenu`: его пункты вызывают тот же метод с конкретным kind. Раздели два действия: кнопка меню определяет намерение автора сцены, а фабрика создаёт набор данных.

После создания проверь сущность в Outliner и Inspector. Если `CrewNpc` виден в Hierarchy, это не доказывает, что у него есть модель, Bullet controller, NavMesh доступность и правильно заданная фракция. Quick Create — начальный профиль, который должен пройти проверку обязательных компонентов. Для нового шаблона не копируй случайную ветку и не оставляй несовместимые поля. Составь «паспорт» созданной Entity: Transform, логика, визуал, collision, связи с комнатой, источник питания, save fields. Если у профиля только логика, Inspector должен это явно показать.

1.2. Изменение параметра и его владелец

`RenderComponentManager` в `EditorUI.cpp` собирает интерфейс для компонентов выбранной Entity. Перед добавлением нового поля выясни, кто его меняет в Play. Авторский параметр `maxHealth` редактируется до игры; текущий `health` может изменяться системами каждый кадр. Если дать редактировать live значение и затем сохранить сцену в неожиданном моменте, автор случайно запишет состояние после боя как стартовое. Раздели UI на «настройка» и «текущее состояние». Для расчётного `powerRatio` лучше read-only текст с единицей и источником, чем ползунок, который энергосистема перезапишет через один тик.

Одна величина должна иметь название, единицу, диапазон и default. Например, `repairDurationSeconds=10` означает время при skill=1 и фиксированном requiredProgress; `repairDistanceMeters=1.5` — горизонтальный радиус работы, а не размер collision shape. Если в UI будет просто «Repair 10», следующий разработчик не поймёт, что меняется. Для полей, влияющих на математику, показывай вычисленный результат: при выбранном skill ожидаемое время ремонта; при заданной мощности ожидаемый powerRatio.

1.3. Критерий «объект работает»

Создай в Quick Create одного инженера. Проверь последовательно: он есть в Registry; у него есть нужные компоненты; у него есть визуальная модель или ясно указано, что профиль логический; collider создан и находится у ног модели; он видит доступную работу; маршрут ведёт к рабочей позиции; progress изменяется; завершение отражается в мире. Для оборудования проверь WorkZone, если он задан; текущая ветка пробоины использует другую целевую точку без WorkZone. Любой отказ должен иметь первый нарушенный контракт. Такой тест занимает меньше времени, чем настройка красивой анимации у NPC, которому ни разу не выдали задачу.

2. `SaveScene` построчно: почему в JSON попадает не всё

2.1. Заголовок файла и список объектов

Около строки 3270 `SceneManager::SaveScene(const std::string& filename)` создаёт `json scene`, задаёт `scene_name`, `version=2` и пустой массив `objects`. Номер версии — контракт с загрузчиком: изменение формата данных должно учитывать старые сцены. Следом создаются `int id=0` и `savedEntities`. Лямбда `saveEntity` проверяет `IsAlive`, отбрасывает `NpcWeaponVisualComponent` и `ShipPartComponent`, исключает повторение Entity через `savedEntities.insert(entity).second` и вызывает `SerializeObject(entity,id++)`. У отбрасывания есть смысл: визуал оружия и импортированные части корабля восстанавливаются из более высокого источника, поэтому их сериализация создала бы дубликаты.

Почему функция не просто обходит все живые Entity? Потому что часть сущностей является runtime или производными от GLB root, а часть логических объектов не имеет mesh. Дальше SaveScene проходит несколько pools: корни кораблей, обычные объекты с mesh, помещения и комнаты без mesh. При чтении каждого цикла спроси: какую категорию объектов он добавляет, какие уже были добавлены и какую категорию может забыть. `savedEntities` защищает от дубля, если одна Entity встречается в нескольких pools. Но объект с новым компонентом, не попавший ни в один из этих обходов, может вообще не сохраниться. При добавлении системы это одна из первых проверок.

2.2. `SerializeObject` и граница authoring/runtime

`SerializeObject` начинается около строки 3428. Он превращает выбранную Entity в JSON. Читая его, для каждого поля подпиши, откуда взялось значение и должно ли оно переживать reload. Позиция модели, стартовые параметры генератора, фракция и роль NPC обычно нужны для новой сцены. Указатели на D3D buffers, `btRigidBody*`, временный результат raycast и очередь job не имеют смысла в файле. Если сериализация пропускает новый компонент, он исчезнет после загрузки даже при успешном сохранении. Если записывает runtime pointer как число, файл будет выглядеть заполненным, но после запуска значение станет недействительным.

Практика: сохрани сцену с двумя объектами с различными Transform. Открой JSON как текст, найди version и обе записи. Измени только один авторский параметр в Editor, сохрани снова и сравни файлы. Если в диффе изменилось множество не связанных runtime полей, формат сцены захватывает лишнее. Если нужный параметр не изменился, проверь Inspector и `SerializeObject` — возможно, UI редактирует копию или не тот компонент.

3. `LoadScene` построчно: пересоздание мира

3.1. Открытие и очистка

Около строки 4301 `LoadScene` принимает путь, D3D device и device context. `std::ifstream file(filename)` открывает файл; если он не открыт, метод возвращает false. Затем JSON считывается, вызывается `Clear()`, создаются таблица `remap` старых ID к новым и набор `restoredEntities`. Если файл синтаксически повреждён и исключение из JSON возникает до `Clear()`, состояние мира может остаться прежним; если после Clear — уже частично очищенным. Для пользовательской игры полезна подготовительная проверка файла до разрушения текущей сцены и понятная ошибка загрузки. Это проектное улучшение, не описание гарантии текущего кода.

`LoadScene` читает блок `world`, если он присутствует, и восстанавливает fog/skybox параметры с fallback на текущие defaults. Для объектов проходит JSON массив. Корень импортированного корабля обрабатывается отдельно: `ImportShipFromGlb` строит структуру из asset, затем возвращаются Transform и параметры power grid/генераторов. Обычный объект создаётся через `DeserializeObject`. После создания любой успешный Entity заносится в `restoredEntities`, а старый `entity_id` связывается с новым в `remap`. Здесь особенно важно отличать сохранённый номер от текущего: Registry выдаёт новые ID, и ссылка `repairTarget=42` не обязана всё ещё указывать на 42.

3.2. Восстановление ссылок и Bullet

После цикла вызывается `RemapEntityReferences(remap,restoredEntities)`. Комментарий в коде объясняет тонкость: remap применяется только к компонентам, наполненным из сохранённых полей. Вспомогательные Entity импортированного GLB уже указывают на новые runtime ID; слепое повторное remap может испортить иерархию, если новый ID случайно совпал со старым. Затем `LoadScene` обходит `RigidBodyComponent`: если `body` отсутствует, вызывает `PhysicsSystem::CreateRigidBody`. Это верный порядок зависимости — физическое тело создаётся после того, как существуют модели и ссылки. В конце разрешаются repair zones и соединения комнат.

Проверь round trip на одном NPC и пробоине. До сохранения запиши ID обоих. В текущем `SaveScene` сохраняются авторские параметры NPC, но не его runtime назначение на ремонт: после загрузки он должен получить новую задачу от AI, а не сохранить старый `repairTarget`. Старые и новые Entity ID могут различаться; всякая действительно сериализованная ссылка должна пройти `remap` на живую цель. Bullet collider должен существовать и отвечать на raycast; если объект виден, но после загрузки можно пройти насквозь, проверь восстановление физики. Повторная загрузка дважды не должна создавать дублированные тела или второй визуал оружия.

3.3. Scene file и сохранение прогресса игры

`SaveScene` хранит авторское устройство уровня и часть настроек мира. Сохранение прохождения требует отдельного договора: здоровье после боя, использованный материал, запас батареи, текущий этап миссии, живые аварии. Не стоит бездумно записывать в authoring сцену все runtime значения: тогда тестовое прохождение изменит исходный уровень. Для save game можно хранить исходную сцену плюс изменения или полный snapshot постоянных компонентов, но в обоих случаях нужен version, Entity remap и восстановление runtime ресурсов. Начни с маленького эпизода: одна дверь, одна пробоина, один NPC, один враг. Сохрани после ремонта и урона, закрой процесс, загрузи и сравни конкретные инварианты.

4. Практикум: один новый компонент проходит весь путь

4.1. Формулируем учебную систему

Пусть у корабельного прибора есть датчик перегрева. Его авторские поля: room ID, порог температуры, название. Runtime поля: текущая температура, статус срабатывания, время последнего перехода. Система читает температуру комнаты, сравнивает с порогом и создаёт событие только при переходе через него. Показать число в Inspector недостаточно: оно должно появиться у созданной Entity, сохраниться, загрузиться и повлиять на игру. Важное математическое замечание: если датчик просто копирует текущую температуру комнаты, `dt` не нужно умножать ни на что. Если он реагирует с задержкой, введи физически или игрово понятную постоянную времени, тогда формула обновления зависит от dt.

4.2. Где менять код

Сначала добавь структуру компонента рядом с другими компонентами проекта и default значения. Затем добавь систему, которую можно вызвать на одной Entity или на `ForEach`. Включи её в проект Visual Studio, если новые `.cpp` не подхватываются автоматически. В `ApplicationClass::Frame` поставь вызов после обновления комнатной температуры и до системы, потребляющей тревогу. В Editor добавь возможность создать датчик и изменять authoring поля; в `SerializeObject` и `DeserializeObject` — сохранение и загрузку этих полей. В debug выводи Entity, room ID, текущее значение, порог и последний переход. Наконец, сделай тестовый мир с двумя датчиками в разных комнатах, чтобы обнаружить ошибочный общий `static` state.

Составь таблицу зависимостей своими словами: источник температуры — RoomSystem; автор порога — Inspector/JSON; вычислитель статуса — SensorSystem; потребитель события — HUD или тревога; сохранение — SceneManager; runtime очистка — при удалении Entity. Если одну строку не

можешь заполнить, пока не подключай следующий слой. Пустое место в таблице позже превратится в «необъяснимый» баг.

4.3. Проверка за 15 минут

Создай датчик с порогом 60°C в комнате A и второй с порогом 80°C в комнате B. Сохрани сцену и загрузи её. Установи температуры 65 и 65: первый сработал, второй нет. Подними B до 85: второй сработал один раз. Опусти A до 55: выбери правило возврата, например гистерезис — выключить тревогу ниже 55°C, чтобы показатель 59↔61 не мигал на границе. Удали первый датчик и проверь, что событие/указатель не осталось в UI. Если этот путь пройден без редактирования JSON вручную, ты действительно добавил систему в движок, а не только написал функцию.

5. Завершённый эпизод вместо бесконечного набора механик

5.1. Состояния миссии

Собери один уровень: игрок входит на корабль, узнаёт о неисправной двери, включает генератор, отдаёт приказ инженеру устранить пробоину, отбивается от врага и достигает эвакуации. Переходы миссии определяются фактическими состояниями мира: потребитель двери запитан, пробоина закрыта, необходимый враг побеждён, игрок достиг выходной зоны. HUD сообщает цель и причину, почему следующий этап ещё не доступен. Если пробоина починена раньше генератора, миссия должна иметь явное правило: засчитать сразу, дождаться этапа или разрешить альтернативный путь. Нельзя оставлять это на порядок `if` в одном кадре без дизайна.

Наблюдение от внешнего игрока является проверкой качества. Если он не понял, какой NPC чинит, почему дверь не открывается или откуда пришёл урон, в игре отсутствует важная обратная связь. При этом нужно отличать скрытую игровую информацию от непонятного интерфейса. Враг вправе искать игрока по последнему шуму, но UI не обязан раскрывать точный AI score; достаточно, чтобы реакция выглядела причинной и последовательной.

5.2. Проверяемая сборка и рабочий каталог

`scripts/verify_engine.ps1` собирает Debug/Release, проверяет shader compilation и core contracts. Скрипт выполняется из корня проекта; успешный результат не доказывает, что игру можно перенести. После него собери отдельный каталог с executable, нужными DLL, assets, shaders, scenes и настройками. Запусти из этого каталога после переименования и перемещения. На другой машине или чистом профиле проверь первый запуск, одну миссию, save/load и выход. Если модель исчезла только вне репозитория, начни с поиска asset path и текущего working directory, а не переписывания vertex shader.

5.3. Что записать после внешнего прогона

Для каждого дефекта укажи последовательность действий, начальную сцену, ожидаемое состояние, фактический результат, кадр/время, затронутые Entity и лог. Ошибка «NPC иногда тупит» не даёт следующего шага; запись «после отключения питания двери D инженер E держит claim на пробоине B, routeStatus=DoorUnpowered, не освобождает его 30 с» сразу указывает на контракт назначения. Исправляй блокирующие проблемы запуска, потери сохранений и невозможности закончить миссию прежде декоративной полировки. Такая дисциплина превращает учебный проект в почти готовую игру.

Новая механика и дорога к выпуску

Эта часть превращает знания о движке в повторяемый способ разработки. Любую новую механику — пожар, кислород, абордаж, медицинскую помощь или систему миссий — можно добавить одним и тем же порядком: сформулировать правило для игрока, определить данные и владельца, вставить вычисление в кадр, соединить с редактором и сохранением, показать обратную связь, проверить инварианты и только потом масштабировать на большой уровень.

Разбор 44. Спецификация новой механики до кода

Возьмём пожар в машинном отсеке. Игровое правило: после повреждения может появиться очаг; он расходует горючее и кислород, нагревает комнату и портит генератор; экипаж при наличии огнетушителя может тушить его, игрок может выдать ручной приказ; закрытая дверь меняет путь NPC и обмен воздухом. Запиши состояния `Igniting`, `Burning`, `Dying`, `Extinguished`, а также события переходов. Затем укажи единицы: интенсивность безразмерна или в кВт тепловыделения? Топливо в кг или условных единицах? Сколько расходуется за секунду? Без этих вопросов даже компилирующийся FireSystem даст непредсказуемый баланс.

Данные раздели по владельцу. `FireComponent` хранит состояние очага и room ID; room state хранит температуру/кислород; инвентарь NPC хранит заряд; claim на очаге фиксирует назначенного исполнителя; mission state хранит, относится ли пожар к цели. FireSystem изменяет огонь и комнату, coordinator выбирает исполнителя, `StateSystem` исполняет тушение, HUD подписывается на переходы. Не держи глобальный `currentFire` в singleton: второй независимый очаг сразу обнаружит ошибку модели. Не записывай указатель на комнату в файл сцены — сохраняй ID и восстанавливай связь.

Разбор 45. Вставка в кадр и интеграционные границы

Зависимости определяют порядок. Если FireSystem обновляет интенсивность, а coordinator выбирает работу по интенсивности, FireSystem выполняется раньше coordinator либо координатор сознательно использует значение предыдущего тика. Если дверь получает питание от grid, power update выполняется до door action. Если NPC тушит огонь, ресурс списывается до или вместе с уменьшением интенсивности; нельзя уменьшить пожар, а потом обнаружить, что огнетушитель пуст. Если новый эффект требует GPU, render preparation выполняется после gameplay update, но отсутствие GPU ресурса не должно отменять физическое завершение тушения.

Для асинхронного перестроения mesh/physics не передавай worker указатели на компоненты, которые main thread может удалить. Worker получает копию входных данных с generation/revision, строит результат, а main thread проверяет, что Entity ещё жив и revision совпадает, затем применяет результат. Если за время работы job пробоину уже закрыли, результат устарел и отбрасывается. Такой протокол нужен для резки меша, обновления collision и дорогой подготовки ресурсов; без него редкий scene reload станет аварией времени жизни.

Разбор 46. Сохранение и восстановление состояния игры

Авторская сцена и игровой save отвечают на разные вопросы. Scene authoring хранит начальную расстановку и параметры: тип NPC, роль, модель, порог пожара, позицию двери. Save game хранит прогресс: здоровье, уже потраченный ресурс, открытые двери, текущий пожар, состояние миссии. Одно поле может присутствовать в обоих форматах, но причина должна быть ясной. Bullet pointers, GPU handles, transient path queries, claims с прошедшим временем и worker jobs нельзя восстанавливать копированием памяти. После загрузки создай runtime ресурсы, затем remap ссылок Entity и пересчитай derived state.

Проведи round trip на конкретном эпизоде. В Play игрок открыл дверь, инженер потратил 2 единицы материала, пожар снизился наполовину, враг получил урон. Сохрани, закрой процесс, запусти заново и загрузи. Ожидаемые данные — ровно эти постоянные изменения. Не должно появиться второго огня, второго Bullet тела или второго claim на ту же работу. Если AI после загрузки выбирает новую задачу, это допустимо, если его память и назначения считаются runtime и правила книги это оговаривают. Игроку важен сохранённый результат, а не повторение каждого промежуточного pointer.

Разбор 47. UI и звук как часть правил

Игрок принимает решения по информации, которую показывает игра. Если дверь не открывается из-за питания, HUD должен сообщить «нет энергии», а не только оставить кнопку неактивной. Если инженер не идёт к пробоине, статус показывает «маршрут недоступен» или «нет материала». Если враг потерял игрока, звук поиска и анимация должны соответствовать lastKnownPosition, а не скрытой точной позиции. UI и audio подписываются на состояния/события gameplay, но не вычисляют их сами. Так один и тот же мир остаётся корректным при отключённом звуке или закрытом HUD.

Событие мира имеет тип, время, позицию, источник и важные параметры. Выстрел создаёт gameplay событие, из него могут появиться звук, muzzle flash, AI слух и журнал. Не воспроизводи звук внутри одной ветки и отдельно создавай событие слуха в другой: они разойдутся по координатам или времени. При пожаре ограничь частоту повторяющихся событий, иначе один очаг заполнит лог и запустит сотни одинаковых звуков. Для доступности дублируй критичный звуковой сигнал визуальным индикатором, а важный цветовой статус — текстом или формой.

Разбор 48. Производство ресурсов

Каждая модель должна иметь понятный паспорт: исходный файл, экспортированный GLB, масштаб, ось forward/up, pivot, материал, текстуры, число треугольников, collision shape, skeleton, clips и socket names. Если персонаж импортирован с forward −Z, а AI считает +Z, исправь один явный offset при импорте или создании prefab. Не подгоняй вручную yaw каждой сцены. Render mesh и collision geometry имеют разные бюджеты: красивый корпус может быть сложным, физический корпус — составлен из простых shapes. Для NPC скелет, клипы Idle/Walk/Run/Aim/Fire/Reload/Repair и muzzle socket проверяй в небольшой preview сцене до включения в большую миссию.

Текстуры PBR проверяй по назначению каналов и цветовым пространствам. После импорта сначала выведи albedo без света, затем normals, roughness и metallic в debug material. Если новый asset работает только в исходном каталоге, путь к текстуре или shader зависит от рабочей папки. Выпускная сборка должна содержать весь manifest файлов и запускаться после переноса в другую директорию. Игрок не будет устанавливать Blender и открывать Visual Studio, чтобы увидеть корабль.

Разбор 49. Миссия и проверяемый вертикальный срез

Один завершённый эпизод лучше десятка незаконченных механик. Для космического симулятора сценарий может быть таким: игрок приходит на повреждённый корабль, восстанавливает питание, закрывает аварийную секцию, отдаёт инженеру приказ, отбивает врага и запускает эвакуацию. У миссии есть состояния `Intro`, `PowerRestored`, `BreachContained`, `AttackRepelled`, `EvacuationReady`, `Won` и варианты поражения. Переходы основаны на фактах мира, а не на том, показана ли подсказка. Если игрок починил корпус раньше восстановления питания, правило должно явно решить, засчитывается ли этап позже. Если NPC погиб, миссия должна оставаться проходимой альтернативным способом или честно завершаться поражением.

Для каждого этапа выпиши действие игрока, обратную связь и условие проверки. `PowerRestored` — grid сообщает достаточную мощность критическому устройству, HUD меняет статус, дверь получает команду открыть. `BreachContained` — утечка закрыта, давление перестало падать, событие ремонта подтверждено. `AttackRepelled` — все нужные враги мертвы или отступили; отсутствие draw call врага не считается победой. Сохрани и загрузи после каждого этапа. Эта работа обнаруживает интеграционные ошибки лучше отдельного теста одной функции, потому что миссия связывает ECS, NPC, физику, рендер и UI в наблюдаемую причинную цепь.

Разбор 50. Сборка, внешний запуск и критерий завершения

После функционального прогона собери Release, создай отдельный каталог с executable, DLL, shaders, assets, scenes и настройками. Запускай из этого каталога, не из корня репозитория. Переименуй каталог и повтори: относительный путь не должен случайно зависеть от имени папки исходников. Проверь сохранение в месте, доступном обычному пользователю, а не рядом с защищённым executable. На другой машине или чистом Windows профиле воспроизведи запуск, миссию, save/load и выход. Сохраняй лог ошибок и версии ресурсов.

Перед выпуском исправь блокирующие дефекты: аварийный выход, непроходимая миссия, потеря сохранения, невидимая геометрия, дублирование Bullet объектов, NPC с бесконечным ожиданием без причины. Затем измерь производительность типичной и тяжёлой сцены, проверь читаемость HUD на целевых разрешениях, управление мышью/клавиатурой и fallback при отсутствии необязательного asset. Оставшиеся дефекты запиши с приоритетом и воспроизводимым сценарием. «Почти до релиза» означает, что внешний человек способен запустить и пройти законченную игру, а список оставшейся работы конечен и понятен.

Производственная часть игры

Эта часть отвечает на вопрос, который не решается одной правильной системой: как превратить инженерный прототип в повторяемый игровой опыт. Она не вводит второй календарный план. Разделы ниже можно применять к каждому уровню и каждому кандидату сборки. Пометки «добавить» означают работу над проектом, а не уже готовый функционал.

Урок 17. Конвейер моделей, материалов и анимации

17.1. Один asset проходит несколько пространств и владельцев

Модель создаётся во внешнем редакторе, экспортируется в glTF/GLB, читается загрузчиком, получает mesh/material, появляется как Entity в сцене, рисуется рендером и при необходимости имеет collision shape и skeleton. Ошибка может возникнуть на любом шаге. Например, корабельная панель видна в Blender, но невидима в игре из-за неверного winding; видна, но не выбирается лучом из-за отсутствующего collider; выбирается, но недоступна NPC, потому что её WorkZone находится внутри корпуса.

Для каждого игрового asset веди мини-паспорт: имя файла, источник и условия использования, масштаб в метрах, направление «вперёд», pivot, material slots, текстуры с цветовым пространством, collision strategy, skeleton, анимационные клипы, socket имена и ожидаемые размеры bounding box. Этот паспорт дешевле повторной диагностики после каждого переэкспорта. Если модель не влезает в дверь, сначала сравни её scale и capsule, а не меняй NavMesh параметры на всей карте.

17.2. Геометрия и физика имеют разные бюджеты

Видимый mesh может содержать множество полигонов ради нормалей, силуэта и детализации. Collision shape должен описывать нужные контакты с меньшей стоимостью. Для большого неподвижного интерьера годится статическая геометрия; для динамического ящика — выпуклая форма; для персонажа — капсула. Раздели LOD рендера и физическую точность. Замена далёкой модели на низкий LOD не должна менять collision, если игрок может взаимодействовать с ней с близкого расстояния. Если runtime cutting создаёт реальную пробоину, обновление физической формы должно быть частью операции или явно ограниченной возможностью.

Границы также важны для culling. Ошибочный AABB может скрыть объект, хотя mesh правильный. После импорта проверь min/max координаты модели, world bounds после Transform и debug фрустум. Для анимированного персонажа bounds должны охватывать движение костей и оружие либо обновляться; иначе рука или вся фигура могут исчезать у края камеры.

17.3. Материалы и текстуры

PBR материал состоит не из одной картинки. Albedo задаёт базовый цвет, normal — направление микроповерхности, roughness — ширину блика, metallic — характер отражения металла, emissive — собственное свечение. Не подставляй roughness map в albedo slot: даже если объект виден, свет станет физически бессмысленным. Проверь, как импортёр и MaterialClass задают defaults при отсутствии карты. Ошибка «всё чёрное» может быть следствием отсутствующего SRV, нулевой normal, неверного sRGB или NaN в BRDF.

Для каждой текстуры запиши: является ли она цветом или данными; нужны ли mipmaps; допустим ли alpha; какой sampler нужен; можно ли её загружать асинхронно. Копирование данных на GPU остаётся на главном потоке по контракту проекта. Фоновый worker может декодировать файл, но результат должен применяться только если сцена и материал ещё живы.

17.4. Skeleton и клипы NPC

Перед подключением AI убедись, что rig и анимации самостоятельны. Один и тот же rest pose, названия костей, направление модели и scale должны сохраняться при экспорте Idle, Walk, Run, Aim, Fire, Reload, Repair и Death. У клипа есть длительность, loop policy и момент события. Для выстрела gameplay решение остаётся у WeaponSystem; событие анимации помогает синхронизировать вспышку, звук и recoil, но не должно бесконтрольно дублировать пулю. Для ремонта progress обычно идёт по игровому времени, а клип является визуальным циклом; при прерывании работы анимация прекращается без потери или с сохранением прогресса согласно правилу.

Проверяй sockets после blend: оружие в руке, дуло, точка инструмента и камера first-person. Скриншот одной позы недостаточен. Просмотри переходы и крайние моменты: начало Reload, конец Aim, поворот на месте, бег с оружием, смерть во время ремонта. Если socket меняет сторону или проваливается в тело, ищи порядок bone transforms и разницу локального/мирового пространства.

17.5. Проверка нового asset до включения в уровень

Открой его в маленькой тестовой сцене с белым светом, сеткой метров и debug axes. Проверь размер, pivot, нормали, текстуры, alpha, shadow, collision, picking, LOD и сохранение ссылки на файл. Для NPC добавь capsule и один клип, затем все остальные. Только после этого помещай asset в большой корабль. Такой gate не запрещает экспериментировать; он делает ошибку дешёвой для локализации.

Урок 18. Миссия и правила игрового эпизода

18.1. Цель игрока задаёт смысл системам

В космическом симуляторе можно бесконечно чинить панели, но игра требует причины. Пример миссии: удержать корабль в работоспособном состоянии, пока вычисляется маршрут эвакуации. Игроку нужны три понятных показателя: время до расчёта, критические системы и угроза противника. Аварии делают выбор сложнее: пробоина требует инженера, повреждённый генератор отключает дверь, враг контролирует коридор. Механики связаны, потому что действие с одной меняет возможности других.

Опиши миссию состояниями, а не набором произвольных bool. `NotStarted`, `AlarmRaised`, `PowerRestoration`, `EvacuationReady`, `Completed`, `Failed` — учебный пример. Для каждого перехода назови событие, условие и действие. Например, `PowerRestoration → EvacuationReady`, когда двигатель/связь получили питание и расчёт времени завершён. Не делай переход в UI рендере; игровая система должна принять событие, записать состояние и сохранить его. Триггерная зона может выдать «игрок вошёл в мостик», но не должна напрямую считать все ресурсы и переключать пять экранов.

18.2. Победа, поражение и повторение

Победа должна иметь наблюдаемый результат: короткое завершение эпизода, запись в save и понятный переход дальше. Поражение бывает несколькими путями: игрок погиб, критическое давление или кислород упали ниже порога, экипаж потерял возможность восстановить важную систему. Реши, является ли необратимая потеря возможной: если игра допускает «мёртвый корабль», покажи предупреждение до того, как ситуация станет безнадёжной. Если игрок может всегда восстановиться, дай логичный путь возврата, а не скрытую debug кнопку.

Повторение сценария проверяет детерминизм там, где он нужен. Одинаковые условия не обязаны приводить к идентичной перестрелке, если есть случайный spread, но событие «дверь без питания» должно давать одинаковый отказ. Для случайных событий логируй seed и причины выбора, чтобы ошибку можно было воспроизвести. Не подменяй сложность случайным наказанием игрока вне его наблюдения.

18.3. Уровень учит механикам через последствия

Первый отсек знакомит с терминалом и питанием без врага. Второй показывает пробоину и инженера. Третий заставляет использовать укрытие и услышать врага до контакта. Это порядок подачи информации внутри одной миссии, а не отдельная дорожная карта разработки. На каждом этапе игрок должен понимать, чему его научил опыт. Если после первой пробоины появляется только число «pressure 0,4», он может не понять, что открытая дверь распространила опасность на другую комнату. Дай звук, движение воздуха, подсказку терминала и запись события.

Не рассчитывай на знание читателя учебника у будущего игрока. Проведи испытание на человеке, который не видел код. Отмечай, что он пробует, где останавливается и какие слова интерфейса неверно трактует. Исправление может быть в цели, маркере, звуке, timing события или правилах NPC, а не в дополнительном текстовом окне.

Урок 19. Наблюдаемость и диагностика без догадок

19.1. Для каждой системы нужна причина отказа

Успешное действие обычно легко увидеть. Отказ без причины выглядит как баг. Поэтому у двери — причина отказа открытия; у NPC — причина отсутствия маршрута; у оружия — `rangedStatus`; у электросети — requested/supplied; у ремонта — состояние цели и progress; у загрузки asset — путь файла и этап. Сообщение должно называть конкретное условие, а не универсальное «failed». В release UI эти причины переводятся в понятный игроку язык; в debug выводятся точные поля и Entity ID.

Пример трассы ремонтной задачи: «Breach 84 created in Room 5», «Engineer 12 score 1,30», «claim 84→12», «path via Door 9», «Door 9 requires power», «route blocked», «claim released», «Engineer 12 considers next task». Если есть только последняя строка «NPC idle», восстановить причину невозможно. Не обязательно хранить весь журнал вечно; кольцевой буфер на последние события и фильтр по Entity часто достаточно.

19.2. Метод локализации ошибки

Начни с первого неправильного значения в цепочке. Если экран пуст, проверь: Entity есть в Registry? Transform верный? asset загружен? render candidate создан? culling оставил объект? shader получил правильную матрицу? depth не скрывает? Не меняй exposure, пока кандидат вообще не дошёл до draw. Если NPC не стреляет, проверь: target допустим? visible? brain в Attacking? анимация и muzzle готовы? союзник на линии? cooldown? WeaponSystem вернул true? Такой порядок переводит огромный проект в последовательность проверок.

После нахождения причины создай минимальную сцену, где симптом воспроизводится. Исправь там, затем верни сложную сцену. Минимальная сцена нужна потому, что в большом корабле одновременно меняются двери, давление, power и AI. Для багов от FPS повтори при фиксированной simulation speed и разных render FPS. Для багов загрузки повтори clean start, а не только hot reload после старого состояния.

19.3. Диагностика утечки

Считай сущности, pool sizes, Bullet bodies/actions/constraints/shapes, COM resources, jobs в очереди и активные audio voices. При переходе сцены ожидаемые значения могут временно подняться во время импорта, но после завершения должны вернуться к устойчивому уровню. Если каждое переключение добавляет один body или shader resource, ищи владельца по типу. Не полагайся только на общий расход памяти Windows: allocator может держать освобождённые блоки, а маленькая утечка может долго не расти заметно. Счётчики объектов точнее связывают проблему с конкретным lifecycle.

Урок 20. Сборка, упаковка и внешний прогон

20.1. Что уже проверяет проект

`scripts/verify_engine.ps1` ищет MSBuild, проверяет shader entry points, собирает x64 Debug и Release, запускает core contract tests и проверяет наличие `assimp-vc143-mt.dll` рядом с exe. Это хороший технический входной gate. Но `scripts/run_engine.ps1` выполняет `Push-Location` в корень проекта и запускает exe оттуда. Поэтому его успех не доказывает, что отдельная папка с игрой содержит все assets и shaders. Не выдавай build из Visual Studio за релизный package.

20.2. Манифест runtime файлов

Составь список файлов, которые нужны игре: exe; runtime DLL; shader source/compiled assets в зависимости от текущей загрузки; модели, текстуры, анимации, сцены, конфигурации; лицензии сторонних компонентов и контента, если они требуются распространением. Для каждого файла запиши, где код его ищет: относительно process working directory, exe directory или настроенного asset root. Один согласованный asset root лучше набора случайных относительных путей. Если FileSystem создаёт несуществующий каталог, это не должно молча скрыть отсутствие обязательных игровых ресурсов.

Упаковка должна быть повторяемой. Скрипт копирует ровно перечисленные файлы в чистый каталог, проверяет хэши или наличие, запускает exe с рабочей директорией, отличной от проекта, и пишет лог ошибок загрузки. Не копируй весь исходный репозиторий: это скрывает недостающий манифест и увеличивает размер. Не удаляй важные файлы из package на глаз, если сначала не измерил обращения при загрузке миссии и всех материалов.

20.3. Чистая машина как другой мир

На машине разработчика уже установлены Visual Studio, SDK, DLL, кэш шейдеров и модели в исходных папках. У игрока этого нет. Поэтому полезна чистая Windows VM или другой компьютер без среды разработки. Проверь старт, меню, новую игру, загрузку save, смену разрешения, звук, ввод, игровой цикл и выход. Если игра падает, лог должен показать путь последней операции, а не только exit code. Для графических проблем запиши GPU, драйвер, режим окна и разрешение. Не делай высокоуровневых выводов по одному компьютеру.

20.4. Кандидат к выпуску

Кандидат имеет зафиксированный исходный commit, список assets, версию save и записи известных ограничений. После исправления критической ошибки собирай новый кандидат целиком и повторяй обязательные сценарии: новая игра, загрузка старого save, ремонт, бой, гибель NPC, пауза, смена сцены, завершение миссии. Не копируй один обновлённый exe поверх старого package без пересборки манифеста: shader или config мог измениться вместе с кодом. Отдельно храни символы отладки для диагностики падений, не обязательно включая их в пакет игрока.

20.5. Что ещё отличает почти релиз от технической демонстрации

Игрок понимает управление и цели без разработчика рядом. Сохранение и повторное открытие работают. Критические действия имеют звук и визуальное подтверждение. Камера, анимация, столкновения и UI не спорят друг с другом. Производительность измерена на целевой машине. Сцена не требует debug команд для выхода из тупика. Все используемые модели и звуки имеют понятные права использования. Известные ошибки перечислены честно, а блокирующие падения и потеря сохранений исправлены. Это практическое определение готовности, которое можно проверить, а не абстрактное «полировка».

Математический задачник для движка

Задачник нужен как рабочая тетрадь. Сначала реши пример вручную, затем найди соответствующий метод в коде и сравни числа с debug выводом. Запись формулы без единиц и пространства координат считается неполным решением. В каждом примере ниже объясняется не только вычисление, но и причина выбора метода.

Задача 1. Движение NPC к рабочей точке

Дано: позиция NPC A=(2; 0; 3) м, точка ремонта B=(8; 0; 11) м, скорость 2 м/с, шаг симуляции 0,1 с, допустимый радиус ремонта 1,5 м. Сначала вычти точки: B−A=(6; 0; 8) м. Длина 10 м. Единичное направление n=(0,6; 0; 0,8). Скорость как вектор — n·2=(1,2; 0; 1,6) м/с. Желаемое смещение за шаг — v·0,1=(0,12; 0; 0,16) м. Если нет препятствий, до границы рабочего радиуса осталось 8,5 м, то есть примерно 4,25 с движения.

Почему нельзя взять только `B.x-A.x` и `B.z-A.z` как готовую скорость? Тогда объект на диагонали двигался бы быстрее: длина вектора `(6;8)` равна 10, а не 1. Нормализация отделяет направление от скорости. Почему для ремонта игнорируем Y? `StateSystem::UpdateRepairTask` оценивает рабочую дистанцию по палубе. Для трёхмерного выстрела Y игнорировать нельзя: цель этажом выше может иметь нулевую горизонтальную дистанцию, но быть далеко по прямой.

Проверка в игре: временно отобрази A, B, длину d, текущий waypoint и горизонтальную скорость. Если персонаж не дошёл за 4,25 с, это не обязательно ошибка формулы: реальный путь может быть длиннее, дверь может ждать открытия, а Bullet может остановить капсулу у препятствия. Раздели «время по прямой» и «время по маршруту».

Задача 2. Угол зрения через dot product

Дано: forward=(0; 0; 1), цель в направлении (3; 0; 4). Длина направления равна 5, поэтому unitToTarget=(0,6; 0; 0,8). Dot(forward,unitToTarget)=0,8. Угол arccos(0,8)≈36,87°. При полном FOV 70° половина равна 35°, cos35°≈0,819. Условие `dot >= cos(halfFov)` не выполняется: цель немного за пределом. При полном FOV 90° порог cos45°≈0,707 и цель проходит.

Здесь особенно легко перепутать полный угол с половиной. Если поставить порог cos70°≈0,342 для поля зрения 70°, фактически разрешишь цели примерно до 70° в каждую сторону, то есть конус 140°. Другая ошибка — сравнить dot с 35 без cos. Dot всегда в диапазоне −1…1 для единичных направлений, поэтому сравнение с 35 никогда не выполнится. Третья ошибка — забыть нормализацию: dot с исходным `(3;0;4)` равен 4 и пройдёт почти любой порог.

Проверка: поставь цель строго впереди, на половинном угле и за спиной. Значения dot должны быть примерно 1, cos(halfFov) и −1. После FOV raycast всё равно нужен: цель за стеной внутри конуса не видна.

Задача 3. Выбор правильной стороны поворота

Дано: NPC смотрит вдоль +Z, цель находится справа по +X. Геометрически требуется повернуть корпус на 90° к цели. В какой знак yaw это превратится, зависит от соглашения вращения конкретного Transform и исходной оси модели. Поэтому не выводи знак из слова «справа» по памяти. Возьми тестовый basis vector +Z, примени к нему матрицу поворота на +π/2, посмотри, куда он попал в world space. Если в +X, знак найден; если в −X, нужен обратный. После этого проверь модель: её импортированный «forward» может быть −Z, и визуально персонаж окажется спиной к цели даже при верной мировой матрице.

Сравни два способа. `atan2(dx,dz)` часто используют для направления на плоскости XZ, но порядок аргументов и нулевое направление важны. В состоянии `Attacking` реальный `StateSystem.h` вычисляет

`atan2(dx,dz)+π`: добавка учитывает базовую ориентацию модели в этом участке. Альтернатива — строить целевое вращение из forward и desired direction, затем интерполировать кватернион. В обоих случаях при A=B угол не определён: сохраняй прежний поворот. При интерполяции через границу −π/π от 179° к −179° выбирай короткий ход 2°, а не полный оборот 358°.

Проверка: четыре цели по +Z,+X,−Z,−X и диагонали. Debug стрелка логического forward должна совпадать с направлением модели и ствола. Если стрелка верна, а визуальная модель неверна, исправляй model offset, а не AI.

Задача 4. Порядок матриц на точке

Дано: локальная точка p=`(1;0;0;1)`. Масштаб по X равен 2; потом объект переносится на +5 по X. При контракте вектор-строка `p·S·T` сначала даёт `(2;0;0;1)`, затем `(7;0;0;1)`. Если применить `p·T·S`, сначала будет `(6;0;0;1)`, затем `(12;0;0;1)`. Оба результата математически правильны для разных последовательностей действий; игровой объект должен соответствовать задуманному порядку.

Теперь добавь поворот. Для локальной точки `(1;0;0)` после масштаба 2 и поворота на 90° вокруг Y её расстояние от pivot остаётся 2, только направление меняется. После переноса на `(5;0;0)` расстояние от новой точки до `(5;0;0)` равно 2. Этот инвариант работает даже если конкретный знак Z зависит от соглашения осей. Если расстояние 7 или 12, перенос попал не туда в цепочке.

В проекте HLSL `pbr.vs` использует `mul(finalPos, worldMatrix)`, потом view, потом projection. CPU транспонирует матрицы перед записью в shader buffer. Не меняй геометрический порядок из-за слова transpose. Для диагностики запиши локальную точку, затем её world и view координаты, а не только экранный пиксель.

Задача 5. Нормаль при неравномерном масштабе

Возьми поверхность с касательной t=`(1;1;0)` и нормалью n=`(1;−1;0)`, нормализованными по желанию. Dot(t,n)=0. Применим к точкам масштаб S=`(2;1;1)`. Касательная станет t'=`(2;1;0)`. Если нормаль тоже просто умножить на S, получим n_bad=`(2;−1;0)`, и dot(t',n_bad)=4−1=3: нормаль перестала быть перпендикулярной. Применение inverse transpose к нормали даёт n_correct=`(1/2;−1;0)`, и dot(t',n_correct)=1−1=0. После этого нормаль нужно нормализовать.

Причина обратной матрицы не в «особом шейдерном трюке»: нормаль задаёт плоскость, а её уравнение должно оставаться согласованным после преобразования поверхности. Для rigid rotation inverse transpose совпадает с самим вращением, поэтому ошибка может годами не проявляться. Она становится видна на растянутых мешах. Проверь tangent и binormal: TBN должен быть согласован с новой нормалью, иначе normal map освещается в неверном направлении.

Задача 6. Экранная точка и picking ray

Дано: viewport 1920×1080, курсор `(1440;270)`. Нормализованные координаты: `x_ndc=2·1440/1920−1=0,5`, `y_ndc=1−2·270/1080=0,5`. В NDC Y растёт вверх, хотя экранная Y растёт вниз. Для центра `(960;540)` результат `(0;0)`. Это только координаты экрана: их нельзя напрямую передать Bullet как мировое направление.

Построй две точки в clip/NDC на ближней и дальней плоскости, восстанови world position через обратные projection и view, раздели на w. Затем `direction=normalize(farWorld−nearWorld)`. Если используешь перспективную камеру, origin луча обычно берут из положения камеры или nearWorld в зависимости от задачи, но направление должно соответствовать выбранной экранной точке. Для ортографической камеры направления всех экранных лучей параллельны, а origin различается. Способ с двумя мировыми точками и началом `nearWorld` работает для обоих типов проекции; способ с началом в позиции камеры требует учитывать тип проекции.

Проверка: щёлкни по четырём углам и центру, нарисуй лучи. Центральный луч совпадает с camera forward; верхний левый идёт через верхний левый угол фрустума. Если по вертикали всё отражено, потерян знак в формуле y_ndc.

Задача 7. Порог прибытия и гистерезис

Пусть NPC стоит в 1,49 м от WorkZone, а порог Repairing=1,5 м. При коррекции физики следующего тика расстояние стало 1,51 м; простое правило переключит его обратно в MoveToRepair. Затем движение уменьшит расстояние до 1,49, и цикл повторится. Визуально персонаж дёргается, а ремонт прогрессирует рывками. Сделай разные пороги входа и выхода: входить при ≤1,5 м, выходить при >1,8 м. Промежуток 0,3 м — зона устойчивого состояния.

Почему нельзя просто добавить большой `epsilon` ко всем расстояниям? Он может разрешить ремонт через стену. Гистерезис решает только переключение состояния; LOS, room, достижимость и рабочая точка остаются отдельными условиями. Подбирай пороги относительно радиуса капсулы и расположения WorkZone. Тот же приём полезен для attack/chase границы и выключения/включения питания при почти равном supply.

Задача 8. Полезность ремонта и цена маршрута

Две пробоины имеют одинаковую severity=0,8. До A по прямой 3 м, но путь через дверь 20 м; до B по прямой 8 м и путь 9 м. При detectionRange=20 м текущая близость A равна 1−3/20=0,85, B равна 0,6. Текущая база score A: 0,8·0,65+0,85·0,35=0,8175. B: 0,8·0,65+0,6·0,35=0,73. С одинаковыми ролью и skill A выигрывает. Но если заменить геометрическую дистанцию оценкой пути, близость A станет 0, B — 0,55, и базы будут 0,52 против 0,7125. B станет разумнее.

Не обязательно выполнять дорогой path query для каждой пары NPC–задача каждый кадр. Используй грубый геометрический фильтр, кэш длины маршрута с revision NavMesh/дверей и обновляй выбор при изменении мира. Недостижимая цель получает не просто «очень длинный путь», а явный статус. Проверь, что сохранённое назначение не удерживается бесконечно после закрытия двери.

Задача 9. Прицеливание и движение цели

Базовый hitscan в текущем оружии проверяет луч в момент выстрела; поэтому для него не нужно рассчитывать баллистическое упреждение пули с временем полёта. Если позже добавишь физический снаряд со скоростью s, цель движется со скоростью v, стрелок в p, цель в q. Нужно найти t≥0, для которого `|q+v·t−p|=s·t`. Квадрат обеих сторон даёт `(v·v−s²)t² + 2(r·v)t + r·r = 0`, где r=q−p. Выбирай наименьший неотрицательный корень. Если коэффициент при `t²` равен нулю, решай оставшееся линейное уравнение; если и оно вырождено, отдельно проверь случай r=0. Если неотрицательного действительного решения нет, снаряд не может догнать цель при принятых постоянных скоростях.

Простой случай: цель в 10 м и движется прямо от стрелка со скоростью 2 м/с, снаряд летит 12 м/с. Время перехвата t=10/(12−2)=1 с, точка прицеливания на 12 м от стрелка. Прицеливание в текущие 10 м даст промах позади цели. Но использовать эту формулу для hitscan означало бы искусственно вести прицел впереди без необходимости. Сначала определи физическую модель оружия, потом выбирай математику.

Задача 10. Физика взрыва и импульс

Взрыв даёт телу массой 20 кг импульс J=40 Н·с. Из `J=m·Δv` следует Δv=2 м/с в направлении импульса. Если тело в 5 м от центра, а второе в 10 м, выбери закон ослабления, например `factor=max(0,1−r/R)` для радиуса R=15 м. Тогда факторы 2/3 и 1/3. Это игровая модель, а не

физическое следствие самого слова «взрыв»; её надо явно настроить. Не используй `1/r²` без epsilon и верхнего ограничения: при r≈0 импульс станет огромным.

Если взрыв вызван разгерметизацией, количество движения воздуха и корпуса должно иметь понятный источник, но движок может применять упрощённый импульс ради игры. Документируй допущение, чтобы позже не перепутать его с точной симуляцией. Проверяй массы: одинаковый импульс меньше меняет скорость тяжёлого тела. Проверь точку приложения: импульс не через центр масс добавляет вращение.

Задача 11. Аккумулятор и мощность

Источник выдаёт 3 МВт в течение 10 с. Отданная энергия `E=P·t=30 МДж`. Если аккумулятор хранит 0,01 МВт·ч, это `0,01·3,6·10⁹=36 МДж`; его достаточно примерно на 12 с при постоянных 3 МВт без потерь. Число 0,01 может выглядеть маленьким, но единица МВт·ч крупная. В коде не смешивай МВт, МДж и Вт·ч в одном float без названия единицы. Разряд должен зависеть от dt и не уходить ниже нуля.

Если потребитель требует 4 МВт, а получает 3, powerRatio=0,75. В текущей сети `hasPower` истинно лишь около полного обеспечения. Поэтому устройство с частичной выдачей может считаться выключенным; это предметное правило. Для нового устройства реши: оно останавливается ниже 100%, работает с 75% производительности или имеет порог 50%. Правило должно быть одинаковым в gameplay, HUD и save.

Задача 12. Согласованность симуляции после сохранения

Перед сохранением пробоина выпускает 0,2 единицы массы воздуха в секунду, давление падает, инженер имеет claim, дверь закрыта. После загрузки сохранённые authoring и runtime данные восстанавливаются, но временные claims и Bullet pointers создаются заново. Если прогресс ремонта 0,4 сохранился, новая работа должна продолжиться с 0,4 либо по явно выбранному правилу начаться с нуля. Оба поведения возможны, но смешение — ошибка: UI может показывать 40%, а ApplyRepairProgress читать 0.

Составь контрольную таблицу до/после: число живых Entity; room links; health генератора; water/air mass; дверь; repair progress; текущая цель NPC; число Bullet bodies и GPU resources. Для физических величин сравни с погрешностью, например $|a-b| \le 1e-4 \cdot \max(1,|a|)$, а не битовое равенство float. Для идентификаторов после remap сравни логические связи, а не прежние runtime числа Entity. Этот приём превращает «после загрузки всё странно» в список проверяемых расхождений.

Практикум создания космического симулятора

Следующие лабораторные работы не заменяют четырнадцатидневный план в конце книги. Они показывают, как выполнить законченный результат по любому удобному графику. Каждая работа начинается с наблюдаемого поведения, затем проходит по данным и коду, а заканчивается проверкой отказов. Если базовая сцена не воспроизводится, вернись к Уроку 3 и сократи число объектов.

Лабораторная 1. Генератор питает дверь и кислородный прибор

Результат для игрока

На терминале видно, что генератор выдаёт мощность; дверь и прибор работают. После повреждения генератора мощность падает, дверь не открывается, а прибор перестаёт поддерживать атмосферу. После ремонта состояние возвращается. Игрок должен понять причину через интерфейс, а не по случайному изменению цвета лампы.

Данные и подготовка

Создай один источник мощности и два потребителя. Для источника проверь GeneratorComponent и его включённость/health; для потребителей — PowerConsumerComponent с требуемой мощностью и приоритетом. Связь с конкретной сетью должна быть определена данными сцены, иначе два независимых корабля начнут делить одну мощность. Поставь `DamageableComponent` и `RepairableComponent` на генератор, а WorkZone перед панелью, куда может подойти инженер. Сделай дверь частью графа двух комнат и добавь требование питания.

Пусть генератор производит 5 МВт, дверь требует 1 МВт, кислородный прибор 3 МВт. Запас 1 МВт остаётся для будущих систем. При повреждении генератор выдаёт 2 МВт: согласно приоритетам одним устройствам может достаться полное питание, другим частичное. Не ожидай, что оба просто переключатся одинаково; читай `ShipPowerGridSystem::SupplyEvenly` и реальные приоритеты. Для каждого потребителя показывай `requiredMw`, `suppliedMw`, `powerRatio`, `hasPower`. Если у прибора есть `hasPower=false` при powerRatio 0,66, это соответствует текущему почти полному порогу, а не ошибке HUD.

Путь реализации

Сначала создай предметы в сцене и проверь PowerGrid без урона. Затем вызови повреждение через существующий DamageSystem, а не прямую запись health в панели UI. После изменения health проверь, что GeneratorSystem действительно меняет production, а PowerGrid пересчитывает выдачу в следующем нужном тике. Теперь привяжи DoorSystem и OxygenGeneratorSystem к результату питания. Последним добавь ремонт: координатор назначает инженера, StateSystem доводит до WorkZone, RepairSystem восстанавливает оборудование. В этом порядке каждое звено можно диагностировать отдельно.

Проверка и признаки ошибки

Выключи генератор: дверь должна объяснить отказ, а прибор — показать отсутствие питания. Включи аккумулятор: результат должен соответствовать мощности и времени разряда, а не постоянному бесплатному питанию. Удали генератор: система не должна читать указатель на уничтоженный компонент. Сохрани и загрузи сцену: связи потребителей с сетью и рабочей зоны с генератором сохраняются через remap. Поставь второй корабль: энергосети не смешиваются. Если прибор работает при нулевой выдаче, ищи кэш прошлого кадра или обход `hasPower`.

Лабораторная 2. Две пробоины и два инженера

Результат для игрока

После двух повреждений корпуса на разных палубах инженеры получают разные аварийные работы, если обе цели входят в `detectionRange` и их score выше `idleScore`. Один путь может быть длиннее, но доступен, другой может быть закрыт дверью без питания. Сейчас `Blocked` показывает причину, но само по себе не освобождает `CrewTaskClaimComponent`: координатор продолжает удерживать валидное назначение. Автоматическое освобождение недостижимой задачи — упражнение на доработку. После успешного ремонта утечка прекращается, а работник может получить другую полезную задачу; чтобы `WorldEventLog` сообщил о ремонте именно NPC, проведи его в `StateSystem::UpdateRepairTask` и передай в `RepairSystem::ApplyRepairProgress` вместе со временем и источником.

Постройка сцены

Сделай три комнаты, две двери и две пробоины с различными leakRate. Один инженер стоит рядом с серьёзной пробоиной, второй — далеко, но внутри своего `detectionRange`. Задай обоим Active, CrewMember onDuty, NpcBrain, Movement, Transform и корректный controller. Начни с открытых дверей. Прежде чем запускать AI, проверь NavMesh debug: стартовые позиции NPC и пути подхода к пробоинам должны быть достижимы. `WorkZoneComponent` текущий `StateSystem` ищет для оборудования, а не для пробоины. Координатор не проверяет достижимость пробоины: даже точку за стеной он может счесть пригодной по компонентам и прямому расстоянию, а движение затем застрянет. Поставь пробоины там, где персонаж может подойти на `repairDistance`, и зафиксируй этот предел как отдельный тест.

Прогон назначения

На первом кадре `NpcUtilitySystem` вычисляет repairScore; затем `CrewTaskCoordinator` собирает кандидатов и назначает их в устойчивом порядке Entity ID. Запиши score каждой пары работник–задача, занятые claims и причину выбора. При two tasks и two workers второй не должен выбрать уже заявленную цель. Не меняй коэффициенты наугад, если распределение кажется странным: в score входит геометрическое расстояние и роль, но пока не гарантирована длина маршрута. Для обходного пути нужен отдельный улучшенный критерий.

Когда один инженер получает ручную команду Stop, его automatic claim освобождается. Другой работник сможет взять работу, если она подходит. После ResumeAutomatic первый должен получить новое решение без использования старого `doorRoute`. Во время ремонта удаление пробоины заставляет StateSystem и Coordinator безопасно перечитать Entity. Не держи в debug UI указатель на старый компонент после завершения.

Сложные случаи

Для закрытой двери без питания ожидай `Blocked`: граф комнат может не найти проход, а `DoorComponent::TryOpen` не откроет запитанный затвор без энергии. Для отдельного опыта оставь питание и наблюдай `WaitingForDoor`, пока `openProgress<0,65` возле двери. Убей назначенного инженера: на следующем проходе координатора claim должен освободиться. Перемести пробоину во время движения: `StateSystem` перечитает `worldPosition`, а при включённом NavMesh локальный путь перестроится при смещении цели больше 0,60; `NpcCommand::revision` от одного перемещения пробоины не увеличивается. Если цель перешла в другую комнату, обнови и `HullBreachComponent::room`. Сделай `repairDuration` очень малой: завершение не должно дважды удалить Entity. Поставь паузу: progress остаётся постоянным. Повтори при 30 и 120 FPS: длительность ремонта в игровых секундах примерно совпадает.

Лабораторная 3. Враг видит, ищет и стреляет честно

Результат для игрока

Враг замечает игрока на открытой палубе, наводится и стреляет; при стрельбе на ходу тактический компонент ограничивает огонь первой короткой очередью. Текущий `movingBurstPauseTimer` после неё не уменьшается, поэтому перед проверкой повторных очередей исправь этот таймер или зафиксируй известный отказ в отчёте. Затем враг теряет контакт за укрытием, идёт к последней известной точке и ищет. Он не поворачивается мгновенно к скрытому игроку. Если союзник пересекает линию огня, стрелок удерживает выстрел. Если путь от груди до дула закрыт стенкой, выстрел блокируется.

Состав сущности

Враг должен быть Active, иметь NPCComponent, NpcBrainComponent, NPCMovementComponent, Transform, FactionComponent, NPCCombatComponent, физический controller и визуальную модель с корректной анимацией. Для дальнего боя нужно экипированное оружие с WeaponComponent, корректным `rightHandBone` и локальной `muzzlePosition`: из хвата, кости и мировой матрицы система вычисляет мировое дуло. Отсутствие любого из ключевых компонентов имеет наблюдаемый отказ. Не используй старый `isHostile=true` как единственное разрешение боя: отношение конкретной пары задаёт FactionComponent.

План диагностики одного контакта

Сначала проверяй фракцию и существование цели. Затем distance² и FOV. Затем нарисуй луч зрения от головы к выбранной части цели. По текущему условию видимости первое пересечение должно быть самой целью либо луч не должен встретить коллайдер до проверяемой точки. Стена как первое пересечение означает `targetVisible=false`. Смотри `targetVisible`, `visibleAimPoint`, `lastKnownPosition`. Далее смотри `brain.state` и приоритет укрытия. После анимации нарисуй muzzle origin и направление. Наконец смотри `rangedStatus` и WeaponSystem cooldown. Если враг не стреляет, «оружие сломано» — последняя гипотеза, когда предыдущие проверки пройдены.

При потере LOS запиши старую lastKnownPosition. Перемести игрока за стеной в другое место. Враг не должен изменить маршрут на новую позицию без нового наблюдения или честного звукового события. Если изменил, найди чтение живого Transform скрытой цели в поиске/cover. При появлении игрока снова память должна обновиться; старые search sectors сбрасываются, иначе враг после второго контакта может считать, что уже осмотрел все точки.

Проверка огневой дисциплины

Поставь второго NPC той же фракции между стрелком и игроком. Центральный луч от дула должен первым вернуть союзника и удержать огонь. В результате нет вызова `WeaponSystem::TryFire` для этого кадра. После ухода союзника повторно проверь личную видимость, прицеливание, cooldown, reload и `movingBurstPauseTimer`: при непрерывной стрельбе на ходу последний сейчас может оставаться положительным бесконечно. Проверь также наклон модели и высоту укрытия: глаз может видеть цель, а дуло быть закрыто. Для полной защиты от friendly fire можно проектировать более широкий коридор или учитывать разброс, но пометь такую доработку как отдельное правило.

Лабораторная 4. Новый пожар и тушение экипажем

Правила мира до кода

Очаг начинается с intensity 0,2. При достаточном кислороде растёт на 0,05/с до 1; при недостатке кислорода убывает на 0,08/с; тушение инженером уменьшает на `0,15·skill` в секунду. Эти числа — учебные игровые параметры, а не физическая модель горения. В сценарии огонь нагревает устройство и может снизить его health, но за один тик не должен создавать новый пожар на самом

себе. Прежде чем писать визуализацию, проверь за 10 секунд на бумаге: без тушения intensity вырастет примерно до 0,7; при skill 1 и тушении в течение 2 секунд уменьшится на 0,3 без учёта одновременного роста. Явно выбери, суммируются ли рост и тушение.

Компоненты и порядок кадра

Добавь FireComponent с интенсивностью, комнатой, коэффициентами и ссылкой на источник. `FireSystem` читает текущий кислород/температуру, рассчитывает следующий шаг и публикует пожарное событие при переходе порога. Не отправляй тревогу каждый кадр, иначе журнал и звук заполнятся дублями. Оборудование получает damage через единый DamageSystem. CrewTaskCoordinator видит новый тип задачи после появления очага; StateSystem доводит назначенного работника до WorkZone; действие тушения уменьшает intensity. Renderer и particle system читают intensity после симуляции.

Определи порядок относительно атмосферы. Если FireSystem на шаге k читает кислород после расчёта комнаты, а расход записывает на шаг k+1, это одна явная задержка. Если нужно одновременное решение, разбей обновление на фазы «собрать потоки» и «применить». Не заставляй FireSystem напрямую вызывать RoomSystem посреди его обхода. В debug выводи кислород до шага, после шага, рост, тушение и итог intensity, иначе отрицательный огонь придётся искать визуально.

Расширение AI и отказов

В `CrewTaskType` добавь ExtinguishFire. Score может зависеть от intensity, близости, ценности соседнего оборудования и доступного ресурса. Claim хранится на очаге, как ремонтный claim на цели. Для ручного приказа добавь метод `NpcCommandSystem::ExtinguishEntity`, который вызывает общий Begin и сбрасывает маршрут; StateSystem выполняет новую команду. NPC с пустым огнетушителем не должен навсегда занимать очаг. При исчезновении огня маршрут и claim освобождаются, а анимация тушения прекращается. При закрытой двери с отсутствием питания задача получает ясный Blocked статус.

Контроль результата

Проверь: один очаг и один инженер; два очага и два инженера; очаг без кислорода; нет ресурса; смерть работника; удаление источника; save/load; пауза; очаг вне камеры. Только после этого добавляй красивый дым и свет. Если частицы есть, а FireComponent не существует, это визуальный декор, а не системная авария. Если FireComponent существует, но игрок не понимает, что делать, нужен интерфейс и сценарная подача.

Лабораторная 5. Релизная сборка и проверка вне исходников

Технический gate проекта

Скрипт `scripts/verify_engine.ps1` собирает x64 Debug и Release по MSBuild, вызывает проверку shader entry points и core contracts; параметр `-Configuration` позволяет выбрать Debug, Release или All. `scripts/run_engine.ps1` проверяет exe, `assimp-vc143-mt.dll` и assets, но запускает игру из корня проекта через `Push-Location`. Поэтому успешный `run_engine.ps1` доказывает работоспособность рабочего дерева, но не доказывает переносимость отдельно упакованной игры.

Для кандидата к выпуску создай отдельный каталог с exe, нужными DLL, shaders, assets, сценами и конфигурацией. Путь к нему должен быть определён манифестом сборки, а не ручным копированием «что вспомнил». Запусти exe из другого current working directory и проверь, что FileSystem корректно находит runtime root. Если ресурс отсутствует, приложение должно показать точную ошибку и имя файла. Обучающие временные assets и огромные исходные blend файлы не должны автоматически попадать в package, если игра их не использует.

Игровой gate

Пройди выпускную работу Урока 16 с чистым профилем и без редактора. Сохранись перед боем, выйди из игры, снова загрузи save. Убедись, что инженер не получает вторую копию своей задачи, дверь остаётся в правильном состоянии, пробоина не появляется снова, патроны и health согласованы. Поставь симуляцию на паузу и вернись. Убей NPC во время навигации. Сменяй сцену при фоновой загрузке модели. Несколько раз перезапусти game loop. Каждая из этих проверок пересекает несколько слоёв и потому лучше простого запуска главного меню.

Полезный отчёт об ошибке

Записывай build configuration, commit, имя сцены, seed при наличии случайности, действия игрока, ожидаемый и фактический результат, значения debug статусов, скриншот или короткое видео. «NPC тупит» нельзя воспроизвести; «после команды Stop claim на пробоине Entity 84 остаётся у NPC Entity 12, второй инженер не берёт задачу» уже является технической задачей. Исправление готово, когда сценарий повторно проходит и не ломает соседние случаи.

Единственный план развития: 14 дней

Это единственный календарный план книги. Он предполагает, что учебная сцена и базовый движок уже запускаются. Все дни продолжают одну игру на одной палубе корабля: так ошибки интеграции становятся видимыми сразу. Текущий код отличай от предложенных расширений: пожар, слух и полноценная тактика отряда в этом плане являются задачами разработки. В конце каждого дня оставляй запускаемую сборку, короткое видео или журнал прогона и запись о том, что осталось неизвестным. Если задача оказалась больше дня, сохрани работающий вариант и сократи объём следующего шага, не пропуская проверку.

78. День 1. Воспроизводимая сцена и базовая диагностика

Собери две комнаты и соединяющую их дверь. В первой находятся генератор, потребитель энергии, один инженер и оператор; во второй — пробоина, укрытие, игрок и два противника. Для каждого объекта запиши Entity, необходимые компоненты, начальное состояние и ожидаемую реакцию на команды. Закрепи координаты и параметры сцены в файле, чтобы повторный запуск не менял исходную расстановку. Не подключай новые механики, пока не можешь повторить текущий сбой.

Сделай один экран диагностики: NPC state, текущая цель, claim, следующий waypoint, расстояние до WorkZone, видимость, lastKnownPosition, статус оружия, текущие HP, состояние двери и её питания. Статус должен отвечать на вопрос «почему не действует», а не только показывать число. Например, `NoPath`, `DoorUnpowered`, `MuzzleBlocked` и `Reloading` дают разные следующие действия разработчику.

Прогони четыре сценария: автоматический ремонт, ручной приказ ремонта, обнаружение игрока на открытом поле и потеря видимости за дверью. Сохрани начальные значения и итоговые состояния. День готов, когда другой человек может загрузить сцену и получить те же переходы, а каждое бездействие NPC имеет наблюдаемую причину.

79. День 2. Назначение и освобождение ремонта

Разбери полный путь от `RepairableComponent` до завершения `RepairSystem`. Координатор выбирает кандидата и фиксирует claim; utility AI выбирает действие; `StateSystem` идёт к рабочей точке и наращивает прогресс. Для двух инженеров и двух пробоин проверь четыре перестановки расстояний и severity. Сравни рассчитанный score с выведенным в UI, чтобы исключить невидимую ошибку нормализации расстояния или skill.

Определи контракт claim: кто его создаёт, когда он продлевается и в каких случаях освобождается. Смерть инженера, ручной приказ, удаление цели, завершение ремонта, перезагрузка сцены и потеря маршрута должны иметь явную ветку. Если два исполнителя законно работают вместе, введи capacity цели; иначе одна цель имеет одного владельца. После `ApplyRepairProgress` перечитывай Entity: завершение может удалить пробоину и старую ссылку нельзя использовать.

День готов, когда пробоина ремонтируется без дублирования работы, второй инженер берёт свободную задачу, а каждый сценарий отмены освобождает claim в тот же игровой цикл или по документированному тайм-ауту. Проверь случай, когда более близкая цель уже занята: NPC должен выбрать другую или объяснить ожидание.

80. День 3. Достижимость и двери до выбора цели

Прямая дистанция не равна длине пути: соседняя через стену пробоина может быть недоступна, а дальняя по коридору доступна. Добавь дешёвую проверку room connection и состояния двери перед окончательным назначением. Для оставшихся кандидатов используй NavMesh путь или кэшированную приблизительную цену маршрута. Записывай, какой фильтр исключил каждую цель. Не запускай десятки дорогих path queries каждый кадр: обновляй кэш при изменении двери, NavMesh revision или позиции цели.

Раздели три состояния: маршрута нет, маршрут есть, но дверь закрыта, и путь есть, но Bullet не даёт пройти из-за динамического препятствия. Для первого освободи claim и возьми другую работу; для второго попробуй открыть дверь и проверь питание; для третьего включи stuck timer и ограниченное перепланирование. Избегай бесконечного повторения `TryOpen` или path query без задержки. Рабочую точку проецируй на доступную область, а не на геометрический центр пробоины.

День готов на сцене с двумя авариями: близкая за запертой дверью и дальняя доступная. Экипаж выбирает доступную. Когда питание двери восстановлено, приоритет пересчитывается; NPC проходит после фактического открытия collider. Отладочный маршрут и физическое положение должны совпадать с объяснением решения.

81. День 4. Роли экипажа и расход материалов

Определи минимум четыре роли: инженер, оператор, медик и охрана. Для каждой укажи подходящие задачи, скорость, порог отказа и случаи, когда роль может временно заменить другую. Не полагайся только на множитель score: role affinity ниже единицы не должен запрещать аварийную помощь, если специалиста нет. Сначала зафиксируй правило выбора, потом меняй веса. Пусть действия имеют стоимость: ремонт корпуса тратит материал, тушение позже потратит заряд огнетушителя, лечение — аптечку.

Ресурс проверяй при выборе и перед фактическим применением. Если ресурс исчез между назначением и началом работы, освободи claim, покажи `MissingMaterial` и разреши NPC выбрать снабжение или другое действие. Не списывай предмет каждый кадр: прогресс и расход должны быть связаны с завершением шага, а сохранение — хранить уже списанный объём. UI отображает роль, назначение, инвентарь и причину приоритета, чтобы игрок мог переопределить работу осознанно.

День готов, когда инженер ремонтирует корпус быстрее оператора, медик не занимает работу без нужного ресурса, ручной приказ имеет понятный результат, а отсутствие специалиста не блокирует критическую задачу навсегда. После reload инвентарь и незавершённый прогресс согласованы.

82. День 5. Пожар как часть симуляции корабля

Добавь один очаг с состояниями `Igniting`, `Burning`, `Dying` и `Extinguished`. Храни интенсивность, температуру или тепловой запас, запас горючего, room ID и время последнего обновления. Определи, чем питается огонь: при отсутствии кислорода или топлива интенсивность не растёт бесконечно. Выбери простую, ограниченную модель: за шаг огонь расходует часть топлива и кислорода, нагревает помещение и наносит урон находящемуся рядом устройству. Значения и единицы запиши рядом с компонентом.

Вычисление зависит от simulation dt, а не от частоты рендера. Сначала проверь один очаг без NPC при 30, 60 и 120 FPS: через одинаковое игровое время температура и расход топлива близки. Затем закрой дверь и выключи вентиляцию; условия должны изменить рост пожара по заранее записанному правилу. Систему поставь в кадре до выбора NPC, если решение о тушении использует свежую интенсивность, или объясни задержку на один тик.

День готов, когда пожар влияет на устройство, энергию или атмосферу, виден в UI, продолжается вне камеры и не создаёт отрицательное топливо. Останови игру, сохрани и загрузи сцену: пожар либо продолжается с тем же состоянием, либо явно начинается заново согласно выбранному формату сохранения.

83. День 6. Экипаж тушит пожар

Для нового типа работы сначала создай минимальную вертикальную цепочку: `FireComponent` → доступный WorkZone → score координатора → claim → маршрут → состояние тушения → расход заряда → снижение интенсивности → завершение. Начни с одного очага и одного NPC; только

после работающего цикла добавляй нескольких исполнителей. Укажи, разрешено ли совместное тушение: если да, очаг должен иметь capacity и разделять рабочие точки, иначе второй NPC не должен занимать тот же подход.

Сделай ручную команду `Extinguish` и возврат в автоматический режим. Отмена приказа освобождает claim и не оставляет NPC в старом состоянии движения. Если огнетушитель пуст, NPC получает явный статус и ищет ресурс или другую работу; огонь не должен уменьшаться из-за одной анимации. Если путь перекрыт дверью без питания, применяй тот же контракт ожидания и отказа, что при ремонте. После тушения не используй уничтоженный Entity очага через сохранённую ссылку.

День готов, когда автоматическое и ручное тушение дают одинаковые физические последствия, расходуют одинаковый ресурс и завершаются одинаковым событием. На повторном прогоне два NPC не дублируют claim и не замирают после удаления очага.

84. День 7. Восприятие врага без всеведения

Разведи личное зрение, последний контакт и слуховое предположение как разные факты с разной точностью. Существующая проверка FOV и raycast отвечает только за личную видимость. После потери контакта сохрани последнюю видимую точку и время; последующее перемещение скрытого игрока не обновляет их. Стекло, низкое укрытие, дверь и высота глаз требуют отдельных тестов. Начало raycast должно соответствовать актуальной позе или документированному fallback.

Поставь игрока за закрытой дверью, затем открой её, затем уйди за укрытие. Запиши последовательность `visible`, `aimPoint`, `lastKnownPosition`, `state`. Враг сначала не видит, затем видит, затем идёт к последней видимой позиции и ищет там. Если он разворачивается прямо к скрытому игроку, отследи, какая система переписала память. При сообщении от товарища передавай источник и точность; чужое наблюдение не превращается в личный флаг `targetVisible`.

День готов, когда игрок способен обмануть врага скрытым перемещением, а debug показывает источник знания. Не настраивай сложность повышением точности сверх информации, которой NPC реально располагает.

85. День 8. Огонь, анимация и перезарядка

Проследи один выстрел от решения `NpcCombatSystem` до `NpcRangedCombatSystem`, `WeaponSystem::TryFire`, Bullet raycast и `DamageSystem`. Покажи отдельно луч зрения от глаз и луч выстрела от дула. Состояние `Attacking` не означает, что выстрел разрешён: требуются патроны, завершённая перезарядка, истёкший cooldown, личная видимость, допустимый угол, свободный muzzle ray и отсутствие союзника на линии. Выводи первый отказ, чтобы «NPC не стреляет» перестало быть загадкой.

Обновление позы должно предшествовать вычислению muzzle socket. Проверь повороты модели по четырём сторонам и стрельбу из-за низкого укрытия. Случайный разброс задавай одним источником для реального луча и визуального эффекта, иначе трассер покажет попадание в одно место, а урон придёт в другое. Проверь empty magazine: выстрелы прекращаются, начата reload, после задержки магазин пополнен из резерва, дальнейшие очереди снова используют общий cooldown. Отдельно исправь текущий `movingBurstPauseTimer`: после первой очереди он выставляется, но не уменьшается в обновлении, поэтому стрельба на ходу может остаться в `Moving: pause between bursts`. Считай таймер по `dt` и проверь две последовательные очереди при 30, 60 и 120 FPS.

День готов, когда открытая цель получает ожидаемый урон, стена блокирует дуло, союзник удерживает огонь, а каждый отказ отображён. Измени FPS вдвое: число выстрелов за игровую секунду остаётся согласованным.

86. День 9. Укрытия и выход из тупиков

Определи для точки укрытия две позиции: безопасная стоянка и точка выглядывания. Обе должны быть достижимы капсулой; из второй дуло должно видеть цель. Оценка укрытия учитывает ближайшую угрозу, длину маршрута, занятость, направление входящего огня и запасной выход. Резерв освобождается при смерти, смене цели, отмене тактики, разрушении укрытия и уходе из сцены. Храни причину последнего отказа, а не только ID выбранной точки.

Поставь два укрытия рядом и одно за непроходимой стеной. Прогони входящий огонь, потерю цели, смерть владельца резерва и перекрытие пути ящиком. Добавь минимальное время удержания выбора и разные пороги смены, чтобы NPC не перебегал между точками каждый кадр. Stuck timer измеряет фактическое продвижение Bullet, а не лишь наличие waypoint. После ограниченного числа попыток NPC получает запасное действие: отступить, стрелять с текущей позиции или искать другой путь.

День готов, когда NPC способен объяснимо занять, использовать и освободить укрытие и не остаётся навсегда в `MoveToCover` при недостижимой цели.

87. День 10. Первая командная тактика

В текущем `ApplicationClass.cpp` вызов `NpcSquadCombatSystem::Update` закомментирован. Включай его лишь после успешных индивидуальных тестов дней 7–9. Начни с двух бойцов и одного контракта обмена сообщениями: наблюдатель сообщает последнюю увиденную комнату или точку с временем, второй использует её для поиска, но сам не получает `targetVisible=true`. Ограничь время жизни сообщения и показывай его источник в debug.

Затем введи две роли: один удерживает угол, другой меняет позицию по доступному маршруту. Роль назначается на короткий интервал, чтобы оба NPC не переключались на каждом кадре; при смерти участника оставшийся возвращается к индивидуальному AI. Избегай общего указателя на текущую позицию игрока: отряд должен координировать имеющиеся сведения, а не читать игровое состояние без наблюдения. Проверяй договор с `NpcCoverSystem`, чтобы две системы не перезаписывали один brain.state без приоритета.

День готов, когда одиночный враг ведёт себя как прежде, пара распределяет направления и не занимает одну точку, а потерянная цель остаётся потерянной для обоих до нового свидетельства.

88. День 11. Авария корабля меняет решения

Соедини уже существующие давление, затопление, энергетическую сеть, ремонт и дверь в одну цепь причин. Пробоина или пожар выключает важный прибор; дверь без питания блокирует часть экипажа; команда игрока или восстановление питания возвращает доступ. Для каждой стрелки цепи укажи владельца данных и момент кадра. Следи за единицами: power в ваттах, энергия аккумулятора в джоулях или Wh, давление в паскалях, приток воды в объёме за секунду. Отрицательные значения и деление на нуль ограничивай на границе системы.

Добавь приоритет спасения: критичная комната повышает ценность ремонта, но не отменяет достижимость и наличие ресурса. Если все кандидаты недоступны, экипаж должен сообщить причину и продолжить другую полезную работу. Враги могут использовать физически открывшийся проход, но не обязаны знать невидимое состояние аварии. HUD показывает последствие: упавшее питание, изменение давления, затопление и назначенного работника.

День готов, когда одна авария меняет минимум два решения NPC и одно правило мира, а отключение любого звена воспроизводимо объясняет получившийся результат.

89. День 12. События, звук и тревога

Добавь события мира для выстрела, удара, пробоины, пожара и окончания ремонта. Событие содержит тип, время, позицию, интенсивность, источник и срок жизни. Это общие данные, а

воспроизведение звука является отдельным потребителем: отсутствие audio устройства не должно выключать реакцию NPC. Если в проекте нет полноценного звукового слоя, сначала реализуй журнал и debug визуализацию радиусов, затем подключай аудио.

Слух NPC проверяет расстояние, помещение, дверь и затухание. Он возвращает предполагаемую область, а не точный Entity игрока. Одинаковый звук может быть не слышен через герметичную дверь и слышен через открытый проход; выбери правило и сохрани его. Общекорабельная тревога передаёт известный тип аварии и комнату, но не сообщает местоположение скрытого врага без источника данных. Ограничь повтор событий, чтобы один длинный пожар не порождал тысячи одинаковых сообщений в секунду.

День готов, когда враг идёт к приблизительной области звука, после личного контакта прицеливается точнее, а в журнале видно различие источников сведений. Выключи звук на компьютере: игровой AI продолжает реагировать на события мира.

90. День 13. Сохранение, нагрузка и устойчивость

Раздели authoring параметры сцены и runtime состояние продолжаемой игры. Сохраняй фракции, роли, инвентарь, HP, постоянные аварии, ресурс аккумулятора и прогресс задач только там, где определён владелец. Не сериализуй `btRigidBody*`, GPU handles, временные claims, кэш пути или ссылки на кадр. После загрузки выполни Entity remap, восстанови Bullet тела и runtime кэши, затем проверь инварианты: на одну эксклюзивную задачу один claim, мёртвый NPC не стреляет, дверь и атмосфера согласованы.

Запусти сцену с несколькими NPC и очагами, измерь CPU время AI, NavMesh, физики и обновления симуляции, а также время рендера. Измеряй отдельно paused Editor и Play. Самый дорогой путь оптимизируй только после профиля: например, кэширование запросов пути или редкое обновление дальних NPC, но не отключение всей симуляции вне камеры. Прогоняй одинаковый сценарий на 30, 60 и 120 FPS; мировые результаты должны оставаться близкими.

День готов, когда save/load дважды подряд не создаёт копий пожара или claim, scene reload не оставляет старых Bullet объектов, а таблица профиля указывает измеренные, а не предполагаемые узкие места.

91. День 14. Вертикальный срез и кандидат к выпуску

Сыграй эпизод от входа до победы или поражения без редакторского вмешательства. В начале игрок получает понятную цель; затем происходит пробоина и пожар, экипаж делит работу, один ресурс заканчивается, враги обнаруживают и теряют игрока, энергия и атмосфера меняют доступность помещений. Игрок отдаёт ручной приказ, отменяет его, сохраняет игру, загружает её и завершает миссию. Зафиксируй время прохождения, непонятные решения NPC и места, где UI не объясняет событие.

Собери Release и упакуй executable, DLL, shaders, assets, scenes и настройки в отдельный каталог. Проверь запуск из этого каталога после переименования или переноса, а затем на другой машине или чистом Windows профиле. Внешний тестировщик должен пройти сценарий по игровым подсказкам. Запиши каждый дефект с шагами повторения, ожидаемым и фактическим результатом; устрани ошибки запуска, сохранения и невозможности завершить миссию до полировки эффектов.

День готов, когда есть один завершённый игровой эпизод, проверенная независимая сборка и короткий список оставшихся задач с приоритетами. Это кандидат к выпуску, если игрок может понять цель, действовать и закончить игру без разработчика рядом.